算法模板板子

生成时间: 2026-10-14

默认省略通用 C++ 头文件与空壳 main

目录

1 通用模板 5
1.1 cpp 模板 5
2 数据结构 6
2.1 01tire 6
2.2 [应用]Treap 维护数对 7
2.3 BIT(树状数组,0base) 9
2.4 BIT(树状数组,1base) 9
2.5 DSU(并查集) 10
2.6 FHQTreap(Info+Tag+Key 版) 10
2.7 FHQTreap(无旋平衡树) 13
2.8 jiangly 线段树(info) 15
2.9 jiangly 线段树(改) 16
2.10 st 表(0-based) 19
2.11 st 表(1-based) 19
2.12 zkw 线段树(单点修区间查) 20
2.13 主席树(例 2) 21
2.14 主席树 22
2.15 可并堆(平板电视) 22
2.16 回撤并查集 & 可持久化并查集(离线) 23
2.17 平板电视 Set(含多重) 24
2.18 普通莫队 25
2.19 李超线段树 26
2.20 珂朵莉树(ODT) 27
2.21 笛卡尔树(新版) 28
2.22 线段树二分 28
2.23 线段树分治 31
3 图论 33
3.1 2-sat 33
3.2 dfs&bfs 34
3.3 DSU on tree(树上启发式合并) 35
3.4 eDCC (边双联通分量，tarjan） 36
3.5 johnson 最短路 37
3.6 MCS（最大势算法） 38
3.7 SCC（低注释） 39
3.8 SCC（强联通分量，缩点，tarjan） 40
3.9 ShortestPath（Bellman_Ford,SPFA) 42
3.10 ShortestPath（dijkstra） 43
3.11 ShortestPath（Floyed） 44
3.12 vBCC（点双连通分量，tarjan.点划分） 45
3.13 vBCC（点双连通分量，tarjan.边划分） 46
3.14 二分图最大匹配 46
3.15 二分图染色 48
3.16 分层图 49
3.17 差分约束 50
3.18 找环(topsort) 51
3.19 拓扑排序 51
3.20 无向图三元环计数 52
3.21 最大流(dinic) 52
3.22 最大费用可行流 53
3.23 最小斯坦纳树 55
3.24 最小生成树（boruvka,标） 55
3.25 最小生成树（boruvka） 57
3.26 最小生成树（kruskal） 58
3.27 最小生成树（prim） 59
3.28 最小费用最大流(dinic) 60
3.29 最小费用最大流(dinic,浮点) 61
3.30 最小费用最大流(原始对偶) 62
3.31 最近公共祖先（LCA）(tarjan,静态) 63
3.32 最近公共祖先（LCA）(倍增) 65
3.33 最近公共祖先（LCA）(欧拉序 &dfs 序(O(1))) 66
3.34 树上倍增(点.边) 67
3.35 树上前缀和(点.边.倍增 lca.ver.) 68
3.36 树上基本处理(带欧拉序) 69

3.37 树上基本处理 70
3.38 树上差分(点,边.树剖 lca.ver.) 71
3.39 树链剖分（线段树 ver.） 72
3.40 欧拉回路(路径,Hierholzer 法) 74
3.41 点分树 75
3.42 点分治 76
3.43 线段树优化建图 77
3.44 虚树(可拓展版) 78
3.45 虚树(带边权) 80
4 字符串 82
4.1 AC 自动机(dp 版) 82
4.2 AC 自动机(可拓展版) 83
4.3 exKMP 84
4.4 kmp 84
4.5 Manacher 85
4.6 tiretree 85
4.7 后缀自动机(SAM,低注释) 86
4.8 后缀自动机(SAM,高注释) 88
4.9 回文树(PAM,回文自动机) 90
4.10 广义后缀自动机(GSAM) 91
4.11 防 hack 的 umap,gp_hash_table 92
4.12 随机底数固定模数单 hash 92
4.13 随机底数固定模数双 hash 93
5 动态规划 94
5.1 数位 dp(例题 1,数位和) 94
6 数论 96
6.1 (ex)CRT((扩展)中国剩余定理)(新版) 96
6.2 (ex)CRT((扩展)中国剩余定理)(旧版) 97
6.3 (最小)原根 98
6.4 FFT(快速傅里叶变换) 99
6.5 FMT&FWT(快速莫比乌斯 & 沃尔什变换) 100
6.6 NTT(快速数论变换) 101
6.7 乘法逆元 103
6.8 安全取模类(精简版) 103
6.9 安全取模类 103
6.10 拉格朗日插值 104
6.11 数论预处理 105
6.12 整除分块 108
6.13 矩阵快速幂(新版) 108
6.14 矩阵快速幂(旧板) 110
6.15 筛法求积性函数 110
6.16 线性基(gauss) 111
6.17 线性基(贪心法) 112
6.18 组合数预处理 113
7 计算几何(全整数版) 116
7.1 三分 116
7.2 三角剖分 116
7.3 三角剖分 2 117
7.4 凸包 118
7.5 凸包闵和 120
7.6 凸包面积并 121
7.7 向量 122
7.8 旋转卡壳 123
7.9 极角排序 126
7.10 点在多边形内判定 126
8 计算几何(浮点数版) 129
8.1 三分 129
8.2 三角剖分 130
8.3 三角剖分 2 132
8.4 凸包 133
8.5 凸包闵和 134
8.6 凸包面积并 136
8.7 向量 137
8.8 旋转卡壳 138
8.9 极角排序 140

9 其他 141
9.1 bitset 操作手册 141
9.2 SWAG(滑动窗口聚合) 141
9.3 动态 bitset(0base) 142
9.4 动态 bitset(1base) 144
9.5 整体二分 145
9.6 离散化 146

1 通用模板

1.1 cpp 模板

```cpp
#include <algorithm>
#include <array>
#include <assert.h>
#include <bitset>
#include <cassert>
#include <chrono>
#include <climits>
#include <cmath>
#include <complex>
#include <cstdint>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ctime>
#include <deque>
#include <functional>
#include <iomanip>
#include <iostream>
#include <limits>
#include <map>
#include <numeric>
#include <queue>
#include <random>
#include <ranges> // C++20
#include <set>
#include <stack>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>
// 特殊头：PBDS/gp_hash_table 需要，普通题可删
#include <ext/pb_ds/assoc_container.hpp>
// PBDS priority_queue/pairing_heap 需要
#include <ext/pb_ds/priority_queue.hpp>
// PBDS tree/order statistics tree 需要
#include <ext/pb_ds/tree_policy.hpp>
// 万能头
#include <bits/stdc++.h>

// C++23 注意：this auto&& 递归 lambda 需要 C++23，不是头文件能解决的问题。
// 若目标是 C++17/20，请把对应代码改成普通递归 lambda / std::function。
//#define int long long //赫赫 要不要龙龙呢
using ll=long long;
using namespace std;
using namespace __gnu_pbds;

signed main()
{
    //freopen("in.txt","r",stdin);
    //freopen("out.txt","w",stdout);
    //ios::sync_with_stdio(false),cin.tie(0),cout.tie(0);

    return 0;
}
//编译指令：
// g++ -std=c++23 -O2 -Wall "-Wl,--stack=1073741824" a.cpp -o a (win)
// g++ -std=c++23 -O2 -Wall a.cpp -o a (linux)
```

2 数据结构

2.1 01tire

```
class O1Tire{
    public:
        struct node
        {
            int ch[2];
            int cnt;
            node():ch{0,0},cnt(0){}
        };
        vector<node> trie;
        int tot,root;
        O1Tire(int len):trie(len+5),tot(0),root(0){}
        //len:节点数,此处一般是32*n
        void set(uint32_t x,int t)//从高到低建树
        {
            int p=root;
            for(int i=31;i>=0;i--)
            {
                int d=(x>>i)&1;
                if(!trie[p].ch[d])
                    trie[p].ch[d]=++tot;
                trie[trie[p].ch[d]].cnt+=t;
                p=trie[p].ch[d];
            }
        }
        uint32_t findMax(uint32_t x)//从高到低找,贪心选择,求解x对tire中所有数的最大异或值
        {
            int p=root;
            uint32_t res=0;
            for(int i=31;i>=0;i--)
            {
                int d=(x>>i)&1;
                if(trie[p].ch[d^1]&&trie[trie[p].ch[d^1]].cnt)
                    p=trie[p].ch[d^1],res+=(1u<<i);
                else
                    p=trie[p].ch[d];
                if(!p)
                    return res;
            }
            return res;
        }
        //求解x对tire中所有数的xor中<=k的个数
        int qry(uint32_t x,uint32_t k){
            int res=0,p=root;
            for(int i=31;i>=0;i--)
            {
                int d=(x>>i)&1;
                int kd=(k>>i)&1;
                if(kd){
                    if(trie[p].ch[d]) res+=trie[trie[p].ch[d]].cnt;
                    if(!trie[p].ch[d^1]) return res;
                    p=trie[p].ch[d^1];
                }
                else{
                    if(!trie[p].ch[d]) return res;
                    p=trie[p].ch[d];
                }
            }
            //加上=k的情况
            res+=trie[p].cnt;
            return res;
        }
        //其他逻辑反着来即可
        //求解01trie中与x异或的第k小的异或值(k从1开始)
        uint32_t kth(uint32_t x,int k)
        {
            int p=root;
            uint32_t res=0;
            for(int i=31;i>=0;i--)
            {
                int d=(x>>i)&1;
                int c=trie[p].ch[d]?trie[trie[p].ch[d]].cnt:0;
                if(k<=c) p=trie[p].ch[d];
                else k-=c,p=trie[p].ch[d^1],res+=(1u<<i);
                if(!p) return res;
            }
            return res;
        }
        void clear(){
            for(auto &i:trie){
                i.cnt=0;
                i.ch[0]=i.ch[1]=0;
            }
            tot=0;
            root=0;
        }
};
```

用法示例:

```
    int t;cin>>t;
    while(t--)
    {
        int n;uint32_t k;cin>>n>>k;
        vector<uint32_t> a(n);
        for(int i=0;i<n;i++)
            cin>>a[i];
        O1Tire tire(n*32);
        int ans=0xffffffff;
        for(int i=0,j=0;i<n;i++)
        {
            tire.set(a[i],1);
            while(j<=i&&tire.findMax(a[i])>=k)
            {
                ans=min(ans,i-j+1);
                tire.set(a[j],-1);
                j++;
            }
        }
        if(ans==0xffffffff) cout<<-1<<endl;
        else cout<<ans<<endl;
    }
```

2.2 [应用]Treap 维护数对

```
class FHQTreap{
    //无旋Treap：1.满足二叉搜索树性质(val) 2.满足堆性质（优先级）
    //树堆：BST+Heap
    public:
        struct Node{
            int l,r;
            int size=0;
            int priority;//随机数
            Node *left, *right;
            Node(int l,int r):l(l),r(r),priority(rand()),left(NULL),right(NULL),size(1){}
        };
        bool cmp(Node *a,pair<int,int> val){
            if(a->l==val.first) return a->r<=val.second;
            return a->l<val.first;
        }
        Node *root;
        FHQTreap():root(NULL){}
        void merge(Node *&root,Node *a,Node *b){
            //val a<=val b(内部满足Treap)
            if(!a)root=b;
            else if(!b)root=a;
            else{
                if(a->priority>b->priority){//a的优先级大
                    root=a;//a作为根(为了满足Heap(大))
                    merge(a->right,a->right,b);//b合并到a的右子树（为了满足BST：a的右子树的所有节点都大于a)
                }
                else{
                    root=b;
                    merge(b->left,a,b->left);
                }
            }
            if(root)
            {
                root->size=1;
                if(root->left)root->size+=root->left->size;
                if(root->right)root->size+=root->right->size;
                //cout<<root->val<<' '<<root->size<<endl;
            }
        }
        void split(Node *root,Node *&a,Node *&b,pair<int,int> val){
            //将root按照val分割为a,b两部分
            //a的val都小于等于val，b的val都大于val
            if(!root){
                a=b=NULL;
                return;
            }
            if(cmp(root,val)){
                a=root;
                split(root->right,a->right,b,val);
            }
            else{
                b=root;
                split(root->left,a,b->left,val);
            }
            if(root)
            {
                root->size=1;
                if(root->left)root->size+=root->left->size;
                if(root->right)root->size+=root->right->size;
                //cout<<root->val<<' '<<root->size<<endl;
            }
        }
        void insert(pair<int,int> val){
            Node *a,*b;
            split(root,a,b,val);//将root按照val分割为a,b两部分
            merge(a,a,new Node(val.first,val.second));//将val插入到a中
```

```
            merge(root,a,b);//将a,b合并为root
            //偶还能这样
        }
        void erase(pair<int,int> val){
            Node *a,*b,*c;
            split(root,a,b,val);//将root按照val分割为a,b两部分
            split(a,a,c,{val.first,val.second-1});//将a按照val-1分割为a,c两部分
            if(c)
            {
                merge(a,a,c->right);//将c的右子树合并到a中(删除一个节点)
                merge(a,a,c->left);//将c的左子树合并到a中(删除一个节点)
            }
            merge(root,a,b);//将a,b合并为root
        }
        pair<int,int> findMax(Node *root){
            if(!root)return {-1,-1};
            while(root->right)root=root->right;
            return {root->l,root->r};
        }
        pair<int,int> findMin(Node *root){
            if(!root)return {-1,-1};
            while(root->left)root=root->left;
            return {root->l,root->r};
        }
        pair<int,int> pre(pair<int,int> val){
            Node *a,*b;
            split(root,a,b,{val.first,val.second-1});//将root按照val-1分割为a,b两部分
            pair<int,int> res=findMax(a);
            merge(root,a,b);
            return res;
        }
        pair<int,int> next(pair<int,int> val){
            Node *a,*b;
            split(root,a,b,val);//将root按照val分割为a,b两部分
            pair<int,int> res=findMin(b);
            merge(root,a,b);
            return res;
        }
        bool find(pair<int,int> val)
        {
            Node *a,*b;
            split(root,a,b,val);
            bool res=a&&findMax(a).second==val.second&&findMax(a).first==val.first;
            merge(root,a,b);
            return res;
        }
        int size()
        {
            return root?root->size:0;
        }

};
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
```

用法示例:

```
    cin.tie(0);
    cout.tie(0);
    int t;cin>>t;
    FHQTreap tree;
    while(t--)
    {
        char op;
        cin>>op;
        if(op=='B')
        {
            cout<<tree.size()<<'\n';
```

```
        }
        else if(op=='A')
        {
            int x,y;
            cin>>x>>y;
            auto check=[](pair<int,int> a,pair<int,int> b)->bool{
                vector<int> v1={a.first,a.second,b.first,b.second};
                sort(v1.begin(),v1.end());
                return ((v1[0]==a.first&&v1[1]==a.second&&v1[2]==b.first&&v1[3]==b.second)||
(v1[0]==b.first&&v1[1]==b.second&&v1[2]==a.first&&v1[3]==a.second))&&v1[1]!=v1[2];
            };
            int ans=0;
            if(tree.find({x,y}))
            {
                tree.erase({x,y});
                ans++;
            }
            while(tree.size()&&tree.pre({x,y}).first!=-1&&!check(tree.pre({x,y}),{x,y}))
            {
                ans++;
                tree.erase(tree.pre({x,y}));
            }
            while(tree.size()&&tree.next({x,y}).first!=-1&&!check(tree.next({x,y}),{x,y}))
            {
                ans++;
                tree.erase(tree.next({x,y}));
            }
            cout<<ans<<'\n';
            tree.insert({x,y});
        }
    }
```

2.3 BIT(树状数组,0base)

```
class BIT{
    public:
    vector<int> tr;int n;
    BIT(int n):n(n),tr(n+1,0){}
    void add(int x,int v){
        for(;x<=n;x|=x+1) tr[x]+=v;//tr[x]=max(tr[x],v);
    }
    int pre(int x){
        int res=0;
        for(;x>=0;x=(x&(x+1))-1) res+=tr[x];//res=max(res,tr[x]);
        return res;
    }
    int query(int l,int r){
        return pre(r)-(l?pre(l-1):0);
    }
};
```

2.4 BIT(树状数组,1base)

```
//进阶用法1.维护差分数组
//进阶用法2.把数组离散化后按照值域建树状数组，可以用来求逆序对(第K大)
//e.g.val[1,16,9,10,3]->dis[1,5,3,4,2]->bit[1,1,1,1,1]
//    BIT bit(5);
//    //bit.update(1,1);bit.update(2,1);bit.update(3,1);bit.update(4,1);bit.update(5,1);
//    val_i[1,3]即更新为[1,0,1,0,1] 9即为第2大 即bit.query(1,3)=2
//求逆序对，how to do? 即[1,r]中比a[r]大的数的个数
```

```
class BIT{
    private:
        int n;
        vector<int> tree;//tree[i] 是[i-lowbit(i)+1,i]的和,[1,n]存储
        int lowbit(int x){
            return x&(-x);
        }
    public:
        BIT(int n): n(n),tree(n+1,0){}
        void update(int i,int val)//单点修改 a[i]+=val
        {
            while(i<=n){
                tree[i]+=val;
                i+=lowbit(i);//跳到后一个lowbit(x)的位置
            }
        }
        int pre(int x){
            int res=0;
            while(x>0){
                res+=tree[x];
                x-=lowbit(x);//跳到前一个lowbit(x)的位置
            }
            return res;
        }
        int query(int l,int r)//区间查询 [l,r]的和
        {
            return pre(r)-pre(l-1);
        }
        int query_diff(int i)//单点查询 a_diff[i] (维护差分数组)=sum[1,i]
```

```
        {
            return query(1,i);
        }
        void update_diff(int l,int r,int val)//区间修改 (维护差分数组) a_diff[l]+=val,a_diff[r+1]-=val
        {
            update(l,val);
            update(r+1,-val);
        }
        void init(vector<int> a)//初始化
        {
            vector<int> presum(a.size()+1,0);
            for(int i=1;i<=a.size();i++)
            {
                presum[i]=presum[i-1]+a[i-1];
                tree[i]=presum[i]-presum[i-lowbit(i)];//按定义
            }
        }
};
```

用法示例:

```
    //test
    int n=10; vector<int> a={1,3,2,4,2,1,5,4,3,2},a_diff={1,2,-1,2,-2,-1,4,-1,-1,-1};//a_diff[i]=a[i]-a[i-1]
    BIT bit(n),bit_diff(n);
    bit.init(a);bit_diff.init(a_diff);
    cout<<bit.query(1,5)<<endl;
    bit.update(1,5);
    cout<<bit.query(1,5)<<endl;
    bit_diff.update_diff(1,5,2);
    cout<<bit_diff.query_diff(5)<<endl;
    cout<<bit_diff.query_diff(6)<<endl;
    //test end
```

2.5 DSU(并查集)

```
class DSU{
    public:
        int n;vector<int> fa,sz;
        vector<vector<int>> ans;
        DSU(int n):n(n)
        {
            srand(time(NULL));
            fa.resize(n+1);
            sz.resize(n+1);
            for(int i=1;i<=n;i++)
            {
                fa[i]=i;
                sz[i]=1;
            }
        }
        int find(int u){
            return fa[u]==u?u:fa[u]=find(fa[u]);
        }
        void merge(int a,int b)
        {
            int u=find(a),v=find(b);
            if(u==v) return;
            fa[u]=v;
            sz[v]+=sz[u];
        }
        int same(int a,int b)
        {
            return find(a)==find(b);
        }
        int size(int u){
            return sz[find(u)];
        }
        vector<vector<int>> get(){
            ans.resize(n+1);
            for(auto &i:ans) i.clear();
            for(int i=1;i<=n;i++)
            {
                ans[find(i)].push_back(i);
            }
            ans.erase(remove(ans.begin(),ans.end(),vector<int>()),ans.end());
            return ans;
        }
};
```

用法示例:

```
    srand(time(NULL));
```

2.6 FHQTreap(Info+Tag+Key 版)

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
template<class Key,class Info,class kTag,class iTag>
class FHQTreap{
    public:
    struct Node{
        Key key;
        Info info;
        kTag ktag;
```

```
    iTag itag;
    int sz,rnd,l,r,id;
};
vector<Node> tr;
int rt,tot;
FHQTreap(int n){
    tr.resize(n+5);
    tr[0]={Key(),Info(),kTag(),iTag(),0,0,0,0,0};
    rt=tot=0;
}
int nw(Key kval,Info ival,int id=0){
    tr[++tot]={kval,ival,kTag(),iTag(),1,(int)rng(),0,0,id};
    return tot;
}
inline void pushup(int u){
    tr[u].sz=1+tr[tr[u].l].sz+tr[tr[u].r].sz;
    tr[u].key.pushup(tr[tr[u].l].key,tr[tr[u].r].key);
    tr[u].info.pushup(tr[tr[u].l].info,tr[tr[u].r].info,tr[u].key);
}
inline void apply(int u,const kTag &kt,const iTag &it){
    if(!u)return;
    if(kt.has_tag()){
        tr[u].key.apply(kt,tr[u].sz);
        tr[u].ktag.apply(kt);
    }
    if(it.has_tag()){
        tr[u].info.apply(it,tr[u].sz);
        tr[u].itag.apply(it);
    }
}
inline void pushdown(int u){
    if(!tr[u].ktag.has_tag()&&!tr[u].itag.has_tag())return;
    apply(tr[u].l,tr[u].ktag,tr[u].itag);
    apply(tr[u].r,tr[u].ktag,tr[u].itag);
    tr[u].ktag.clear();tr[u].itag.clear();
}
void merge(int &u,int a,int b){
    if(!a||!b){u=a|b;return;}
    pushdown(a);pushdown(b);
    if(tr[a].rnd>tr[b].rnd){
        u=a;merge(tr[a].r,tr[a].r,b);
    }else{
        u=b;merge(tr[b].l,a,tr[b].l);
    }
    pushup(u);
}
void split(int u,int &a,int &b,Key val,bool strict=false){
    if(!u){a=b=0;return;}
    pushdown(u);
    bool go_right=strict?(tr[u].key<val):!(val<tr[u].key);
    if(go_right){
        a=u;split(tr[u].r,tr[a].r,b,val,strict);
    }else{
        b=u;split(tr[u].l,a,tr[b].l,val,strict);
    }
    pushup(u);
}
void insert(Key val,Info ival=Info(),int id=0){
    int a,b;split(rt,a,b,val,false);
    merge(a,a,nw(val,ival,id));
    merge(rt,a,b);
}
void erase(Key val){
    int a,b,c;
    split(rt,a,b,val,false); // a存<=val
    split(a,a,c,val,true);   // a存<val,c存==val
    if(c){
        merge(a,a,tr[c].r);
        merge(a,a,tr[c].l);
    }
    merge(rt,a,b);
}
void range_add_key(Key lim,const kTag &t){
    int x,y;split(rt,x,y,lim,true); // x存<lim, y存>=lim
    if(y)apply(y,t,iTag());
    merge(rt,x,y);
}
void range_add_info(Key l,Key r,const iTag &t){
    int x,y,z;
    split(rt,x,y,l,true); // x存<l
    split(y,y,z,r,false); // y存[l,r], z存>r
    if(y)apply(y,kTag(),t);
    merge(y,y,z);merge(rt,x,y);
}
Key findMax(int u){
    if(!u)return tr[0].key;
    pushdown(u);
    while(tr[u].r){u=tr[u].r;pushdown(u);}
    return tr[u].key;
}
Key findMin(int u){
    if(!u)return tr[0].key;
    pushdown(u);
```

```
        while(tr[u].l){u=tr[u].l;pushdown(u);}
        return tr[u].key;
    }
    Key pre(Key val){
        int a,b;split(rt,a,b,val,true); // 按<分裂
        Key res=findMax(a);
        merge(rt,a,b);return res;
    }
    Key next(Key val){
        int a,b;split(rt,a,b,val,false); //按<=分裂, b中自然就是>的部分
        Key res=findMin(b);
        merge(rt,a,b);return res;
    }
    int rank(Key val){
        int a,b;split(rt,a,b,val,true);
        int res=(a?tr[a].sz:0)+1;
        merge(rt,a,b);return res;
    }
    Key QueryKth(int k){return KthQuery(rt,k);}
    Key KthQuery(int u,int k){
        if(!u)return tr[0].key;
        pushdown(u);
        int lsz=tr[tr[u].l].sz;
        if(k<=lsz)return KthQuery(tr[u].l,k);
        if(k==lsz+1)return tr[u].key;
        return KthQuery(tr[u].r,k-lsz-1);
    }
    bool find(Key val){
        int a,b,c;
        split(rt,a,b,val,false);
        split(a,a,c,val,true);
        bool res=(c!=0);
        merge(a,a,c);merge(rt,a,b);
        return res;
    }
    void get_ans(int u,vector<ll>& ans){
        if(!u)return;
        pushdown(u);
        get_ans(tr[u].l,ans);
        if(tr[u].id)ans[tr[u].id]=tr[u].key.val;
        get_ans(tr[u].r,ans);
    }
    vector<ll> get_ans(){
        vector<ll> ans(tot+1);
        get_ans(rt,ans);return ans;
    }
};

struct kTag{
    ll add;
    kTag():add(0){}
    void apply(const kTag &v){add+=v.add;}
    bool has_tag()const{return add!=0;}
    void clear(){add=0;}
};
struct iTag{
    ll add;
    iTag():add(0){}
    void apply(const iTag &v){add+=v.add;}
    bool has_tag()const{return add!=0;}
    void clear(){add=0;}
};
struct Key{
    ll val,sum;
    Key(ll v=0):val(v),sum(v){}
    bool operator<(const Key &o)const{return val<o.val;}
    void pushup(const Key &l,const Key &r){sum=val+l.sum+r.sum;}
    void apply(const kTag &t,int sz){val+=t.add,sum+=t.add*sz;}
};
struct Info{
    ll ex_val,ex_sum;
    Info(ll v=0):ex_val(v),ex_sum(v){}
    void pushup(const Info &l,const Info &r,const Key &k){
        ex_sum=ex_val+l.ex_sum+r.ex_sum;
    }
    void apply(const iTag &t,int sz){
        ex_val+=t.add,ex_sum+=t.add*sz;
    }
};
//可以额外维护 key的sum,>=lim的key整体加正值(保证单调性)
//以及 Key范围[l,r]内额外的Info区间加 info的子树和 之类的
/*
struct kTag{
    void apply(const kTag &){}
    bool has_tag()const{return false;}
    void clear(){}
};
struct iTag{
    void apply(const iTag &){}
    bool has_tag()const{return false;}
    void clear(){}
};
struct Info{
    Info(ll v=0){}
```

```
    void pushup(const Info &,const Info &,const Key &){}
    void apply(const iTag &,int){}
};
struct Key{
    ll val,sum;
    Key(ll v=0):val(v),sum(v){}
    bool operator<(const Key &o)const{return val<o.val;}
    void pushup(const Key &l,const Key &r){sum=val+l.sum+r.sum;}
    void apply(const kTag &,int){}
};
//只需维护key的模式（将其他结构体置空即可无缝套用）。
*/
```

2.7 FHQTreap(无旋平衡树)

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
class FHQTreap{
    //无旋Treap：1.满足二叉搜索树性质(val) 2.满足堆性质（优先级）
    //树堆：BST+Heap
    public:
        struct Node{
            ll val,sum,tag;
            int sz,rnd,l,r,id;
        };
        vector<Node> tr;
        int rt,tot;

        bool cmp(ll a,ll b){
            return a<=b;
        }

        FHQTreap(int n){
            srand(time(0));
            tr.resize(n+5); //预分配空间
            tr[0]={0,0,0,0,0,0,0};
            rt=tot=0;
        }

        int nw(ll val,int id=0){
            tot++;
            tr[tot]={val,val,0,1,(int)rng(),0,0,id};
            return tot;
        }

        void pushup(int u){
            tr[u].sz=1;
            tr[u].sum=tr[u].val;
            if(tr[u].l){
                tr[u].sz+=tr[tr[u].l].sz;
                tr[u].sum+=tr[tr[u].l].sum;
            }
            if(tr[u].r){
                tr[u].sz+=tr[tr[u].r].sz;
                tr[u].sum+=tr[tr[u].r].sum;
            }
        }

        void pushdown(int u){
            if(tr[u].tag){
                ll t=tr[u].tag;
                if(tr[u].l){
                    tr[tr[u].l].val+=t;
                    tr[tr[u].l].sum+=t*tr[tr[u].l].sz;
                    tr[tr[u].l].tag+=t;
                }
                if(tr[u].r){
                    tr[tr[u].r].val+=t;
                    tr[tr[u].r].sum+=t*tr[tr[u].r].sz;
                    tr[tr[u].r].tag+=t;
                }
                tr[u].tag=0;
            }
        }

        void merge(int &u,int a,int b){
            //val a<=val b(内部满足Treap)
            if(!a||!b){
                u=a|b;
                return;
            }
            pushdown(a); pushdown(b); //下传标记
            if(tr[a].rnd>tr[b].rnd){//a的优先级大
                u=a;//a作为根(为了满足Heap(大))
                merge(tr[a].r,tr[a].r,b);//b合并到a的右子树（为了满足BST：a的右子树的所有节点都大于a)
            }
            else{
                u=b;
                merge(tr[b].l,a,tr[b].l);
            }
            pushup(u);
        }
```

```
void split(int u,int &a,int &b,ll val){
    //将root按照val分割为a,b两部分
    //a的val都小于等于val，b的val都大于val
    if(!u){
        a=b=0;
        return;
    }
    pushdown(u); //下传标记
    if(cmp(tr[u].val,val)){
        a=u;
        split(tr[u].r,tr[a].r,b,val);
    }
    else{
        b=u;
        split(tr[u].l,a,tr[b].l,val);
    }
    pushup(u);
}

void insert(ll val,int id=0){
    int a,b;
    split(rt,a,b,val);//将root按照val分割为a,b两部分
    int node=nw(val,id);
    merge(a,a,node);//将val插入到a中
    merge(rt,a,b);//将a,b合并为root
    //偶还能这样
}

void erase(ll val){
    int a,b,c;
    split(rt,a,b,val);//将root按照val分割为a,b两部分
    split(a,a,c,val-1);//将a按照val-1分割为a,c两部分
    if(c)
    {
        merge(a,a,tr[c].r);//将c的右子树合并到a中(删除一个节点)
        merge(a,a,tr[c].l);//将c的左子树合并到a中(删除一个节点)
    }
    merge(rt,a,b);//将a,b合并为root
}

//对>=lim的节点进行加值操作，需保证加完后仍满足左半区<=右半区
void range_add(ll lim,ll add){
    int x,y;
    split(rt,x,y,lim-1);
    if(y){
        tr[y].val+=add;
        tr[y].sum+=add*tr[y].sz;
        tr[y].tag+=add;
    }
    merge(rt,x,y);
}

void print(int u){
    if(!u)return;
    pushdown(u);
    print(tr[u].l);
    cout<<tr[u].val<<" ";
    print(tr[u].r);
}
ll findMax(int u){
    if(!u)return -1;
    pushdown(u);
    while(tr[u].r){
        u=tr[u].r;
        pushdown(u);
    }
    return tr[u].val;
}
ll findMin(int u){
    if(!u)return -1;
    pushdown(u);
    while(tr[u].l){
        u=tr[u].l;
        pushdown(u);
    }
    return tr[u].val;
}
ll pre(ll val){
    int a,b;
    split(rt,a,b,val-1);//将root按照val-1分割为a,b两部分
    ll res=findMax(a);
    merge(rt,a,b);
    return res;
}
ll next(ll val){
    int a,b;
    split(rt,a,b,val);//将root按照val分割为a,b两部分
    ll res=findMin(b);
    merge(rt,a,b);
    return res;
}
```

```
        int rank(ll val){
            int a,b;
            split(rt,a,b,val-1);//将root按照val-1分割为a,b两部分
            int res=(a?tr[a].sz:0)+1;
            merge(rt,a,b);
            return res;
        }
        ll QueryKth(int k){
            return KthQuery(rt,k);
        }
        ll KthQuery(int u,int k){
            if(u==0) return -1;
            pushdown(u);
            int lsz=tr[u].l?tr[tr[u].l].sz:0;
            if(k<=lsz) return KthQuery(tr[u].l,k);
            else if(k==lsz+1) return tr[u].val;
            else return KthQuery(tr[u].r,k-lsz-1);
        }
        bool find(ll val){
            int a,b;
            split(rt,a,b,val);//将root按照val分割为a,b两部分
            bool res=a&&findMax(a)==val;
            merge(rt,a,b);
            return res;
        }

        //中序遍历获取答案
        void get_ans(int u,vector<ll>& ans){
            if(!u) return;
            pushdown(u);
            get_ans(tr[u].l,ans);
            if(tr[u].id) ans[tr[u].id]=tr[u].val;
            get_ans(tr[u].r,ans);
        }

        vector<ll> get_ans(){
            vector<ll> ans(tot+1);
            get_ans(rt,ans);
            return ans;
        }
};
```

用法示例:

```
    int t;cin>>t;
    while(t--){
        int n;cin>>n;
        vector<int> a(n+1);
        FHQTreap fhq(n+1);
        for(int i=1;i<=n;i++) cin>>a[i];
        for(int i=1;i<=n;i++){
            fhq.range_add(a[i],a[i]);
            fhq.insert(a[i],i);
        }
        auto ans=fhq.get_ans();
        for(int i=1;i<=n;i++) cout<<ans[i]<<" ";
        cout<<endl;
    }
```

2.8 jiangly 线段树(info)

```
template<class Info, class Tag>
struct LazySegmentTree {
    const int n;
    std::vector<Info> info;
    std::vector<Tag> tag;
    LazySegmentTree(int n) : n(n), info(4 << std::__lg(n)), tag(4 << std::__lg(n)) {}
    LazySegmentTree(std::vector<Info> init) : LazySegmentTree(init.size()) {
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                info[p] = init[l];
                return;
            }
            int m = (l + r) / 2;
            build(2 * p, l, m);
            build(2 * p + 1, m, r);
            pull(p);
        };
        build(1, 0, n);
    }
    void pull(int p) {
        info[p] = info[2 * p] + info[2 * p + 1];
    }
    void apply(int p, const Tag &v) {
        info[p].apply(v);
        tag[p].apply(v);
    }
    void push(int p) {
        apply(2 * p, tag[p]);
        apply(2 * p + 1, tag[p]);
        tag[p] = Tag();
    }
    void modify(int p, int l, int r, int x, const Info &v) {
```

```
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int m = (l + r) / 2;
        push(p);
        if (x < m) {
            modify(2 * p, l, m, x, v);
        } else {
            modify(2 * p + 1, m, r, x, v);
        }
        pull(p);
    }
    void modify(int p, const Info &v) {
        modify(1, 0, n, p, v);
    }
    Info rangeQuery(int p, int l, int r, int x, int y) {
        if (l >= y || r <= x) {
            return Info();
        }
        if (l >= x && r <= y) {
            return info[p];
        }
        int m = (l + r) / 2;
        push(p);
        return rangeQuery(2 * p, l, m, x, y) + rangeQuery(2 * p + 1, m, r, x, y);
    }
    Info rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }
    void rangeApply(int p, int l, int r, int x, int y, const Tag &v) {
        if (l >= y || r <= x) {
            return;
        }
        if (l >= x && r <= y) {
            apply(p, v);
            return;
        }
        int m = (l + r) / 2;
        push(p);
        rangeApply(2 * p, l, m, x, y, v);
        rangeApply(2 * p + 1, m, r, x, y, v);
        pull(p);
    }
    void rangeApply(int l, int r, const Tag &v) {
        return rangeApply(1, 0, n, l, r, v);
    }
    void half(int p, int l, int r) {
        if (info[p].act == 0) {
            return;
        }
        if ((info[p].min + 1) / 2 == (info[p].max + 1) / 2) {
            apply(p, {-(info[p].min + 1) / 2});
            return;
        }
        int m = (l + r) / 2;
        push(p);
        half(2 * p, l, m);
        half(2 * p + 1, m, r);
        pull(p);
    }
    void half() {
        half(1, 0, n);
    }
};
struct Tag {
    //tag清空态
    void apply(Tag t) {
        //tag t下发对tag的影响
    }
};
struct Info {
    //维护啥信息
    void apply(Tag t) {
        //tag t下发对info的影响
    }
};
Info operator + (Info a, Info b) {
    Info c;
    //info a和info b合并
    return c;
}
//tip:[l,r)区间->传入[l,r]改为[l,r+1)
```

2.9 jiangly 线段树(改)

```
#define lc(p) (p<<1)
#define rc(p) (p<<1|1)
template<class Info,class Tag>
class SegTree{
    public:
    int n;
```

```
    vector<Info> info;
    vector<Tag> tag;
    SegTree(int n):n(n),info((n<<2)+5),tag((n<<2)+5){}
    SegTree(const vector<Info> &a):n(a.size()-1){
        //a 1-Based
        info.resize((n<<2)+5);
        tag.resize((n<<2)+5);
        bd(1,1,n,a);
    }
    inline void pushup(int p){
        info[p]=info[lc(p)]+info[rc(p)];
    }
    inline void apply(int p,int l,int r,const Tag &v){
        info[p].apply(l,r,v);
        tag[p].apply(v);
    }
    inline void pushdown(int p,int l,int r){
        if(!tag[p].has_tag()) return;
        int m=(l+r)>>1;
        apply(lc(p),l,m,tag[p]);
        apply(rc(p),m+1,r,tag[p]);
        tag[p]=Tag();
    }
    void bd(int p,int l,int r,const vector<Info> &a){
        if(l==r){
            info[p]=a[l];
            return;
        }
        int m=(l+r)>>1;
        bd(lc(p),l,m,a);
        bd(rc(p),m+1,r,a);
        pushup(p);
    }
    void upd(int p,int l,int r,int x,int y,const Tag &v){
        if(x>r||y<l||x>y) return;
        if(x<=l&&r<=y){
            apply(p,l,r,v);
            return;
        }
        pushdown(p,l,r);
        int m=(l+r)>>1;
        if(x<=m) upd(lc(p),l,m,x,y,v);
        if(m<y) upd(rc(p),m+1,r,x,y,v);
        pushup(p);
    }
    void mdf(int p,int l,int r,int x,const Info &v){
        if(l==r){
            info[p]=v;
            return;
        }
        pushdown(p,l,r);
        int m=(l+r)>>1;
        if(x<=m) mdf(lc(p),l,m,x,v);
        else mdf(rc(p),m+1,r,x,v);
        pushup(p);
    }
    Info qry(int p,int l,int r,int x,int y){
        if(x>r||y<l||x>y) return Info();
        if(x<=l&&r<=y) return info[p];
        pushdown(p,l,r);
        int m=(l+r)>>1;
        Info res=Info();
        if(x<=m) res=res+qry(lc(p),l,m,x,y);
        if(m<y) res=res+qry(rc(p),m+1,r,x,y);
        return res;
    }
    int findfirst(int p,int l,int r,int x,int y,
        Info &v,const function<bool(const Info&)> &chk){
        if(r<x||y<l) return n+1;
        if(x<=l&&r<=y){
            Info cmb=v+info[p];
            if(!chk(cmb)) {
                v=cmb;
                return n+1;
            }
            if(l==r) return l;
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=findfirst(lc(p),l,m,x,y,v,chk);
            if(res!=n+1) return res;
            return findfirst(rc(p),m+1,r,x,y,v,chk);
        }
        pushdown(p,l,r);
        int m=(l+r)>>1;
        int res=findfirst(lc(p),l,m,x,y,v,chk);
        if(res!=n+1) return res;
        return findfirst(rc(p),m+1,r,x,y,v,chk);
    }
    int findlast(int p,int l,int r,int x,int y,
        Info &v,const function<bool(const Info&)> &chk){
            if(r<x||y<l) return 0;
            if(x<=l&&r<=y){
                Info cmb=v+info[p];
                if(!chk(cmb)) {
```

```
                    v=cmb;
                    return 0;
                }
                if(l==r) return l;
                pushdown(p,l,r);
                int m=(l+r)>>1;
                int res=findlast(rc(p),m+1,r,x,y,v,chk);
                if(res!=0) return res;
                return findlast(lc(p),l,m,x,y,v,chk);
            }
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=findlast(rc(p),m+1,r,x,y,v,chk);
            if(res!=0) return res;
            return findlast(lc(p),l,m,x,y,v,chk);
    }
    int _findfirst(int p,int l,int r,int x,int y,
        const function<bool(const Info&)> &chk){
            if(r<x||y<l) return n+1;
            if(!chk(info[p])) return n+1;
            if(l==r) return l;
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=_findfirst(lc(p),l,m,x,y,chk);
            if(res!=n+1) return res;
            return _findfirst(rc(p),m+1,r,x,y,chk);
    }
    int _findlast(int p,int l,int r,int x,int y,
        const function<bool(const Info&)> &chk){
            if(r<x||y<l) return 0;
            if(!chk(info[p])) return 0;
            if(l==r) return l;
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=_findlast(rc(p),m+1,r,x,y,chk);
            if(res!=0) return res;
            return _findlast(lc(p),l,m,x,y,chk);
    }
    void upd(int l,int r,const Tag &v){
        upd(1,1,n,l,r,v);
    }
    void mdf(int x,const Info &v){
        mdf(1,1,n,x,v);
    }
    Info qry(int l,int r){
        return qry(1,1,n,l,r);
    }
    //寻找在[l,r]的第一个[l,k] 满足Info{l,k}满足chk e.g.[1,4]的[1,2]满足sum(1,2)<10
    //异常值: n+1
    int findfirst(int l,int r,const function<bool(const Info&)> &chk){
        Info tp=Info();
        return findfirst(1,1,n,l,r,tp,chk);
    }
    //寻找在[l,r]的最后一个[k,r] 满足Info{k,r}满足chk e.g.[1,4]的[3,4]满足sum(3,4)<10
    //异常值: 0
    int findlast(int l,int r,const function<bool(const Info&)> &chk){
        Info tp=Info();
        return findlast(1,1,n,l,r,tp,chk);
    }
    //寻找在[l,r]的第一个k 满足Info k满足chk e.g.[1,4]的第一个k=2满足info k<10
    //异常值: n+1
    int _findfirst(int l,int r,const function<bool(const Info&)> &chk){
        return _findfirst(1,1,n,l,r,chk);
    }
    //寻找在[l,r]的最后一个k 满足Info k满足chk e.g.[1,4]的最后一个k=3满足info k<10
    //异常值: 0
    int _findlast(int l,int r,const function<bool(const Info&)> &chk){
        return _findlast(1,1,n,l,r,chk);
    }
};
// Tag 结构体: 定义懒标记
// 需要实现:
// 1. 成员变量: 存储懒标记信息
// 2. 默认构造函数: 表示无标记状态
// 3. apply(const Tag& v): 将另一个标记 v 合并到当前标记
// 4. has_tag(): 判断当前是否是无标记状态
struct Tag{
    int tag;
    Tag():tag(0){}
    void apply(const Tag &v){

    }
    bool has_tag(){
        return tag!=0;
    }
};
// Info 结构体: 定义节点信息
// 需要实现:
// 1. 成员变量: 存储节点维护的信息
// 2. 默认构造函数: Info 的单位元 (例如求和的0, 求积的1)
```

```
// 3. apply(int l, int r, const Tag& v): 将懒标记 v 应用到当前节点信息上
// 4. operator+(const Info& other): 合并两个子节点的信息
struct Info{
    //...
    int info;
    Info():info(0){}
    void apply(int l,int r,const Tag &v){

    }
};
Info operator+(const Info &a,const Info &b){
    //...
    Info c;
    return c;
}
```

2.10 st 表(0-based)

```
class st{
    public:
        vector<vector<int>> dp;
        int inf(int a,int b){return max(a,b);}
        //0-based ST表, 输入数组为 a[0..n-1], 查询区间为 [l,r]
        void init(vector<int>& a,int n)
        {
            if(!n) return;
            int len=__lg(n)+1;
            dp.assign(len,vector<int>(n));
            for(int i=0;i<n;i++) dp[0][i]=a[i];
            for(int j=1;j<len;j++)
                for(int i=0;i+(1<<j)<=n;i++)
                    dp[j][i]=inf(dp[j-1][i],dp[j-1][i+(1<<(j-1))]);
        }
        int query(int l,int r)
        {
            int k=__lg(r-l+1);
            return inf(dp[k][l],dp[k][r-(1<<k)+1]);
        }
        st(vector<int>& a,int n){init(a,n);}
};
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
```

2.11 st 表(1-based)

```
class st{
    public:
        vector<vector<int>> dp;
        int inf(int a,int b){return max(a,b);}
        void init(vector<int>& a,int n)
        {
            if(!n) return;
            int len=__lg(n)+1;
            dp.assign(len,vector<int>(n+1));
            for(int i=1;i<=n;i++) dp[0][i]=a[i];
            for(int j=1;j<len;j++)
                for(int i=1;i+(1<<j)-1<=n;i++)
                    dp[j][i]=inf(dp[j-1][i],dp[j-1][i+(1<<(j-1))]);
        }
        int query(int l,int r)
        {
            int k=__lg(r-l+1);
            return inf(dp[k][l],dp[k][r-(1<<k)+1]);
        }
        st(vector<int>& a,int n){init(a,n);}
};
int read()
```

```
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
```

用法示例:

```
    int n=read(),m=read();
    vector<int> nums(n+1);
    for(int i=1;i<=n;i++)
    {
        nums[i]=read();
    }
    st s(nums,n);
    for(int i=0;i<m;i++)
    {
        int l=read(),r=read();
        write(s.query(l,r));
        putchar('\n');
    }
```

2.12 zkw 线段树(单点修区间查)

```
template<class Info>
class zkw{
    public:
    int n,m;
    vector<Info> info;
    zkw(const vector<Info> &a):n(a.size()-1){
        //a 1-Based
        for(m=1;m<=n+1;m<<=1);
        info.resize((m<<1)+5);
        for(int i=1;i<=n;++i) info[m+i]=a[i];
        for(int i=m-1;i;--i) pushup(i);
    }
    inline void pushup(int p){
        info[p]=info[p<<1]+info[p<<1|1];
    }
    void mdf(int x,const Info &v){
        for(info[x+=m]=v,x>>=1;x;x>>=1) pushup(x);
    }
    Info qry(int l,int r){
        Info rl=Info(),rr=Info();
        for(l+=m-1,r+=m+1;l^r^1;l>>=1,r>>=1){
            if(~l&1) rl=rl+info[l^1];
            if(r&1) rr=info[r^1]+rr;
        }
        return rl+rr;
    }
};
// Info 结构体：定义节点信息
// 需要实现：
// 1. 成员变量：存储节点维护的信息
// 2. 默认构造函数：Info 的单位元（例如求和的0，求积的1，最大值的-INF）
// 3. operator+(const Info& other)：合并两个子节点的信息
struct Info{
    //...
    int info;
    Info():info(0){}
    Info(int x):info(x){}
};

Info operator+(const Info &a,const Info &b){
    //...（注意非交换律运算时，顺序为 左子节点+右子节点）
    Info c;
    c.info=a.info+b.info;
    return c;
}
```

2.13 主席树(例 2)

```
//https://codeforces.com/contest/893/problem/F
//主席树维护一个前缀结构 解决二维偏序
```

```
#define lc(x) tr[x].l
#define rc(x) tr[x].r
class HJTree{
    public:
    struct node
    {
        int l,r,minn;
        //左右儿子, 区间minn
    };
    vector<node> tr;
    vector<int> rt,rt_k;
    int tot,n;
    HJTree(int n,vector<vector<array<int,2>>>& q,int& maxdep):tot(0),n(n)
    {
        //a 1-based
        rt.resize(n+5);
        tr.resize((log2(n)+4)*n+5);
        rt_k.resize(maxdep+5);
        bd(rt[0],1,n);
        int cur=1;
        for(int i=1;i<=maxdep;i++)
        {
            for(auto [pos,val]:q[i])
            {
                ins(rt[cur-1],rt[cur],1,n,pos,val);
                cur++;
            }
            rt_k[i]=cur-1;
        }
    };
    void bd(int &x,int l,int r)
    {
        x=++tot; tr[x].minn=2e9;
        if(l==r) return ;
        int m=(l+r)>>1;
        bd(lc(x),l,m);
        bd(rc(x),m+1,r);
    }
    void ins(int x,int &y,int l,int r,int p,int val)
    {
        y=++tot; tr[y]=tr[x]; tr[y].minn=min(tr[y].minn,val);
        if(l==r) return ;
        int m=(l+r)>>1;
        if(p<=m) ins(lc(x),lc(y),l,m,p,val);
        else ins(rc(x),rc(y),m+1,r,p,val);
    }
    int qry(int rt,int l,int r,int s,int e){
        if(l>e||r<s) return 2e9;
        if(l>=s&&r<=e) return tr[rt].minn;
        int m=(l+r)>>1;
        return min(qry(lc(rt),l,m,s,e),qry(rc(rt),m+1,r,s,e));
    }
    int qry(int k,int s,int e)
    {
        return qry(rt[rt_k[k]],1,n,s,e);
    }
};
```

用法示例:

```
    int n,r;cin>>n>>r;
    vector<int> val(n+1);
    for(int i=1;i<=n;i++) cin>>val[i];
    vector<vector<int>> tre(n+1);
    for(int i=1;i<n;i++){
        int u,v;cin>>u>>v;
        tre[u].push_back(v);
        tre[v].push_back(u);
    }
    vector<int> dep(n+1,0),dfn(n+1,0),out(n+1,0);
    int idx=0,maxdep=0;
    auto dfs=[&](this auto&& dfs,int u,int fa)->void{
        dfn[u]=++idx;
        dep[u]=dep[fa]+1,maxdep=max(maxdep,dep[u]);
        for(auto v:tre[u])
            if(v!=fa) dfs(v,u);
        out[u]=idx;
    };
    dep[r]=1;
    dfs(r,0);
    //在[1,n]建主席树 维护dep<=k的版本
    vector<vector<array<int,2>>> q(maxdep+1);
    for(int i=1;i<=n;i++)
    {
        q[dep[i]].push_back({dfn[i],val[i]});
    }
    int last=0,m;cin>>m;
    HJTree hjt(n,q,maxdep);
    while(m--)
```

```
    {
        int p,q;cin>>p>>q;
        int x=(p+last)%n+1;
        int k=(q+last)%n;
        int ans=hjt.qry(min(maxdep,dep[x]+k),dfn[x],out[x]);
        cout<<ans<<'\n';
        last=ans;
    }
```

2.14 主席树

```
//主席树 (静态区间第k小)
//利用权值线段树,维护a[1]-a[n] n次插入的历史版本
//于是可以利用前缀和思想,求出任意区间第k小
//时间复杂度qry(logn) 空间复杂度nlogn+2*n
//本质上是做了一个单点更新 保存历史版本 维护一个前缀结构 以此可以处理很多二维偏序问题
```

```
#define lc(x) tr[x].l
#define rc(x) tr[x].r
class HJTree{
    public:
    struct node
    {
        int l,r,s;
        //左右儿子, 区间数频次
    };
    vector<node> tr;
    vector<int> b,rt;
    int tot,n,bn;
    HJTree(int n,const vector<int>& a):tot(0),n(n){
        //a 1-based
        rt.resize(n+5);
        tr.resize((log2(n)+4)*n+5);
        b.resize(n);
        //注意空间是2*n+(ceil(log2(n))+1)*n
        b.assign(a.begin()+1,a.end());
        sort(b.begin(),b.end());
        b.erase(unique(b.begin(),b.end()),b.end());
        bn=b.size();
        bd(rt[0],1,bn);
        for(int i=1;i<=n;i++) ins(rt[i-1],rt[i],1,bn,getid(a[i]));
    };
    int getid(int x){
        //离散化 ->[1,n]
        return lower_bound(b.begin(),b.end(),x)-b.begin()+1;
    }
    void bd(int &x,int l,int r)
    {
        x=++tot; tr[x].s=0;
        if(l==r) return ;
        int m=(l+r)>>1;
        bd(lc(x),l,m);
        bd(rc(x),m+1,r);
    }
    void ins(int x,int &y,int l,int r,int tar)
    {
        y=++tot; tr[y]=tr[x]; tr[y].s++;
        if(l==r) return ;
        int m=(l+r)>>1;
        if(tar<=m) ins(lc(x),lc(y),l,m,tar);
        else ins(rc(x),rc(y),m+1,r,tar);
    }
    int qry(int x,int y,int l,int r,int tar){
        if(l==r) return l;
        int m=(l+r)>>1;
        int s=tr[lc(y)].s-tr[lc(x)].s;
        if(tar<=s) return qry(lc(x),lc(y),l,m,tar);
        else return qry(rc(x),rc(y),m+1,r,tar-s);
    }
    int qry(int l,int r,int k){
        return b[qry(rt[l-1],rt[r],1,bn,k)-1];
    }
};
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<int> a(n+1);
    for(int i=1;i<=n;i++) cin>>a[i];
    HJTree hjt(n,a);
    while(m--){
        int l,r,k;cin>>l>>r>>k;
        cout<<hjt.qry(l,r,k)<<'\n';
    }
```

2.15 可并堆(平板电视)

```
//可并堆: pairing_heap_tag,配对堆 插入,合并均摊O(1),删除堆顶(logn)
```

用法示例:

```
    int n,q;cin>>n>>q;
    vector<int> fa(n+1),dep(n+1,0),nfa(n+1),ndep(n+1,0);
    for(int i=2;i<=n;i++){
        int x;cin>>x;fa[i]=x,dep[i]=dep[x]+1;
    }
    auto cmp=[&](int x,int y){
        return dep[x]<dep[y];
    };
    __gnu_pbds::priority_queue<int,decltype(cmp),__gnu_pbds::pairing_heap_tag> pq(cmp);
    vector<__gnu_pbds::priority_queue<int,decltype(cmp),__gnu_pbds::pairing_heap_tag>> nq(n+1,pq);
    for(int i=1;i<=q;i++){
        int u,v;cin>>u>>v;
        nq[u].push(v);
    }
    for(int i=n;i>=2;i--){
        if(nq[i].empty()) nfa[i]=1;
        else{
            int d=nq[i].top();nq[i].pop();nfa[i]=d;
            while(!nq[i].empty()&&nq[i].top()==d){
                nq[i].pop();
            }
            nq[d].join(nq[i]);
        }
    }
    ll ans=0;
    for(int i=2;i<=n;i++){
        ndep[i]=ndep[nfa[i]]+1;
        ans+=ndep[i];
    }
    cout<<ans<<endl;
```

2.16 回撤并查集 & 可持久化并查集(离线)

```
class REDSU{
public:
    vector<int> fa,sz;
    int n;vector<array<int,2>> st;
    REDSU(int n):n(n){
        fa.resize(n+5);
        sz.resize(n+5);
        st.reserve(n+5);
        for(int i=1;i<=n;i++){
            fa[i]=i;
            sz[i]=1;
        }
    }
    int find(int x){
        while(x!=fa[x]) x=fa[x];
        return x;
    }
    bool same(int x,int y){
        return find(x)==find(y);
    }
    void merge(int x,int y){
        x=find(x),y=find(y);
        if(x==y)
        {
            st.push_back({0,y});
            return;
        }
        if(sz[x]<sz[y]) swap(x,y); //sz[x]>=sz[y]
        st.push_back({1,y});sz[x]+=sz[y];fa[y]=x;
    }
    int size(int x){return sz[find(x)];}
    void back(){
        if(!st.empty()){
            auto [fg,y]=st.back();st.pop_back();
            if(!fg) return;
            sz[fa[y]]-=sz[y];fa[y]=y;
        }
    }
    void back_k(int k){
        while(k--) back();
    }
};
```

用法示例:

```
    int n,m;cin>>n>>m;
    REDSU rsu(n);
    vector<array<int,3>> op(m+1);
    vector<vector<int>> optr(m+1);
    op[0]={2,-1,-1};
    for(int i=1;i<=m;i++){
        int x;cin>>x;
        if(x!=2){
            int a,b;cin>>a>>b;
            op[i]={x,a,b};
            optr[i].push_back(i-1);
            optr[i-1].push_back(i);
        }
        else{
            int a;cin>>a;
            op[i]={x,a,-1};
```

```
            optr[i].push_back(a);
            optr[a].push_back(i);
        }
    }
    vector<int> ans(m+1,-1);
    auto dfs=[&](this auto&& dfs,int u,int f)->void{
        if(op[u][0]!=2){
            if(op[u][0]==1) rsu.merge(op[u][1],op[u][2]);
            else ans[u]=rsu.same(op[u][1],op[u][2]);
        }
        for(int y:optr[u])
            if(y!=f) dfs(y,u);
        if(op[u][0]==1) rsu.back();
    };
    dfs(0,0);
    for(int i=1;i<=m;i++){
        if(op[i][0]==3) cout<<ans[i]<<endl;
    }
```

2.17 平板电视 Set(含多重)

```
template<typename T>
//多重集
class multiSet{
    private:
        typedef tree<pair<T,int>,null_type,less<pair<T,int>>,
            rb_tree_tag,tree_order_statistics_node_update> pb_tree;
        pb_tree tr; int tot;
        const int INF=1e9;
        const T bad=-1;
        //此处记得贴合T的类型
        //INF 插入元素个数的最大值
        //bad 没有前驱或后继时的返回值
    public:
        multiSet():tot(0){}
        //插入元素
        void insert(T x){ tr.insert({x,++tot}); }
        //查询元素的排名
        int rank(T x){ return tr.order_of_key({x,0})+1; }
        //查询元素的前驱
        T pre(T x){
            auto it=tr.lower_bound({x,0});
            if(it==tr.begin()) return bad;
            --it; return it->first;
        }
        //查询元素的后继
        T next(T x){
            auto it=tr.lower_bound({x,INF});
            if(it==tr.end()) return bad;
            return it->first;
        }
        T kth(int k){
            auto it=tr.find_by_order(k-1);
            if(it==tr.end()) return bad;
            return it->first;
        }
        bool find(T x){
            auto it=tr.lower_bound({x,0});
            return it!=tr.end()&&it->first==x;
        }
        int size(){ return tr.size(); }
        bool empty(){ return tr.empty(); }
        //删除元素
        void erase(T x){
            auto it=tr.lower_bound({x,0});
            if(it!=tr.end()&&it->first==x) tr.erase(it);
        }
};

template<typename T>
//不可重集
class Set{
    private:
        typedef tree<T,null_type,less<T>,
            rb_tree_tag,tree_order_statistics_node_update> pb_tree;
        pb_tree tr;
        const T bad=-1;
        //此处记得贴合T的类型
        //bad 没有前驱或后继时的返回值
    public:
        Set(){}
        //插入元素
        void insert(T x){ tr.insert(x); }
        //查询元素的排名
        int rank(T x){ return tr.order_of_key(x)+1; }
        //查询元素的前驱
        T pre(T x){
            auto it=tr.lower_bound(x);
            if(it==tr.begin()) return bad;
            --it; return *it;
```

```
        }
        //查询元素的后继
        T next(T x){
            auto it=tr.upper_bound(x);
            if(it==tr.end()) return bad;
            return *it;
        }
        T kth(int k){
            auto it=tr.find_by_order(k-1);
            if(it==tr.end()) return bad;
            return *it;
        }
        bool find(T x){
            return tr.find(x)!=tr.end();
        }
        int size(){ return tr.size(); }
        bool empty(){ return tr.empty(); }
        //删除元素
        void erase(T x){
            tr.erase(x);
        }
};
```

2.18 普通莫队

```
class BIT{
    private:
        int n;
        vector<int> tree;//tree[i] 是[i-lowbit(i)+1,i]的和,[1,n]存储
        int lowbit(int x){
            return x&(-x);
        }
    public:
        BIT(int n): n(n),tree(n+1,0){}
        void update(int i,int val)//单点修改 a[i]+=val
        {
            while(i<=n){
                tree[i]+=val;
                i+=lowbit(i);//跳到后一个lowbit(x)的位置
            }
        }
        int query(int l,int r)//区间查询 [l,r]的和
        {
            int res=0;
            while(r>=l){
                res+=tree[r];
                r-=lowbit(r);//跳到前一个lowbit(x)的位置
            }
            return res;
        }
        void init(vector<int> a)//初始化
        {
            vector<int> presum(a.size()+1,0);
            for(int i=1;i<=a.size();i++)
            {
                presum[i]=presum[i-1]+a[i-1];
                tree[i]=presum[i]-presum[i-lowbit(i)];//按定义
            }
        }
};
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
unordered_map<int,int> dis(vector<int> a)
{
    sort(a.begin(),a.end());
    unordered_map<int,int> mp;
    for(int i=0;i<a.size();i++)
```

```
    {
        mp[a[i]]=i+1;
    }
    return mp;
}
```

用法示例:

```
    int t=read();
    while(t--)
    {
        int n=read(),m=read();
        vector<int> a(n+1);
        for(int i=1;i<=n;i++)
        {
            a[i]=read();
        }
        vector<array<int,4>> q(m);
        for(int i=0;i<m;i++)
        {
            q[i][0]=read();
            q[i][1]=read();
            q[i][2]=read();
            q[i][3]=i;
        }
        //一般来说 我们最优的块长应该是 序列长度/sqrt(q)
        int block=static_cast<int>(sqrt(n))+1;//按值域分块
        auto cmp=[block](array<int,4> a,array<int,4> b)
        {
            if(a[0]/block!=b[0]/block) return a[0]/block<b[0]/block;//按块排序
            else{
                if(a[0]/block%2==0) return a[1]<b[1];//按值排序
                else return a[1]>b[1];//按块排序
            }
        };
        sort(q.begin(),q.end(),cmp);
        vector<int> ans(m,0);
        int l=1,r=0;
        BIT bit(n+1);
        for(auto [ql,qr,x,idx]:q)//暴力
        {
            while(r<qr) bit.update(a[++r],1);
            while(l>ql) bit.update(a[--l],1);
            while(r>qr) bit.update(a[r--],-1);
            while(l<ql) bit.update(a[l++],-1);
            //cout<<l<<" "<<r<<endl;
            ans[idx]=bit.query(1,a[x])+l-1;
        }
        for(auto i:ans) write(i),putchar('\n');
    }
```

2.19 李超线段树

```
//李超线段树插入O(logDlogD)查询O(logD)
//解决：插入一条直线，查询某个点的最大/最小值
//插入先划分logn区间，再懒标记下放(logn)
//维护的是直线中点的最大/最小值
//证明，对每个局部，区间中值最大代表着这个局部最优，于是可以遍历获得全局最优
//空间复杂度O(nlogDlogD)
```

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class LCSegTree{
public:
    const int L=1,R=1e9;
    const double eps=1e-9;
    struct Line{
        long double a,b;
        int id;
        Line(long double a=0,long double b=/*-1e18*/1e18,int id=0):a(a),b(b),id(id){}
        long double val(int x)const{return (long double)1.0*a*x+b;}
    };
    struct Node{
        Line l;
        Node *lc,*rc;
        Node():l(),lc(nullptr),rc(nullptr){}
    };
    Node *rt;
    LCSegTree(){
        rt=nullptr;
    }
    bool cmp(long double a,long double b){
        // return a-b>eps;//max
        return b-a>eps;//min
    }
    bool cmp1(Line a,Line b,int x){
        long double va=a.val(x),vb=b.val(x);
        return cmp(va,vb)||(fabs(va-vb)<eps&&a.id<b.id);
    }
    bool cmp2(pair<long double,int> a,long double b,int cid){
        return cmp(a.first,b)||(fabs(a.first-b)<eps&&a.second<cid);
```

```
    }
    void ins(Node*&o,int l,int r,int ql,int qr,Line v){
        if(qr<l||r<ql)return;
        if(!o)o=new Node();
        if(ql<=l&&r<=qr){
            int mid=(l+r)>>1;
            bool LB=cmp1(v,o->l,l),MB=cmp1(v,o->l,mid),RB=cmp1(v,o->l,r);
            if(MB)swap(o->l,v);
            if(l==r)return;
            if(LB!=MB)ins(o->lc,l,mid,ql,qr,v);
            else ins(o->rc,mid+1,r,ql,qr,v);
            return;
        }
        int mid=(l+r)>>1;
        ins(o->lc,l,mid,ql,qr,v);
        ins(o->rc,mid+1,r,ql,qr,v);
    }
    pair<long double,int> qry(Node*o,int l,int r,int x){
        if(!o)return {/*-1e18*/1e18,0};
        long double cur=o->l.val(x);
        int cid=o->l.id;
        int mid=(l+r)>>1;
        if(x<=mid){
            auto res=qry(o->lc,l,mid,x);
            if(cmp2(res,cur,cid))return res;
        }else{
            auto res=qry(o->rc,mid+1,r,x);
            if(cmp2(res,cur,cid))return res;
        }
        return {cur,cid};
    }
    void add(int x1,int y1,int x2,int y2,int id){
        if(x1==x2){
            int y=cmp(y1,y2)?y1:y2;
            ins(rt,L,R,x1,x1,Line(0,y,id));
        }else{
            if(x1>x2)swap(x1,x2),swap(y1,y2);
            long double a=1.0*(y2-y1)/(x2-x1),b=1.0*y1-a*x1;
            ins(rt,L,R,x1,x2,Line(a,b,id));
        }
    }
    void add(Line v){
        ins(rt,L,R,L,R,v);
    }
    pair<long double,int> ask(int x){return qry(rt,L,R,x);}
};
```

2.20 珂朵莉树(ODT)

```
//ODT(珂朵莉树)
//处理区间查询后立即覆盖问题
//修改/查询的一次为O(logn) 一次查询m个区间 覆盖后最多产生3个区间，并减少m左右个区间
//一次操作的代价是随机变量di 那么q次操作的期望是 n乘一个小常数
//所以期望是 均摊O(nlogn)
```

```
class ODT{
    public:
        struct node
        {
            int l,r;
            mutable int val;
            node(int l,int r,int val):l(l),r(r),val(val){}
            bool operator<(const node &o)const {
                return l<o.l;
            }
        };
        set<node> s;
        int n;
        //0-based
        ODT(vector<int> &a):n(a.size()){
            for(int i=0;i<a.size();i++){
                s.insert(node(i,i,a[i]));
            }
        }
        //把[l,r]区间分割成[l,mid)和[mid,r]两个区间
        auto split(int pos){
            assert(pos>=0);
            if(pos>=n) return s.end();
            auto it=s.lower_bound(node(pos,0,0));
            if(it!=s.end()&&it->l==pos) return it;
            --it;
            int l=it->l,r=it->r,val=it->val;
            s.erase(it);
            s.insert(node(l,pos-1,val));
            return s.insert(node(pos,r,val)).first;
        }
        //把[l,r]区间赋值为val
        void assign(int l,int r,int val){
            assert(0<=l&&l<=r&&r<n);
            auto itr=split(r+1),itl=split(l);
            s.erase(itl,itr);
            s.insert(node(l,r,val));
```

```
        }
        //对区间操作
        void perform(int l,int r)
        {
            assert(0<=l&&l<=r&&r<n);
            auto itr=split(r+1),itl=split(l);
            for(auto it=itl;it!=itr;++it)
            {
                //perform
            }
        }
};
```

2.21 笛卡尔树(新版)

```
class DKRTr{
    public:
    vector<array<int,4>> tr;
    //[val,idx,lc,rc]
    int root,n;
    DKRTr(vector<int> a,int n):tr(a.size()+5,{0,0,0,0}),n(n){
        //a 1-based
        stack<int> s;
        for(int i=1;i<=n;i++){
            tr[i]={a[i],i,0,0};
            int last=0;
            while(!s.empty()&&tr[s.top()][0]>tr[i][0]){
                //最小堆
                last=s.top();
                s.pop();
            }
            if(!s.empty()) tr[s.top()][3]=i;
            if(last) tr[i][2]=last;
            s.push(i);
        }
        while(!s.empty()){
            root=s.top();
            s.pop();
        }
    }
    //获取每个节点的管辖区间
    //x[i]是往左找<=x的第一个数，y[i]是往右找<x的第一个数(均不包含)
    array<vector<int>,2> get(){
        vector<int> x(n+1),y(n+1);
        auto dfs=[&](auto&& dfs,int u,int l,int r)->void{
            if(!u) return;
            x[u]=l,y[u]=r;
            dfs(dfs,tr[u][2],l,u-1);
            dfs(dfs,tr[u][3],u+1,r);
        };
        dfs(dfs,root,1,n);
        return {x,y};
    }
};
```

2.22 线段树二分

```
//e.g 区间mex->
//把询问离线，然后从左往右扫，每次把当前数最后一次出现下标加入权值线段树，然后处理每个[li,r]
//询问每个最小的x lastidx<l 维护min树即可
```

```
#define lc(p) (p<<1)
#define rc(p) (p<<1|1)
template<class Info,class Tag>
class SegTree{
    public:
    int n;
    vector<Info> info;
    vector<Tag> tag;
    SegTree(int n):n(n),info((n<<2)+5),tag((n<<2)+5){}
    SegTree(const vector<Info> &a):n(a.size()-1){
        //a 1-Based
        info.resize((n<<2)+5);
        tag.resize((n<<2)+5);
        bd(1,1,n,a);
    }
    inline void pushup(int p){
        info[p]=info[lc(p)]+info[rc(p)];
    }
    inline void apply(int p,int l,int r,const Tag &v){
        info[p].apply(l,r,v);
        tag[p].apply(v);
    }
    inline void pushdown(int p,int l,int r){
        if(!tag[p].has_tag()) return;
        int m=(l+r)>>1;
        apply(lc(p),l,m,tag[p]);
        apply(rc(p),m+1,r,tag[p]);
        tag[p]=Tag();
    }
    void bd(int p,int l,int r,const vector<Info> &a){
```

```
        if(l==r){
            info[p]=a[l];
            return;
        }
        int m=(l+r)>>1;
        bd(lc(p),l,m,a);
        bd(rc(p),m+1,r,a);
        pushup(p);
    }
    void upd(int p,int l,int r,int x,int y,const Tag &v){
        if(r<x||y<l||x>y) return;
        if(x<=l&&r<=y){
            apply(p,l,r,v);
            return;
        }
        pushdown(p,l,r);
        int m=(l+r)>>1;
        if(x<=m) upd(lc(p),l,m,x,y,v);
        if(m<y) upd(rc(p),m+1,r,x,y,v);
        pushup(p);
    }
    void mdf(int p,int l,int r,int x,const Info &v){
        if(l==r){
            info[p]=v;
            return;
        }
        pushdown(p,l,r);
        int m=(l+r)>>1;
        if(x<=m) mdf(lc(p),l,m,x,v);
        else mdf(rc(p),m+1,r,x,v);
        pushup(p);
    }
    Info qry(int p,int l,int r,int x,int y){
        if(r<x||y<l||x>y) return Info();
        if(x<=l&&r<=y) return info[p];
        pushdown(p,l,r);
        int m=(l+r)>>1;
        Info res=Info();
        if(x<=m) res=res+qry(lc(p),l,m,x,y);
        if(m<y) res=res+qry(rc(p),m+1,r,x,y);
        return res;
    }
    int findfirst(int p,int l,int r,int x,int y,
        Info &v,const function<bool(const Info&)> &chk){
        if(r<x||y<l) return n+1;
        if(x<=l&&r<=y){
            Info cmb=v+info[p];
            if(!chk(cmb)) {
                v=cmb;
                return n+1;
            }
            if(l==r) return l;
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=findfirst(lc(p),l,m,x,y,v,chk);
            if(res!=n+1) return res;
            return findfirst(rc(p),m+1,r,x,y,v,chk);
        }
        pushdown(p,l,r);
        int m=(l+r)>>1;
        int res=findfirst(lc(p),l,m,x,y,v,chk);
        if(res!=n+1) return res;
        return findfirst(rc(p),m+1,r,x,y,v,chk);
    }
    int findlast(int p,int l,int r,int x,int y,
        Info &v,const function<bool(const Info&)> &chk){
            if(r<x||y<l) return 0;
            if(x<=l&&r<=y){
                Info cmb=v+info[p];
                if(!chk(cmb)) {
                    v=cmb;
                    return 0;
                }
                if(l==r) return l;
                pushdown(p,l,r);
                int m=(l+r)>>1;
                int res=findlast(rc(p),m+1,r,x,y,v,chk);
                if(res!=0) return res;
                return findlast(lc(p),l,m,x,y,v,chk);
            }
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=findlast(rc(p),m+1,r,x,y,v,chk);
            if(res!=0) return res;
            return findlast(lc(p),l,m,x,y,v,chk);
    }
    int _findfirst(int p,int l,int r,int x,int y,
        const function<bool(const Info&)> &chk){
            if(r<x||y<l) return n+1;
            if(!chk(info[p])) return n+1;
            if(l==r) return l;
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=_findfirst(lc(p),l,m,x,y,chk);
```

```
            if(res!=n+1) return res;
            return _findfirst(rc(p),m+1,r,x,y,chk);
    }
    int _findlast(int p,int l,int r,int x,int y,
        const function<bool(const Info&)> &chk){
            if(r<x||y<l) return 0;
            if(!chk(info[p])) return 0;
            if(l==r) return l;
            pushdown(p,l,r);
            int m=(l+r)>>1;
            int res=_findlast(rc(p),m+1,r,x,y,chk);
            if(res!=0) return res;
            return _findlast(lc(p),l,m,x,y,chk);
    }
    void upd(int l,int r,const Tag &v){
        assert(1<=l&&l<=r&&r<=n);
        upd(1,1,n,l,r,v);
    }
    void mdf(int x,const Info &v){
        assert(1<=x&&x<=n);
        mdf(1,1,n,x,v);
    }
    Info qry(int l,int r){
        assert(1<=l&&l<=r&&r<=n);
        return qry(1,1,n,l,r);
    }
    //寻找在[l,r]的第一个[l,k] 满足Info{l,k}满足chk e.g.[1,4]的[1,2]满足sum(1,2)<10
    //异常值: n+1
    int findfirst(int l,int r,const function<bool(const Info&)> &chk){
        assert(1<=l&&l<=r&&r<=n);
        Info tp=Info();
        return findfirst(1,1,n,l,r,tp,chk);
    }
    //寻找在[l,r]的最后一个[k,r] 满足Info{k,r}满足chk e.g.[1,4]的[3,4]满足sum(3,4)<10
    //异常值: 0
    int findlast(int l,int r,const function<bool(const Info&)> &chk){
        assert(1<=l&&l<=r&&r<=n);
        Info tp=Info();
        return findlast(1,1,n,l,r,tp,chk);
    }
    //寻找在[l,r]的第一个k 满足Info k满足chk e.g.[1,4]的第一个k=2满足info k<10
    //异常值: n+1
    int _findfirst(int l,int r,const function<bool(const Info&)> &chk){
        assert(1<=l&&l<=r&&r<=n);
        return _findfirst(1,1,n,l,r,chk);
    }
    //寻找在[l,r]的最后一个k 满足Info k满足chk e.g.[1,4]的最后一个k=3满足info k<10
    //异常值: 0
    int _findlast(int l,int r,const function<bool(const Info&)> &chk){
        assert(1<=l&&l<=r&&r<=n);
        return _findlast(1,1,n,l,r,chk);
    }
};
// Tag 结构体: 定义懒标记
// 需要实现:
// 1. 成员变量: 存储懒标记信息
// 2. 默认构造函数: 表示无标记状态
// 3. apply(const Tag& v): 将另一个标记 v 合并到当前标记
// 4. has_tag(): 判断当前是否是无标记状态
struct Tag{
    Tag(){}
    void apply(const Tag &v){

    }
    bool has_tag(){
        return false;
    }
};
// Info 结构体: 定义节点信息
// 需要实现:
// 1. 成员变量: 存储节点维护的信息
// 2. 默认构造函数: Info 的单位元 (例如求和的0, 求积的1)
// 3. apply(int l, int r, const Tag& v): 将懒标记 v 应用到当前节点信息上
// 4. operator+(const Info& other): 合并两个子节点的信息
struct Info{
    //...
    int minn;
    Info():minn(0){}
    Info(int x):minn(x){}
    void apply(int l,int r,const Tag &v){

    }
};
Info operator+(const Info &a,const Info &b){
    //...
    Info c;
    c.minn=min(a.minn,b.minn);
    return c;
}
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<int> a(n+1);
    for(int i=1;i<=n;i++) cin>>a[i];
    vector<vector<array<int,2>>> q(n+1);
    vector<int> ans(m+1);
    for(int i=1;i<=m;i++){
        int l,r;cin>>l>>r;
        q[r].push_back({l,i});
    }
    //[0,n]->[1,n+1]
    SegTree<Info,Tag> seg(n+1);
    for(int r=1;r<=n;r++)
    {
        seg.mdf(a[r]+1,{r});
        for(const auto& [l,id]:q[r])
        {
            ans[id]=seg._findfirst(1,n+1,[&](const Info &v)->bool{
                return v.minn<l;
            })-1;
        }
    }
    for(int i=1;i<=m;i++) cout<<ans[i]<<'\n';
```

2.23 线段树分治

```
//线段树分治：把区间信息(一般是时间信息)挂到线段树的点上 然后dfs线段树，维护一些东西
//挂信息的复杂度是logn的
```

```
#define lc(x) (x<<1)
#define rc(x) (x<<1|1)
class REDSU{
public:
    vector<int> fa,sz;
    int n,tag;vector<array<int,3>> st;
    REDSU(int n):n(n){
        tag=1;
        fa.resize(n+5);
        sz.resize(n+5);
        st.reserve(n+5);
        for(int i=1;i<=n;i++){
            fa[i]=i;
            sz[i]=1;
        }
    }
    int find(int x){
        while(x!=fa[x]) x=fa[x];
        return x;
    }
    bool same(int x,int y){
        return find(x)==find(y);
    }
    void merge(int x,int y){
        x=find(x),y=find(y);
        if(x==y)
        {
            st.push_back({0,-1,tag});
            return;
        }
        if(sz[x]<sz[y]) swap(x,y); //sz[x]>=sz[y]
        st.push_back({1,y,tag});sz[x]+=sz[y];fa[y]=x;
    }
    int size(int x){return sz[find(x)];}
    void back(){
        if(!st.empty()){
            auto [fg,y,prvtag]=st.back();st.pop_back();
            tag=prvtag;
            if(!fg) return;
            sz[fa[y]]-=sz[y];fa[y]=y;
        }
    }
    void back_k(int k){
        while(k--) back();
    }
};
class SegDiv{
public:
    struct info
    {
        vector<array<int,2>> ed;
    };
    int n;vector<info> tr;
    vector<int> ans;
    SegDiv(int n):n(n){
        tr.resize((n<<2)+5);
        ans.resize(n+5);
    }
    // p:根节点为1
    // l,r:线段树值域范围 ql,qr:插入的区间
    void ins(int p,int l,int r,int ql,int qr,array<int,2> d){
        if(ql<=l&&r<=qr){
            tr[p].ed.push_back(d);
            return;
```

```
        }
        int mid=(l+r)>>1;
        if(ql<=mid) ins(lc(p),l,mid,ql,qr,d);
        if(mid<qr) ins(rc(p),mid+1,r,ql,qr,d);
    }
    // p:根节点为1
    // l,r:线段树值域范围,注意一个性质:在[l,r]的时候, 所有[l,r]的信息是没有的,如果你挂的信息是排除W的。
    void q(int p,int l,int r,REDSU &dsu){
        int k=0;
        for(const auto &[u,v]:tr[p].ed){
            if(dsu.same(u,v)){
                dsu.st.push_back({0,-1,dsu.tag}),k++;
                if(dsu.tag==1) dsu.tag=0;
            }
            dsu.merge(u,v+n);
            dsu.merge(v,u+n);
            k+=2;
        }
        if(l==r){
            ans[l]=dsu.tag;
        }
        else{
            int mid=(l+r)>>1;
            q(lc(p),l,mid,dsu);
            q(rc(p),mid+1,r,dsu);
        }
        dsu.back_k(k);
    }
};
```

用法示例:

```
    int n,m,k;cin>>n>>m>>k;
    REDSU dsu(2*n+5);
    SegDiv segdiv(k+5);
    for(int i=1;i<=m;i++){
        int u,v,l,r;cin>>u>>v>>l>>r;
        //[l,r) 0-> [l+1,r+1)->[l+1,r]
        segdiv.ins(1,1,k,l+1,r,{u,v});
    }
    segdiv.q(1,1,k,dsu);
    for(int i=1;i<=k;i++) cout<<(segdiv.ans[i]==1?"Yes":"No")<<'\n';
```

3 图论

3.1 2-sat

```
//2-sat
//处理n个命题变元的赋值问题，形式上判断形如(p->q) and (!p->q)是否可永真赋值
//即判断是否存在一种赋值使得p->q和!p->q同时为真
//很显然若p->q,q->p均成立,则p,q在一个scc里
//2026/8/22 什么狗屎注释 看都看不懂
//2-sat处理这样的东西：考虑n个命题变元：0表示此命题为假，1表示此命题为真
//然后点就是这个命题，边表示由此命题可以推出另外一个命题(即前命题成立，后命题一定成立)，注意此时逆否命题也要搞，即p->q和!q->!p都要搞
//然后跑tarjan，若一个命题和它的逆命题在一个scc里，则无解，否则有解，并且可以求出一种解法
//我们直接选拓扑序靠后的节点作为答案即可
//常见一阶逻辑
//1. a or b= !a->b and !b->a (a,b至少发生一个)
//2. !(a and b) =a->!b and b->!a (a,b不能同时发生)
//3. a->b = a->b and !b->!a (a发生，b一定发生)
//4. a=b = a->b and b->a and !a->!b and !b->!a(a,b要么一起发生，要么一起不发生)
//5. a!=b = !a->b and !b->a and a->!b and b->!a (a,b一个发生，一个不发生)
//6. a=True = !a->a (a一定发生)
//7. a=False = a->!a (a一定不发生)
```

```
pair<vector<int>,int> tarjan(vector<vector<int>> &mp,int n)
{
    vector<int> bel(n+1,-1);//bel[i]:i属于哪个强连通分量
    vector<int> dfn(n+1,-1),low(n+1,-1);
    stack<int> st;int cnt=0,scc_cnt=0;
    auto dfs=[&](auto dfs,int u)->void{
        dfn[u]=low[u]=++cnt; //时间戳+1
        st.push(u); //inst[u]=1; //入栈
        for(int v:mp[u])
        {
            if(dfn[v]==-1)//case1:u的邻接点v未被访问过
            {
                dfs(dfs,v);
                low[u]=min(low[u],low[v]);
            }
            else if(bel[v]==-1)//v所属的强连通分量还未被确定（等价于case2）
            {
                low[u]=min(low[u],dfn[v]);
            }
            //case3:u的邻接点v不在栈中,且访问过
            //说明v已经确定在某个强连通分量中，所以u的low不需要更新
        }
        if(dfn[u]==low[u])
        {
            scc_cnt++;
            while(true)
            {
                int v=st.top();
                st.pop();
                bel[v]=scc_cnt;
                if(v==u) break;
            }
        }
    };
    //图有可能不是强联通的
    for(int i=1;i<=n;i++)
    {
        if(dfn[i]==-1)
        {
            dfs(dfs,i);
        }
    }
    return {bel,scc_cnt};
}
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<int>> mp(2*n+1);
    for(int i=0;i<m;i++)
    {
        int u,b1,v,b2;
        cin>>u>>b1>>v>>b2;
        //u=b1 or v=b2
        //=>u!=b1->v=b2 and v!=b2->u=b1
        mp[u+(!b1)*n].push_back(v+b2*n);
        mp[v+(!b2)*n].push_back(u+b1*n);
        //u=b1-> u+b1*n x
    }
    auto [bel,scc_cnt]=tarjan(mp,2*n);
    vector<int> ans(n+1,-1);
    int flag=1;
    for(int i=1;i<=n;i++)
    {
        if(bel[i]==bel[i+n]) {flag=0;break;}
        else ans[i]=bel[i]>bel[i+n];
```

```
    }
    //此处处理的是i的正确性
    //当bel[u==0]>bel[u==1]时,u==0的拓扑序小,i应当被赋值为false
    //因为i->!i为永真式的前提为i=0
    //此处i的含义是命题变元i的取值=0
    //所以ans[i]=1
    if(flag)
    {
        cout<<"POSSIBLE"<<endl;
        for(int i=1;i<=n;i++)
        {
            cout<<ans[i]<<" ";
        }
        cout<<endl;
    }
    else cout<<"IMPOSSIBLE"<<endl;
```

3.2 dfs&bfs

```
//preview:2024.12.29 23:01
```

```
vector <int> edge[100000+5];
queue <int> q;int vis[100000+5]={0},sum=0;
int ans[100000+5];
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
void dfs(int x)
{
    write(x);putchar(' ');vis[x]=1;
    for(int i=0;i<edge[x].size();i++)
    {
        if(!vis[edge[x][i]]) dfs(edge[x][i]);
    }
    return ;
}
int bfs(int x)
{
    q.push(x);
    while(!q.empty())
    {
        int temp=q.front(); q.pop();
        if(vis[temp]) continue;
        else{
            vis[temp]=1;sum++;
        }
        for(int i=0;i<edge[temp].size();i++)
        {
            q.push(edge[temp][i]);
            if(!vis[edge[temp][i]]) ans[edge[temp][i]]=ans[temp];
        }
        // cout<<q.size()<<endl;
        // for(int i=0;i<=q.size();i++)
        // {
        //     cout<<q.front()<<" ";
        //     q.pop();
        // }
    }
    return sum;
}
```

用法示例:

```
    int n=read(),m=read();
    for(int i=0;i<m;i++)
    {
        int u=read(),v=read();
        edge[v].push_back(u);
    }
    for(int i=1;i<=n;i++)
```

```
    {
        ans[i]=i;
    }
    for(int i=n;i>=1;i--)
    {
        bfs(i);
        if(sum==n) break;
    }
    for(int i=1;i<=n;i++)
    {
        write(ans[i]);putchar(' ');
    }
    putchar('\n');
```

3.3 DSU on tree(树上启发式合并)

```
//树上启发式合并(dsu on tree)
//时间复杂度O(nlogn)
//考虑将树上的问题转化为集合合并信息的问题
//想到子树，就想到dsu on tree
//考虑把小集合合并到大集合里，这样小集合的大小至少变成原来的两倍，这样合并的次数就变少了
//当然可以不用dsu on tree 来暴力合并，不过要一些数据结构支持/时空复杂度会多一个log
//这边加上-撤销贡献的操作是为了保证空间复杂度
//同时保证了此时s数组是空的，所以不会影响后续的合并操作
//注意：这里撤销贡献的操作是必须的，如果不撤销贡献，空间复杂度会退化到O(n^2)
//考虑时间复杂度证明
//考虑一个节点被作为轻儿子做出贡献的次数
//实际上就转化为从该节点到根节点路径上的轻边数量
//why?路径上有轻边意味着该节点作为轻儿子的子树被合并到该节点上
//撤销操作和合并操作是互反的
//所以轻边数量就是该节点被作为轻儿子做出贡献的次数
//根据HLD的结论：该节点到根节点路径上的轻边数量<logn
//所以时间复杂度是O(nlogn),Q.E.D
```

用法示例:

```
    int n,m;cin>>n;
    vector<int> sz(n+5,0),hson(n+5,0),fa(n+5,0);
    vector<int> dfn(n+5,0),id(n+5,0),out(n+5,0);
    //子树大小,重儿子,父节点,dfs序,dfs序对应的节点,出栈序
    vector<int> c(n+5,0),s(n+5,0),ans(n+5,0);
    vector<vector<int>> tr(n+5);
    int tot=0,cnt=0;
    for(int i=1;i<n;i++)
    {
        int u,v;cin>>u>>v;
        tr[u].push_back(v);
        tr[v].push_back(u);
    }
    for(int i=1;i<=n;i++) cin>>c[i];
    auto dfs1=[&](this auto&& dfs1,int u,int f)->void{
        dfn[u]=++tot;
        id[tot]=u,sz[u]=1;
        for(auto v:tr[u])
        {
            if(v==f) continue;
            dfs1(v,u);
            sz[u]+=sz[v];
            if(sz[v]>sz[hson[u]]) hson[u]=v;
        }
        out[u]=tot;
    };//预处理一些东西
    dfs1(1,0);
    auto dfs2=[&](this auto& dfs2,int u,int f,bool keep)->void{
        for(auto v:tr[u]) //先遍历轻儿子，不保留其对集合的影响
        {
            if(v==f||v==hson[u]) continue;
            dfs2(v,u,0);
        }
        if(hson[u]) dfs2(hson[u],u,1);// 然后遍历重儿子，保留其对集合的影响
        if(!s[c[u]]) ++cnt,s[c[u]]=1; // 加入根结点对集合的贡献
        for(auto v:tr[u])
        {
            if(v==f||v==hson[u]) continue;
            for(int i=dfn[v];i<=out[v];i++) //遍历轻儿子的子树
            {
                int x=id[i];
                if(!s[c[x]]) ++cnt,s[c[x]]=1; //加入轻儿子的贡献
            }
        }
        ans[u]=cnt;
        if(!keep) //如果当前节点不是重儿子，则撤销当前节点的贡献
        {
            for(int i=dfn[u];i<=out[u];i++)
            {
                int x=id[i];
                s[c[x]]=0;
            }
            cnt=0;
```

```
        }
    };
    dfs2(1,0,1);
    cin>>m;
    for(int i=1;i<=m;i++)
    {
        int x;cin>>x;
        cout<<ans[x]<<'\n';
    }
```

3.4 eDCC (边双联通分量，tarjan)

```
vector<vector<int>> tarjan(vector<vector<pair<int,int>>> &mp,int n)
{
    //边双联通分量：无向图中，边双联通分量是指一个极大子图，删除该子图中的任意一条边，该子图仍然连通
    //连接边双联通分量的边称为桥
    vector<int> low(n+1,-1),dfn(n+1,-1);
    //low:从当前点出发能到达的最早时间戳
    //dfn:当前点的时间戳
    vector<vector<int>> dccs;
    stack<int> st; int cnt=0;
    auto dfs=[&](auto dfs,int u,int fre)->void{
        //fre:来时的边
        dfn[u]=low[u]=++cnt;
        st.push(u);
        for(auto [v,rev]:mp[u])
        {
            if(dfn[v]==-1)//case1:未访问
            {
                dfs(dfs,v,rev);
                low[u]=min(low[u],low[v]);//更新low[u]
            }
            else if(rev!=(fre^1))//case2:已访问且不是该边的反边
            {
                //阻断向父节点更新的可能
                //多重边可能有一种特殊的组合让v!=fa失效
                //eg.1-2,1-2,2-3,2-3
                low[u]=min(low[u],dfn[v]);//更新low[u]
            }
        }
        if(dfn[u]==low[u])//case3:u的子树中不存在能到达u的祖先的边
        //u的子树全为dcc
        {
            vector<int> dcc;
            while(true){
                int t=st.top();st.pop();
                dcc.push_back(t);
                if(t==u) break;
            }
            dccs.push_back(dcc);
        }
    };
    for(int i=1;i<=n;i++)
    {
        if(dfn[i]==-1)
        {
            dfs(dfs,i,-1);
        }
    }
    return dccs;
}
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<pair<int,int>>> mp(n+1);
    vector<int> val(n+1);
    int tot=0;
    for(int i=0;i<m;i++)
    {
        int u,v;cin>>u>>v;
        if(u==v) continue;
        mp[u].push_back({v,tot+1});
        mp[v].push_back({u,tot});
        tot+=2;//存各自的边的编号
    }
    vector<vector<int>> bccs=tarjan(mp,n);
    cout<<bccs.size()<<endl;
    for(auto bcc: bccs)
    {
        cout<<bcc.size()<<' ';
        for(auto x: bcc)
        {
            cout<<x<<" ";
        }
        cout<<endl;
    }
```

3.5 johnson 最短路

```
//johnson全源最短路算法：O(nmlogm)
//重新标记边权后,u-v两点的任意路径一定有hu-hv项，最短路不变
//由于三角形不等式，所以重新标记边权，边权一定非负
//所以重新标记边权后，可以使用Dijkstra算法求最短路
```

```
#define int long long
int has_neg=0;
vector<int> SPFA(vector<vector<pair<int,int>>>& mp,int s,int n)
{
    vector<int> dis(n+1,1e9);
    vector<int> vis(n+1,0);
    vector<int> cnt(n+1,0);
    dis[s]=0;queue<int> q;
    q.push(s);vis[s]=1;cnt[s]=0;
    while(!q.empty())
    {
        int u=q.front();
        q.pop();vis[u]=0;
        for(auto [v,w]:mp[u])
        {
            if(dis[v]>dis[u]+w)//松弛
            {
                dis[v]=dis[u]+w;
                cnt[v]=cnt[u]+1;
                if(cnt[v]>n-1)//存在负权环
                {
                    //1-n的节点，最短路最多经过n-1条边，如果经过n条边，说明存在负权环
                    has_neg=1;
                    return dis;
                }
                if(!vis[v])
                {
                    q.push(v);
                    vis[v]=1;
                }
            }
        }
    }
    return dis;
    //SPFA：形式上Bellman_Ford是一棵树，很显然，只有上一次被松弛的节点u，才有可能对v进行松弛，所以可以采用SPFA
    //为啥只有上一次被松弛的节点u，才有可能对v进行松弛？
    //手玩一下就好了（悲
    //考虑简单图，他可以是个递推的过程
}
vector<int> dijkstra(int n,vector<vector<pair<int,int>>>& mp,int s)
{
    vector<int> dis(n+1,1e9);//初始化距离为无穷大
    dis[s]=0;//起点到起点的距离为0
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> pq;
    pq.push({0,s});//将起点加入优先队列
    while(!pq.empty())
    {
        int u=pq.top().second;//取出当前距离最小的点
        int d=pq.top().first;//取出当前距离最小的点的距离
        pq.pop();
        if(d>dis[u]) continue;//u已经被更新过
        if(mp[u].empty()) continue;
        for(auto it:mp[u])
        {
            int v=it.first;
            int w=it.second;
            if(dis[v]>dis[u]+w)//更新s->v的最短距离(min(s->v,s->u->v))
            {
                dis[v]=dis[u]+w;
                pq.push({dis[v],v});
            }
        }
    }
    //正确性证明：
    //假设目前更新s->t(=3),假设存在s->u->t(=2),则s->u<s->t,而s->u一定在之前被更新过，所以s->u->t一定在之前被更新过，与假设矛盾。
    //单源最短路(正边权)
    //时间复杂度O(ElogV),E为边数,V为点数(二叉堆)
    //使用斐波那契堆的 Dijkstra 算法的时间复杂度为 O(E+VlogV)。
    //不用堆优化：O(v^2+E)
    //当E<<v^2时，使用堆优化
    //当E~v^2时，不用堆优化
    return dis;
}
vector<vector<int>> johnson(vector<vector<pair<int,int>>>& mp,int n)
{
    //1.添加一个虚拟节点0，连接到所有节点，边权为0
    for(int i=1;i<=n;i++)
    {
        mp[0].push_back({i,0});
    }
    //2.使用Bellman-Ford算法计算从虚拟节点0到所有节点的最短路径
```

```
    vector<int> h=SPFA(mp,0,n+1);
    if(has_neg==1)
    {
        cout<<-1<<endl;
        exit(0);
    }
    //3.删除虚拟节点0，并更新所有边的权重
    for(int i=1;i<=n;i++)
    {
        for(auto& it:mp[i])
        {
            it.second+=h[i]-h[it.first];
        }
    }
    //4.对每个节点i，使用Dijkstra算法计算从i到所有节点的最短路径
    vector<vector<int>> dis(n+1,vector<int>(n+1,1e18));
    for(int i=1;i<=n;i++)
    {
        dis[i]=dijkstra(n,mp,i);
        for(int j=1;j<=n;j++)
        {
            if(dis[i][j]==1e18) dis[i][j]=1e9;
            else dis[i][j]-=h[i]-h[j];
        }
    }
    //5.更新所有边的权重
    for(int i=1;i<=n;i++)
    {
        for(auto& it:mp[i])
        {
            it.second+=h[it.first]-h[i];
        }
    }
    return dis;
}
```

用法示例:

```
    int n,m;
    cin>>n>>m;
    vector<vector<pair<int,int>>> mp(n+1);
    for(int i=0;i<m;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        mp[u].push_back({v,w});
    }
    vector<vector<int>> dis=johnson(mp,n);
    for(int i=1;i<=n;i++)
    {
        int ans=0;
        for(int j=1;j<=n;j++)
        {
            //cout<<dis[i][j]<<" ";
            ans+=j*dis[i][j];
        }
        //cout<<endl;
        cout<<ans<<endl;
    }
```

3.6 MCS（最大势算法）

```
class MCS{
public:
    vector<vector<int>> mp;
    int n,mxlab,tim;
    int omg,alp;//最大团/色数+最大独立集
    vector<int> lab,vis,peo,mark,rk;
    vector<vector<int>> st,chklis;
    MCS(int n,vector<vector<int>>& mp):
        n(n),vis(n+1,0),lab(n+1,0),mp(mp),
        mxlab(0),peo(n+1,0),tim(0),chklis(n+1),
        mark(n+1,0),rk(n+1,0),omg(0),alp(0),st(n+1){
            mcs();
    }
    void mcs(){
        st[0].resize(n);
        iota(st[0].begin(),st[0].end(),1);
        for(int i=n;i>=1;i--){
            int u=-1;
            while(true){
                while(mxlab>0&&st[mxlab].empty()){
                    mxlab--;
                }
                int cur=st[mxlab].back();
                st[mxlab].pop_back();
                if(!vis[cur]){
                    u=cur;
                    break;
                }
            }
            peo[i]=u,vis[u]=1,rk[u]=i;
            omg=max(omg,lab[u]+1);
```

```
            for(auto v:mp[u]){
                if(!vis[v]){
                    lab[v]++;
                    st[lab[v]].push_back(v);
                    mxlab=max(mxlab,lab[v]);
                }
            }
        }
    }
    bool chk(){
        for(int u=1;u<=n;u++){
            int p=0,minrk=n+1;
            for(auto v:mp[u]){
                if(rk[v]>rk[u]&&rk[v]<minrk){
                    minrk=rk[v];
                    p=v;
                }
            }
            if(p){
                chklis[p].push_back(u);
            }
        }
        for(int p=1;p<=n;p++){
            if(chklis[p].empty()) continue;
            tim++;
            for(auto v:mp[p]){
                mark[v]=tim;
            }
            for(auto u:chklis[p]){
                for(auto v:mp[u]){
                    if(rk[v]>rk[u]){
                        if(p==v) continue;
                        if(mark[v]!=tim){
                            return false;
                        }
                    }
                }
            }
        }
        return true;
    }
    void getalp(){
        fill(vis.begin(),vis.end(),0);
        for(int i=1;i<=n;i++){
            int u=peo[i];
            if(!vis[u]){
                alp++;
                vis[u]=1;
                for(auto v:mp[u]){
                    vis[v]=1;
                }
            }
        }
    }
};
```

用法示例:

```
    int t;cin>>t;
    while(t--){
        int n,m;cin>>n>>m;
        vector<vector<int>> mp(n+1);
        for(int i=1;i<=m;i++){
            int u,v;cin>>u>>v;
            mp[u].push_back(v);
            mp[v].push_back(u);
        }
        MCS mcs(n,mp);
        if(mcs.chk()){
            cout<<"Yes"<<endl;
            mcs.getalp();
            for(int i=1;i<=n;i++){
                cout<<mcs.peo[i]<<" ";
            }
            cout<<endl;
            cout<<mcs.omg<<' '<<mcs.omg<<' '<<mcs.alp<<'\n';
        }
        else cout<<"No"<<endl;
    }
```

3.7 SCC（低注释）

```
pair<vector<int>,int> tarjan(vector<vector<int>> &mp,int n)
{
    vector<int> bel(n+1,-1);//bel[i]:i属于哪个强连通分量
    vector<int> dfn(n+1,-1),low(n+1,-1);
    stack<int> st;int cnt=0,scc_cnt=0;
    auto dfs=[&](auto dfs,int u)->void{
        dfn[u]=low[u]=++cnt; //时间戳+1
        st.push(u); //inst[u]=1; //入栈
        for(int v:mp[u])
        {
            if(dfn[v]==-1)//case1:u的邻接点v未被访问过
```

```
            {
                dfs(dfs,v);
                low[u]=min(low[u],low[v]);
            }
            else if(bel[v]==-1)//v所属的强连通分量还未被确定（等价于case2）
            {
                low[u]=min(low[u],dfn[v]);
            }
            //case3:u的邻接点v不在栈中,且访问过
            //说明v已经确定在某个强连通分量中，所以u的low不需要更新
        }
        if(dfn[u]==low[u])
        {
            scc_cnt++;
            while(true)
            {
                int v=st.top();
                st.pop();
                bel[v]=scc_cnt;
                if(v==u) break;
            }
        }
    };
    //图有可能不是强联通的
    for(int i=1;i<=n;i++)
    {
        if(dfn[i]==-1)
        {
            dfs(dfs,i);
        }
    }
    return {bel,scc_cnt};
}
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<int>> mp(n+1);
    vector<int> val(n+1);
    for(int i=1;i<=n;i++) cin>>val[i];
    for(int i=0;i<m;i++)
    {
        int u,v;cin>>u>>v;
        mp[u].push_back(v);
    }
    auto [bel,cnt]=tarjan(mp,n);
    vector<vector<int>> mp2(cnt+1);
    vector<int> val2(cnt+1,0);
    vector<int> in(cnt+1,0);
    vector<int> dp(cnt+1,0);
    for(int i=1;i<=n;i++)
    {
        val2[bel[i]]+=val[i];
    }
    for(int i=1;i<=n;i++)
    {
        for(int v:mp[i])
        {
            if(bel[i]!=bel[v])
            {
                mp2[bel[i]].push_back(bel[v]);
                in[bel[v]]++;
            }
        }
    }
    queue<int> q;
    for(int i=1;i<=cnt;i++)
    {
        if(in[i]==0) q.push(i),dp[i]=val2[i];
    }
    while(!q.empty())
    {
        int u=q.front();q.pop();
        for(int v:mp2[u])
        {
            dp[v]=max(dp[v],dp[u]+val2[v]);
            in[v]--;
            if(in[v]==0) q.push(v);
        }
    }
    cout<<*max_element(dp.begin()+1,dp.end())<<endl;
```

3.8 SCC（强联通分量，缩点，tarjan）

```
///vector<vector<int>> tarjan(vector<vector<int>> &mp,int n,int m)
pair<vector<int>,int> tarjan(vector<vector<int>> &mp,int n)
{
    //求强连通分量，强连通分量是有向图中的极大顶点子集，其中任意两个顶点都是互相可达的
    ///vector<vector<int>> scc;//强连通分量
    vector<int> bel(n+1,-1);//bel[i]:i属于哪个强连通分量
    vector<int> dfn(n+1,-1),low(n+1,-1);
    //vector<int> inst(n+1,0);
```

```
    //dfn:时间戳 (dfs序), low:从i开始能到达的最小时间戳, inst:是否在栈中
    stack<int> st;int cnt=0,scc_cnt=0;
    //st:未放到scc的点, cnt: 计时器, 初始为0
    auto dfs=[&](auto dfs,int u)->void{
        dfn[u]=low[u]=++cnt; //时间戳+1
        st.push(u); //inst[u]=1; //入栈
        for(int v:mp[u])
        {
            if(dfn[v]==-1)//case1:u的邻接点v未被访问过
            {
                dfs(dfs,v);
                low[u]=min(low[u],low[v]);//用v的low更新u的low
            }
            // else if(inst[v])//case2:u的邻接点v在栈中,且访问过
            // {
            //     low[u]=min(low[u],dfn[v]);
            //     //有可能存在一个环
            // }
            else if(bel[v]==-1)//v所属的强连通分量还未被确定 (等价于case2)
            {
                low[u]=min(low[u],dfn[v]);
            }
            //case3:u的邻接点v不在栈中,且访问过
            //说明v已经确定在某个强连通分量中, 所以u的low不需要更新
        }
        if(dfn[u]==low[u])
        {
            //u是某个强连通分量的根 (第一个被访问的结点)
            //why,low[u]==dfn[u]说明u没有指向自己的边, 所以u是某个强连通分量的根
            //某个强连通分量的根的low值不会被更新
            // vector<int> s;
            // while(true)
            // {
            //     int v=st.top();
            //     st.pop();inst[v]=0;
            //     s.push_back(v);
            //     if(v==u) break;
            // }
            // scc.push_back(s);
            scc_cnt++;
            while(true)
            {
                int v=st.top();
                st.pop();
                bel[v]=scc_cnt;
                if(v==u) break;
            }
        }
    };
    //图有可能不是强联通的
    for(int i=1;i<=n;i++)
    {
        if(dfn[i]==-1)
        {
            dfs(dfs,i);
        }
    }
    //return scc;
    return {bel,scc_cnt};
}
//证明: 如果结点 u 是某个强连通分量在搜索树中遇到的第一个结点, 那么这个强连通分量的其余结点肯定是在搜索树中以 u 为根的子树中。
//结点 u 被称为这个强连通分量的根。
//反证法: 假设有个结点 v 在该强连通分量中但是不在以 u 为根的子树中, 那么 u 到 v 的路径中肯定有一条离开子树的边。
//但是这样的边只可能是横叉边或者反祖边, 然而这两条边都要求指向的结点已经被访问过了, 这就和 v 不在以 u 为根的子树中矛盾了。得证。
//其实手玩一下, 若有个结点 v 在该强连通分量中但是不在以 u 为根的子树中, 他的与u形成的那个环, 其实是v一定在u的子树中, 从u的dfs一定能够遍历到v
//画个图就知道了
//并且, 很容易想到, 对于一个连通分量图, 有且只有一个根, 即第一个被访问的结点
//所以算法正确性显然
//时间复杂度: O(n+m)
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<int>> mp(n+1);
    vector<int> val(n+1);
    for(int i=1;i<=n;i++) cin>>val[i];
    for(int i=0;i<m;i++)
    {
        int u,v;cin>>u>>v;
        mp[u].push_back(v);
    }
    auto [bel,cnt]=tarjan(mp,n);
    vector<vector<int>> mp2(cnt+1);
    vector<int> val2(cnt+1,0);
    vector<int> in(cnt+1,0);
    vector<int> dp(cnt+1,0);
    for(int i=1;i<=n;i++)
    {
```

```
        val2[bel[i]]+=val[i];
    }
    for(int i=1;i<=n;i++)
    {
        for(int v:mp[i])
        {
            if(bel[i]!=bel[v])
            {
                mp2[bel[i]].push_back(bel[v]);
                in[bel[v]]++;
            }
        }
    }
    queue<int> q;
    for(int i=1;i<=cnt;i++)
    {
        if(in[i]==0) q.push(i),dp[i]=val2[i];
    }
    while(!q.empty())
    {
        int u=q.front();q.pop();
        for(int v:mp2[u])
        {
            dp[v]=max(dp[v],dp[u]+val2[v]);
            in[v]--;
            if(in[v]==0) q.push(v);
        }
    }
    cout<<*max_element(dp.begin()+1,dp.end())<<endl;
```

3.9 ShortestPath（Bellman_Ford,SPFA)

```
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
vector<int> Bellman_Ford(vector<vector<pair<int,int>>>& mp,int s,int n)
{
    vector<int> dis(n+1,0x7fffffff);
    dis[s]=0;
    for(int i=1;i<=n-1;i++)
    {
        for(int j=1;j<=n;j++)
        {
            for(auto [u,w]:mp[j])
            {
                if(dis[j]!=0x7fffffff&&dis[j]+w<dis[u])
                {
                    dis[u]=dis[j]+w;
                }
            }
        }
    }
    for(int j=1;j<=n;j++)
    {
        for(auto [u,w]:mp[j])
        {
            if(dis[j]!=0x7fffffff&&dis[j]+w<dis[u])
            {
                cout<<"negative cycle!"<<endl;
            }
        }
    }
    return dis;
    //Bellman_Ford 对所有的边进行n-1次松弛操作，如果在进行第n次松弛操作时，仍然存在边可以松弛，则说明图中存在负权环（从s点出发存在负权环）
    //时间复杂度：O(nm),形式上就是暴力）
    //第i次循环，我们能找到经历i条边到达的点的最短距离
    //所以第n次循环，我们能找到经历n条边到达的点的最短距离，如果存在负权环，那么一定能在第n次循环找到经历n条边到达的点的最短距离
}
```

```
vector<int> SPFA(vector<vector<pair<int,int>>>& mp,int s,int n)
{
    vector<int> dis(n+1,0x7fffffff);
    vector<int> vis(n+1,0);
    vector<int> cnt(n+1,0);
    dis[s]=0;queue<int> q;
    q.push(s);vis[s]=1;cnt[s]=0;
    while(!q.empty())
    {
        int u=q.front();
        q.pop();vis[u]=0;
        for(auto [v,w]:mp[u])
        {
            if(dis[v]>dis[u]+w)//松弛
            {
                dis[v]=dis[u]+w;
                cnt[v]=cnt[u]+1;
                if(cnt[v]>n-1)//存在负权环
                {
                    //1-n的节点，最短路最多经过n-1条边，如果经过n条边，说明存在负权环
                    cout<<"negative cycle!"<<endl;
                    return dis;
                }
                if(!vis[v])
                {
                    q.push(v);
                    vis[v]=1;
                }
            }
        }
    }
    return dis;
    //SPFA：形式上Bellman_Ford是一棵树，很显然，只有上一次被松弛的节点u，才有可能对v进行松弛，所以可以采用SPFA
    //为啥只有上一次被松弛的节点u，才有可能对v进行松弛？
    //手玩一下就好了（悲
    //考虑简单图，他可以是个递推的过程
}
```

用法示例:

```
    int n=read(),m=read();
    vector<vector<pair<int,int>>> mp(n+1);
    for(int i=0;i<m;i++)
    {
        int u=read(),v=read(),w=read();
        mp[u].push_back({v,w});
    }
```

3.10 ShortestPath（dijkstra）

```
/*
朴素dijk核心步骤
初始化：dis 数组全设为无穷大，dis[S] = 0，vis 数组全为 0。
迭代 $V$ 次（外层循环）：找最小值：遍历所有节点 $1 \dots V$，在所有 vis[i] == 0 的点中，找到 dis[i] 最小的节点 $u$。
标记：将 $u$ 标记为已确定最短路，即 vis[u] = 1。
松弛：遍历 $u$ 的所有出边（如果是邻接矩阵，则遍历所有节点 $v$），若 dis[v] > dis[u] + w(u, v)，则更新 dis[v]。
*/
```

```
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
vector<int> dijkstra(int n,vector<vector<pair<int,int>>>mp,int s)
{
    vector<int> dis(n+1,0x7fffffff);//初始化距离为无穷大
    dis[s]=0;//起点到起点的距离为0
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> pq;
```

```
    pq.push({0,s});//将起点加入优先队列
    while(!pq.empty())
    {
        int u=pq.top().second;//取出当前距离最小的点
        int d=pq.top().first;//取出当前距离最小的点的距离
        pq.pop();
        if(d>dis[u]) continue;//u已经被更新过
        if(mp[u].empty()) continue;
        for(auto it:mp[u])
        {
            int v=it.first;
            int w=it.second;
            if(dis[v]>dis[u]+w)//更新s->v的最短距离(min(s->v,s->u->v))
            {
                dis[v]=dis[u]+w;
                pq.push({dis[v],v});
            }
        }
    }
    //正确性证明：
    //假设目前更新s->t(=3),假设存在s->u->t(=2),则s->u<s->t,而s->u一定在之前被更新过，所以s->u->t一定在之前被更新过，与假设矛盾。
    //单源最短路(正边权)
    //时间复杂度O(ElogV),E为边数,V为点数(二叉堆)
    //使用斐波那契堆的 Dijkstra 算法的时间复杂度为 O(E+VlogV)。
    //不用堆优化：O(v^2+E)
    //当E<<v^2时，使用堆优化
    //当E~v^2时，不用堆优化
    return dis;
}
```

用法示例:

```
    int n=read(),m=read(),s=read();
    vector<vector<pair<int,int>>> mp(n+1);
    while(m--)
    {
        int u=read(),v=read(),w=read();
        mp[u].push_back({v,w});
    }
    vector<int> dis=dijkstra(n,mp,s);
    for(int i=1;i<=n;i++)
        write(dis[i]),putchar(' ');
    int T_end=clock();
```

3.11 ShortestPath（Floyed）

```
//Floyed 算法
//处理任意两点之间的最短路径(无负环)
//时间复杂度O(n^3)
```

```
const int MAXN=1e4;
int Graph[MAXN][MAXN];
int dp[MAXN][MAXN];
```

用法示例:

```
    int T=10;
    srand(time(NULL));
    for(int i=1;i<=T;i++)
    {
        for(int j=1;j<=T;j++)
        {
            int op=rand()%3;
            if(false) Graph[i][j]=0;
            else if(op==1) Graph[i][j]=0x3f3f3f3f;
            else Graph[i][j]=(rand()*10+rand())%10;
            //Graph[i][j]=(rand()*10+rand())%10;
        }
        Graph[i][i]=0;
    }
    for(int i=1;i<=T;i++)
    {
        for(int j=1;j<=T;j++)
        {
            dp[i][j]=Graph[i][j];
        }
    }
    for(int i=1;i<=T;i++)
    {
        for(int j=1;j<=T;j++)
        {
            cout<<dp[i][j]<<" ";
        }
        cout<<endl;
    }
    cout<<endl;
    for(int i=1;i<=T;i++)
    {
        for(int j=1;j<=T;j++)
        {
```

```
            for(int k=1;k<=T;k++)
            {
                dp[j][k]=min(dp[j][k],dp[j][i]+dp[i][k]);
            }
        }
    }
    for(int i=1;i<=T;i++)
    {
        for(int j=1;j<=T;j++)
        {
            cout<<dp[i][j]<<" ";
        }
        cout<<endl;
    }
    int T_end=clock();
```

3.12 vBCC（点双连通分量，tarjan.点划分）

```
vector<vector<int>> tarjan(vector<vector<int>> &mp,int n)
{
    //点双连通分量：无割点，且任意两点间至少有两条路径
    vector<int> low(n+1,-1),dfn(n+1,-1);
    //low:从当前点出发能到达的最早时间戳
    //dfn:当前点的时间戳
    vector<vector<int>> bccs;
    stack<int> st;
    int cnt=0;
    auto dfs=[&](auto dfs,int u,int fa)->void{
        int ch=0; //儿子数
        dfn[u]=low[u]=++cnt;
        st.push(u);
        for(auto v:mp[u])
        {
            if(dfn[v]==-1)//case1:未访问
            {
                ch++;
                dfs(dfs,v,u);
                low[u]=min(low[u],low[v]);//更新low[u]
                if((fa==-1&&ch>1)||(fa!=-1&&low[v]>=dfn[u]))//是割点,v以及他的被处理过的子树是一个bcc
                {
                    vector<int> bcc;
                    while(1)
                    {
                        int x=st.top();st.pop();
                        bcc.push_back(x);
                        if(x==v)break;//处理到v
                    }
                    bcc.push_back(u);//把割点也加入bcc:割点有可能在多个bcc中
                    bccs.push_back(bcc);
                }
            }
            else if(v!=fa)//case2:已访问且不是父节点
            {
                low[u]=min(low[u],dfn[v]);//更新low[u]
            }
        }
        // if(fa==-1&&ch==0) {
        //  bccs.push_back({u});
        // }
    };
    for(int i=1;i<=n;i++)
    {
        if(dfn[i]==-1)
        {
            dfs(dfs,i,-1);
            //处理剩下的bcc
            vector<int> bcc;
            while(!st.empty())
            {
                int x=st.top();st.pop();
                bcc.push_back(x);
            }
            if(!bcc.empty()) bccs.push_back(bcc);
        }
    }
    return bccs;
}
//无向图中割点：删除该点后，图的bcc数增加
//一个图中割点的判断
//1.对于某个顶点 u，如果存在至少一个顶点 v（u 的儿子），使得low[v]>=dfn[u] ，即只能回到祖先（到不了dfn更早的点），那么 u 点为割点。
//2.对于搜索的起始点，如果它的儿子数大于等于 2，那么它就是割点。
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<int>> mp(n+1);
    vector<int> val(n+1);
    for(int i=0;i<m;i++)
    {
        int u,v;cin>>u>>v;
        mp[u].push_back(v);
```

```
        mp[v].push_back(u);
    }
    vector<vector<int>> bccs=tarjan(mp,n);
    cout<<bccs.size()<<endl;
    for(auto bcc: bccs)
    {
        cout<<bcc.size()<<' ';
        for(auto x: bcc)
        {
            cout<<x<<" ";
        }
        cout<<endl;
    }
```

3.13 vBCC（点双连通分量，tarjan.边划分）

```
vector<vector<int>> tarjan(const vector<vector<array<int,2>>>&g,int n){
    vector<int> dfn(n+1,0),low(n+1,0);
    vector<vector<int>> bccs;
    stack<int> st;int cnt=0;
    auto dfs=[&](auto&& dfs,int u,int in)->void{
        dfn[u]=low[u]=++cnt;
        for(auto[v,id]:g[u]){
            if(!dfn[v]){
                st.push(id);
                dfs(dfs,v,id);
                low[u]=min(low[u],low[v]);
                if(low[v]>=dfn[u]){
                    vector<int> bcc;
                    while(true){
                        int e=st.top();st.pop();
                        bcc.push_back(e);
                        if(e==id)break;
                    }
                    bccs.push_back(bcc);
                }
            }else if(id!=in&&dfn[v]<dfn[u]){
                st.push(id);
                low[u]=min(low[u],dfn[v]);
            }
        }
    };
    for(int i=1;i<=n;i++){
        if(!dfn[i]) dfs(dfs,i,0);
    }
    return bccs;
}
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<array<int,2>>> g(n+1);
    for(int i=1;i<=m;i++){
        int u,v;cin>>u>>v;
        g[u].push_back({v,i});
        g[v].push_back({u,i});
    }
    auto bccs=tarjan(g,n);
    cout<<bccs.size()<<"\n";
    for(const auto&bcc:bccs){
        cout<<bcc.size()<<" ";
        for(int id:bcc)cout<<id<<" ";
        cout<<"\n";
    }
```

3.14 二分图最大匹配

```
//二分图最大匹配 最大流O(n^1/2*m)
//二分图的划分可以用二分图染色进行
//二分图最大匹配：设有若干男生,若干女生,若干配对关系,求最大匹配,即求出最多的配对关系
//二分图最小点覆盖：在一张无向图中选择最少的顶点，满足每条边至少有一个端点被选
//->二分图中，最小点覆盖中的顶点数量等于最大匹配中的边数量。
//从网络流的角度看，最小点覆盖问题就是最小割问题：选择左部点，相当于切割它与源点的连边；选择右部点，相当于切割它与汇点的连边。
//why?因为一条边被割掉，意味着原二分图上这个点的配对点无法跑一条流->最小割
//[引] 最小割：把图分为S集和t集 s->t的边为割边，割边的最小权值和为最小割
//最大独立集问题：在一张无向图中选择最多的顶点，满足两两之间互不相邻。
//->二分图中，最大独立集中的顶点数量等于n-最小点覆盖中的顶点数量
//[引理1] 图G(V,E)中V的子集S为点覆盖<=>V/S为独立集
//证明：S为点覆盖=>V/S为独立集 假设V/S不是独立集，则存在v1,v2∈V/S，v1,v2相邻，但是v1-v2这条边的两个端点都不在S中 ->矛盾
//V/S为独立集=>S为点覆盖 要证 对所有u-v∈E，至少有一个端点在S中
//假设u-v都不在S中，则u,v都在V/S中 但此时V/S不是独立集 ->矛盾
//[推论1] 图G(V,E)中V的子集S为最大点覆盖<=>V/S为最小独立集
//有向无环图最小路径覆盖：在一张有向图中，选择最少数量的简单路径，使得所有顶点都恰好出现在一条路径中。
//->有向无环图的最小路径覆盖数等于顶点数减去最大匹配数
//通过dag构造的二分图如下：
//将每个顶点拆成两个顶点，v_in v_out
//对于原图中的每条有向边u->v，在二分图中连边v_in-u_out
//此证明为构造性的：二分图的每个匹配对应这dag中不交的各个路径
```

```
//考虑最极端的平凡图 n个点 路径是n条 增加一个匹配->路径数减少1
//Q.E.D
```

```
class maxflow{
    public:
        struct node
        {
            int to,cap,id;
        };
        vector<vector<node>> mp;
        vector<int> dep,cur;//dep:层次图, cur:当前弧优化
        int n,m,s,t;
        maxflow(int n,int m,int s,int t,vector<array<int,3>>& eds):
        mp(n+1),n(n),m(m),s(s),t(t),dep(n+1),cur(n+1){
            //u->v capacity
            for(auto [u,v,cap]:eds){
                int uid=mp[u].size();
                int vid=mp[v].size();
                mp[u].push_back({v,cap,vid});
                mp[v].push_back({u,0,uid});
                //建反边
            }
        }
        bool bfs(){
            fill(dep.begin(),dep.end(),-1);
            fill(cur.begin(),cur.end(),0);
            queue<int> q;
            q.push(s);
            dep[s]=0;
            while(!q.empty()){
                int u=q.front();
                q.pop();
                for(auto [v,cap,id]:mp[u]){
                    if(cap>0&&dep[v]==-1){
                        dep[v]=dep[u]+1;
                        q.push(v);
                    }
                }
            }
            return dep[t]!=-1;
        }
        int dfs(int u,int lim)//到u点的最大流量lim
        {
            if(u==t) return lim;
            int sum=0;//u点流出的流量
            for(int &i=cur[u];i<mp[u].size();i++){
                //当前弧优化,考虑u->v有重边,那么这个优化会使
                //被榨干过的v的出边不再被访问
                auto [v,cap,id]=mp[u][i];
                if(cap>0&&dep[v]==dep[u]+1){
                    int f=dfs(v,min(lim,cap));
                    mp[u][i].cap-=f;
                    mp[v][id].cap+=f;
                    sum+=f;
                    lim-=f;
                    if(lim==0) break;
                }
            }
            if(sum==0) dep[u]=-1;//无增广路
            return sum;
        }
        int dinic(){
            int res=0;
            while(bfs()){
                res+=dfs(s,INT_MAX);
            }
            return res;
        }
};
```

用法示例:

```
    int n,m,e;cin>>n>>m>>e;
    vector<array<int,3>> eds;
    int s=n+m+1,t=n+m+2;
    //s->left cap 1
    for(int i=1;i<=n;i++){
        eds.push_back({s,i,1});
    }
    //right->t cap 1
    for(int i=1;i<=m;i++){
        eds.push_back({i+n,t,1});
    }
    //u->v cap 1
    for(int i=1;i<=e;i++){
        int u,v;cin>>u>>v;
        eds.push_back({u,v+n,1});
    }
    maxflow mf(n+m+2,e,s,t,eds);
    cout<<mf.dinic()<<endl;
```

3.15 二分图染色

```
struct node{
    int v;
    int w;
};
vector<node> mp[20005];
bool vis[20005]={false};
int dyed[20005]={0};
int Data[100005];
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
bool dye(int start,int mid)
{
    queue<int> q;
    q.push(start);
    vis[start]=1;dyed[start]=1;
    while(!q.empty())
    {
        int temp=q.front();
        q.pop();
        for(auto i:mp[temp])
        {
            if(i.w>=mid)
            {
                if(!vis[i.v])
                {
                    q.push(i.v);
                    vis[i.v]=true;
                    dyed[i.v]=3-dyed[temp];
                }
                else if(dyed[i.v]==dyed[temp]) return false;
            }
        }
    }
    return true;
}
bool isBinGraph(int n,int mid)
{
    memset(vis,0,sizeof(vis));
    memset(dyed,0,sizeof(dyed));
    for(int i=1;i<=n;i++)
    {
        if(!vis[i])
        {
            if(!dye(i,mid)) return false;
        }
    }
    return true;
}
```

用法示例:

```
    // freopen("in.txt","r",stdin);
    // freopen("out.txt","w",stdout);
    int n=read(),m=read();
    for(int i=0;i<m;i++)
    {
        int u=read(),v=read(),w=read();
        mp[u].push_back({v,w});
        mp[v].push_back({u,w});
        Data[i]=w;
    }
    sort(Data,Data+m);
    // for(int i=0;i<m;i++)
    // {
    //     cout<<Data[i]<<endl;
    // }
    if(isBinGraph(n,0))
    {
```

```
        cout<<"0"<<endl;
    }
    else
    {
        int l=0,r=m;
        while(l<=r)
        {
            int mid=(l+r)>>1;
            if(!isBinGraph(n,Data[mid])) l=mid+1;
            else r=mid-1;
        }
        cout<<Data[r]<<endl;
    }
```

3.16 分层图

```
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
int dij(vector<vector<pair<int,int>>>& mp,int s,int n,int t,int kk)
{
    vector<int> vis((kk+1)*n+1,0x7fffffff);
    vis[s]=0;
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> pq;
    pq.push({0,s});
    while(!pq.empty())
    {
        auto [val,k]=pq.top();
        pq.pop();
        if(val>vis[k]) continue;
        for(auto i:mp[k])
        {
            auto [v,w]=i;
            if(vis[v]>vis[k]+w)
            {
                vis[v]=vis[k]+w;
                pq.push({vis[v],v});
            }
        }
    }
    int ans=0x7fffffff;
    for(int i=0;i<=kk;i++)
    {
        //i表示免费次数
        ans=min(ans,vis[i*n+t]);
    }
    return ans;
}
```

用法示例:

```
    //分层图：解决k次免费（有代价）最短路问题
    int n=read(),m=read(),k=read();
    int s=read(),t=read();
    vector<vector<pair<int,int>>> mp((k+1)*n+1);
    while(m--)
    {
        int u,v,w;
        u=read(),v=read(),w=read();
        for(int i=0;i<=k;i++)
        {
            mp[i*n+u].push_back({i*n+v,w});
            mp[i*n+v].push_back({i*n+u,w});
            if(i!=k)
            {
                mp[i*n+u].push_back({(i+1)*n+v,0});
                mp[i*n+v].push_back({(i+1)*n+u,0});//分层图连边
            }
        }
```

```
    }
    cout<<dij(mp,s,n,t,k)<<endl;
```

3.17 差分约束

```
//对一个差分约束系统，判断是否存在一组解，使得所有约束条件都成立。
//ex. x1-x2<=3
//    x2-x3<=-2
//    x1-x3<=1
//将xn看作超级源点（到所有点的权值为w=0）到n的最短路
//那么第一个式子的意义就是x1<=x2+3,0到1的最短路<=3+0到2的最短路
//在图上的意义就是建2->1的边权为3的边，0->1,0->2的边权为0的边
//0->1,0->2的边权为0的边也是添加了以下条件
//x1-x0<=0
//x2-x0<=0
//x0=0
//那么整个系统就转化为了一张图
//求xn即求0到n的最短路，如果存在负环，则无解，否则有解
//负环还原的形式为
//x1-x2<=-1...1
//x2-x3<=-4...2
//x3-x1<=-5...3
//1+2+3->0<=-10,不成立
//还有结论，设定w即求x1,x2..xn<=w的最大解
//如果差分约束系统换换不等号，求最长路，spfa改一下即可
//结论形式证明
//假设X0是定死的；X1到Xn在满足所有约束的情况下可以取到的最大值分别为M1、M2、……、Mn（当然我们不知道它们的值是多少）；解出的源点到每个点的最短路径长度为D1、D2、……、Dn。
//基本的Bellman-Ford算法是一开始初始化D1到Dn都是无穷大。然后检查所有的边对应的三角形不等式，一但发现有不满足三角形不等式的情况，则更新对应的D值。最后求出来的D1到Dn就是源点到每个点的最短路径长度。
//如果我们一开始初始化D1、D2、……、Dn的值分别为M1、M2、……、Mn，则由于它们全都满足三角形不等式（我们刚才已经假设M1到Mn是一组合法的解），则Bellman-Ford算法不会再更新任合D值，则最后得出的解就是M1、M2、……、Mn。
//好了，现在知道了，初始值无穷大时，算出来的是D1、D2、……、Dn；初始值比较小的时候算出来的则是M1、M2、……、Mn。大家用的是同样的算法，同样的计算过程，总不可能初始值大的算出来的结果反而小吧。所以D1、D2、……、Dn就是M1、M2、……、Mn。
```

```
int flag=0;
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
vector<int> SPFA(vector<vector<pair<int,int>>>& mp,int s,int n)
{
    vector<int> dis(n+1,0x7fffffff);
    vector<int> vis(n+1,0);
    vector<int> cnt(n+1,0);
    dis[s]=0;queue<int> q;
    q.push(s);vis[s]=1;cnt[s]=0;
    while(!q.empty())
    {
        int u=q.front();
        q.pop();vis[u]=0;
        for(auto [v,w]:mp[u])
        {
            if(dis[v]>dis[u]+w)//松弛
            {
                dis[v]=dis[u]+w;
                cnt[v]=cnt[u]+1;
                if(cnt[v]>=n+1)
                {
                    flag=1;
                    return vector<int>(n+1,-1);
                }
                if(!vis[v])
                {
                    q.push(v);
                    vis[v]=1;
```

```
                }
            }
        }
    }
    return dis;
}
```

用法示例:

```
    int n=read(),m=read();
    vector<vector<pair<int,int>>> mp(n+1);
    for(int i=1;i<=m;i++)
    {
        int v=read(),u=read(),w=read();
        mp[u].push_back(make_pair(v,w));
    }
    for(int i=1;i<=n;i++)
    {
        mp[0].push_back(make_pair(i,0));
    }
    vector<int>ans=SPFA(mp,0,n);
    if(flag==1)
    {
        printf("NO\n");
    }
    else
    {
        //printf("YES\n");
        for(int i=1;i<=n;i++)
        {
            printf("%d ",ans[i]);
        }
        printf("\n");
    }
```

3.18 找环(topsort)

```
//无向图,跑完topsort后,度数>1的点一定在环上
//有向图只要把入度=0的入队,跑完topsort后,入度>0的点一定在环上
//同理topsort可以用来判环
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<int>> mp(n+1);
    vector<int> deg(n+1,0);
    for(int i=1;i<=m;i++)
    {
        int u,v;cin>>u>>v;
        mp[u].push_back(v);
        mp[v].push_back(u);
        deg[u]++,deg[v]++;
    }
    queue<int> q;
    for(int i=1;i<=n;i++)
    {
        if(deg[i]==1) q.push(i);
    }
    while(!q.empty())
    {
        int u=q.front();q.pop();
        for(int v:mp[u])
        {
            deg[v]--;
            if(deg[v]==1) q.push(v);
        }
    }
```

3.19 拓扑排序

```
#define MOD 80112002
vector<int> edge[5005];
int _to[5005]={0},_in[5005]={0};
long long ans[5005]={0};queue<int> q;
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
```

```
    int top=0;
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
```

用法示例:

```
    int n=read(),m=read();
    for(int i=0;i<m;i++)
    {
        int u=read(),v=read();
        edge[v].push_back(u);
        _to[u]++;_in[v]++;
    }
    for(int i=1;i<=n;i++)
    {
        if(!_to[i])
        {
            q.push(i);
            ans[i]=1;
        }
    }
    while(!q.empty())
    {
        int temp=q.front();
        //cout<<temp<<endl;
        q.pop();
        for(int i=0;i<edge[temp].size();i++)
        {
            //cout<<temp<<' '<<edge[temp][i]<<' '<<ans[temp]<<' '<<ans[edge[temp][i]]<<endl;
            ans[edge[temp][i]]=(ans[edge[temp][i]]+ans[temp])%MOD;
            _to[edge[temp][i]]--;
            if(!_to[edge[temp][i]]) q.push(edge[temp][i]);
        }
    }
    long long res=0;
    for(int i=1;i<=n;i++)
    {
        if(!_in[i])
        {
            //cout<<i<<' '<<ans[i]<<endl;
            res=(res+ans[i])%MOD;
        }
    }
    write(res),putchar('\n');
```

3.20 无向图三元环计数

```
//无向图三元环计数 O(msqrt(m))
//由 无向图把边定向：度数小的指向度数大的点，然后每个点的出度<=sqrt(2m) 保证
//可用根号分治证明
//至于不重不漏的原因，某个三元环定向后一定是算法所示的形态，按度数最小的分类就是对的
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<int>> mp(n+1);
    vector<int> deg(n+1,0),vis(n+1,0);
    for(int i=0;i<m;i++){
        int u,v;cin>>u>>v;
        mp[u].push_back(v);
        mp[v].push_back(u);
        deg[u]++,deg[v]++;
    }
    vector<vector<int>> g(n+1);
    for(int u=1;u<=n;u++){
        for(int v:mp[u]){
            if(deg[u]>deg[v]||(deg[u]==deg[v]&&u<v)){
                g[u].push_back(v);
            }
        }
    }
    int ans=0;
    for(int u=1;u<=n;u++){
        for(auto v:mp[u]) vis[v]=1;
        for(auto v:g[u]){
            for(auto w:g[v]){
                if(vis[w]) ans++;
            }
        }
        for(auto v:mp[u]) vis[v]=0;
    }
    cout<<ans<<endl;
```

3.21 最大流(dinic)

```
//最大流，解决从有向图源点到汇点的最大流量问题(假定源点流量无限)
//dinic算法，时间复杂度O(n^2*m)
//增广路：是从源点到汇点的路径，其上所有边的残余容量均大于0
//初级思路：贪心选择所有增广路，然后更新边权，引入反向边进行反悔贪心
```

```
//基本思路：每次bfs把图变成一个带层数的DAG(限制dfs深度)
//然后找到极大增广流量，更新图，重复上述过程
```

```
#define int long long
class maxflow{
    public:
        const int INF=1e18;
        struct node
        {
            int to,cap,id;
        };
        vector<vector<node>> mp;
        vector<int> dep,cur;//dep:层次图，cur:当前弧优化
        int n,m,s,t;
        maxflow(int n,int m,int s,int t,vector<array<int,3>>& eds):
        mp(n+1),n(n),m(m),s(s),t(t),dep(n+1),cur(n+1){
            //u->v capacity
            for(auto [u,v,cap]:eds){
                int uid=mp[u].size();
                int vid=mp[v].size();
                mp[u].push_back({v,cap,vid});
                mp[v].push_back({u,0,uid});
                //建反边
            }
        }
        bool bfs(){
            fill(dep.begin(),dep.end(),-1);
            fill(cur.begin(),cur.end(),0);
            queue<int> q;
            q.push(s);
            dep[s]=0;
            while(!q.empty()){
                int u=q.front();
                q.pop();
                for(auto [v,cap,id]:mp[u]){
                    if(cap>0&&dep[v]==-1){
                        dep[v]=dep[u]+1;
                        q.push(v);
                    }
                }
            }
            return dep[t]!=-1;
        }
        int dfs(int u,int lim)//到u点的最大流量lim
        {
            if(u==t) return lim;
            int sum=0;//u点流出的流量
            for(int &i=cur[u];i<mp[u].size();i++){
                //当前弧优化,考虑u->v有重边,那么这个优化会使
                //被榨干过的v的出边不再被访问
                auto [v,cap,id]=mp[u][i];
                if(cap>0&&dep[v]==dep[u]+1){
                    int f=dfs(v,min(lim,cap));
                    mp[u][i].cap-=f;
                    mp[v][id].cap+=f;
                    sum+=f;
                    lim-=f;
                    if(lim==0) break;
                }
            }
            if(sum==0) dep[u]=-1;//无增广路
            return sum;
        }
        int dinic(){
            int res=0;
            while(bfs()){
                res+=dfs(s,INF);
            }
            return res;
        }
};
```

用法示例:

```
    int n,m,s,t;
    cin>>n>>m>>s>>t;
    vector<array<int,3>> eds(m);
    for(auto &[u,v,cap]:eds){
        cin>>u>>v>>cap;
    }
    maxflow mf(n,m,s,t,eds);
    cout<<mf.dinic()<<endl;
```

3.22 最大费用可行流

```
class MC{
public:
    struct node{
        int to;
        int cap;
        int cost;
        int rev;
```

```
    };
    int n,s,t;
    int maxf=0,maxc=0;
    const int INF=1e9;
    vector<vector<node>> mp;
    vector<int> dis,cur,inq,vis;
    MC(int n,int s,int t,vector<array<int,4>>& eds):
    n(n),s(s),t(t),mp(n+1),dis(n+1),
    cur(n+1),inq(n+1,0),vis(n+1,0){
        for(auto [u,v,cap,w]:eds){
            int uid=mp[u].size();
            int vid=mp[v].size();
            mp[u].push_back({v,cap,w,vid});
            mp[v].push_back({u,0,-w,uid});
            //反边的费用是负的
        }
    }

    bool spfa(){
        fill(dis.begin(),dis.end(),-INF);
        fill(inq.begin(),inq.end(),0);
        deque<int> q;dis[s]=0,inq[s]=1;
        q.push_back(s);
        while(!q.empty()){
            int u=q.front();q.pop_front();
            inq[u]=0;
            for(auto [v,cap,w,rev]:mp[u]){
                if(cap>0&&dis[u]+w>dis[v]){
                    dis[v]=dis[u]+w;
                    if(!inq[v]){
                        if(!q.empty()&&dis[v]>dis[q.front()]){
                            q.push_front(v);
                        }else{
                            q.push_back(v);
                        }
                        inq[v]=1;
                    }
                }
            }
        }
        return dis[t]>0;
    }

    int dfs(int u,int f){
        if(u==t)return f;
        vis[u]=1;
        int res=0;
        for(int &i=cur[u];i<mp[u].size();i++){
            auto [v,cap,w,rev]=mp[u][i];
            if(!vis[v]&&cap>0&&dis[u]+w==dis[v]){
                int tmp=dfs(v,min(f,cap));
                f-=tmp;
                res+=tmp;
                mp[u][i].cap-=tmp;
                mp[v][rev].cap+=tmp;
                maxc+=tmp*w;
                if(!f)break;
            }
        }
        vis[u]=0;
        return res;
    }

    void dinic(){
        while(spfa()){
            fill(vis.begin(),vis.end(),0);
            fill(cur.begin(),cur.end(),0);
            maxf+=dfs(s,INF);
        }
    }

};
```

用法示例:

```
    int n,m,k;cin>>n>>m>>k;
    vector<vector<int>> mp(n+1,vector<int>(m+1));
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
            cin>>mp[i][j];
        }
    }
    auto ch=[&](int x,int y){
        return (x-1)*m+y;
    };
    auto chk=[&](int x,int y){
        return x>=1&&x<=n&&y>=1&&y<=m&&mp[x][y]!=-1;
    };
    int dx[]={0,0,1,-1};
    int dy[]={1,-1,0,0};
    vector<array<int,4>> eds;
    //拆点, 入点ch(i,j) 出点ch(i,j)+n*m
    for(int i=1;i<=n;i++){
        for(int j=1;j<=m;j++){
```

```
            if(mp[i][j]==0) eds.push_back({ch(i,j),ch(i,j)+n*m,1,-1});
            else if(mp[i][j]==1) eds.push_back({ch(i,j),ch(i,j)+n*m,1,0});
            if(mp[i][j]!=-1){
                for(int d=0;d<4;d++){
                    int nx=i+dx[d],ny=j+dy[d];
                    if(chk(nx,ny)){
                        eds.push_back({ch(i,j)+n*m,ch(nx,ny),1,0});
                    }
                }
            }
        }
    }
    int s=2*n*m+1,t=2*n*m+2;
    for(int i=1;i<=k;i++){
        int x,y;cin>>x>>y;
        eds.push_back({s,ch(x,y),1,0});
    }
    for(int i=1;i<=k;i++){
        int x,y;cin>>x>>y;
        eds.push_back({ch(x,y)+n*m,t,1,100});
    }
    MC mc(2*n*m+2,s,t,eds);
    mc.dinic();
    cout<<mc.maxc<<endl;
```

3.23 最小斯坦纳树

```
//最小斯坦纳树 给定一个图 和k个关键点 求一个包含所有关键点的最小生成树(可以用其他点)
//时间复杂度(n*3^k+2^k*mlogm)
```

```
#define int long long //赫赫 要不要龙龙呢
const int INF=1e18;
using namespace std;
```

用法示例:

```
    int n,m,k;cin>>n>>m>>k;
    vector<vector<array<int,2>>> mp(n+1);
    for(int i=1;i<=m;i++)
    {
        int u,v,w;cin>>u>>v>>w;
        mp[u].push_back({v,w});
        mp[v].push_back({u,w});
    }
    vector<vector<int>> dp((1<<k),vector<int>(n+1,INF));
    //dp[i][j]表示以i为集合，j为根的最小贡献
    vector<int> sp(k+1);
    for(int i=1;i<=k;i++)
    {
        cin>>sp[i];
        dp[1<<(i-1)][sp[i]]=0;
    }
    for(int st=1;st<(1<<k);st++){
        for(int t=st;t;t=(t-1)&st){
            for(int i=1;i<=n;i++)
            {
                dp[st][i]=min(dp[st][i],dp[t][i]+dp[st-t][i]);
            }
        }//枚举子集，合并 只保证了v节点的状态是最优的
        priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> q;
        vector<int> vis(n+1,0);
        for(int i=1;i<=n;i++){
            if(dp[st][i]!=INF) q.push({dp[st][i],i});
        }
        while(!q.empty()){
            int u=q.top().second;q.pop();
            if(vis[u]) continue;
            vis[u]=1;
            for(auto [v,w]:mp[u]){
                if(dp[st][v]>dp[st][u]+w){
                    dp[st][v]=dp[st][u]+w;
                    q.push({dp[st][v],v});
                }
            }
        }//状态传播
    }
    cout<<dp[(1<<k)-1][sp[1]]<<endl;
    //状态传播完了，由于树的性质，所以sp[1]一定是根节点
```

3.24 最小生成树（boruvka,标）

```
//比较标准的boruvka算法
```

```
using ull=unsigned long long;
using namespace std;
class DSU{
    public:
        int n;vector<int> fa,sz;
        DSU(int n):n(n)
        {
```

```
            srand(time(NULL));
            fa.resize(n+1);
            sz.resize(n+1);
            for(int i=1;i<=n;i++)
            {
                fa[i]=i;
                sz[i]=1;
            }
        }
        int find(int u){
            return fa[u]==u?u:fa[u]=find(fa[u]);
        }
        void merge(int a,int b)
        {
            int u=find(a),v=find(b);
            if(u==v) return;
            fa[u]=v;
            sz[v]+=sz[u];
        }
        int same(int a,int b)
        {
            return find(a)==find(b);
        }
        int size(int u){
            return sz[find(u)];
        }
        vector<vector<int>> get(){
            vector<vector<int>> ans(n+1);
            for(int i=1;i<=n;i++)
            {
                ans[find(i)].push_back(i);
            }
            ans.erase(remove(ans.begin(),ans.end(),vector<int>()),ans.end());
            return ans;
        }
};
```

用法示例:

```
    const int INF=2e9+5;
    int t;cin>>t;
    while(t--)
    {
        int n,m;cin>>n>>m;
        vector<int> a(n+1),p(n+1),ip(n+1);
        for(int i=1;i<=n;i++) cin>>a[i];
        iota(p.begin()+1,p.end(),1);
        sort(p.begin()+1,p.end(),[&](int x,int y){return a[x]<a[y];});
        for(int i=1;i<=n;i++) ip[p[i]]=i;
        sort(a.begin()+1,a.end());
        vector<vector<int>> g(n+1);
        for(int i=1;i<=m;i++)
        {
            int u,v;cin>>u>>v;
            u=ip[u],v=ip[v];
            g[u].push_back(v);
            g[v].push_back(u);
        }
        for(int i=1;i<=n;i++) sort(g[i].begin(),g[i].end());
        auto find=[&](int u,int v){
            auto it=lower_bound(g[u].begin(),g[u].end(),v);
            if(it==g[u].end()||*it!=v) return 0;
            return 1;
        };
        DSU dsu(n);
        vector<int> cur(n+1,1);
        ll ans=0;
        auto boruvka=[&](){
            int cnt=0;
            vector<int> r(n+1); r[n]=n;
            for(int i=n-1;i>=1;i--){
                if(dsu.same(i,i+1)) r[i]=r[i+1];
                else r[i]=i;
            }
            auto now=dsu.get();
            vector<array<int,2>> best(now.size(),{INF,-1});
            for(int i=0;i<now.size();i++){
                auto &vec=now[i];
                for(auto x:vec){
                    int j=cur[x];
                    while(j<=n){
                        if(dsu.same(x,j)){
                            j=r[j]+1;
                            continue;
                        }
                        if(find(x,j)){
                            j+=1;
                            continue;
                        }
                        if(best[i][0]>a[x]+a[j]){
                            best[i]={a[x]+a[j],j};
                        }
                        break;
                    }
                    cur[x]=j;
```

```
                }
            }
            for(int i=0;i<now.size();i++){
                if(best[i][1]!=-1){
                    if(!dsu.same(now[i][0],best[i][1])){
                        ans+=best[i][0];
                        dsu.merge(now[i][0],best[i][1]);
                        cnt++;
                    }
                }
            }
            return cnt;
        };
        while(boruvka());
        if(dsu.size(1)!=n) cout<<"-1\n";
        else cout<<ans<<"\n";
    }
```

3.25 最小生成树（boruvka）

```
//boruvka算法
//考虑对多个联通块找最短的出边，然后合并，每次合并能使联通块个数/2，
//所以时间复杂度是O(nlogn*p),其中p是每次找出边的复杂度
//处理完全图好
//此处是对每一轮边合并边找，标准的写法是对每一轮先找出所有边，然后合并边（这个不容易有错）
```

```
class DSU{
    public:
        int n;vector<int> fa,sz;
        DSU(int n):n(n)
        {
            srand(time(NULL));
            fa.resize(n+1);
            sz.resize(n+1);
            for(int i=1;i<=n;i++)
            {
                fa[i]=i;
                sz[i]=1;
            }
        }
        int find(int u){
            return fa[u]==u?u:fa[u]=find(fa[u]);
        }
        void merge(int a,int b)
        {
            int u=find(a),v=find(b);
            if(u==v) return;
            fa[u]=v;
            sz[v]+=sz[u];
        }
        int same(int a,int b)
        {
            return find(a)==find(b);
        }
        int size(int u){
            return sz[find(u)];
        }
        vector<vector<int>> get(){
            vector<vector<int>> ans(n+1);
            for(int i=1;i<=n;i++)
            {
                ans[find(i)].push_back(i);
            }
            ans.erase(remove(ans.begin(),ans.end(),vector<int>()),ans.end());
            return ans;
        }
};
```

用法示例:

```
    int t;cin>>t;
    while(t--)
    {
        int n,m;cin>>n>>m;
        vector<int> a(n+1);
        vector<vector<int>> g(n+1);
        for(int i=1;i<=n;i++) cin>>a[i];
        for(int i=1;i<=m;i++){
            int u,v;cin>>u>>v;
            g[u].push_back(v);
            g[v].push_back(u);
        }
        set<array<int,2>> s;
        for(int i=1;i<=n;i++)
            s.insert({a[i],i});
        ll ans=0;
        DSU dsu(n);
        auto boruvka=[&](){
            int cnt=0;
            vector<vector<int>> now(n+1);
            for(int i=1;i<=n;i++) now[dsu.find(i)].push_back(i);
            for(auto &vec:now){
```

```
                int nm=2e9+5,idx=-1;
                for(auto &x:vec) s.erase({a[x],x});
                for(auto &u:vec){
                    for(auto &v:g[u]){
                        if(dsu.find(v)==dsu.find(u)) continue;
                        s.erase({a[v],v});
                    }
                    if(s.size()){
                        auto [val,id]=*s.begin();
                        if(val+a[u]<nm) nm=val+a[u],idx=id;
                    }
                    for(auto &v:g[u]){
                        if(dsu.find(v)==dsu.find(u)) continue;
                        s.insert({a[v],v});
                    }
                }
                if(idx!=-1&&!dsu.same(idx,vec[0])){
                    cnt++;
                    ans+=nm;
                    dsu.merge(idx,vec[0]);
                }
                for(auto &x:vec) s.insert({a[x],x});
            }
            return cnt;
        };
        while(boruvka());
        if(dsu.size(1)!=n) cout<<"-1\n";
        else cout<<ans<<'\n';
    };
```

3.26 最小生成树（kruskal）

```
class BSU{
    public:
        int n;vector<int> fa;
        BSU(int n):n(n)
        {
            fa.resize(n+1);
            for(int i=1;i<=n;i++)
            {
                fa[i]=i;
            }
        }
        int find(int u){
            return fa[u]==u?u:fa[u]=find(fa[u]);
        }
        void merge(int a,int b)
        {
            int op=rand()%2;
            if(op==0) fa[find(a)]=find(b);
            else fa[find(b)]=find(a);
        }
};
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
int kruskal(vector<array<int,3>> &edge,int m,int n)
{
    sort(edge.begin(),edge.end(),[](auto a,auto b)->bool{
        return a[2]<b[2];
    });
    BSU bsu(n);int cnt=0;int ans=0;
    for(auto [u,v,w]:edge)
    {
        if(bsu.find(u)!=bsu.find(v))
        {
            bsu.merge(u,v);
            cnt++;ans+=w;
            //cout<<u<<" "<<v<<endl;
        }
```

```
        if(cnt==n-1) break;
    }
    return cnt==n-1?ans:-1;
}
//时间复杂度O(mlogm)，证明同prim
```

用法示例:

```
    srand(time(NULL));
    int n=read(),m=read();
    vector<array<int,3>> edge(m);
    for(int i=0;i<m;i++)
    {
        int u=read(),v=read(),w=read();
        edge[i]={u,v,w};
    }
    int ans=kruskal(edge,m,n);
    if(ans==-1) puts("orz");
    else cout<<ans<<endl;
```

3.27 最小生成树（prim）

```
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
vector<int> prim(vector<vector<pair<int,int>>> &mp,int s,int n)
{
    vector<int> dis(n+1,0x7fffffff);//点离当前生成树的距离
    vector<int> in(n+1,0);//点是否在生成树中
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> pq;
    dis[s]=0;pq.push({0,s});
    while(!pq.empty())
    {
        auto [_,u]=pq.top();//找到最小生成树连的边中未加入生成树的边权最小的边
        pq.pop();
        if(in[u]) continue;
        in[u]=true;//进入最小生成树
        for(auto [v,w]:mp[u])
        {
            if(dis[v]>w&&!in[v])//更新不在当前生成树中的点离生成树的距离
            {
                dis[v]=w;
                pq.push({dis[v],v});
            }
        }
    }
    return dis;
}
//和dij一样，时间复杂度O((n+m)logn),暴力prim时间复杂度O(n^2),看看稀疏图和稠密图哪个更快
//正确性证明：反证法：假设prim生成的不是最小生成树
// 1).设prim生成的树为G0
// 2).假设存在Gmin使得cost(Gmin)<cost(G0)    则在Gmin中存在<u,v>不属于G0
// 3).将<u,v>加入G0中可得一个环，且<u,v>不是该环的最长边(这是因为<u,v>∈Gmin)
// 4).这与prim每次生成最短边矛盾
// 5).故假设不成立，命题得证.
```

用法示例:

```
    int n=read(),m=read();
    vector<vector<pair<int,int>>> mp(n+1);
    for(int i=1;i<=m;i++)
    {
        int u=read(),v=read(),w=read();
        mp[u].push_back({v,w});
        mp[v].push_back({u,w});
    }
    vector<int> ans=prim(mp,1,n);
    int sum=0;
```

```
    for(int i=1;i<=n;i++)
    {
        if(ans[i]==0x7fffffff)
        {
            puts("不连通！");
        }
        sum+=ans[i];
    }
    cout<<sum<<endl;
```

3.28 最小费用最大流(dinic)

```
//最小费用最大流，O(nmf)
//基本思路：找到最短增广路，然后增广，直到找不到为止
//最短增广路：spfa(slf优化)，每次找到最短路径，然后更新，直到找不到为止
//增广：用dinic思路在最短路上多路增广
```

```
#define int long long
class Mcmf{
public:
    struct node{
        int to;
        int cap;
        int cost;
        int rev;
    };
    int n,s,t;
    int maxf=0,minc=0;
    const int INF=1e18;
    vector<vector<node>> mp;
    vector<int> dis,cur,inq,vis;
    Mcmf(int n,int s,int t,vector<array<int,4>>& eds):
    n(n),s(s),t(t),mp(n+1),dis(n+1),
    cur(n+1),inq(n+1,0),vis(n+1,0){
        for(auto [u,v,cap,w]:eds){
            int uid=mp[u].size();
            int vid=mp[v].size();
            mp[u].push_back({v,cap,w,vid});
            mp[v].push_back({u,0,-w,uid});
            //反边的费用是负的
        }
    }

    bool spfa(){
        fill(dis.begin(),dis.end(),INF);
        fill(inq.begin(),inq.end(),0);
        deque<int> q;dis[s]=0,inq[s]=1;
        q.push_back(s);
        while(!q.empty()){
            int u=q.front();q.pop_front();
            inq[u]=0;
            for(auto [v,cap,w,rev]:mp[u]){
                if(cap>0&&dis[u]+w<dis[v]){
                    dis[v]=dis[u]+w;
                    if(!inq[v]){
                        if(!q.empty()&&dis[v]<dis[q.front()]){
                            q.push_front(v);
                        }else{
                            q.push_back(v);
                        }
                        inq[v]=1;
                    }
                }
            }
        }
        return dis[t]!=INF;
    }

    int dfs(int u,int f){
        if(u==t)return f;
        vis[u]=1;
        int res=0;
        for(int &i=cur[u];i<mp[u].size();i++){
            auto [v,cap,w,rev]=mp[u][i];
            if(!vis[v]&&cap>0&&dis[u]+w==dis[v]){
                int tmp=dfs(v,min(f,cap));
                f-=tmp;
                res+=tmp;
                mp[u][i].cap-=tmp;
                mp[v][rev].cap+=tmp;
                minc+=tmp*w;
                if(!f)break;
            }
        }
        vis[u]=0;
        return res;
    }

    void dinic(){
        while(spfa()){
            fill(vis.begin(),vis.end(),0);
            fill(cur.begin(),cur.end(),0);
```

```
            maxf+=dfs(s,INF);
        }
    }

};
```

用法示例:

```
    int n,m,s,t;
    cin>>n>>m>>s>>t;
    vector<array<int,4>> eds;
    for(int i=0;i<m;i++){
        int u,v,cap,w;
        cin>>u>>v>>cap>>w;
        eds.push_back({u,v,cap,w});
    }
    Mcmf mcmf(n,s,t,eds);
    mcmf.dinic();
    cout<<mcmf.maxf<<" "<<mcmf.minc<<endl;
```

3.29 最小费用最大流(dinic,浮点)

```
//最小费用最大流，O(nmf)
//基本思路：找到最短增广路，然后增广，直到找不到为止
//最短增广路：spfa(slf优化)，每次找到最短路径，然后更新，直到找不到为止
//增广：用dinic思路在最短路上多路增广
//浮点数比较
//a==b->abs(a-b)<1e-10
//a<b->a+eps<b
//a>b->a>b+eps
```

```
#define int long long
struct ta{
    int u,v;
    int cap;
    double w;
};
class Mcmf{
public:
    struct node{
        int to;
        int cap;
        double cost;
        int rev;
    };
    int n,s,t;
    int maxf=0;double minc=0;
    const int INF=1e18;
    vector<vector<node>> mp;
    vector<int> cur,inq,vis;
    vector<double> dis;
    Mcmf(int n,int s,int t,vector<ta>& eds):
    n(n),s(s),t(t),mp(n+1),dis(n+1),
    cur(n+1),inq(n+1,0),vis(n+1,0){
        for(auto [u,v,cap,w]:eds){
            int uid=mp[u].size();
            int vid=mp[v].size();
            mp[u].push_back({v,cap,w,vid});
            mp[v].push_back({u,0,-w,uid});
            //反边的费用是负的
        }
    }

    bool spfa(){
        fill(dis.begin(),dis.end(),INF);
        fill(inq.begin(),inq.end(),0);
        deque<int> q;dis[s]=0,inq[s]=1;
        q.push_back(s);
        while(!q.empty()){
            int u=q.front();q.pop_front();
            inq[u]=0;
            for(auto [v,cap,w,rev]:mp[u]){
                if(cap>0&&dis[u]+w+1e-10<dis[v]){
                    dis[v]=dis[u]+w;
                    if(!inq[v]){
                        if(!q.empty()&&dis[v]+1e-10<dis[q.front()]){
                            q.push_front(v);
                        }else{
                            q.push_back(v);
                        }
                        inq[v]=1;
                    }
                }
            }
        }
        return dis[t]!=INF;
    }

    int dfs(int u,int f){
        if(u==t)return f;
        vis[u]=1;
        int res=0;
```

```
        for(int &i=cur[u];i<mp[u].size();i++){
            auto [v,cap,w,rev]=mp[u][i];
            if(!vis[v]&&cap>0&&abs(dis[u]+w-dis[v])<1e-10){
                int tmp=dfs(v,min(f,cap));
                f-=tmp;
                res+=tmp;
                mp[u][i].cap-=tmp;
                mp[v][rev].cap+=tmp;
                minc+=1.0*tmp*w;
                if(!f)break;
            }
        }
        vis[u]=0;
        return res;
    }

    void dinic(){
        while(spfa()){
            fill(vis.begin(),vis.end(),0);
            fill(cur.begin(),cur.end(),0);
            maxf+=dfs(s,INF);
        }
    }

};
```

用法示例:

```
    int n;cin>>n;
    vector<array<int,2>> xy(n+1);
    for(int i=1;i<=n;i++){
        cin>>xy[i][0]>>xy[i][1];
    }
    int s=0,t=2*n+1;
    vector<ta> eds;
    for(int i=1;i<=n;i++)
    {
        eds.push_back({s,i,2,0});
    }
    for(int i=1;i<=n;i++)
    {
        eds.push_back({i+n,t,1,0});
    }
    auto dis=[&](int u,int v)->double{
        return sqrt(1.0*(xy[u][0]-xy[v][0])*(xy[u][0]-xy[v][0])+1.0*(xy[u][1]-xy[v][1])*(xy[u][1]-xy[v][1]));
    };
    for(int u=1;u<=n;u++)
    {
        for(int v=1;v<=n;v++)
        {
            if(xy[u][1]>xy[v][1])
            {
                eds.push_back({u,v+n,1,dis(u,v)});
            }
        }
    }
    Mcmf mc(2*n+2,s,t,eds);
    mc.dinic();
    if(mc.maxf==n-1) cout<<fixed<<setprecision(10)<<mc.minc<<endl;
    else cout<<-1<<endl;
```

3.30 最小费用最大流(原始对偶)

```
//Primal-Dual 原始对偶算法,O(F*E*logE)
//利用johnson最短路，将每条边的权值加上一个常数，使得每条边的权值非负，从而可以使用dijkstra算法
```

```
#define int long long
class Mcmf{
public:
    struct node{
        int to;
        int cap;
        int cost;
        int rev;
    };
    int n,s,t;
    int maxf=0,minc=0;
    const int INF=1e18;
    vector<vector<node>> mp;
    vector<int> dis,h,prev,previd;
    Mcmf(int n,int s,int t,vector<array<int,4>>& eds):
    n(n),s(s),t(t),mp(n+1),dis(n+1),
    h(n+1,INF),prev(n+1),previd(n+1){
        //only 1-based
        for(auto [u,v,cap,w]:eds){
            int uid=mp[u].size();
            int vid=mp[v].size();
            mp[u].push_back({v,cap,w,vid});
            mp[v].push_back({u,0,-w,uid});
            //反边的费用是负的
        }
    }
    bool dijk(){
```

```
        fill(dis.begin(),dis.end(),INF);
        dis[s]=0;
        priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>> pq;
        pq.push({0,s});
        while(!pq.empty()){
            auto [d,u]=pq.top();
            pq.pop();
            if(dis[u]<d) continue;
            for(int i=0;i<mp[u].size();i++){
                auto [v,cap,w,rev]=mp[u][i];
                int cost=w+h[u]-h[v];
                if(cap>0&&dis[u]+cost<dis[v]){
                    dis[v]=dis[u]+cost;
                    prev[v]=u;//记录前驱
                    previd[v]=i;//记录当前弧
                    pq.push({dis[v],v});
                }
            }
        }
        return dis[t]<INF;
    }

    void SPFA(){
        h[s]=0;
        queue<int> q;
        vector<bool> inq(n+1,0);
        q.push(s),inq[s]=1;
        while(!q.empty()){
            int u=q.front();
            q.pop();
            inq[u]=0;
            for(auto [v,cap,w,rev]:mp[u]){
                if(cap>0&&h[u]+w<h[v]){
                    h[v]=h[u]+w;
                    if(!inq[v]){
                        q.push(v);
                        inq[v]=1;
                    }
                }
            }
        }
    }

    void PD(){
        SPFA();
        while(dijk())
        {
            //now dis(u-v)=dis(u,v)(real)+h[u]-h[v]
            //when u=0, dis(u,v)=dis(u,v)(real)-h[v]
            int d=INF;
            for(int i=t;i!=s;i=prev[i])
            {
                d=min(d,mp[prev[i]][previd[i]].cap);
            }
            //计算增广路径上的最小流量
            maxf+=d;
            minc+=d*(dis[t]+h[t]);
            for(int i=t;i!=s;i=prev[i])
            {
                mp[prev[i]][previd[i]].cap-=d;
                mp[i][mp[prev[i]][previd[i]].rev].cap+=d;
            }
            //更新残余网络
            for(int i=1;i<=n;i++)
            {
                if(dis[i]<INF)
                {
                    h[i]+=dis[i];
                }
            }
        }
    }
};
```

用法示例:

```
    int n,m,s,t;
    cin>>n>>m>>s>>t;
    vector<array<int,4>> eds;
    for(int i=0;i<m;i++){
        int u,v,cap,w;
        cin>>u>>v>>cap>>w;
        eds.push_back({u,v,cap,w});
    }
    Mcmf mcmf(n,s,t,eds);
    mcmf.PD();
    cout<<mcmf.maxf<<" "<<mcmf.minc<<endl;
```

3.31 最近公共祖先（LCA）(tarjan,静态)

```
const int MaxN=5e5+5;
vector<int> tree[MaxN];
vector<pair<int,int>> q[MaxN];
```

```
int vis[MaxN],ans[MaxN];
int fa[MaxN];
void prepare_tree(int n)
{
    for(register int i=1;i<=n;i++)
    {
        fa[i]=i;
    }
}
int find(int G)
{
    if(G==fa[G]) return G;
    else
    {
        fa[G]=find(fa[G]);
        return fa[G];
    }
    //return G==fa[G]? G:(fa[G]=find(fa[G]));
}
void merge(int a,int b)//合并
{
    fa[find(a)]=find(b);//有时路径压缩可能破坏rank'(rank->树深)
    /*register int x=find(a),y=find(b);
    Rank[x]<=Rank[y]?fa[x]=y:fa[y]=x;
    if(Rank[x]==Rank[y]&&x!=y) Rank[y]++;*/

}
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
void dfs(int node)
{
    vis[node]=1;
    for(auto child:tree[node])
    {
        if(!vis[child])
        {
            dfs(child);
            fa[child]=node;//调换顺序会使路径压缩到child的父节点，此时子树还没遍历完
        }
    }
    for(auto i:q[node])
    {
        if(vis[i.first])//node及其子树已经dfs完了,如果此时i已经搜到，显然，根据dfs原则，find(i)是lca(i,node)
        {
            ans[i.second]=find(i.first);
        }
    }
}
```

用法示例:

```
    int n=read(),m=read(),s=read();
    for(int i=0;i<n-1;i++)
    {
        int u=read(),v=read();
        tree[v].push_back(u);
        tree[u].push_back(v);
    }
    for(int i=0;i<m;i++)
    {
        int u=read(),v=read();
        q[v].push_back(make_pair(u,i));
        q[u].push_back(make_pair(v,i));
    }
    prepare_tree(n);
    for(int i=1;i<=n;i++)
    {
        vis[i]=0;
    }
```

```
    dfs(s);
    for(int i=0;i<m;i++)
    {
        write(ans[i]);
        putchar('\n');
    }
```

3.32 最近公共祖先（LCA）(倍增)

```
const int MAXN=5e5+5;
const int LOG=25;//MAXN<=2^LOG
vector<int> tree[MAXN];
int dep[MAXN],st[MAXN][LOG];//节点深度，st表，st[i][j]=i的2^j级祖先
int read()
{
    int s=0,f=1;
    char ch=getchar();
    while(ch<'0'||ch>'9')
    {
        if(ch=='-') f=-1;
        ch=getchar();
    }
    while(ch>='0'&&ch<='9')
    {
        s=(s<<3)+(s<<1)+ch-'0';
        ch=getchar();
    }
    return s*f;
}
inline void write(int x)
{
    static int sta[35];
    int top=0;
    if(x<0&&x!=-2147483648) {putchar('-');x=-x;}
    if(x==-2147483648) {printf("-2147483648");return;}
    do{
      sta[top++]=x%10, x/=10;
      }while(x);
    while(top) putchar(sta[--top]+48);
}
void init(int node,int parent)//用dfs预处理dep和st
{
    dep[node]=(parent==-1)?0:dep[parent]+1;
    st[node][0]=parent;//一级祖先为自身
    for(int i=1;i<LOG;i++)//更新node的祖先表
    {
        if(st[node][i-1]!=-1)
        {
            st[node][i]=st[st[node][i-1]][i-1];
            //node的2^j级祖先为node的2^j-1祖先的2^j-1祖先
        }
        else st[node][i]=-1;//你的码的码没了，你还有码？(可删吗？)
    }
    for(auto child:tree[node])
    {
        if(child!=parent)
        {
            init(child,node);//从父节点向下dfs
        }
    }
}
int lca(int u,int v)
{
    if(dep[u]<dep[v]) swap(u,v);//确保u比v深
    int diff=dep[u]-dep[v];
    for(int i=0;i<LOG;i++)
    {
        if((diff>>i)&1)
        {
            u=st[u][i];//u向上跳转2^i,其中i为diff的二进制表示中第i位为一
        }
    }
    if(u==v) return u;//深度相等，可能找到
    //不相等，假设他们与lca(u,v)的距离为diff
    //注意到5=4+1，5-4=1
    //7=4+2+1,7-4-2=1
    //6=4+2,6-4-1=1
    //12=8+4,12-8-2-1
    //做以下操作总能使diff=1
    // for(int i=LOG-1;i>=0;i--)
    // {
    //  if(st[u][i]!=st[v][i])
    //  {
    //      u=st[u][i];
    //      v=st[v][i];
    //  }
    // }
    // return st[u][0];
    //优化版
    for(int i=LOG-1;i>=0;i--)
    {
```

```
        if(st[u][i]!=st[v][i])
        {
            u=st[u][i];
            v=st[v][i];
        }
    }
    return st[u][0];
}
```

用法示例:

```
    int n=read(),m=read(),s=read();//n个点, n-1条边,m个询问, s为根
    for(int i=0;i<n-1;i++)
    {
        int u=read(),v=read();
        tree[u].push_back(v);
        tree[v].push_back(u);//存树
    }
    for(int i=1;i<=n;i++)
    {
        dep[i]=-1;
        for(int j=0;j<LOG;j++)
        {
            st[i][j]=-1;
        }
    }
    init(s,-1);
    while(m--)
    {
        write(lca(read(),read()));
        putchar('\n');
    }
```

3.33 最近公共祖先（LCA）(欧拉序 &dfs 序(O(1)))

```
//O(nlogn)-O(1) lca 通过拍成欧拉序|dfs序然后st表得到
//dfs序常数更小
```

```
class StLca{
    public:
        vector<vector<int>> dp,tr;
        int n,tim;
        vector<int> dep,eul,fir;
        int inf(int a,int b){return dep[a]<dep[b]?a:b;}
        void init(vector<int>& a,int m)
        {
            if(!m) return;
            int len=__lg(m)+1;
            dp.assign(len,vector<int>(m+1));
            for(int i=1;i<=m;i++) dp[0][i]=a[i];
            for(int j=1;j<len;j++)
                for(int i=1;i+(1<<j)-1<=m;i++)
                    dp[j][i]=inf(dp[j-1][i],dp[j-1][i+(1<<(j-1))]);
        }
        int query(int l,int r)
        {
            int k=__lg(r-l+1);
            return inf(dp[k][l],dp[k][r-(1<<k)+1]);
        }
        StLca(int n,const vector<vector<int>>& tr):tr(tr),n(n){
            dep.assign(n+1,0),fir.resize(n+1);
            eul.resize(2*n);tim=0;dfs(1,0);
            init(eul,2*n-1);
        }
        void dfs(int u,int fa){
            tim++,eul[tim]=u,fir[u]=tim;
            dep[u]=dep[fa]+1;
            for(auto v:tr[u]){
                if(v==fa) continue;
                dfs(v,u);
                eul[++tim]=u;
            }
        }
        int lca(int u,int v){
            if(fir[u]>fir[v]) swap(u,v);
            return query(fir[u],fir[v]);
        }
        int dis(int u,int v){
            return dep[u]+dep[v]-2*dep[lca(u,v)];
        }
};
class StLca{
    public:
        vector<vector<int>> dp;
        const vector<vector<int>>& tr;
        int n,tim;
        vector<int> dep,eul,fa,dfn,seq;
        int inf(int a,int b){return dep[a]<dep[b]?a:b;}
        void init(vector<int>& a,int m)
        {
            if(!m) return;
            int len=__lg(m)+1;
            dp.assign(len,vector<int>(m+1));
```

```
            for(int i=1;i<=m;i++) dp[0][i]=a[i];
            for(int j=1;j<len;j++)
                for(int i=1;i+(1<<j)-1<=m;i++)
                    dp[j][i]=inf(dp[j-1][i],dp[j-1][i+(1<<(j-1))]);
        }
        int query(int l,int r)
        {
            int k=__lg(r-l+1);
            return inf(dp[k][l],dp[k][r-(1<<k)+1]);
        }
        StLca(int n,const vector<vector<int>>& tr,int rt):tr(tr),n(n){
            dep.assign(n+1,0),dfn.resize(n+1),seq.resize(n+1);
            fa.resize(n+1);tim=0;dfs(rt,0);
            init(seq,n);
        }
        void dfs(int u,int f){
            dfn[u]=++tim;fa[u]=f;
            dep[u]=dep[f]+1;seq[tim]=u;
            for(auto v:tr[u]){
                if(v==f) continue;
                dfs(v,u);
            }
        }
        int lca(int u,int v){
            if(u==v) return u;
            if(dfn[u]>dfn[v]) swap(u,v);
            return fa[query(dfn[u]+1,dfn[v])];
        }
        int dis(int u,int v){
            return dep[u]+dep[v]-2*dep[lca(u,v)];
        }
};
```

3.34 树上倍增(点.边)

```
class TreeEinf{
    public:
    vector<vector<array<int,2>>> tr;
    vector<vector<int>> fa,inf;
    vector<int> dep;
    int n,k,INF,op;
    int nop(int a,int b){
        if(op==1) return max(a,b);
        else return min(a,b);
    }
    TreeEinf(int n,vector<vector<array<int,2>>> &g,int op):
        tr(g),n(n),k(__lg(n)+1),dep(n+1,0),op(op),
        fa(__lg(n)+2,vector<int>(n+1,0)){
            //inf j点向上2^i步的最值
            if(op==1) INF=-1e9;
            else INF=1e9;
            inf.assign(k+1,vector<int>(n+1,INF));
            dfs(1,0,INF);
    }
    void dfs(int u,int f,int fw){
        dep[u]=dep[f]+1;
        fa[0][u]=f;
        inf[0][u]=fw;
        for(int i=1;i<=k;i++){
            fa[i][u]=fa[i-1][fa[i-1][u]];
            inf[i][u]=nop(inf[i-1][u],inf[i-1][fa[i-1][u]]);
        }
        for(auto [v,w]:tr[u]){
            if(v!=f){
                dfs(v,u,w);
            }
        }
    }
    int q(int u,int v){
        int ans=INF;
        if(dep[u]<dep[v]) swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[i][u]]>=dep[v]){
                ans=nop(ans,inf[i][u]);
                u=fa[i][u];
            }
        }
        if(u==v) return ans;
        for(int i=k;i>=0;i--){
            if(fa[i][u]!=fa[i][v]){
                ans=nop(ans,inf[i][u]);
                ans=nop(ans,inf[i][v]);
                u=fa[i][u],v=fa[i][v];
            }
        }
        ans=nop(ans,inf[0][u]);
        ans=nop(ans,inf[0][v]);
        return ans;
    }
};
class TreeDinf{
    public:
    vector<vector<int>> tr;
```

```
    vector<vector<int>> fa,inf;
    vector<int> dep,val;
    int n,k,INF,op;
    int nop(int a,int b){
        if(op==1) return max(a,b);
        else return min(a,b);
    }
    TreeDinf(int n,vector<vector<int>> &g,
        int op,vector<int> &val):
        tr(g),n(n),k(__lg(n)+1),dep(n+1,0),op(op),
        fa(__lg(n)+2,vector<int>(n+1,0)),val(val){
            //inf j点向上2^i步的最值
            if(op==1) INF=-1e9;
            else INF=1e9;
            inf.assign(k+1,vector<int>(n+1,INF));
            dfs(1,0);
    }
    void dfs(int u,int f){
        dep[u]=dep[f]+1;
        fa[0][u]=f;
        inf[0][u]=(f==0?val[u]:nop(val[f],val[u]));
        for(int i=1;i<=k;i++){
            fa[i][u]=fa[i-1][fa[i-1][u]];
            inf[i][u]=nop(inf[i-1][u],inf[i-1][fa[i-1][u]]);
        }
        for(auto v:tr[u]){
            if(v!=f){
                dfs(v,u);
            }
        }
    }
    int q(int u,int v){
        int ans=INF;
        if(dep[u]<dep[v]) swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[i][u]]>=dep[v]){
                ans=nop(ans,inf[i][u]);
                u=fa[i][u];
            }
        }
        if(u==v) return nop(ans,val[u]);
        for(int i=k;i>=0;i--){
            if(fa[i][u]!=fa[i][v]){
                ans=nop(ans,inf[i][u]);
                ans=nop(ans,inf[i][v]);
                u=fa[i][u],v=fa[i][v];
            }
        }
        ans=nop(ans,inf[0][u]);
        ans=nop(ans,inf[0][v]);
        return ans;
    }
};
```

3.35 树上前缀和(点.边.倍增 lca.ver.)

```
class TreeVsum{
    public:
    vector<vector<int>> fa;
    vector<int> pre,val,dep;//pre是1到u的点权和
    int n,k;

    TreeVsum(int n,vector<vector<int>>& tr,vector<int>& a):
        n(n),k(__lg(n)+1),pre(n+1,0),val(a),
        fa(__lg(n)+2,vector<int>(n+1,0)),dep(n+1,0){
        //a 1-base
        dfs(1,0,tr);
    }
    void dfs(int u,int f,vector<vector<int>>& tr){
        pre[u]=pre[f]+val[u];
        dep[u]=dep[f]+1;
        fa[0][u]=f;
        for(int i=1;i<=k;i++){
            fa[i][u]=fa[i-1][fa[i-1][u]];
        }
        for(auto v:tr[u]){
            if(v==f)continue;
            dfs(v,u,tr);
        }
    }
    int lca(int u,int v){
        if(dep[u]<dep[v])swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[i][u]]>=dep[v])
                u=fa[i][u];
        }
        if(u==v)return u;
        for(int i=k;i>=0;i--){
            if(fa[i][u]!=fa[i][v]){
                u=fa[i][u];
                v=fa[i][v];
            }
        }
```

```
        return fa[0][u];
    }
    int q(int u,int v){
        int lc=lca(u,v);
        return pre[u]+pre[v]-pre[lc]-pre[fa[0][lc]];
    }
};
class TreeEsum{
    public:
    vector<vector<int>> fa;
    vector<int> pre,dep;//pre是1到u的边权和
    int n,k;

    TreeEsum(int n,vector<vector<array<int,2>>>& tr):
        n(n),k(__lg(n)+1),pre(n+1,0),dep(n+1,0),
        fa(__lg(n)+2,vector<int>(n+1,0)){
        dfs(1,0,0,tr);
    }
    void dfs(int u,int f,int w,vector<vector<array<int,2>>>& tr){
        pre[u]=pre[f]+w;
        dep[u]=dep[f]+1;
        fa[0][u]=f;
        for(int i=1;i<=k;i++){
            fa[i][u]=fa[i-1][fa[i-1][u]];
        }
        for(auto [v,w]:tr[u]){
            if(v==f)continue;
            dfs(v,u,w,tr);
        }
    }
    int lca(int u,int v){
        if(dep[u]<dep[v])swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[i][u]]>=dep[v])
                u=fa[i][u];
        }
        if(u==v)return u;
        for(int i=k;i>=0;i--){
            if(fa[i][u]!=fa[i][v]){
                u=fa[i][u];
                v=fa[i][v];
            }
        }
        return fa[0][u];
    }
    int q(int u,int v){
        int lc=lca(u,v);
        return pre[u]+pre[v]-2*pre[lc];
    }
};
```

3.36 树上基本处理(带欧拉序)

```
class tree{
    public:
    vector<vector<int>> tr;
    vector<vector<int>> fa;
    vector<int> dfn,dep,siz,in,out,eul;
    int n,k,tot,tim;
    tree(int n,vector<vector<int>>& tr):
    tr(tr),n(n),k(__lg(n)+1),tim(0),dfn(n+1,0),dep(n+1,0),tot(0),siz(n+1,0),
    fa(n+1,vector<int>(__lg(n)+2,0)),in(n+1,0),out(n+1,0),eul(2*n+1,0){
        dfs(1,0);
    }

    int dfs(int u,int f){
        dfn[u]=++tot,in[u]=++tim;eul[tim]=u;
        dep[u]=dep[f]+1;
        fa[u][0]=f;siz[u]=1;
        for(int i=1;i<=k;i++){
            fa[u][i]=fa[fa[u][i-1]][i-1];
        }
        for(auto v:tr[u]){
            if(v==f) continue;
            siz[u]+=dfs(v,u);
        }
        out[u]=++tim,eul[tim]=u;
        return siz[u];
    }
    //求u,v的最近公共祖先
    int lca(int u,int v){
        if(dep[u]<dep[v]) swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[u][i]]>=dep[v]) u=fa[u][i];
        }
        if(u==v) return u;
        for(int i=k;i>=0;i--){
            if(fa[u][i]!=fa[v][i]){
                u=fa[u][i];
                v=fa[v][i];
            }
        }
        return fa[u][0];
```

```
    }
    //求u,v距离
    int dis(int u,int v){
        return dep[u]+dep[v]-2*dep[lca(u,v)];
    }
    //判断x是否在s,t的路径上
    bool con(int s,int t,int x){
        return dis(s,x)+dis(t,x)==dis(s,t);
    }
};
```

用法示例:

```
    int n,q;cin>>n>>q;
    vector<int> a(n+1),vis(n+1,0),g(n+5,0),cnt(n+1,0);
    for(int i=1;i<=n;i++) cin>>a[i];
    vector<vector<int>> tr(n+1);
    for(int i=1;i<n;i++){
        int u,v;cin>>u>>v;
        tr[u].push_back(v);
        tr[v].push_back(u);
    }
    tree t(n,tr);
    vector<array<int,6>> qry(q);
    vector<int> ans(q);
    for(int i=0;i<q;i++){
        int u,v,qa,qb;cin>>u>>v>>qa>>qb;
        if(t.in[u]>t.in[v]) swap(u,v);
        if(t.lca(u,v)==u){
            qry[i]={t.in[u],t.in[v],qa,qb,0,i};
        }
        else{
            qry[i]={t.in[u]+1,t.in[v],qa,qb,t.lca(u,v),i};
        }
    }
    const int B=2*n/static_cast<int>(sqrt(q));
    auto cmp=[&](array<int,6> a,array<int,6> b){
        if(a[0]/B!=b[0]/B) return a[0]/B<b[0]/B;
        else{
            if((a[0]/B)&1) return a[1]>b[1];
            else return a[1]<b[1];
        }
    };
    sort(qry.begin(),qry.end(),cmp);
    int l=1,r=0;
    auto f=[&](int x){
        int c=a[x];
        if(vis[x]){
            g[cnt[c]]--;
            cnt[c]--;
        }
        else{
            cnt[c]++;
            g[cnt[c]]++;
        }
        vis[x]^=1;
    };
    for(auto [ql,qr,qa,qb,lca,i]:qry){
        while(r<qr) f(t.eul[++r]);
        while(l>ql) f(t.eul[--l]);
        while(r>qr) f(t.eul[r--]);
        while(l<ql) f(t.eul[l++]);
        if(lca) f(lca);
        ans[i]=g[qa]-g[qb+1];
        if(lca) f(lca);
    }
    for(int i=0;i<q;i++) cout<<ans[i]<<'\n';
```

3.37 树上基本处理

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class tree{
    public:
    vector<vector<int>> tr;
    vector<vector<int>> fa;
    vector<int> dfn,dep,siz;
    int n,k,tot;
    tree(int n,vector<vector<int>>& tr):
    tr(tr),n(n),k(__lg(n)+1),dfn(n+1,0),dep(n+1,0),tot(0),siz(n+1,0),
    fa(n+1,vector<int>(__lg(n)+2,0)){
        dfs(1,0);
    }

    int dfs(int u,int f){
        dfn[u]=++tot;
        dep[u]=dep[f]+1;
        fa[u][0]=f;siz[u]=1;
        for(int i=1;i<=k;i++){
            fa[u][i]=fa[fa[u][i-1]][i-1];
        }
        for(auto v:tr[u]){
            if(v==f) continue;
```

```
            siz[u]+=dfs(v,u);
        }
        return siz[u];
    }
    //求u,v的最近公共祖先
    int lca(int u,int v){
        if(dep[u]<dep[v]) swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[u][i]]>=dep[v]) u=fa[u][i];
        }
        if(u==v) return u;
        for(int i=k;i>=0;i--){
            if(fa[u][i]!=fa[v][i]){
                u=fa[u][i];
                v=fa[v][i];
            }
        }
        return fa[u][0];
    }
    //求u,v距离
    int dis(int u,int v){
        return dep[u]+dep[v]-2*dep[lca(u,v)];
    }
    //判断x是否在s,t的路径上
    bool con(int s,int t,int x){
        return dis(s,x)+dis(t,x)==dis(s,t);
    }
};
```

3.38 树上差分(点,边.树剖 lca.ver.)

```
class TreeDiff{
    public:
    vector<int> cnt,dep,fa;
    vector<int> siz,hson,top;
    vector<vector<int>> tr;
    //cnt在点差分的时候代表点上的数值
    //cnt在边差分的时候代表该点和父亲连边上的数值
    int n;

    TreeDiff(int n,vector<vector<int>>& tr):
        n(n),cnt(n+1,0),dep(n+1,0),siz(n+1,0),
        hson(n+1,-1),top(n+1,-1),fa(n+1,0),tr(tr){
        //a 默认根为1
        dfs1(1,0,tr);
        top[1]=1;
        dfs2(1,tr);
    }
    void dfs1(int u,int f,vector<vector<int>>& tr){
        dep[u]=dep[f]+1;
        fa[u]=f;siz[u]=1;
        for(auto v:tr[u]){
            if(v==f)continue;
            dfs1(v,u,tr);
            siz[u]+=siz[v];
            if(hson[u]==-1||siz[hson[u]]<siz[v])
                hson[u]=v;
        }
    }
    void dfs2(int u,vector<vector<int>>& tr){
        if(hson[u]!=-1){
            top[hson[u]]=top[u];
            dfs2(hson[u],tr);
        }
        for(auto v:tr[u]){
            if(v==fa[u]||v==hson[u])
                continue;
            top[v]=v;
            dfs2(v,tr);
        }
    }
    int lca(int u,int v){
        while(top[u]!=top[v]){
            if(dep[top[u]]<dep[top[v]])
                swap(u,v);
            u=fa[top[u]];
        }
        return dep[u]<dep[v]?u:v;
    }
    void Dadd(int u,int v,int w){
        cnt[u]+=w,cnt[v]+=w;
        cnt[lca(u,v)]-=w;
        if(fa[lca(u,v)]!=0)
            cnt[fa[lca(u,v)]]-=w;
    }
    void Eadd(int u,int v,int w){
        cnt[u]+=w,cnt[v]+=w;
        cnt[lca(u,v)]-=2*w;
    }
    void q(int u,int f){
        for(auto v:tr[u]){
            if(v==f)continue;
```

```
            q(v,u);
            cnt[u]+=cnt[v];
        }
    }
    vector<int> dq() {return cnt;}
    vector<array<int,4>> eq(){
        vector<array<int,4>> res;
        auto dfs=[&](auto self,int u,int f,int w)->void{
            for(auto v:tr[u]){
                if(v==f)continue;
                self(self,v,u,w);
            }
            res.push_back({u,f,w,cnt[u]});
        };
        dfs(dfs,1,0,0);
        return res;
    }
};
```

3.39 树链剖分（线段树 ver.）

```
#define int long long
const int N=1e5+5;
int dep[N],fa[N],hson[N],top[N],siz[N],dfn[N],rk[N],val[N];
int mod;
class SegTree{
    public:
        struct Node
        {
            int sum;
            int s,e;
            int lazy=0;
            Node* lt;
            Node* rt;
            Node(int sum,int s,int e):s(s),e(e),sum(sum),lt(nullptr),rt(nullptr){}
        };
        Node* root;
        Node* buildtree(vector<int> &nums,int l,int r)
        {
            if(l>r) return nullptr;
            if(l==r) return new Node(nums[l],l,l);
            int mid=(l+r)>>1;
            Node* root=new Node(0,l,r);
            Node* lc=buildtree(nums,l,mid);
            Node* rc=buildtree(nums,mid+1,r);
            if(lc) root->lt=lc,root->sum=(root->sum+lc->sum)%mod;
            if(rc) root->rt=rc,root->sum=(root->sum+rc->sum)%mod;
            return root;
        }
        void init(vector<int>nums)
        {
            root=buildtree(nums,0,nums.size()-1);
            return;
        }
        void taglazy(Node* root,int val)
        {
            if(root==nullptr) return;
            val%=mod;
            root->lazy=(root->lazy+val)%mod;
            root->sum=(root->sum+(root->e-root->s+1)%mod*val)%mod;
        }
        void pushdown(Node* root)
        {
            if(!root) return ;
            if(root->lazy)
            {
                taglazy(root->lt,root->lazy);
                taglazy(root->rt,root->lazy);
                root->lazy=0;
            }
        }
        void update(Node* root,int l,int r,int val)
        {
            if(!root) return ;
            if(root->s>r||root->e<l) return ;
            if(root->s>=l&&root->e<=r)
            {
                taglazy(root,val);
                return;
            }
            pushdown(root);
            update(root->lt,l,r,val);
            update(root->rt,l,r,val);
            root->sum=((root->lt?root->lt->sum:0)+(root->rt?root->rt->sum:0))%mod;
            return ;
        }
        void update(int l,int r,int val)
        {
            update(root,l,r,val);
            return ;
        }
        int query(Node* root,int l,int r)
        {
            pushdown(root);
```

```
            if(!root) return 0;
            if(root->s>r||root->e<l) return 0;
            if(root->s>=l&&root->e<=r) return root->sum;
            return query(root->lt,l,r)+query(root->rt,l,r);
        }
        int query(int l,int r)
        {
            return query(root,l,r);
        }
};
class cutTree
{
    //树链剖分，把树剖分成若干条链，每条链上维护一个线段树
    //可以支持链上修改和查询，也可以支持树上修改和查询
    //还可以求lca
    //重链剖分有一个重要的性质：对于节点数为n的树，从任意节点向上走到根节点，经过的轻边数量不超过logn。这是因为，如果一个节点连向父节点的边是轻边，
    //就必然存在子树不小于它的兄弟节点，那么父节点对应子树的大小一定超过该节点的两倍(由dfs1可得)。每经过一条轻边，子树大小就翻倍，所以最多只能经过logn条。
    public:
        int n,tot,s;
        //s:根节点
        vector<vector<int>> tree;
        //dep:树深,fa:父节点,hson:i的重儿子,top:重链顶端,siz:子树大小,dfn:dfs序,rk:dfs序对应的节点
        SegTree seg;
        void dfs1(int u,int f)
        {
            //cntt++;cout<<cntt<<endl;
            dep[u]=dep[f]+1;//更新树深
            fa[u]=f;siz[u]=1;
            for(auto v:tree[u])
            {
                if(v==f)continue;
                dfs1(v,u);
                siz[u]+=siz[v];
                if(hson[u]==-1||siz[v]>siz[hson[u]]) hson[u]=v;
                //u的重儿子是所有子树大小最大的儿子
            }
        }
        void dfs2(int u)
        {
            dfn[u]=++tot;rk[tot]=u;
            //优先访问重儿子,保证重链顶端的dfn最小
            if(hson[u]!=-1)
            {
                top[hson[u]]=top[u];
                //重儿子的top是它所在重链的顶端
                dfs2(hson[u]);
            }
            for(auto v:tree[u])
            {
                if(v==fa[u]||v==hson[u])//跳过父节点和重儿子
                    continue;
                top[v]=v;//轻儿子的top是自己
                dfs2(v);
            }
        }
        void init()
        {
            tot=0;
            dfs1(s,0);
            dfs2(s);
        }
        int lca(int u,int v)
        {
            while(top[u]!=top[v])//不在同一条重链上
            {
                if(dep[top[u]]<dep[top[v]])swap(u,v);
                u=fa[top[u]];
                //链头深度大的往上跳
            }
            return dep[u]<dep[v]?u:v;
        }
        int queryPath(int u,int v)
        {
            int ans=0;
            while(top[u]!=top[v])//遍历所有的边
            {
                if(dep[top[u]]<dep[top[v]])swap(u,v);
                ans=(ans+seg.query(dfn[top[u]],dfn[u]))%mod;
                u=fa[top[u]];
            }
            if(dep[u]>dep[v])swap(u,v);
            ans=(ans+seg.query(dfn[u],dfn[v]))%mod;
            return ans;
        }
        void updatePath(int u,int v,int val)
        {
            while(top[u]!=top[v])
            {
                if(dep[top[u]]<dep[top[v]])swap(u,v);
```

```
                seg.update(dfn[top[u]],dfn[u],val);
                u=fa[top[u]];
            }
            if(dep[u]>dep[v])swap(u,v);
            seg.update(dfn[u],dfn[v],val);
        }
        void updateSub(int u,int val)
        {
            //子树的dfn一定是连续的
            seg.update(dfn[u],dfn[u]+siz[u]-1,val);
        }
        int querySub(int u)
        {
            return seg.query(dfn[u],dfn[u]+siz[u]-1);
        }
    cutTree(int n,vector<vector<int>> tree,int s):n(n),tree(tree),s(s)
    {
        for(int i=0;i<=n;i++)
        {
            dep[i]=0;fa[i]=-1;hson[i]=-1;top[i]=-1;
            siz[i]=0;dfn[i]=-1;rk[i]=-1;
        }
        top[s]=s;init(); vector<int> inf(n+1,0);
        for(int i=1;i<=n;i++)inf[dfn[i]]=val[i]%mod;
        seg.init(inf);
    }
};
```

用法示例:

```
    cin.tie(0);
    int n,m,s;
    cin>>n>>m>>s>>mod;
    vector<vector<int>> tree(n+1);
    for(int i=1;i<=n;i++)
    {
        cin>>val[i];
    }
    for(int i=1;i<n;i++)
    {
        int u,v;
        cin>>u>>v;
        tree[u].push_back(v);
        tree[v].push_back(u);
    }
    cutTree ct(n,tree,s);
    while(m--)
    {
        int op,x,y,z;
        cin>>op;
        if(op==1)
        {
            cin>>x>>y>>z;
            ct.updatePath(x,y,z);
        }
        else if(op==2)
        {
            cin>>x>>y;
            cout<<ct.queryPath(x,y)%mod<<endl;
        }
        else if(op==3)
        {
            cin>>x>>y;
            ct.updateSub(x,y);
        }
        else if(op==4)
        {
            cin>>x;
            cout<<ct.querySub(x)%mod<<endl;
        }
    }
    int T_end=clock();
    //cout<<"time: "<<(double)(T_end-T_start)/CLOCKS_PER_SEC<<"s"<<endl;
```

3.40 欧拉回路(路径,Hierholzer 法)

```
//有向图的情况,注意一下此处假设图的基图全联通，未考虑孤立点
//即此处只考虑处理一个联通分量的情况
class HierholzerD{
    public:
        vector<vector<int>> mp;
        vector<int> in,out,del;
        int n;
        HierholzerD(vector<vector<int>> &mp,int n):
        mp(mp),n(n),in(n+1,0),out(n+1,0),del(n+1,0){
            for(int i=1;i<=n;i++){
                for(auto &j:mp[i]){
                    out[i]++;
                    in[j]++;
                }
            }
        }
        vector<int> get(){
```

```
            int s=-1,t=-1,cnts=0,cntt=0;
            for(int i=1;i<=n;i++){
                if(out[i]==in[i]) continue;
                if(out[i]-in[i]==1) s=i,cnts++;
                else if(in[i]-out[i]==1) t=i,cntt++;
                else return {};
            }
            if(!((cnts==0&&cntt==0)||(cnts==1&&cntt==1))) return {};
            if(s==-1) s=1;
            vector<int> res;
            auto dfs=[&](auto&& dfs,int u)->void{
                for(int &i=del[u];i<mp[u].size();){
                    int v=mp[u][i++];
                    dfs(dfs,v);
                }
                res.push_back(u);
            };
            dfs(dfs,s);
            reverse(res.begin(),res.end());
            return res;
        }
        //返回欧拉回路/欧拉路径
};
//无向图的情况,注意一下此处假设图的基图全联通, 未考虑孤立点
//即此处只考虑处理一个联通分量的情况
//注意存双向边的时候存下编号, eg.u->v 编号为0, v->u编号为1
class HierholzerN{
    public:
        vector<vector<array<int,2>>> mp;
        vector<int> deg,del,vis;
        int n,m;
        HierholzerN(vector<vector<array<int,2>>> &mp,int n,int m):
        mp(mp),n(n),deg(n+1,0),del(n+1,0),vis(2*m+5){
            for(int i=1;i<=n;i++){
                deg[i]=mp[i].size();
            }
        }
        vector<int> get(){
            int s=-1,odd=0,even=0;
            for(int i=1;i<=n;i++){
                if(deg[i]&1) odd++,s=i;
                else even++;
            }
            if(!(odd==0||odd==2)) return {};
            if(s==-1) s=1;
            vector<int> res;
            auto dfs=[&](auto&& dfs,int u)->void{
                for(int &i=del[u];i<mp[u].size();){
                    auto [v,id]=mp[u][i++];
                    if(vis[id]) continue;
                    vis[id]=vis[id^1]=1;
                    dfs(dfs,v);
                }
                res.push_back(u);
            };
            dfs(dfs,s);
            reverse(res.begin(),res.end());
            return res;
        }
        //返回欧拉回路/欧拉路径
};
```

3.41 点分树

```
//点分树, 处理一类树上带修路径问题
//把点分治的分治重心连成一条树, 这样就可以用点分治的思路处理树上带修路径问题了
//如果在点分树上, $x$ 是 $u$ 和 $v$ 的最近公共祖先 (LCA), 那么 $x$ 一定在原树 $u$ 到 $v$ 的路径上。
//换句话说 在点分树上以x子树的不同分支的点对 一一对应 以x为最早分治中心的那些路径
//换句话说 点分树 把所有路径做了如下划分: 按路径上作为分治中心最早的那个点进行分类
//换句话说 点分树 从u向上跳到根 等价于 访问了所有以u为端点的路径
//注意: 在点分树上, 当你从 $u$ 跳到祖先 $x$, 并利用 $x$ 的信息去计算 $u$ 到其他分支的路径时,
//你会多算“和 $u$ 在 $x$ 的同一个子树分支里的点”, 可以容斥掉或者用答案不当规避掉
```

```
class DivTree{
public:
    vector<vector<int>> tr,ntr;
    vector<int> sz,vis,fa;
    int n,frt;
    DivTree(int n,const vector<vector<int>>& tr):
    n(n),sz(n+1,0),vis(n+1,0),fa(n+1,0),tr(tr),ntr(n+1){
        build();
    }
    int getsz(int u,int p){
        sz[u]=1;
        for(auto v:tr[u]){
            if(v==p||vis[v]) continue;
            sz[u]+=getsz(v,u);
        }
        return sz[u];
    }
```

```
    int getrt(int u,int p,int szrt){
        for(auto v:tr[u]){
            if(v==p||vis[v]) continue;
            if(sz[v]>szrt/2) return getrt(v,u,szrt);
        }
        return u;
    }
    void div(int u){
        vis[u]=1;
        for(auto v:tr[u]){
            if(vis[v]) continue;
            getsz(v,0);
            int subrt=getrt(v,0,sz[v]);
            fa[subrt]=u;
            ntr[u].push_back(subrt);
            ntr[subrt].push_back(u);
            div(subrt);
        }
    }
    void build(){
        getsz(1,0);
        frt=getrt(1,0,sz[1]);
        div(frt);
    }
};
```

3.42 点分治

```
//淀粉质：把树上路径问题转化为子树分治问题
//把树按重心划分，那么树高（或树的大小）不超过n/2，递归深度不超过logn(最坏：退化为链)，于是可以暴力处理子树
//根据实现方式的不同，时间复杂度可以做到O(nlogn)或O(nlog^2n)
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<vector<pair<int,int>>> tr(n+1);
    for(int i=1;i<n;i++)
    {
        int u,v,w;
        cin>>u>>v>>w;
        tr[u].push_back({v,w});
        tr[v].push_back({u,w});
    }
    vector<int> siz(n+1,0),q(m+1),vis(n+1,0),ans(m+1,0);
    for(int i=1;i<=m;i++) cin>>q[i];
    auto getsz=[&](auto getsz,int u,int p=-1)->int
    {
        siz[u]=1;
        for(auto [v,w]:tr[u])
        {
            if(v==p||vis[v])continue;
            siz[u]+=getsz(getsz,v,u);
        }
        return siz[u];
    };//统计以u为根的子树大小
    auto getrt=[&](auto getrt,int u,int p=-1,int sizrt)->int
    {
        for(auto [v,w]:tr[u])
        {
            if(v==p||vis[v])continue;
            if(siz[v]>sizrt/2)return getrt(getrt,v,u,sizrt);
        }
        return u;
    };//寻找重心
    //重心：对于一棵树，如果存在一个顶点，其子树中最大的子树大小不超过整棵树大小的一半，则称该顶点为这棵树的重心。
    auto clac=[&](auto clac,int uu,int dis,int p=-1,vector<int>& tpd)->void
    {
        tpd.push_back(dis);
        for(auto [vv,ww]:tr[uu])
        {
            if(vv==p||vis[vv])continue;
            clac(clac,vv,dis+ww,uu,tpd);
        }
    };
    auto div=[&](auto div,int u)->void{
        vis[u]=1;
        unordered_set<int> s{0};
        for(auto [v,w]:tr[u])
        {
            if(vis[v])continue;
            vector<int> tpd;
            clac(clac,v,w,u,tpd);
            for(auto d:tpd)
            {
                for(int i=1;i<=m;i++)
                {
                    if(!ans[i]&&d<=q[i]&&s.find(q[i]-d)!=s.end())
                    {
                        ans[i]=1;
                    }
                }
            }
            for(auto d:tpd)s.insert(d);
```

```
        }
        for(auto [v,w]:tr[u])
        {
            //用重心划分u的子树
            if(vis[v])continue;
            getsz(getsz,v);
            int subrt=getrt(getrt,v,-1,siz[v]);
            div(div,subrt);
        }
    };//处理以u为根的子树
    getsz(getsz,1);
    int rt=getrt(getrt,1,-1,siz[1]);
    div(div,rt);
    for(int i=1;i<=m;i++)
    {
        if(ans[i])cout<<"AYE\n";
        else cout<<"NAY\n";
    }
```

3.43 线段树优化建图

```
//线段树优化建图：边数爆炸的时候可以用
//要求：存在点->区间/区间->点/区间->区间的关系
```

```
class SegGraph{
public:
    struct node
    {
        int lc,rc;
    };
    int n,m;
    int rtin,rtout;

    vector<node> tr;
    vector<vector<int>> adj;
    vector<int> deg,val;
    int nw(){
        tr.push_back({0,0});
        adj.push_back({});
        deg.push_back(0);
        return tr.size()-1;
    }
    void add(int u,int v){
        adj[u].push_back(v);
        deg[v]++;
    }
    int build(int l,int r,int typ){
        int u=nw();
        if(l==r) return u;
        int mid=(l+r)>>1;
        tr[u].lc=build(l,mid,typ);
        tr[u].rc=build(mid+1,r,typ);
        if(!typ) add(u,tr[u].lc),add(u,tr[u].rc); //向下的树
        else add(tr[u].lc,u),add(tr[u].rc,u); //向上的树
        return u;
    }
    SegGraph(int n,int m):n(n),m(m){
        //根据实际情况计算：tr,deg,adj:点数
        tr.reserve(4*m+n+5);
        adj.reserve(4*m+n+5);
        deg.reserve(4*m+n+5);
        for(int i=0;i<=n;i++) nw(); //原始点 1-based 使用
        rtin=build(1,m,0); //向下的树
        rtout=build(1,m,1); //向上的树
        //初始化完了只初始化了两棵线段树
    };
    //区间分解
    void qry(int u,int l,int r,int ql,int qr,int tar,int typ){
        if(ql<=l&&r<=qr){
            if(!typ) add(tar,u); //向下树连边
            else add(u,tar);
            return;
        }
        int mid=(l+r)>>1;
        if(ql<=mid) qry(tr[u].lc,l,mid,ql,qr,tar,typ);
        if(mid<qr) qry(tr[u].rc,mid+1,r,ql,qr,tar,typ);
    }
    //找叶子
    int getleaf(int u,int l,int r,int rk){
        if(l==r) return u;
        int mid=(l+r)>>1;
        if(rk<=mid) return getleaf(tr[u].lc,l,mid,rk);
        else return getleaf(tr[u].rc,mid+1,r,rk);
    }
    //下树叶子->图的原始点
    void set1(int u,int p){
        add(getleaf(rtin,1,m,p),u);
    }
    //图的原始点->上树叶子
    void set2(int u,int p){
```

```
        add(u,getleaf(rtout,1,m,p));
    }
    //点向[l,r]连边
    void dot_range(int u,int l,int r){
        qry(rtin,1,m,l,r,u,0);
    }
    //[l,r]向点连边
    void range_dot(int u,int l,int r){
        qry(rtout,1,m,l,r,u,1);
    }
    //[l1,r1]向[l2,r2]连边
    void range_range(int l1,int r1,int l2,int r2){
        int vt=nw();
        range_dot(vt,l1,r1);
        dot_range(vt,l2,r2);
    }
};
```

用法示例:

```
    int n;cin>>n;
    vector<int> s(n+1),t(n+1),p;
    for(int i=1;i<=n;i++) {
        cin>>s[i]>>t[i];
        p.push_back(s[i]);
        p.push_back(t[i]);
    }
    sort(p.begin(),p.end());
    p.erase(unique(p.begin(),p.end()),p.end());
    auto getid=[&](int x){
        return lower_bound(p.begin(),p.end(),x)-p.begin()+1;
    };
    SegGraph sg(n,2*n);
    for(int i=1;i<=n;i++){
        sg.set2(i,getid(s[i]));
        sg.set1(i,getid(t[i]));
    }
    for(int i=1;i<=n;i++){
        int l=getid(min(s[i],t[i])),r=getid(max(s[i],t[i]));
        if(l+1<=r-1){
            sg.dot_range(i,l+1,r-1);
            sg.range_dot(i,l+1,r-1);
        }
    }
    queue<int> q;
    vector<int> ans;
    for(int i=1;i<sg.deg.size();i++){
        if(sg.deg[i]==0) q.push(i);
    }
    while(!q.empty()){
        int u=q.front();q.pop();
        if(u<=n&&u>0) ans.push_back(u);
        for(int v:sg.adj[u]){
            sg.deg[v]--;
            if(sg.deg[v]==0) q.push(v);
        }
    }
    if(ans.size()!=n) cout<<"No\n";
    else{
        cout<<"Yes\n";
        for(int i=0;i<n;i++) cout<<ans[i]<<" \n"[i==n-1];
    }
```

3.44 虚树(可拓展版)

```
//虚树，处理q次询问，每次询问给出k个关键点，答案只跟关键点有关的问题
//构建虚树：
//1.将关键点按dfn排序
//2.相邻的关键点的lca加入虚树
//why?虚树的定义是关键点的集合和其两两lca构成的树
//考虑任意两个关键点，其lca一定把这两个关键点分成两个分支，
//而根据dfn的连续性，一定有某个相邻的关键点的lca是这个lca
//Q.E.D
//3.按父子关系构建虚树
```

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class vtree{
    public:
    vector<vector<int>> tr,vtr;
    vector<vector<int>> fa;
    vector<int> dfn,dep;
    int n,k,tot;
    vtree(int n,vector<vector<int>>& tr):
    tr(tr),vtr(n+1),n(n),k(__lg(n)+1),dfn(n+1,0),dep(n+1,0),tot(0),
    fa(n+1,vector<int>(__lg(n)+2,0)){
        dfs(1,0);
    }

    void dfs(int u,int f){
        dfn[u]=++tot;
```

```
        dep[u]=dep[f]+1;
        fa[u][0]=f;
        for(int i=1;i<=k;i++){
            fa[u][i]=fa[fa[u][i-1]][i-1];
        }
        for(auto v:tr[u]){
            if(v==f) continue;
            dfs(v,u);
        }
    }

    int lca(int u,int v){
        if(dep[u]<dep[v]) swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[u][i]]>=dep[v]) u=fa[u][i];
        }
        if(u==v) return u;
        for(int i=k;i>=0;i--){
            if(fa[u][i]!=fa[v][i]){
                u=fa[u][i];
                v=fa[v][i];
            }
        }
        return fa[u][0];
    }

    void getvTree(vector<int>& o){
        sort(o.begin(),o.end(),[&](int a,int b){return dfn[a]<dfn[b];});
        int p=o.size();
        for(int i=1;i<p;i++){
            o.push_back(lca(o[i-1],o[i]));
        }
        sort(o.begin(),o.end(),[&](int a,int b){return dfn[a]<dfn[b];});
        o.erase(unique(o.begin(),o.end()),o.end());
        for(int i=1;i<o.size();i++){
            int tp=lca(o[i-1],o[i]);
            vtr[tp].push_back({o[i]});
            //vtr[o[i]].push_back({tp});
        }
    }
    void clear(vector<int>& o){
        for(auto x:o) vtr[x].clear();
    }
};
```

用法示例:

```
    int t;cin>>t;
    while(t--){
        int n,k;cin>>n>>k;
        vector<vector<int>> tr(n+1),id(k+1);
        vector<int> c(n+1),w(n+1),cnt(n+1,0);
        //c[0]=1;
        for(int i=1;i<=n;i++) cin>>w[i];
        for(int i=1;i<=n;i++) cin>>c[i],id[c[i]].push_back(i);
        for(int i=1;i<n;i++){
            int u,v;cin>>u>>v;
            tr[u].push_back(v);
            tr[v].push_back(u);
        }
        vtree vt(n,tr);
        for(int i=1;i<=k;i++){
            vt.getvTree(id[i]);
            for(auto x:id[i]){
                cnt[x]+=1;
                if(c[x]==0)
                {
                    c[x]=i;
                }
            }
            vt.clear(id[i]);
        }
        int ans=0;
        for(int i=1;i<=n;i++){
            if(cnt[i]>=2) ans+=w[i];
        }
        cout<<ans<<endl;
        auto dfs=[&](this auto& dfs,int u,int f)->void{
            for(auto v:tr[u]){
                if(v==f) continue;
                if(c[v]==0&&c[u]!=0) c[v]=c[u];
                dfs(v,u);
                if(c[u]==0&&c[v]!=0) c[u]=c[v];
            }
            if(c[u]==0) c[u]=1;
        };
        dfs(1,0);
        for(int i=1;i<=n;i++){
            cout<<c[i]<<" ";
        }
        cout<<endl;
    }
```

3.45 虚树(带边权)

```
//虚树，处理q次询问，每次询问给出k个关键点，答案只跟关键点有关的问题
//构建虚树：
//1.将关键点按dfn排序
//2.相邻的关键点的lca加入虚树
//why?虚树的定义是关键点的集合和其两两lca构成的树
//考虑任意两个关键点，其lca一定把这两个关键点分成两个分支，
//而根据dfn的连续性，一定有某个相邻的关键点的lca是这个lca
//Q.E.D
//3.按父子关系构建虚树
//时间复杂度：O(nlogn)
```

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class vtree{
    public:
    vector<vector<array<int,2>>> tr,vtr;
    vector<vector<int>> len,fa;
    vector<int> dfn,dep;
    int n,k,tot;
    vtree(int n,vector<vector<array<int,2>>>& tr):
    tr(tr),vtr(n+1),n(n),k(__lg(n)+1),dfn(n+1,0),dep(n+1,0),tot(0),
    fa(n+1,vector<int>(__lg(n)+2,0)),len(n+1,vector<int>(__lg(n)+2,1e18)){
        dfs(1,0);
    }

    void dfs(int u,int f){
        dfn[u]=++tot;
        dep[u]=dep[f]+1;
        fa[u][0]=f;
        for(int i=1;i<=k;i++){
            fa[u][i]=fa[fa[u][i-1]][i-1];
            len[u][i]=min(len[u][i-1],len[fa[u][i-1]][i-1]);

        }
        for(auto [v,w]:tr[u]){
            if(v==f) continue;
            len[v][0]=w;
            dfs(v,u);
        }
    }

    int lca(int u,int v){
        if(dep[u]<dep[v]) swap(u,v);
        for(int i=k;i>=0;i--){
            if(dep[fa[u][i]]>=dep[v]) u=fa[u][i];
        }
        if(u==v) return u;
        for(int i=k;i>=0;i--){
            if(fa[u][i]!=fa[v][i]){
                u=fa[u][i];
                v=fa[v][i];
            }
        }
        return fa[u][0];
    }

    int w(int u,int v){
        if(dep[u]<dep[v]) swap(u,v);
        int ans=1e18;
        for(int i=k;i>=0;i--){
            if(dep[fa[u][i]]>=dep[v]){
                ans=min(ans,len[u][i]);
                u=fa[u][i];
            }
        }
        return ans;
    }//注意此处仅查询u到v的不跨lca的最小边权
    void getvTree(vector<int>& o){
        sort(o.begin(),o.end(),[&](int a,int b){return dfn[a]<dfn[b];});
        int p=o.size();
        for(int i=1;i<p;i++){
            o.push_back(lca(o[i-1],o[i]));
        }
        sort(o.begin(),o.end(),[&](int a,int b){return dfn[a]<dfn[b];});
        o.erase(unique(o.begin(),o.end()),o.end());
        for(int i=1;i<o.size();i++){
            int tp=lca(o[i-1],o[i]);
            vtr[tp].push_back({o[i],w(o[i],tp)});
            //vtr[o[i]].push_back({tp,w(o[i],tp)});
        }
    }
    void clear(vector<int>& o){
        for(auto x:o) vtr[x].clear();
    }
};
```

用法示例:

```
    int n;cin>>n;
    vector<vector<array<int,2>>> tr(n+1);
```

```
    for(int i=1;i<n;i++){
        int u,v,w;cin>>u>>v>>w;
        tr[u].push_back({v,w});
        tr[v].push_back({u,w});
    }
    vtree vt(n,tr);
    vector<bool> iskey(n+1,0);
    int q;cin>>q;
    while(q--){
        vector<int> o,co;
        int k;cin>>k;
        for(int i=0;i<k;i++){
            int x;cin>>x;
            o.push_back(x);
            iskey[x]=1;
        }
        co=o;
        vt.getvTree(o);
        auto dp=[&](this auto& self,int u)->int{
            if(iskey[u]){
                return vt.len[u][0];
            }
            int ans=0;
            for(auto [v,w]:vt.vtr[u]){
                ans += min(self(v), w);
            }
            return ans;
        };
        int fin=dp(o[0]);
        if(o[0]==1)  cout<<fin<<endl;
        else cout<<min(fin,vt.w(o[0],1))<<endl;
        vt.clear(o);
        for(auto x:co) iskey[x]=0;
    }
```

4 字符串

4.1 AC 自动机(dp 版)

```
//AC自动机 处理多模式串匹配问题
//具体来说 就是给定多个模式串和一个文本串
//求文本串中有多少个模式串的出现
//AC自动机是Trie树和KMP的结合
//时间复杂度构建：O(n+26n) n是模式串总长
//时间复杂度匹配：O(m) m是文本串长度
//时间复杂度查询：O(m*p) m是文本串长度 p是一个串的后缀串的数量
//-->查询可以通过fail树上做树dp/拓扑排序达到O(n+m)的时间复杂度
//ac自动机的结构其实就是一个 trans 函数，而构建好这个函数后，在匹配字符串的过程中，我们会舍弃部分前缀达到最低限度的匹配。
//本质上就是一个状态，接受一个输入，转移到另一个状态，
//注意到fail链构成的图是一个DAG，所以fail链的长度是O(n)的，所以fail指针的构建是O(n)的。
```

```
class AC{
    public:
    vector<vector<int>> ch;
    int n,tot,pidx; //节点数>=模式串总长
    vector<int> ans,ne,idx,deg,sidx,fin;
    //idx:节点的新编号 sidx:原字符串对应的编号 fin:最终答案
    AC(int n):n(n),ch(n+1,vector<int>(26,0)),
    ans(n+1,0),ne(n+1,0),tot(0),pidx(0),
    idx(n+1,0),deg(n+1,0),sidx(n+1,0),fin(n+1,0){}
    void insert(string s,int i){
        int p=0;
        for(auto c:s){
            if(!ch[p][c-'a']) ch[p][c-'a']=++tot;
            p=ch[p][c-'a'];
        }
        if(!idx[p]) idx[p]=++pidx;
        sidx[i]=idx[p];
    }
    //构建tire
    void build(){
        queue<int> q;
        for(int i=0;i<26;i++){
            if(ch[0][i]) q.push(ch[0][i]);
        }
        while(!q.empty()){
            int u=q.front();q.pop();
            for(int i=0;i<26;i++){
                int v=ch[u][i];
                if(v)
                {
                    ne[v]=ch[ne[u]][i];
                    q.push(v);deg[ch[ne[u]][i]]++;
                }//构建回跳边
                else ch[u][i]=ch[ne[u]][i];//构建转移边(压缩fail指针)
            }
        }
    }
    //统计主串中模式串的出现次数
    void query(string s){
        for(int k=0,i=0;k<s.size();k++){
            i=ch[i][s[k]-'a'];//走树边/转移边
            ans[i]++;
        }
    }
    void topu(){
        queue<int> q;
        for(int i=0;i<=tot;i++){
            if(!deg[i]) q.push(i);
        }
        while(!q.empty()){
            int u=q.front();q.pop();
            fin[idx[u]]=ans[u];
            ans[ne[u]]+=ans[u];
            if(--deg[ne[u]]==0) q.push(ne[u]);
        }
    }
    int qans(int i){
        return fin[sidx[i]];
    }
    vector<int> getans(int k){
        vector<int> res(k+1,0);
        for(int i=1;i<=k;i++) res[i]=qans(i);
        return res;
    }
};
```

用法示例:

```
    int n;
    while(true){
        cin>>n;
        if(n==0) break;
        AC ac(n*70);
        vector<string> p(n+1);
```

```
        for(int i=1;i<=n;i++){
            string s;cin>>s;
            p[i]=s;
            ac.insert(s,i);
        }
        ac.build();
        string l;
        cin>>l;ac.query(l);
        ac.topu();
        vector<int> res=ac.getans(n);
        int maxx=*max_element(res.begin(),res.end());
        cout<<maxx<<endl;
        for(int i=1;i<=n;i++){
            if(res[i]==maxx) cout<<p[i]<<endl;
        }
    }
```

4.2 AC 自动机(可拓展版)

```
//AC自动机 处理多模式串匹配问题
//具体来说 就是给定多个模式串和一个文本串
//求文本串中有多少个模式串的出现
//AC自动机是Trie树和KMP的结合
//时间复杂度构建：O(n+26n) n是模式串总长
//时间复杂度匹配：O(m) m是文本串长度
//时间复杂度查询：O(m*p) m是文本串长度 p是一个串的后缀串的数量
//-->查询可以通过fail树上做树dp/拓扑排序达到O(n+m)的时间复杂度
//ac自动机的结构其实就是一个 trans 函数，而构建好这个函数后，在匹配字符串的过程中，我们会舍弃部分前缀达到最低限度的匹配。
//本质上就是一个状态，接受一个输入，转移到另一个状态，
//注意到fail链构成的图是一个DAG，所以fail链的长度是O(n)的，所以fail指针的构建是O(n)的。
//注意匹配到一个点其实就是匹配到这个点及其fail链上所有串。
```

```
class AC{
    public:
    vector<vector<int>> ch;
    int n,tot; //节点数>=模式串总长 根节点0 节点编号1-tot
    vector<int> cnt,ne;
    AC(int n):n(n),ch(n+1,vector<int>(26,0)),
    cnt(n+1,0),ne(n+1,0),tot(0){}
    void insert(string s){
        int p=0;
        for(auto c:s){
            if(!ch[p][c-'a']) ch[p][c-'a']=++tot;
            p=ch[p][c-'a'];
        }
        cnt[p]++;
    }
    //构建tire
    void build(){
        queue<int> q;
        for(int i=0;i<26;i++){
            if(ch[0][i]) q.push(ch[0][i]);
        }
        while(!q.empty()){
            int u=q.front();q.pop();
            //如果要做一个拓扑序的信息合并（例如根到点的串数，串长前缀和（即匹配到的串的这些信息），可以在这里用u和ne[u]合并信息）
            for(int i=0;i<26;i++){
                int v=ch[u][i];
                if(v) ne[v]=ch[ne[u]][i],q.push(v);//构建回跳边
                else ch[u][i]=ch[ne[u]][i];//构建转移边(压缩fail指针)
            }
        }
    }
    //统计主串中出现过多少个不同模式串；会修改cnt，不能重复query
    int query(string s){
        int ans=0;
        for(int k=0,i=0;k<s.size();k++){
            i=ch[i][s[k]-'a'];//走树边/转移边
            for(int j=i;j&&~cnt[j];j=ne[j]){
                ans+=cnt[j],cnt[j]=-1;//统计后缀匹配的个数
            }
        }
        return ans;
    }
};
```

用法示例:

```
    int n;cin>>n;
    AC ac(1e6+5);
    while(n--){
        string s;cin>>s;
        ac.insert(s);
    }
    ac.build();
    string s;cin>>s;
    cout<<ac.query(s)<<endl;
```

4.3 exKMP

```
//exkmp
//z[i]表示s[i,len]与s[1,len]的最长公共前缀长度
//加速盒 右端点最靠右的LCP区间
//p[i]表示s2[i,len]与s[1,len]的最长公共前缀长度
//o(n)
```

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class exkmp{
    public:
    vector<int> z,p;
    string s1,s2;
    int len1,len2;
    exkmp(int n):z(n+5,0),p(n+5,0){}
    //z[i]表示s[i,len]与s[1,len]的最长公共前缀长度
    //加速盒 右端点最靠右的LCP区间
    //p[i]表示s2[i,len]与s[1,len]的最长公共前缀长度
    void getz(string s){
        len1=s.size();
        s="x"+s;s1=s;
        z[1]=len1;
        for(int i=2,l=0,r=0;i<=len1;i++){
            if(i<=r) z[i]=min(z[i-l+1],r-i+1);//case1+2
            //case1:i在l~r内，对称的区间长度<=加速盒
            //case2:i在l~r内，对称的区间长度>加速盒
            //case3:i在l~r外
            while(s[z[i]+1]==s[i+z[i]]) z[i]++;
            //暴力扩展(case2+3)
            if(i+z[i]-1>r) l=i,r=i+z[i]-1;
            //更新加速盒
            //printf("i=%d z=%d [%d %d]\n",i,z[i],l,r);
        }
    }
    void getp(string oths){
        len2=oths.size();
        oths="x"+oths,s2=oths;
        for(int i=1,l,r=0;i<=len2;i++)
        {
            if(i<=r) p[i]=min(z[i-l+1],r-i+1);
            while(i+p[i]<=len2&&1+p[i]<=len1&&s1[p[i]+1]==s2[i+p[i]]) p[i]++;
            if(i+p[i]-1>r) l=i,r=i+p[i]-1;
        }
    }
};
```

用法示例:

```
    exkmp exk(2e7+5);
    string s1,s2;
    cin>>s1>>s2;
    exk.getz(s2);
    exk.getp(s1);
    int ans1=0,ans2=0;
    for(int i=1;i<=s2.size();i++)
        ans1^=(i*(exk.z[i]+1));
    for(int i=1;i<=s1.size();i++)
        ans2^=(i*(exk.p[i]+1));
    cout<<ans1<<endl<<ans2<<endl;
```

4.4 kmp

```
vector<int> prefix_init_f(string s) //前缀函数初始化
// 前缀函数就是，子串s[0..i]最长的相等的真前缀与真后缀的长度。
// 在kmp算法中，前缀函数是核心，它决定了模式串（key）在匹配过程若不匹配应该跳转的位置。
// e.g. abcabc的前缀函数为[0,0,0,1,2,3]
{
    int len=s.length();
    vector<int> dp(len,0);
    dp[0]=0;
    for(int i=1;i<len;i++)
    {
        int j=dp[i-1];
        while(j>0&&s[i]!=s[j]) j=dp[j-1];//如果s[i]和s[j]不相同，j跳到前一个符合的位置
        if(s[i]==s[j]) j++; //如果s[i]和s[j]相同，j+1
        dp[i]=j;
    }
    return dp;
}
void kmp(string s,string key)
{
    if(key.length()==0) return;
    vector<int> dp=prefix_init_f(key);
    int i=0,j=0;
    while(i<s.length())
    {
        if(s[i]==key[j]) {i++;j++;} //如果匹配，接着匹配下一个字符
        else if(j>0) j=dp[j-1]; //如果不匹配，j跳到前一个符合的位置
        else i++;
```

```
        if(j==key.length())
        {
            /*some operation*/
            j=dp[j-1];//匹配成功后，j跳到前一个符合的位置
        }
    }
}
```

4.5 Manacher

```
//Manacher算法 求最长回文子串长度 O(n)
```

```
class Manacher{
    public:
    vector<char> s;
    vector<int> d;
    int k,n;
    Manacher(int n):d(2*n+5,0),s(2*n+5){}
    void manacher(const string& str){
        //str 1-base
        k=0,n=str.size()-1;
        s[0]='$',s[++k]='#';
        for(int i=1;i<=n;i++){
            s[++k]=str[i],s[++k]='#';
        }
        d[1]=1;
        for(int i=2,l=1,r=1;i<=k;i++){
            if(i<=r) d[i]=min(d[r-i+l],r-i+1);
            while(s[i+d[i]]==s[i-d[i]]) d[i]++;
            if(i+d[i]-1>r) l=i-d[i]+1,r=i+d[i]-1;
            //与exkmp一致 不再赘述
        }
    }
    int get_max(){
        int maxn=0;
        for(int i=1;i<=k;i++){
            maxn=max(maxn,d[i]);
        }
        return maxn-1;
    }
};
```

用法示例:

```
    Manacher ma(1.1e7+5);
    string str;
    cin>>str;
    ma.manacher(str);
    cout<<ma.get_max()<<endl;
```

4.6 tiretree

```
class Trie{
public:
    struct Node{
        int son[26],a1,a2;
        Node(){
            memset(son,0,sizeof(son));
            a1=a2=0;
        }
    };
    vector<Node> trie;
    int tot,root;
    long long fin;
    Trie(int len):trie(len+5),tot(0),root(0),fin(0){}
    void ins(string s,int op)
    {
        int p=root;
        for(auto c:s)
        {
            int id=c-'a';
            if(!trie[p].son[id]) trie[p].son[id]=++tot;
            p=trie[p].son[id];
            if(op==1) trie[p].a1++;
            else trie[p].a2++;
        }
    }
    void dfs(int p)
    {
        fin+=1ll*trie[p].a1*trie[p].a2;
        for(int i=0;i<26;i++)
        {
            if(trie[p].son[i])
            {
                dfs(trie[p].son[i]);
            }
        }
    }
};
```

用法示例:

```
    int n;cin>>n;
    Trie trie(1e6+5);
    long long fin=0;
    for(int i=1;i<=n;i++)
    {
        string s;cin>>s;
        fin+=s.size();
        trie.ins(s,1);
        reverse(s.begin(),s.end());
        trie.ins(s,2);
    }
    fin=2ll*n*fin;
    //cout<<fin<<endl;
    trie.dfs(trie.root);
    cout<<fin-2*trie.fin<<endl;
```

4.7 后缀自动机(SAM,低注释)

```
//#define int ll //赫赫 要不要龙龙呢
using ll=long long;
using namespace std;
class SAM{
    public:
    vector<array<int,26>> ch;
    vector<vector<int>> tree;
    //DAG转移边,Parent树
    //parent树是指向根节点(0)的内向树
    int n,tot,last; // 根节点为0, 节点总数tot
    vector<int> len,fa,sz,fpos,is_np,id;
    //sam的每个点都代表这一个诸如此类的等价类：endpos(结束位置)集合相同的子串
    //而且到这个点的所以路径就是这个等价类里面的子串
    //这个等价类是覆盖[minlen,maxlen]的, 而且每个长度有且只有一个子串
    //一个点的后缀链接指向的点的最长串是这个点的最短串的真后缀
    //一个比较显然的例子是：1<-2 1:aba,ba,a 2:caba acaba
    //父节点的endpos集合是子节点的endpos集合的无交集并集
    //O(sigma*|s|)构建
    SAM(int n):n(n),ch(2*n+5,array<int,26>{}),tree(2*n+5),
               len(2*n+5,0),fa(2*n+5,0),sz(2*n+5,0),fpos(2*n+5,0),
               is_np(2*n+5,0),id(2*n+5,0),tot(0),last(0){
                fa[0]=-1;
           }
    void extend(int c){
        int p=last,np=last=++tot;
        len[np]=len[p]+1,sz[np]=1,fpos[np]=len[np],is_np[np]=1;
        for(;~p&&!ch[p][c];p=fa[p]) ch[p][c]=np;
        if(!~p) fa[np]=0;
        else{
            int q=ch[p][c];
            if(len[q]==len[p]+1) fa[np]=q;
            else{
                int nq=++tot;
                len[nq]=len[p]+1,fpos[nq]=fpos[q];
                fa[nq]=fa[q],ch[nq]=ch[q];
                fa[q]=fa[np]=nq;
                for(;~p&&ch[p][c]==q;p=fa[p]) ch[p][c]=nq;
            }
        }
    }
    void insert(string s){for(auto c:s) extend(c-'a');}
    //按len排序天然满足拓扑序 (从根到叶子)
    //O(|s|)
    void build(){
        vector<int> cnt(n+1,0);
        for(int i=0;i<=tot;i++) cnt[len[i]]++;
        for(int i=1;i<=n;i++) cnt[i]+=cnt[i-1];
        for(int i=0;i<=tot;i++) id[cnt[len[i]]--]=i;
        for(int i=tot+1;i>=2;i--) sz[fa[id[i]]]+=sz[id[i]],tree[fa[id[i]]].push_back(id[i]);
        //id[1]是根节点, id是1-based的, 共tot+1个点
        //父节点的sz恰好是子节点的sz之和
    }
    //1.判定子串是否出现
    //O(|t|)
    bool check(string s){
        int p=0;
        for(auto c:s){if(!ch[p][c-'a']) return false;p=ch[p][c-'a'];}
        return true;
    }
    //2.本质不同子串个数 (本质不同是与位置不同相对的)
    //O(|s|)
    ll dist_sub(){
        ll ans=0;
        for(int i=1;i<=tot;i++) ans+=len[i]-len[fa[i]];
        return ans;
    }
    //3.所有本质不同子串总长度
    //O(simga*|s|)
    ll dist_sub_len(){
        vector<ll> c(tot+1,0),ans(tot+1,0);
        for(int i=tot+1;i>=1;i--){
            int u=id[i];c[u]=1;
```

```
        for(int j=0;j<26;j++)
            if(ch[u][j])
                c[u]+=c[ch[u][j]],ans[u]+=ans[ch[u][j]]+c[ch[u][j]];
    }
    return ans[0];
}
//4.字典序第K小的子串(T=0本质不同,T=1位置不同)
//预处理: O(sigma*|s|),查询: O(sigma*|t|)
string kth_sub(int T,ll k){
    //w: 沿一条路径走到这个点代表的子串个数
    //本质不同就是1, 位置不同就是endpos大小
    vector<ll> s(tot+1,0),w(tot+1,0);
    for(int i=tot+1;i>=2;i--) w[id[i]]=T?sz[id[i]]:1;
    w[0]=0;
    for(int i=tot+1;i>=1;i--){
        int u=id[i];s[u]=w[u];
        for(int j=0;j<26;j++) if(ch[u][j]) s[u]+=s[ch[u][j]];
    }
    //跟tire一样的思路, 在sam上跑
    if(k>s[0]) return "-1";
    string res="";
    for(int u=0;k>0;){
        if(k<=w[u]) break;
        k-=w[u];
        for(int j=0;j<26;j++){
            if(int v=ch[u][j]){
                if(k>s[v]) k-=s[v];
                else{res+=(char)(j+'a'),u=v;break;}
            }
        }
    }
    return res;
}
//5.最小循环移位(初始化需insert(S+S),传原串长)
//O(sigma*m)
string min_cyclic(int m){
    string res="";
    for(int p=0,i=0;i<m;i++){
        for(int j=0;j<26;j++)
            if(ch[p][j])
            {
                res+=(char)(j+'a'),p=ch[p][j];
                break;
            }
    }
    return res;
}
//6.最短未出现子串(DAG逆拓扑序DP)
//O(sigma*|s|)
string short_unapp(){
    vector<int> d(tot+1,1e9),nxt(tot+1,-1);
    for(int i=tot+1;i>=1;i--){
        int u=id[i];
        for(int j=0;j<26;j++) if(!ch[u][j]){d[u]=1,nxt[u]=j;break;}// 优先选字典序最小的断层
        if(d[u]==1) continue;
        for(int j=0;j<26;j++) if(d[u]>d[ch[u][j]]+1) d[u]=d[ch[u][j]]+1,nxt[u]=j;
    }
    string res="";
    for(int u=0;;u=ch[u][nxt[u]]){
        res+=(char)(nxt[u]+'a');
        if(!ch[u][nxt[u]]) break;
    }
    return res;
}
//7.求子串出现次数
//就是sz[p]
//O(|t|)
int count(string s){
    int p=0;
    for(auto c:s){if(!ch[p][c-'a']) return 0;p=ch[p][c-'a'];}
    return sz[p];
}
//8.子串首次出现起始位置(1base)
//O(|t|)
int first_pos(string s){
    int p=0;
    for(auto c:s){if(!ch[p][c-'a']) return -1;p=ch[p][c-'a'];}
    return fpos[p]-s.size()+1;
}
//9.子串所有出现起始位置(1base)
//找到以p为节点的子树的所有叶子节点, 收集pos即可
//O(|t|+ans)
void _dfs_pos(int u,int len,vector<int>& res){
    if(is_np[u]) res.push_back(fpos[u]-len+1);
    for(auto v:tree[u]) _dfs_pos(v,len,res);
}
vector<int> all_pos(string s){
    vector<int> res;int p=0;
    for(auto c:s){if(!ch[p][c-'a']) return res;p=ch[p][c-'a'];}
    _dfs_pos(p,s.size(),res);
    return res;
}
```

```
    // 10.两串LCS(t在s的sam上跑匹配)
    //O(|t|)
    int lcs(string t){
        int u=0,l=0,ans=0;
        for(auto c:t){
            int x=c-'a';
            for(;u>0&&!ch[u][x];u=fa[u]) l=len[fa[u]];
            //失配，砍掉一部分前缀，往fa走
            if(ch[u][x]) u=ch[u][x],l++; //长度加一
            else u=0,l=0;//清零
            ans=max(ans,l);
        }
        return ans;
    }
};
```

用法示例:

```
    string s;cin>>s;
    SAM sam(s.size());
    sam.insert(s);
    sam.build();
    ll mx=0;
    for(int i=1;i<=sam.tot;i++){
        if(sam.sz[i]>=2){
            mx=max(mx,1ll*sam.len[i]*sam.sz[i]);
        }
    }
    cout<<mx<<endl;
```

4.8 后缀自动机(SAM,高注释)

```
//#define int ll //赫赫 要不要龙龙呢
using ll=long long;
using namespace std;
class SAM{
    public:
    vector<array<int,26>> ch;
    vector<vector<int>> tree;
    //DAG转移边,Parent树
    //parent树是指向根节点(0)的内向树
    int n,tot,last; // 根节点为0，节点总数tot
    vector<int> len,fa,sz,fpos,is_np,id;
    //sam的每个点都代表这一个诸如此类的等价类：endpos(结束位置)集合相同的子串
    //而且到这个点的所以路径就是这个等价类里面的子串
    //这个等价类是覆盖[minlen,maxlen]的，而且每个长度有且只有一个子串
    //一个点的后缀链接指向的点的最长串是这个点的最短串的真后缀
    //一个比较显然的例子是：1<-2 1:aba,ba,a 2:caba acaba
    //父节点的endpos集合是子节点的endpos集合的无交集并集
    //len:节点内部最长子串长度
    //fa: endpos 集合变大的节点，后缀连接
    //sz: endpos 集合大小
    //fpos: 此状态第一次在原串中出现的结束位置
    //id: 拓扑序,即rk为i的点是哪个
    //is_np:是否是真实节点（非分裂的点）,即如果is_np为真，就代表这个点是代表原串的一个前缀，即parent树的叶子
    //n: 字符数
    //O(sigma*|s|)构建
    SAM(int n):n(n),ch(2*n+5,array<int,26>{}),tree(2*n+5),
               len(2*n+5,0),fa(2*n+5,0),sz(2*n+5,0),fpos(2*n+5,0),
               is_np(2*n+5,0),id(2*n+5,0),tot(0),last(0){
                fa[0]=-1;
            }
    void extend(int c){
        int p=last,np=last=++tot;
        len[np]=len[p]+1,sz[np]=1,fpos[np]=len[np],is_np[np]=1;
        //p为上一个整串代表的节点，np是新加入字符后形成的节点
        //这里的赋值都比较显然。
        for(;~p&&!ch[p][c];p=fa[p]) ch[p][c]=np;
        //沿着后缀链接一直向上走，直到找到一个节点，这个节点包含的字符中有c，如果没有就指向np
        if(!~p) fa[np]=0; //爬到头了
        else{
            int q=ch[p][c];
            if(len[q]==len[p]+1) fa[np]=q; //恰好，直接指向
            else{
                int nq=++tot;
                len[nq]=len[p]+1,fpos[nq]=fpos[q];
                fa[nq]=fa[q],ch[nq]=ch[q];
                //克隆节点nq存len[p]+1的串的信息，信息继承原来的q节点的
                fa[q]=fa[np]=nq;
                //q和np的后缀连接都是nq
                for(;~p&&ch[p][c]==q;p=fa[p]) ch[p][c]=nq;
                //更新以前指向q的转移边
            }
        }
    }
    void insert(string s){for(auto c:s) extend(c-'a');}
    //insert完调用一下，建树
```

```
//按len排序天然满足拓扑序（从根到叶子）
//O(|s|)
void build(){
    vector<int> cnt(n+1,0);
    for(int i=0;i<=tot;i++) cnt[len[i]]++;
    for(int i=1;i<=n;i++) cnt[i]+=cnt[i-1];
    for(int i=0;i<=tot;i++) id[cnt[len[i]]--]=i;
    for(int i=tot+1;i>=2;i--) sz[fa[id[i]]]+=sz[id[i]],tree[fa[id[i]]].push_back(id[i]);
    //id[1]是根节点，id是1-based的，共tot+1个点
    //父节点的sz恰好是子节点的sz之和
}
//1.判定子串是否出现
//O(|t|)
bool check(string s){
    int p=0;
    for(auto c:s){if(!ch[p][c-'a']) return false;p=ch[p][c-'a'];}
    return true;
}
//2.本质不同子串个数（本质不同是与位置不同相对的）
//O(|s|)
ll dist_sub(){
    ll ans=0;
    for(int i=1;i<=tot;i++) ans+=len[i]-len[fa[i]];
    return ans;
}
//3.所有本质不同子串总长度
//在DAG跑反向dp
//c[u]表示从u开始的子串个数，c[u]=1+sum(c[v])，v是u的子节点,含义是仅选u和选u的子节点
//ans[u]表示从u开始的子串总长度，ans[u]=sum(c[v]+ans[v]),含义就是选和不选u
//O(simga*|s|)
ll dist_sub_len(){
    vector<ll> c(tot+1,0),ans(tot+1,0);
    for(int i=tot+1;i>=1;i--){
        int u=id[i];c[u]=1;
        for(int j=0;j<26;j++)
            if(ch[u][j])
                c[u]+=c[ch[u][j]],ans[u]+=ans[ch[u][j]]+c[ch[u][j]];
    }
    return ans[0];
}
//4.字典序第K小的子串(T=0本质不同,T=1位置不同)
//预处理+查询
//预处理：O(sigma*|s|),查询：O(sigma*|t|)
string kth_sub(int T,ll k){
    //w：沿一条路径走到这个点代表的子串个数
    //本质不同就是1，位置不同就是endpos大小
    vector<ll> s(tot+1,0),w(tot+1,0);
    for(int i=tot+1;i>=2;i--) w[id[i]]=T?sz[id[i]]:1;
    w[0]=0;
    //像3一致的反向dp s[u]表示从u开始的子串个数
    for(int i=tot+1;i>=1;i--){
        int u=id[i];s[u]=w[u];
        for(int j=0;j<26;j++) if(ch[u][j]) s[u]+=s[ch[u][j]];
    }
    //跟tire一样的思路，在sam上跑
    if(k>s[0]) return "-1";
    string res="";
    for(int u=0;k>0;){
        if(k<=w[u]) break;
        k-=w[u];
        for(int j=0;j<26;j++){
            if(int v=ch[u][j]){
                if(k>s[v]) k-=s[v];
                else{res+=(char)(j+'a'),u=v;break;}
            }
        }
    }
    return res;
}
//5.最小循环移位(初始化需insert(S+S),传原串长)
//O(sigma*m)
string min_cyclic(int m){
    string res="";
    for(int p=0,i=0;i<m;i++){
        for(int j=0;j<26;j++)
            if(ch[p][j])
            {
                res+=(char)(j+'a'),p=ch[p][j];
                break;
            }
    }
    return res;
}
//6.最短未出现子串(DAG逆拓扑序DP)
//d[u]表示从u出发最短未出现子串的长度，nxt[u]记录第一步走的字符
//O(sigma*|s|)
string short_unapp(){
    vector<int> d(tot+1,1e9),nxt(tot+1,-1);
    for(int i=tot+1;i>=1;i--){
        int u=id[i];
```

```
            for(int j=0;j<26;j++) if(!ch[u][j]){d[u]=1,nxt[u]=j;break;}// 优先选字典序最小的断层
            if(d[u]==1) continue;
            for(int j=0;j<26;j++) if(d[u]>d[ch[u][j]]+1) d[u]=d[ch[u][j]]+1,nxt[u]=j;
        }
        string res="";
        for(int u=0;;u=ch[u][nxt[u]]){
            res+=(char)(nxt[u]+'a');
            if(!ch[u][nxt[u]]) break;
        }
        return res;
    }
    //7.求子串出现次数
    //就是sz[p]
    //O(|t|)
    int count(string s){
        int p=0;
        for(auto c:s){if(!ch[p][c-'a']) return 0;p=ch[p][c-'a'];}
        return sz[p];
    }
    //8.子串首次出现起始位置(1base)
    //O(|t|)
    int first_pos(string s){
        int p=0;
        for(auto c:s){if(!ch[p][c-'a']) return -1;p=ch[p][c-'a'];}
        return fpos[p]-s.size()+1;
    }
    //9.子串所有出现起始位置(1base)
    //找到以p为节点的子树的所有叶子节点，收集pos即可
    //O(|t|+ans)
    void _dfs_pos(int u,int len,vector<int>& res){
        if(is_np[u]) res.push_back(fpos[u]-len+1);
        for(auto v:tree[u]) _dfs_pos(v,len,res);
    }
    vector<int> all_pos(string s){
        vector<int> res;int p=0;
        for(auto c:s){if(!ch[p][c-'a']) return res;p=ch[p][c-'a'];}
        _dfs_pos(p,s.size(),res);
        return res;
    }
    // 10.两串LCS(t在s的sam上跑匹配)
    //O(|t|)
    int lcs(string t){
        int u=0,l=0,ans=0;
        for(auto c:t){
            int x=c-'a';
            for(;u>0&&!ch[u][x];u=fa[u]) l=len[fa[u]];
            //失配，砍掉一部分前缀，往fa走
            if(ch[u][x]) u=ch[u][x],l++; //长度加一
            else u=0,l=0;//清零
            ans=max(ans,l);
        }
        return ans;
    }
};
```

用法示例:

```
    string s;cin>>s;
    SAM sam(s.size());
    sam.insert(s);
    sam.build();
    ll mx=0;
    for(int i=1;i<=sam.tot;i++){
        if(sam.sz[i]>=2){
            mx=max(mx,1ll*sam.len[i]*sam.sz[i]);
        }
    }
    cout<<mx<<endl;
```

4.9 回文树(PAM,回文自动机)

```
//PAM 处理回文串结构的数据结构，时间复杂度O(n),通过势能分析能做到,不表。
```

```
class PAM{
    public:
    vector<vector<int>> ch;
    vector<int> fail,len,cnt;
    int sz,last,idx; string s;
    //PAM(回文树/回文自动机)
    //是这样的东西 0:偶根,代表偶数长度的回文串 1:奇根,代表奇数长度的回文串
    //一个点代表的回文串是 从他出发到根，又回到他的路径上的所有字符(若是奇数长度的回文串,与根节点相邻的边只走一次)
    //fail:节点代表串的最长真回文后缀
    //len:节点代表串的长度
    //cnt:节点代表串的出现次数(需要bulid一次统计)
    //编号>=2的点是真实节点
    PAM(int maxn):ch(maxn+5,vector<int>(26,0)),
    fail(maxn+5,0),len(maxn+5,0),cnt(maxn+5,0){
        sz=1,last=0,idx=0;
        fail[0]=1,fail[1]=1;
        len[0]=0,len[1]=-1;
```

```
        s="$";
    }
    //找到一个形如 s[i-len[x]-1]+x+s[i] 的回文串
    int get_fail(int x,int i){
        while(s[i]!=s[i-len[x]-1]) x=fail[x];
        return x;
    }
    void insert(string str){
        //str 0-based
        for(auto x:str){
            s+=x;idx++;
            int c=x-'a',p=get_fail(last,idx);
            //没有这样的转移边,新建节点
            if(!ch[p][c]){
                //此处可以算某个本质不同回文串的首次出现下标
                //或者以该下标结尾的回文串的个数(即该节点对应的fail树的深度),开个num,用fail[u]更新即可
                int u=++sz;
                fail[u]=ch[get_fail(fail[p],idx)][c];
                len[u]=len[p]+2,ch[p][c]=u;
            }
            last=ch[p][c],cnt[last]++;
            //记录当前本质不同回文串的出现次数
        }
    }
    //构建/合并fail树上的信息(如统计所有回文子串的真实出现频次)
    void build(){
        //拓扑序直接按节点编号倒序即可，因为sz越大的节点一定越晚建立，深度越大
        for(int i=sz;i>=2;i--){
            cnt[fail[i]]+=cnt[i];
        }
    }
};
```

4.10 广义后缀自动机(GSAM)

```
//GSAM:描述tire的子串性质。此处的子串不一定从根节点开始，从任意节点开始的子串都满足性质
```

```
class Trie{
    public:
    vector<array<int,26>> ch;
    vector<int> cnt;
    int tot;
    // len:预估最大节点数
    Trie(int len):ch(len+5,array<int,26>{}),cnt(len+5,0),tot(0){}
    void insert(string s){
        int p=0;
        for(auto c:s){
            if(!ch[p][c-'a']) ch[p][c-'a']=++tot;
            p=ch[p][c-'a'];
            cnt[p]++;//共享前缀要按经过次数计入出现次数
        }
    }
};

class GSAM{
    public:
    vector<array<int,26>> ch;
    vector<vector<int>> tree;
    int tot;
    vector<int> len,fa,sz,id,pos;
    // n为Trie节点数，GSAM最多2n个节点
    GSAM(int n):ch(2*n+5,array<int,26>{}),tree(2*n+5),
                len(2*n+5,0),fa(2*n+5,0),sz(2*n+5,0),id(2*n+5,0),pos(n+5,0),tot(0){
        fa[0]=-1;
    }
    int extend(int c,int last){
        //如果转移边存在的情况
        if(ch[last][c]){
            int q=ch[last][c];
            if(len[q]==len[last]+1) return q;
            int nq=++tot;
            len[nq]=len[last]+1,fa[nq]=fa[q],ch[nq]=ch[q];
            fa[q]=nq;
            for(int p=last;~p&&ch[p][c]==q;p=fa[p]) ch[p][c]=nq;
            return nq;
        }
        //如果转移边不存在的情况，与普通sam一致
        int np=++tot,p=last;
        len[np]=len[p]+1;
        for(;~p&&!ch[p][c];p=fa[p]) ch[p][c]=np;
        if(!~p) fa[np]=0;
        else{
            int q=ch[p][c];
            if(len[q]==len[p]+1) fa[np]=q;
            else{
                int nq=++tot;
                len[nq]=len[p]+1,fa[nq]=fa[q],ch[nq]=ch[q];
                fa[q]=fa[np]=nq;
                for(;~p&&ch[p][c]==q;p=fa[p]) ch[p][c]=nq;
            }
```

```
        }
        return np;
    }
    // 传入建好的Trie跑BFS建机，保证len拓扑序严格递增
    // 如果不想搞tire可以自己建树然后跑bfs,空间复杂度可以少个simga
    // 当然，dfs也是可行的
    void insert(const Trie& tr){
        queue<pair<int,int>> q;
        q.push({0,0});
        pos[0]=0;
        while(!q.empty()){
            auto [tu,su]=q.front();q.pop();
            for(int c=0;c<26;c++){
                if(tr.ch[tu][c]){
                    int tv=tr.ch[tu][c],sv=extend(c,su);
                    pos[tv]=sv;
                    sz[sv]+=tr.cnt[tv];//每个Trie节点按经过次数累加到对应SAM状态
                    q.push({tv,sv});
                }
            }
        }
    }
    void build(){
        vector<int> cnt(tot+1,0);
        for(int i=0;i<=tot;i++) cnt[len[i]]++;
        for(int i=1;i<=tot;i++) cnt[i]+=cnt[i-1];
        for(int i=0;i<=tot;i++) id[cnt[len[i]]--]=i;
        for(int i=tot+1;i>=2;i--) sz[fa[id[i]]]+=sz[id[i]],tree[fa[id[i]]].push_back(id[i]);
    }
    ll dist_sub(){
        ll ans=0;
        for(int i=1;i<=tot;i++) ans+=len[i]-len[fa[i]];
        return ans;
    }
};
```

4.11 防 hack 的 umap,gp_hash_table

```
using ull=unsigned long long;
using ll=long long;
using namespace std;
using namespace __gnu_pbds;
struct Xhash64{
    static ull splitmix64(ull x){
        x+=0x9e3779b97f4a7c15;
        x=(x^(x>>30))*0xbf58476d1ce4e5b9;
        x=(x^(x>>27))*0x94d049bb133111eb;
        return x^(x>>31);
    }
    size_t operator()(ull x) const{
        static const ull SALT=chrono::steady_clock::now().time_since_epoch().count()^(ull)(new char);
        return splitmix64(x+SALT);
    }
};
struct Xhash32{
    static uint32_t splitmix32(uint32_t x){
        x+=0x9e3779b9;
        x=(x^(x>>16))*0x85ebca6b;
        x=(x^(x>>13))*0xc2b2ae35;
        return x^(x>>16);
    }
    size_t operator()(uint32_t x) const{
        static const uint32_t SALT=std::chrono::steady_clock::now().time_since_epoch().count()^(ull)(new char);
        return splitmix32(x+SALT);
    }
};
```

用法示例:

```
    unordered_map<ull,int,Xhash64> mp; //拉链法
    gp_hash_table<ull,int,Xhash64> gp; //二次寻址，缓存更友好，速度更快。
```

4.12 随机底数固定模数单 hash

```
using ull=unsigned long long;
using ui128=__uint128_t;
using namespace std;
class SHASH{
    //特别注意:传入的字符串应该是>=0的 如果传入负数会导致hash值在模数外
    //如果多开hash类 底数在外面生成。
    public:
        ull b;
        vector<ull> h,p;
        const ull mod=(1ull<<61)-1;
        inline ull add(ull a,ull b){ a+=b;return a>=mod?a-mod:a;}
        inline ull sub(ull a,ull b){ return a>=b?a-b:a+mod-b;}
        inline ull mul(ull a,ull b){ ui128 c=ui128(a)*b; return (add(c>>61,c&mod));}
        SHASH(int m):p(m+1,1),h(m+1,0){
            mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count()^(ull)(new char));
            b=rng()%(mod-1313131)+1313131;
            for(int i=1;i<=m;i++) p[i]=mul(p[i-1],b);
```

```
        }
        void calc(const string& s,int n){
            for(int i=1;i<=n;i++) h[i]=add(mul(h[i-1],b),s[i]);
        }
        ull get(int l,int r){
            return sub(h[r],mul(h[l-1],p[r-l+1]));
        }
};
```

4.13 随机底数固定模数双 hash

```
using ull=unsigned long long;
using namespace std;
class SHASH{
    public:
        const int m1=1e9+7,m2=1e9+9;
        int b1,b2,m;
        vector<int> h1,h2;
        vector<int> p1,p2;
        //m 预估的最长字符串长度
        SHASH(int m):p1(m+1,1),p2(m+1,1),h1(m+1,0),h2(m+1,0){
            mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()^(ull)(new char));
            b1=rng()%(m1-1313131)+1313131,b2=rng()%(m2-133331)+133331;
            //b1=131,b2=13331;
            for(int i=1;i<=m;i++){
                p1[i]=(1ll*p1[i-1]*b1)%m1;
                p2[i]=(1ll*p2[i-1]*b2)%m2;
            }
        }
        //s 1based
        void calc(const string& s,int n){
            for(int i=1;i<=n;i++){
                h1[i]=(1ll*h1[i-1]*b1+s[i])%m1;
                h2[i]=(1ll*h2[i-1]*b2+s[i])%m2;
            }
        }
        ull get(int l,int r){
            int len=r-l+1;
            int r1=h1[r]-1ll*h1[l-1]*p1[len]%m1;
            int r2=h2[r]-1ll*h2[l-1]*p2[len]%m2;
            r1<0?r1+=m1:0,r2<0?r2+=m2:0;
            return ((ull)r1<<32)|r2;
        }
};
```

5 动态规划

5.1 数位 dp(例题 1,数位和)

```
//数位dp 计算[l,r]内所有数的数位和
//dfs 形参总结
//
```

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
const int mod=1e9+7;;
template <int MOD>
struct SMC {
    int64_t val;
    constexpr SMC(int64_t v=0){
        val=(v%MOD+MOD)%MOD;
    }
    SMC& operator=(int64_t v){
        val=(v%MOD+MOD)%MOD;
        return *this;
    }
    SMC& operator+=(const SMC &rhs){
        val+=rhs.val;
        if(val>=MOD) val-=MOD;
        return *this;
    }
    SMC& operator-=(const SMC &rhs){
        val-=rhs.val;
        if(val<0) val+=MOD;
        return *this;
    }
    SMC& operator*=(const SMC &rhs){
        val=1LL*val*rhs.val%MOD;
        return *this;
    }
    static int64_t qpow(int64_t a,int64_t b){
        int64_t res=1;
        while(b){
            if(b&1) res=res*a%MOD;
            a=a*a%MOD;
            b>>=1;
        }
        return res;
    }
    SMC pow(int64_t k) const{
        return SMC(qpow(val,k));
    }
    SMC inv() const{
        return pow(MOD-2);
    }
    SMC& operator/=(const SMC &rhs){
        return *this*=rhs.inv();
    }
    friend SMC operator+(SMC a,const SMC &b){ return a+=b;}
    friend SMC operator-(SMC a,const SMC &b){ return a-=b;}
    friend SMC operator*(SMC a,const SMC &b){ return a*=b;}
    friend SMC operator/(SMC a,const SMC &b){ return a/=b;}
    SMC& operator++() { return *this += 1; }
    SMC& operator--() { return *this -= 1; }
    SMC operator++(int32_t dummy) { SMC t=*this; ++*this; return t; }
    SMC operator--(int32_t dummy) { SMC t=*this; --*this; return t; }
    friend bool operator==(const SMC &a,const SMC &b){ return a.val==b.val;}
    friend bool operator<(const SMC &a,const SMC &b){ return a.val<b.val;}
    friend bool operator>(const SMC &a,const SMC &b){ return a.val>b.val;}
    friend bool operator<=(const SMC &a,const SMC &b){ return a.val<=b.val;}
    friend bool operator>=(const SMC &a,const SMC &b){ return a.val>=b.val;}
    friend bool operator!=(const SMC &a,const SMC &b){ return a.val!=b.val;}

    friend std::istream& operator>>(std::istream &in,SMC &a){
        int64_t v;
        in>>v,a=SMC(v);
        return in;
    }

    friend std::ostream& operator<<(std::ostream &out,const SMC &a){
        out<<a.val;
        return out;
    }
    explicit operator long long() const{
        return val;
    }
    SMC operator-() const{
        return SMC(-val);
    }
    SMC& operator+=(int64_t x) { return *this+=SMC(x); }
    SMC& operator-=(int64_t x) { return *this-=SMC(x); }
    SMC& operator*=(int64_t x) { return *this*=SMC(x); }
    SMC& operator/=(int64_t x) { return *this/=SMC(x); }

    friend SMC operator+(SMC a, int64_t b) { return a+=b; }
    friend SMC operator-(SMC a, int64_t b) { return a-=b; }
    friend SMC operator*(SMC a, int64_t b) { return a*=b; }
    friend SMC operator/(SMC a, int64_t b) { return a/=b; }
```

```
    friend SMC operator+(int64_t a, SMC b) { return b+a; }
    friend SMC operator-(int64_t a, SMC b) { return SMC(a)-b; }
    friend SMC operator*(int64_t a, SMC b) { return b*a; }
    friend SMC operator/(int64_t a, SMC b) { return SMC(a)/b; }
};
using Z=SMC<mod>;
```

用法示例:

```
    int t;cin>>t;
    vector<vector<Z>> dp(20,vector<Z>(18*9+5,-1));
    //dp[i][j]表示[i+1,len](除低i位的高位)数位和=j时,[1,i]任选的所有方案的数位和
    while(t--)
    {
        int l,r;cin>>l>>r;
        vector<int> bit;
        auto work=[&](int x)->int{
            bit.clear();
            bit.push_back(0);//1-based
            while(x) bit.push_back(x%10),x/=10;
            return bit.size()-1;
        };
        auto dfs=[&](this auto&& dfs,int pos,bool lim,int sum)->Z{
            //从len位填到pos+1位,lim表示是否受上界限制,sum表示当前数位和
            //现在填pos位 也就是说dfs的含义是[pos+1,len]数位和=sum时,pos位受lim限制的方案数
            if(pos==0) return sum;//第0位,直接返回sum
            if(!lim&&dp[pos][sum]!=-1) return dp[pos][sum];
            int up=lim?bit[pos]:9;
            Z res=0;
            for(int i=0;i<=up;i++)
            {
                res+=dfs(pos-1,lim&&i==up,sum+i);
                //传递受上界限制的状态
            }
            if(!lim) dp[pos][sum]=res;
            return res;
        };
        auto solve=[&](int x)->Z{
            int len=work(x);
            return dfs(len,1,0);
        };
        cout<<solve(r)-solve(l-1)<<endl;
    }
```

6 数论

6.1 (ex)CRT((扩展)中国剩余定理)(新版)

```cpp
using i128=__int128;
using namespace std;
namespace _EX_CRT{
    ll exgcd(ll a,ll b,ll &x,ll &y){
        if(!b) return x=1,y=0,a;
        ll d=exgcd(b,a%b,y,x);
        return y-=a/b*x,d;
    }
    //r,m 1base
    ll crt(const vector<ll> &r,const vector<ll> &m,int n){
        ll M=1,ans=0;
        for(int i=1;i<=n;i++) M*=m[i];
        for(int i=1;i<=n;i++){
            ll mi=M/m[i],x,y;
            exgcd(mi,m[i],x,y);
            x=(x%m[i]+m[i])%m[i];
            ll tp=(ll)((i128)r[i]*mi%M);
            ans+=(ll)((i128)tp*x%M)%M,ans%=M;
        }
        return (ans%M+M)%M;
    }
    array<ll,2> merge(ll r1,ll m1,ll r2,ll m2){
        ll x,y;
        ll g=exgcd(m1,m2,x,y);
        ll c=(r2-r1)%m2;
        if(c<0) c+=m2;
        if(c%g) return {-1,-1};
        ll bg=m2/g;
        x=(x%bg+bg)%bg;
        ll tp=(ll)((i128)x*(c/g)%bg);
        // i128 _nM=(i128)m1/g*m2;
        // if(_nM>(i128)1e18){
        //     ll nR=r1+(ll)((i128)tp*m1);
        //     return {nR,-1};
        // } //这里是防爆模数
        ll nM=m1/g*m2;
        ll nR=(r1+(ll)((i128)tp*m1%nM))%nM;
        return {nR,nM};
    }
    ll excrt(const vector<ll> &r,const vector<ll> &m,int n){
        ll R=r[1],M=m[1];
        for(int i=2;i<=n;i++){
            auto [nR,nM]=merge(R,M,r[i],m[i]);
            if(nR==-1) return -1;
            R=nR,M=nM;
        }
        return R;
    }
}
//使用例:
//vector<ll> r={0,1,2,3},m={0,3,5,7};
//cout<<_EX_CRT::crt(r,m,3)<<endl;
//cout<<_EX_CRT::excrt(r,m,3)<<endl;
//cout<<_EX_CRT::merge(1,3,2,5)[0]<<endl;
```

用法示例:

```cpp
    int t;cin>>t;
    while(t--){
        int n,q;cin>>n>>q;
        vector<vector<array<int,2>>> tr(n+1);
        vector<int> fa(n+1,0),l(n+1,0),qry(q),ans(q);
        for(int i=2;i<=n;i++) cin>>fa[i];
        for(int i=2;i<=n;i++) cin>>l[i];
        for(int i=2;i<=n;i++) tr[fa[i]].push_back({i,l[i]});
        for(int i=1;i<=n;i++) sort(tr[i].begin(),tr[i].end());
        vector<ll> s(n+1,0),m(q);
        for(int i=0;i<q;i++) cin>>m[i];
        iota(qry.begin(),qry.end(),0);
        auto dfs=[&](this auto&& dfs,int u,vector<int> qr,ll nr,ll nm)->void{
            if(qr.empty()) return;
            if(tr[u].empty()){
                for(int i:qr) ans[i]=u;
                return;
            }
            int d=tr[u].size();
            if(nm==-1||nm%d==0){
                ll k=(nr+s[u])%d;
                int v=tr[u][k][0];
                s[v]=s[u]+tr[u][k][1];
                dfs(v,move(qr),nr,nm);
            }
            else{
                vector<vector<int>> nxt(d);
                for(auto id:qr){
                    nxt[(m[id]+s[u])%d].push_back(id);
                }
                for(int k=0;k<d;k++){
                    if(nxt[k].empty()) continue;
                    int v=tr[u][k][0];
```

```
                    s[v]=s[u]+tr[u][k][1];
                    auto [nR,nM]=_EX_CRT::merge(nr,nm,(k-s[u]%d+d)%d,d);
                    if(nM>1e18l) nM=-1;
                    dfs(v,move(nxt[k]),nR,nM);
                }
            }
        };
        dfs(1,move(qry),0,1);
        for(int i:ans) cout<<i<<' ';
        cout<<'\n';
    }
```

6.2 (ex)CRT((扩展)中国剩余定理)(旧版)

```
//exCRT 求解同余方程组
//形式：x≡a1(mod m1),x≡a2(mod m2),...,x≡ak(mod mk)
//其中m1,m2,...,mk互质(CRT)
//其中m1,m2,...,mk不互质(_exCRT)
//时间复杂度：O(nln(amax))
```

```
#define int __int128
// 用于存储 __int128 的字符串表示
std::string to_string(__int128 value) {
    std::string result;
    bool isNegative = value < 0;
    value = isNegative ? -value : value;

    do {
        result.push_back(static_cast<char>(value % 10) + '0');
        value /= 10;
    } while (value > 0);

    if (isNegative) {
        result.push_back('-');
    }

    std::reverse(result.begin(), result.end());
    return result;
}

// 从字符串转换为 __int128
__int128 to_int128(const std::string& str) {
    __int128 result = 0;
    bool isNegative = str[0] == '-';
    size_t start = isNegative ? 1 : 0;

    for (size_t i = start; i < str.size(); ++i) {
        result = result * 10 + (str[i] - '0');
    }

    return isNegative ? -result : result;
}

// 重载 >> 操作符以支持 __int128 输入
std::istream& operator>>(std::istream& in, __int128& value) {
    std::string str;
    in >> str;
    value = to_int128(str);
    return in;
}

// 重载 << 操作符以支持 __int128 输出
std::ostream& operator<<(std::ostream& out, __int128 value) {
    out << to_string(value);
    return out;
}
class exCRT{

    public:
        exCRT(vector<int> r,vector<int> m){
            this->r=r;
            this->m=m;
            n=r.size();
        }
        vector<int> r,m;
        int x,n;
        int exgcd(int a,int b,int &x,int &y)//扩展欧几里得
        {
            if(b==0)
            {
                x=1;y=0;
                return a;
            }
            int d=exgcd(b,a%b,x,y),t=x;
            x=y;y=t-a/b*y;
            return d;
        }
        int CRT()
        {
            int mul=accumulate(m.begin(),m.end(),(__int128)1,
            [](int a,int b){return a*b;}),ans=0;
```

```
            for(int i=0;i<n;i++)
            {
                int M=mul/m[i],b,y;
                exgcd(M,m[i],b,y);
                ans=(ans+r[i]*M%mul*b%mul+mul)%mul;
            }
            return (ans%mul+mul)%mul;
        }
        int _exCRT()
        {
            int M=m[0],ans=r[0];
            for(int i=1;i<n;i++)
            {
                int a=M,b=m[i];
                int c=((r[i]-ans)%b+b)%b;
                int x,y;
                int gcd=exgcd(a,b,x,y);
                int bg=b/gcd;
                if(c%gcd!=0) return -1;
                x=(x%bg+bg)%bg;
                x=(x*c/gcd%bg+bg)%bg;
                ans+=x*M;
                M*=bg;
                ans=(ans%M+M)%M;
            }
            return (ans%M+M)%M;
        }

};
```

用法示例:

```
    int n;cin>>n;
    vector<int> r(n),m(n);
    for(int i=0;i<n;i++) cin>>m[i]>>r[i];
    exCRT ex(r,m);
    cout<<ex.CRT()<<endl;
```

6.3 (最小)原根

```
class Pre{
public:
    int n;
    vector<int> phi,primes,ex,spf;
    Pre(int n):n(n),phi(n+1,0),ex(n+1,0),spf(n+1){
        prep();
    }
    ll qpow(int a,int b,int m)//快速幂
    {
        ll ans=1;
        while(b)
        {
            if(b&1) ans=ans*a%m;
            a=1ll*a*a%m,b>>=1;
        }
        return ans;
    }
    void prep()
    {
        phi[1]=1;
        vector<bool> v(n+1,0);
        for(int i=2;i<=n;i++)
        {
            if(!v[i])
            {
                primes.push_back(i);
                phi[i]=i-1;
                spf[i]=i;
            }
            for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
            {
                int m=primes[j]*i;
                v[m]=1;spf[m]=primes[j];
                if(i%primes[j]==0)
                {
                    phi[m]=phi[i]*primes[j];
                    break;
                }
                else phi[m]=phi[i]*(primes[j]-1);
            }
        }
        if(n>=2) ex[2]=1;
        if(n>=4) ex[4]=1;
        for(auto p:primes){
            if(p&1){
                ll tp=p;
                while(tp<=n){
                    ex[tp]=1;
                    if(2*tp<=n) ex[2*tp]=1;
                    tp*=p;
                }
            }
        }
        return ;
```

```
    }
    //O(m^1/4*logm^2) 求最小原根
    int getr(int m){
        assert(1<=m&&m<=n);//查询值不能超过预处理上限
        if(!ex[m]) return -1;
        vector<int> now;
        int tp=phi[m];
        while(tp>1){
            int p=spf[tp];
            now.push_back(phi[m]/p);
            while(tp%p==0) tp/=p;
        }
        for(int j=1;j<=m;j++){
            if(__gcd(j,m)!=1) continue;
            int flag=1;
            for(auto k:now){
                if(qpow(j,k,m)==1){
                    flag=0;
                    break;
                }
            }
            if(flag) return j;
        }
        return -1;
    }
    //O(phi(m)*log phi(m)) 求m的所有原根
    vector<int> getar(int m){
        assert(1<=m&&m<=n);//查询值不能超过预处理上限
        vector<int> ans;
        int mi=getr(m);
        if(mi==-1) return ans;
        int tp=mi;
        for(int i=1;i<=phi[m];i++){
            if(__gcd(i,phi[m])==1){
                ans.push_back(tp);
            }
            tp=1ll*tp*mi%m;
        }
        sort(ans.begin(),ans.end());
        return ans;
    }
    //O(n^5/4logn) 求1-n的所有最小原根
    vector<int> get()
    {
        vector<int> ans(n+1,-1);
        for(int i=1;i<=n;i++)
        {
            ans[i]=getr(i);
        }
        return ans;
    }
};
```

用法示例:

```
    Pre p(1e6+5);
    int t;cin>>t;
    while(t--)
    {
        int n,d;cin>>n>>d;
        auto now=p.getar(n);
        cout<<now.size()<<'\n';
        for(int i=d-1;i<now.size();i+=d) cout<<now[i]<<" ";
        cout<<'\n';
    }
```

6.4 FFT(快速傅里叶变换)

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
typedef complex<double> Complex;
class FFT{
    public:
    FFT(){}
    // struct complex{
    //     double x,y;
    //     complex(double x=0,double y=0):x(x),y(y){}
    //     complex operator+(const complex &a) const{return complex(x+a.x,y+a.y);}
    //     complex operator-(const complex &a) const{return complex(x-a.x,y-a.y);}
    //     complex operator*(const complex &a) const{return complex(x*a.x-y*a.y,x*a.y+y*a.x);}
    // };
    const double PI=acos(-1);
    vector<int> R;
    void fft(vector<Complex> &a,int n,int op){
        for(int i=0;i<n;i++) R[i]=(R[i>>1]>>1)|((i&1)*(n>>1));
        //=R[i] = R[i/2]/2 + ((i&1)?n/2:0);
        for(int i=0;i<n;i++) if(i<R[i]) swap(a[i],a[R[i]]);
        for(int i=2;i<=n;i<<=1){
            int m=i>>1;
            Complex w1(cos(2*PI/i),op*sin(2*PI/i));
            for(int j=0;j<n;j+=i){
                Complex wk(1,0);
                for(int k=j;k<j+m;k++){
```

```
                    Complex x=a[k],y=wk*a[k+m];
                    a[k]=x+y;a[k+m]=x-y;
                    wk=wk*w1;
                }
            }
        }
    }
    //普通 double FFT 有精度边界, 大系数整数卷积请用 NTT/CRT/拆系数
    vector<int> calc(vector<int> a,vector<int> b){
        int n=a.size(),m=b.size();
        int len=1;
        while(len<n+m-1) len<<=1;
        R.clear();R.resize(len);
        vector<Complex> fa(len),fb(len);
        for(int i=0;i<n;i++) fa[i].real(a[i]);
        for(int i=0;i<m;i++) fb[i].real(b[i]);
        fft(fa,len,1);fft(fb,len,1);
        for(int i=0;i<len;i++) fa[i]=fa[i]*fb[i];
        fft(fa,len,-1);
        vector<int> ans(n+m-1);
        for(int i=0;i<n+m-1;i++) ans[i]=llround(fa[i].real()/len);
        return ans;
    }
};
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<int> a(n+1),b(m+1);
    for(int i=0;i<=n;i++) cin>>a[i];
    for(int i=0;i<=m;i++) cin>>b[i];
    FFT fft;
    vector<int> ans=fft.calc(a,b);
    for(int i=0;i<ans.size();i++) cout<<ans[i]<<" ";
```

6.5 FMT&FWT(快速莫比乌斯 & 沃尔什变换)

```
const int mod=998244353;
template <int MOD>
struct SMC {
    int val;
    SMC(ll v=0) : val(v%MOD) { if (val<0) val+=MOD; }
    SMC& operator+=(const SMC &r) { val+=r.val; if (val>=MOD) val-=MOD; return *this; }
    SMC& operator-=(const SMC &r) { val-=r.val; if (val<0) val+=MOD; return *this; }
    SMC& operator*=(const SMC &r) { val=1LL*val*r.val%MOD; return *this; }
    SMC& operator/=(const SMC &r) { return *this*=r.inv(); }
    friend SMC operator+(SMC a,const SMC &b) { return a+=b; }
    friend SMC operator-(SMC a,const SMC &b) { return a-=b; }
    friend SMC operator*(SMC a,const SMC &b) { return a*=b; }
    friend SMC operator/(SMC a,const SMC &b) { return a/=b; }
    SMC pow(ll k) const {
        SMC res=1,a=*this;
        for (;k;k>>=1,a*=a) if(k&1) res*=a;
        return res;
    }
    SMC inv() const { return pow(MOD-2); }
    friend istream& operator>>(istream &in,SMC &a) { ll v; in>>v; a=v; return in; }
    friend ostream& operator<<(ostream &out,const SMC &a) { return out<<a.val; }
};
using Z=SMC<mod>;
const Z inv2=Z(2).inv();
class FWT{
public:
    FWT(){}
    void trans(vector<Z> &a,int typ,int op){
        // typ: 1 FWT -1 IFWT
        // op: 0 or 1 and 2 xor
        int len=a.size();
        for(int mid=1;mid<len;mid<<=1){
            //枚举块长
            for(int i=0;i<len;i+=(mid<<1)){
                //枚举块
                for(int j=0;j<mid;j++){
                    //枚举偏移
                    Z x=a[i+j],y=a[i+j+mid];
                    //x是y的子集
                    if(op==0){
                        //把子集数据给超集
                        if(typ==1) a[i+j+mid]=x+y;
                        else a[i+j+mid]=y-x;
                    }
                    else if(op==1){
                        //把超集数据给子集
                        if(typ==1) a[i+j]=x+y;
                        else a[i+j]=x-y;
                    }
                    else{
                        if(typ==1) a[i+j]=x+y,a[i+j+mid]=x-y;
                        else a[i+j]=(x+y)*inv2,a[i+j+mid]=(x-y)*inv2;
                    }
                }
            }
```

```
        }
    }
    //返回c作为结果 a,b 0-based; 会把传入的 a,b 长度扩到2的幂
    vector<Z> conv(vector<Z> &a,vector<Z> &b,int op){
        int n=max(a.size(),b.size());
        int len=1;
        while(len<n) len<<=1;
        vector<Z> c(len,0);
        a.resize(len,0),b.resize(len,0);
        trans(a,1,op),trans(b,1,op);
        for(int i=0;i<len;i++) c[i]=a[i]*b[i];
        trans(a,-1,op),trans(b,-1,op),trans(c,-1,op);
        return c;
    }
};
```

用法示例:

```
    int n;cin>>n;
    vector<Z> a(1<<n),b(1<<n);
    for(int i=0;i<(1<<n);i++) cin>>a[i];
    for(int i=0;i<(1<<n);i++) cin>>b[i];
    FWT fwt;
    auto c1=fwt.conv(a,b,0);
    auto c2=fwt.conv(a,b,1);
    auto c3=fwt.conv(a,b,2);
    for(auto i:c1) cout<<i<<" ";
    cout<<endl;
    for(auto i:c2) cout<<i<<" ";
    cout<<endl;
    for(auto i:c3) cout<<i<<" ";
    cout<<endl;
```

6.6 NTT(快速数论变换)

```
constexpr int P1=998244353;
constexpr int P2=1004535809;
constexpr int P3=469762049;
constexpr int P4=167772161;
constexpr ll P5=4179340454199820289LL;
//大质数可用于答案不超过1e18的多项式乘法
//g=3
const int mod=998244353;
//防爆ll的模运算
template <ll MOD>
struct SMC {
    ll val;
    SMC(ll v=0) : val(v%MOD) { if (val<0) val+=MOD; }
    SMC& operator+=(const SMC &r) { val+=r.val; if (val>=MOD) val-=MOD; return *this; }
    SMC& operator-=(const SMC &r) { val-=r.val; if (val<0) val+=MOD; return *this; }
    SMC& operator*=(const SMC &r) { val=(ll)((__int128_t)val*r.val%MOD); return *this; }
    SMC& operator/=(const SMC &r) { return *this*=r.inv(); }
    friend SMC operator+(SMC a,const SMC &b) { return a+=b; }
    friend SMC operator-(SMC a,const SMC &b) { return a-=b; }
    friend SMC operator*(SMC a,const SMC &b) { return a*=b; }
    friend SMC operator/(SMC a,const SMC &b) { return a/=b; }
    SMC pow(ll k) const {
        SMC res=1,a=*this;
        for (;k;k>>=1,a*=a) if(k&1) res*=a;
        return res;
    }
    SMC inv() const { return pow(MOD-2); }
    friend istream& operator>>(istream &in,SMC &a) { ll v; in>>v; a=v; return in; }
    friend ostream& operator<<(ostream &out,const SMC &a) { return out<<a.val; }
};
//1e9级别模数应该优先使用这个
/*
template <int MOD>
struct SMC {
    int val;
    SMC(ll v=0) : val(v%MOD) { if (val<0) val+=MOD; }
    SMC& operator+=(const SMC &r) { val+=r.val; if (val>=MOD) val-=MOD; return *this; }
    SMC& operator-=(const SMC &r) { val-=r.val; if (val<0) val+=MOD; return *this; }
    SMC& operator*=(const SMC &r) { val=1LL*val*r.val%MOD; return *this; }
    SMC& operator/=(const SMC &r) { return *this*=r.inv(); }
    friend SMC operator+(SMC a,const SMC &b) { return a+=b; }
    friend SMC operator-(SMC a,const SMC &b) { return a-=b; }
    friend SMC operator*(SMC a,const SMC &b) { return a*=b; }
    friend SMC operator/(SMC a,const SMC &b) { return a/=b; }
    SMC pow(ll k) const {
        SMC res=1,a=*this;
        for (;k;k>>=1,a*=a) if(k&1) res*=a;
        return res;
    }
    SMC inv() const { return pow(MOD-2); }
    friend istream& operator>>(istream &in,SMC &a) { ll v; in>>v; a=v; return in; }
    friend ostream& operator<<(ostream &out,const SMC &a) { return out<<a.val; }
};
*/
//Montgomery 模乘, 快很多
template<uint32_t MOD>
struct Mont {
    static constexpr uint32_t M_PRIME = []() {
```

```
        uint32_t x = MOD;
        for (int i = 0; i < 4; ++i) x *= 2u - MOD * x;
        return ~x + 1;
    }();
    static constexpr uint32_t R2 = []() {
        uint64_t x = 1ull << 32;
        x %= MOD;
        return (x * x) % MOD;
    }();

    uint32_t val;
    static constexpr uint32_t reduce(uint64_t T) {
        uint32_t m = uint32_t(T) * M_PRIME;
        uint32_t res = (T + (uint64_t)m * MOD) >> 32;
        return res >= MOD ? res - MOD : res;
    }
    constexpr Mont(long long x = 0) : val(reduce((uint64_t)(x % MOD + MOD) % MOD * R2)) {}
    constexpr uint32_t get() const { return reduce(val); }
    constexpr Mont& operator+=(const Mont& rhs) { val += rhs.val; if (val >= MOD) val -= MOD; return *this; }
    constexpr Mont& operator-=(const Mont& rhs) { if (val < rhs.val) val += MOD; val -= rhs.val; return *this; }
    constexpr Mont& operator*=(const Mont& rhs) {
        val = reduce((uint64_t)val * rhs.val);
        return *this;
    }
    friend constexpr Mont operator+(Mont lhs, const Mont& rhs) { return lhs += rhs; }
    friend constexpr Mont operator-(Mont lhs, const Mont& rhs) { return lhs -= rhs; }
    friend constexpr Mont operator*(Mont lhs, const Mont& rhs) { return lhs *= rhs; }
    constexpr Mont pow(uint64_t n) const {
        Mont res(1), a(*this);
        while (n) { if (n & 1) res *= a; a *= a; n >>= 1; }
        return res;
    }
    constexpr Mont inv() const { return pow(MOD - 2); }
    constexpr Mont operator/(const Mont& rhs) const { return *this * rhs.inv(); }
    friend istream& operator>>(istream &in, Mont &a) {
        long long v;
        in >> v;
        a = Mont(v);
        return in;
    }
    friend ostream& operator<<(ostream &out, const Mont &a) {
        return out << a.get();
    }
};
using Z=SMC<mod>;
class NTT{
public:
    const int G=3;
    vector<int> R; vector<Z> rt;
    //多项式的最高项数
    NTT(int len=0){
        int n=1;
        while(n<len*2) n<<=1;
        init(n);
    }
    void init(int n){
        // 998244353 = 119 * 2^23 + 1, NTT length must not exceed 2^23.
        assert(n<=(1<<23));
        if(rt.empty()) rt={0,1};
        if(rt.size()>=n) return;
        for(int i=rt.size();i<n;i<<=1){
            rt.resize(i<<1); Z w=Z(G).pow((mod-1)/(i<<1)); rt[i]=1;
            for(int j=1;j<i;j++) rt[i+j]=rt[i+j-1]*w;
        }
    }
    void ntt(vector<Z>& a,int n,int op){
        for(int i=0;i<n;i++) R[i]=(R[i>>1]>>1)|((i&1)*(n>>1));
        for(int i=0;i<n;i++) if(i<R[i]) swap(a[i],a[R[i]]);
        for(int i=2;i<=n;i<<=1)
            for(int m=i>>1,j=0;j<n;j+=i)
                for(int k=j;k<j+m;k++){
                    Z x=a[k],y=rt[m+k-j]*a[k+m];
                    a[k]=x+y,a[k+m]=x-y;
                }
        if(op==-1){
            reverse(a.begin()+1,a.end()); Z in=Z(n).inv();
            for(int i=0;i<n;i++) a[i]*=in;
        }
    }
    //仅记录ntt计算过程，直接调用可能增加常数开销
    vector<Z> calc(vector<Z> a,vector<Z> b){
        if(a.empty()||b.empty()) return {};
        int sz=a.size()+b.size()-1,len=1;
        while(len<sz) len<<=1;
        init(len); R.resize(len); a.resize(len); b.resize(len);
        ntt(a,len,1); ntt(b,len,1);
        for(int i=0;i<len;i++) a[i]*=b[i];
        ntt(a,len,-1); a.resize(sz);
        return a;
    }
};
```

用法示例:

```
    int n,m;cin>>n>>m;
    vector<Z> a(n+1),b(m+1);
    for(int i=0;i<=n;i++) cin>>a[i];
    for(int i=0;i<=m;i++) cin>>b[i];
    NTT ntt; vector<Z> c=ntt.calc(a,b);
    for(int i=0;i<=n+m;i++) cout<<c[i]<<' ';
```

6.7 乘法逆元

```
int ModQpow(int a,int b,int m)//快速幂
{
    int ans=1;
    while(b)
    {
        if(b&1) ans=ans*a%m;
        a=a*a%m;b>>=1;
    }
    return ans;
}
int invMod1(int a,int m)//a在模m意义下的逆元（费马小定理，m为质数），即a^(m-2)
{
    return ModQpow(a,m-2,m);
}
int exgcd(int a,int b,int &x,int &y)//扩展欧几里得
{
    if(b==0)
    {
        x=1;y=0;
        return a;
    }
    int d=exgcd(b,a%b,x,y),t=x;
    x=y;y=t-a/b*y;
    return d;
}
int invMod2(int a,int m)//a在模m意义下的逆元（扩展欧几里得）
{
    int x,y;
    int d=exgcd(a,m,x,y);
    assert(d==1);//逆元存在当且仅当 gcd(a,m)=1
    return (x%m+m)%m;
}
```

6.8 安全取模类(精简版)

```
const int mod=998244353;
template <int MOD>
struct SMC {
    int val;
    SMC(ll v=0) : val(v%MOD) { if (val<0) val+=MOD; }
    SMC& operator+=(const SMC &r) { val+=r.val; if (val>=MOD) val-=MOD; return *this; }
    SMC& operator-=(const SMC &r) { val-=r.val; if (val<0) val+=MOD; return *this; }
    SMC& operator*=(const SMC &r) { val=1LL*val*r.val%MOD; return *this; }
    SMC& operator/=(const SMC &r) { return *this*=r.inv(); }
    friend SMC operator+(SMC a,const SMC &b) { return a+=b; }
    friend SMC operator-(SMC a,const SMC &b) { return a-=b; }
    friend SMC operator*(SMC a,const SMC &b) { return a*=b; }
    friend SMC operator/(SMC a,const SMC &b) { return a/=b; }
    SMC pow(ll k) const {
        SMC res=1,a=*this;
        for (;k;k>>=1,a*=a) if(k&1) res*=a;
        return res;
    }
    SMC inv() const { return pow(MOD-2); }
    friend istream& operator>>(istream &in,SMC &a) { ll v; in>>v; a=v; return in; }
    friend ostream& operator<<(ostream &out,const SMC &a) { return out<<a.val; }
};
using Z=SMC<mod>;

template<typename T>
struct SZ{
    T v=1;
    int z=0;
    void mul(const T &r) { if(r.val==0) z++; else v*=r; }
    void div(const T &r) { if(r.val==0) z--; else v/=r; }
    T val() const { return z?0:v; }
};
//全局维护乘积有逆元不存在的情况 调用 SZ<Z>
```

6.9 安全取模类

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
const int mod=998244353;
template <int MOD>
struct SMC {
    int64_t val;
    constexpr SMC(int64_t v=0){
        val=(v%MOD+MOD)%MOD;
    }
    SMC& operator=(int64_t v){
```

```
        val=(v%MOD+MOD)%MOD;
        return *this;
    }
    SMC& operator+=(const SMC &rhs){
        val+=rhs.val;
        if(val>=MOD) val-=MOD;
        return *this;
    }
    SMC& operator-=(const SMC &rhs){
        val-=rhs.val;
        if(val<0) val+=MOD;
        return *this;
    }
    SMC& operator*=(const SMC &rhs){
        val=1LL*val*rhs.val%MOD;
        return *this;
    }
    static int64_t qpow(int64_t a,int64_t b){
        int64_t res=1;
        while(b){
            if(b&1) res=res*a%MOD;
            a=a*a%MOD;
            b>>=1;
        }
        return res;
    }
    SMC pow(int64_t k) const{
        return SMC(qpow(val,k));
    }
    SMC inv() const{
        return pow(MOD-2);
    }
    SMC& operator/=(const SMC &rhs){
        return *this*=rhs.inv();
    }
    friend SMC operator+(SMC a,const SMC &b){ return a+=b;}
    friend SMC operator-(SMC a,const SMC &b){ return a-=b;}
    friend SMC operator*(SMC a,const SMC &b){ return a*=b;}
    friend SMC operator/(SMC a,const SMC &b){ return a/=b;}
    SMC& operator++() { return *this += 1; }
    SMC& operator--() { return *this -= 1; }
    SMC operator++(int32_t dummy) { SMC t=*this; ++*this; return t; }
    SMC operator--(int32_t dummy) { SMC t=*this; --*this; return t; }
    friend bool operator==(const SMC &a,const SMC &b){ return a.val==b.val;}
    friend bool operator<(const SMC &a,const SMC &b){ return a.val<b.val;}
    friend bool operator>(const SMC &a,const SMC &b){ return a.val>b.val;}
    friend bool operator<=(const SMC &a,const SMC &b){ return a.val<=b.val;}
    friend bool operator>=(const SMC &a,const SMC &b){ return a.val>=b.val;}
    friend bool operator!=(const SMC &a,const SMC &b){ return a.val!=b.val;}

    friend std::istream& operator>>(std::istream &in,SMC &a){
        int64_t v;
        in>>v,a=SMC(v);
        return in;
    }

    friend std::ostream& operator<<(std::ostream &out,const SMC &a){
        out<<a.val;
        return out;
    }
    explicit operator long long() const{
        return val;
    }
    SMC operator-() const{
        return SMC(-val);
    }
    SMC& operator+=(int64_t x) { return *this+=SMC(x); }
    SMC& operator-=(int64_t x) { return *this-=SMC(x); }
    SMC& operator*=(int64_t x) { return *this*=SMC(x); }
    SMC& operator/=(int64_t x) { return *this/=SMC(x); }

    friend SMC operator+(SMC a, int64_t b) { return a+=b; }
    friend SMC operator-(SMC a, int64_t b) { return a-=b; }
    friend SMC operator*(SMC a, int64_t b) { return a*=b; }
    friend SMC operator/(SMC a, int64_t b) { return a/=b; }

    friend SMC operator+(int64_t a, SMC b) { return b+a; }
    friend SMC operator-(int64_t a, SMC b) { return SMC(a)-b; }
    friend SMC operator*(int64_t a, SMC b) { return b*a; }
    friend SMC operator/(int64_t a, SMC b) { return SMC(a)/b; }
};
using Z=SMC<mod>;
```

6.10 拉格朗日插值

```
const int mod=1e9+7;
template <int MOD>
struct SMC {
    int val;
    SMC(ll v=0) : val(v%MOD) { if (val<0) val+=MOD; }
    SMC& operator+=(const SMC &r) { val+=r.val; if (val>=MOD) val-=MOD; return *this; }
    SMC& operator-=(const SMC &r) { val-=r.val; if (val<0) val+=MOD; return *this; }
    SMC& operator*=(const SMC &r) { val=1LL*val*r.val%MOD; return *this; }
    SMC& operator/=(const SMC &r) { return *this*=r.inv(); }
```

```
    friend SMC operator+(SMC a,const SMC &b) { return a+=b; }
    friend SMC operator-(SMC a,const SMC &b) { return a-=b; }
    friend SMC operator*(SMC a,const SMC &b) { return a*=b; }
    friend SMC operator/(SMC a,const SMC &b) { return a/=b; }
    SMC pow(ll k) const {
        SMC res=1,a=*this;
        for (;k;k>>=1,a*=a) if(k&1) res*=a;
        return res;
    }
    SMC inv() const { return pow(MOD-2); }
    friend istream& operator>>(istream &in,SMC &a) { ll v; in>>v; a=v; return in; }
    friend ostream& operator<<(ostream &out,const SMC &a) { return out<<a.val; }
};
using Z=SMC<mod>;
//给定n个x不相同的点，求次数<=n-1,过这些点的函数f(x),求f(k)
//$$P(x)=\sum_{i=1}^n y_i \prod_{j \neq i} \frac{x-x_j}{x_i-x_j}$$
//O(n^2)
Z clac(vector<array<int,2>> fun,int n,int k){
    Z res=0;
    for(int i=1;i<=n;i++){
        Z now=fun[i][1];
        for(int j=1;j<=n;j++){
            if(i==j) continue;
            now*=Z(k-fun[j][0])/(Z(fun[i][0]-fun[j][0]));
        }
        res+=now;
    }
    return res;
};
//连续点值优化：当xi是连续的，例如1,2,3,...n
//可以用前缀积和后缀积来优化
//$$P(x)=\sum_{i=1}^n y_i \frac{pre_{i-1} \cdot suf_{i+1}}{(i-1)! (n-i)! (-1)^{n-i}}$$
//$$pre_i=\prod_{j=1}^i(x-j)$$,$$suf_i=\prod_{j=i}^n(x-j)$$
//O(n),
Z clac2(vector<array<Z,2>> fun,int n,int k){
    vector<Z> pre(n+1,1),suf(n+2,1),inv(n+1,1),invfac(n+1,1);
    for(int i=1;i<=n;i++) pre[i]=pre[i-1]*Z(k-i);
    for(int i=n;i>=1;i--) suf[i]=suf[i+1]*Z(k-i);
    for(int i=2;i<=n;i++) inv[i]=(mod-mod/i)*inv[mod%i];
    for(int i=1;i<=n;i++) invfac[i]=invfac[i-1]*inv[i];
    Z res=0;
    for(int i=1;i<=n;i++){
        res+=(((n-i)&1)?-1:1)*fun[i][1]*pre[i-1]*suf[i+1]*invfac[i-1]*invfac[n-i];
    }
    return res;
}
```

用法示例:

```
    int n,k;cin>>n>>k;
    vector<array<Z,2>> fun(k+3);
    fun[0]={0,0};
    for(int i=1;i<=k+2;i++){
        fun[i]={i,Z(i).pow(k)+fun[i-1][1]};
    }
    cout<<clac2(fun,k+2,n)<<endl;
```

6.11 数论预处理

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class Pre{
public:
    int n,tag;
    const int mod=1e9+7;
    vector<int> inv,fac,invfac;
    Pre(int n):n(n){}
    Pre(int n,int tag):n(n),tag(tag){
        inv.resize(n+1);
        fac.resize(n+1);
        invfac.resize(n+1);
        preC();
    }
    int ModQpow(int a,int b,int m)//快速幂
    {
        int ans=1;
        while(b)
        {
            if(b&1)ans=ans*a%m;
            a=a*a%m,b>>=1;
        }
        return ans;
    }
    //O(nlnn)求1-n所有数的约数
    vector<vector<int>> divs()
    {
        vector<vector<int>> ans(n+1);
        for(int i=1;i<=n;i++)
        {
            for(int j=i;j<=n;j+=i)
            {
                ans[j].push_back(i);
            }
```

```
    }
    return ans;
}
//O(n)求1-n所有的质数
vector<int> primes()
{
    vector<int> primes;
    vector<bool>v(n+1,0);
    for(int i=2;i<=n;i++)
    {
        if(!v[i])primes.push_back(i);
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            v[primes[j]*i]=1;
            if(i%primes[j]==0)break;
        }
    }
    return primes;
}
//O(n)求0-n的阶乘和阶乘逆元
void preC()
{
    assert(n<mod);//逆元递推要求 mod 为质数且 1..n 均可逆
    inv[1]=1;
    for(int i=2;i<=n;i++)
    {
        inv[i]=(mod-mod/i)*inv[mod%i]%mod;
    }
    fac[0]=invfac[0]=1;
    for(int i=1;i<=n;i++)
    {
        fac[i]=fac[i-1]*i%mod;
        invfac[i]=invfac[i-1]*inv[i]%mod;
    }
}
//组合数C(n,m) n个数中选m个
int C(int n,int m)
{
    if(n<0||m<0||n<m)return 0;
    return fac[n]*invfac[m]%mod*invfac[n-m]%mod;
}
//O(n)求1-n的欧拉函数
vector<int> euler()
{
    vector<int> phi(n+1);
    phi[1]=1;
    vector<int> primes;
    vector<bool>v(n+1,0);
    for(int i=2;i<=n;i++)
    {
        if(!v[i]) primes.push_back(i),phi[i]=i-1;
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            int m=primes[j]*i;
            v[m]=1;
            if(i%primes[j]==0)
            {
                phi[m]=phi[i]*primes[j];
                break;
            }
            else phi[m]=phi[i]*(primes[j]-1);
        }
    }
    return phi;
}
//O(n)求1-n的约数个数
vector<int> d()
{
    vector<int> a(n+1),d(n+1);
    vector<int> primes;
    vector<bool>v(n+1,0);
    d[1]=1;
    for(int i=2;i<=n;i++)
    {
        if(!v[i])
        {
            primes.push_back(i);
            a[i]=1,d[i]=2;
        }
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            int m=primes[j]*i;
            v[m]=1;
            if(i%primes[j]==0)
            {
                a[m]=a[i]+1;
                d[m]=d[i]/(a[i]+1)*(a[m]+1);
                break;
            }
            else
            {
                a[m]=1;
                d[m]=d[i]*2;
```

```
                }
            }
        }
        return d;
    }
    //O(n)求1-n的约数和
    vector<int> sumd()
    {
        vector<int> g(n+1),f(n+1);
        vector<int> primes;
        vector<bool>v(n+1,0);
        g[1]=f[1]=1;
        for(int i=2;i<=n;i++)
        {
            if(!v[i])
            {
                primes.push_back(i);
                f[i]=g[i]=i+1;
            }
            for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
            {
                int m=primes[j]*i;
                v[m]=1;
                if(i%primes[j]==0)
                {
                    g[m]=g[i]*primes[j]+1;
                    f[m]=f[i]*g[m]/g[i];
                    break;
                }
                else
                {
                    g[m]=primes[j]+1;
                    f[m]=f[i]*g[m];
                }
            }
        }
        return f;
    }
    //O(n)求1-n的莫比乌斯函数
    vector<int> mu()
    {
        vector<int> mu(n+1);
        mu[1]=1;
        vector<int> primes;
        vector<bool>v(n+1,0);
        for(int i=2;i<=n;i++)
        {
            if(!v[i])primes.push_back(i),mu[i]=-1;
            for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
            {
                int m=primes[j]*i;
                v[m]=1;
                if(i%primes[j]==0)
                {
                    mu[m]=0;
                    break;
                }
                else mu[m]=-mu[i];
            }
        }
        return mu;
    }
    //O(n*w(n)) 求1-n的不重质因子/质因数分解
    vector<vector<int>> pri(int mulble){
        vector<int> primes;
        vector<bool>v(n+1,0);
        vector<vector<int>> ans(n+1);
        for(int i=2;i<=n;i++)
        {
            if(!v[i])
            {
                primes.push_back(i);
                ans[i].push_back(i);
            }
            for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
            {
                int m=primes[j]*i;
                v[m]=1;
                if(i%primes[j]==0)
                {
                    ans[m]=ans[i];
                    if(mulble==1) ans[m].push_back(primes[j]);
                    break;
                }
                else ans[m]=ans[i],ans[m].push_back(primes[j]);
            }
        }
        return ans;
    }
    //一个常数更小的写法是求出i的最小质因数，然后递归的查，见原根的实现。
};
```

6.12 整除分块

```
#define int long long
using namespace std;
struct blocknode{
    int l;
    int r;
    int val;
};
//对[l,r]的i,floor(n/i)相等
//n%i=n-i*floor(n/i)
//首项n-l*val 公差-val 项数r-l+1
class divb{
public:
    struct node{
        int l,r;
        int val1,val2;
    };
    int s,e;
    vector<node> a;
    divb(int s,int e):s(s),e(e){
        assert(s>=1);
    }
    void b1(int n,int m){
        int lim=min(e,min(n,m));
        for(int l=s,r;l<=lim;l=r+1)
        {
            r=min(n/(n/l),m/(m/l));
            a.push_back({l,r,n/l,m/l});
        }
    }
    void b2(int n)
    {
        int lim=min(e,n);
        for(int l=s,r;l<=lim;l=r+1)
        {
            r=n/(n/l);
            a.push_back({l,r,n/l,0});
        }
    }
};
```

用法示例:

```
    int n,k;cin>>n>>k;
    vector<blocknode>a;
    for(int l=1,r;l<=n;l=r+1)
    {
        blocknode tp;
        r=min(n/(n/l),n);
        tp.l=l;tp.r=r;
        tp.val=n/l;
        a.push_back(tp);
    }
    //for(auto i:a)
    //{
        //cout<<i.l<<' '<<i.r<<' '<<i.val<<endl;
    //}
```

6.13 矩阵快速幂(新版)

```
template<class T>
//0.普通矩阵乘
struct StdOp{
    static T add(const T& a,const T& b) { return a+b; }
    static T mul(const T& a,const T& b) { return a*b; }
    static T zero() { return T(0); }
    static T one() { return T(1); }
};

//1.1 Min-Plus 半环 (最短路)
template<class T>
struct MinPlusOp {
    //除以 2 是为了防止发生 INF + INF 的溢出
    static const T INF=numeric_limits<T>::max()/2;
    static T add(const T&a,const T&b) { return min(a,b); }
    static T mul(const T&a,const T&b) { return a+b; }
    static T zero() { return INF; }
    static T one() { return T(0); }
};

//1.2 Max-Plus 半环 (最长路)
template<class T>
struct MaxPlusOp {
    static const T INF=numeric_limits<T>::lowest()/2;
    static T add(const T&a,const T&b) { return max(a,b); }
    static T mul(const T&a,const T&b) { return a+b; }
    static T zero() { return INF; }
    static T one() { return T(0); }
};

//2.1 Max-Min 半环 (最大瓶颈路)
template<class T>
```

```
struct MaxMinOp{
    static T add(const T&a,const T&b) { return max(a,b); }
    static T mul(const T&a,const T&b) { return min(a,b); }
    static T zero() { return numeric_limits<T>::lowest(); }
    static T one() { return numeric_limits<T>::max(); }
};

//2.2 Min-Max 半环 (最小瓶颈路)
template<class T>
struct MinMaxOp {
    static T add(const T&a,const T&b) { return min(a,b); }
    static T mul(const T&a,const T&b) { return max(a,b); }
    static T zero() { return numeric_limits<T>::max(); }
    static T one() { return numeric_limits<T>::lowest(); }
};

//3. 最短路-计数半环
template<class T,class C>
struct MinPlusCountOp {
    using P=pair<T,C>;
    static const T INF=numeric_limits<T>::max()/2;
    static P add(const P&a, const P&b) {
        if (a.first<b.first) return a;
        if (b.first<a.first) return b;
        return {a.first,a.second+b.second};
    }
    static P mul(const P&a, const P&b) {
        return {a.first+b.first,a.second*b.second};
    }
    static P zero() { return {INF,C(0)}; }
    static P one() { return {0,C(1)}; }
};

template<class T,class Op>
struct Mat
{
    int n,m;
    vector<T> a;
    Mat(int n=0,int m=0):n(n),m(m),a(n*m,Op::zero()) {}
    Mat(int n,int m,const T& val):n(n),m(m),a(n*m,val) {}
    inline T* operator[](int i) { return a.data()+i*m; }
    inline const T* operator[](int i) const { return a.data()+i*m; }
    Mat operator*(const Mat& r) const{
        assert(m==r.n);
        Mat res(n,r.m);
        for(int i=0;i<n;i++){
            for(int k=0;k<m;k++){
                T tmp=(*this)[i][k];
                if(tmp==Op::zero()) continue;
                for(int j=0;j<r.m;j++){
                    //跳过右侧零元, 避免 INF/-INF 参与 mul 出现溢出风险
                    if(r[k][j]==Op::zero()) continue;
                    res[i][j]=Op::add(res[i][j],Op::mul(tmp,r[k][j]));
                }
            }
        }
        return res;
    };
    static Mat I(int _n){
        Mat res(_n,_n);
        for(int i=0;i<_n;i++) res[i][i]=Op::one();
        return res;
    }
    Mat pow(ll k) const{
        assert(n==m);
        Mat res=I(n),x=*this;
        for(;k;k>>=1,x=x*x) if(k&1) res=res*x;
        return res;
    }
};
```

用法示例:

```
    int n,m;cin>>n>>m;
    Mat<int,MinMaxOp<int>> a(n,n);
    for(int i=1;i<=m;i++){
        int u,v;cin>>u>>v;
        u--,v--;
        a[u][v]=0;
    }
    int k;cin>>k;
    for(int i=1;i<=k;i++){
        int u,v;cin>>u>>v;
        u--,v--;
        a[u][v]=min(a[u][v],i);
    }
    int q,w;cin>>q>>w;
    auto res=a.pow(w);
    while(q--){
        int u,v;cin>>u>>v;
        u--,v--;
        if(res[u][v]==numeric_limits<int>::max()) cout<<"-1\n";
        else cout<<res[u][v]<<"\n";
    }
```

6.14 矩阵快速幂(旧板)

```
//矩阵快速幂：处理快速形式变换
//时间复杂度：O(n^3logk)
```

```
const int mod=1e9+7;
template <int MOD>
struct SMC {
    int val;
    SMC(ll v=0) : val(v%MOD) { if (val<0) val+=MOD; }
    SMC& operator+=(const SMC &r) { val+=r.val; if (val>=MOD) val-=MOD; return *this; }
    SMC& operator-=(const SMC &r) { val-=r.val; if (val<0) val+=MOD; return *this; }
    SMC& operator*=(const SMC &r) { val=1LL*val*r.val%MOD; return *this; }
    SMC& operator/=(const SMC &r) { return *this*=r.inv(); }
    friend SMC operator+(SMC a,const SMC &b) { return a+=b; }
    friend SMC operator-(SMC a,const SMC &b) { return a-=b; }
    friend SMC operator*(SMC a,const SMC &b) { return a*=b; }
    friend SMC operator/(SMC a,const SMC &b) { return a/=b; }
    SMC pow(ll k) const {
        SMC res=1,a=*this;
        for (;k;k>>=1,a*=a) if(k&1) res*=a;
        return res;
    }
    SMC inv() const { return pow(MOD-2); }
    friend istream& operator>>(istream &in,SMC &a) { ll v; in>>v; a=v; return in; }
    friend ostream& operator<<(ostream &out,const SMC &a) { return out<<a.val; }
};
using Z=SMC<mod>;
class MatQpow{
    public:
    int n;
    vector<vector<Z>> mat,e;
    MatQpow(int n,vector<vector<Z>> mat):n(n),mat(mat){
        e=vector<vector<Z>>(n,vector<Z>(n,0));
        for(int i=0;i<n;i++) e[i][i]=1;
    }
    static vector<vector<Z>> mul(
        const vector<vector<Z>>& A,
        const vector<vector<Z>>& B,
        int n
    ){
        vector<vector<Z>> C(n,vector<Z>(n,0));
        for(int i=0;i<n;i++)
            for(int k=0;k<n;k++)
            {
                if(!A[i][k].val) continue;
                for(int j=0;j<n;j++)
                {
                    C[i][j]+=A[i][k]*B[k][j];
                }
            }
        return C;
    }
    vector<vector<Z>> qpow(ll k) const{
        vector<vector<Z>> res=e;
        vector<vector<Z>> base=mat;
        while(k){
            if(k&1) res=mul(res,base,n);
            base=mul(base,base,n);
            k>>=1;
        }
        return res;
    }

};
```

用法示例:

```
    int n;ll k;cin>>n>>k;
    vector<vector<Z>> mat(n,vector<Z>(n,0));
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            cin>>mat[i][j];
        }
    }
    MatQpow m(n,mat);
    auto res=m.qpow(k);
    Z ans=0;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            ans+=res[i][j];
        }
    }
    cout<<ans<<endl;
```

6.15 筛法求积性函数

```
//积性函数线性筛骨架：这里只返回 primes, f 的递推需按题目补全
vector<int> primes(int n)
{
    //积性函数gcd(a,b)=1,f(ab)=f(a)f(b)
    //当f(p^k),p为质数时时的函数值可以快速求出，
    //即可以通过递推求出所有积性函数
```

```
    vector<int> primes;
    vector<bool>v(n+1,0);
    for(int i=2;i<=n;i++)
    {
        if(!v[i])
        {
            primes.push_back(i);
            //考虑f(p)=...
            //单个质数的情况
        }
        for(int j=0;j<primes.size()&&primes[j]*i<=n;j++)
        {
            v[primes[j]*i]=1;
            int m=primes[j]*i;
            if(i%primes[j]==0)
            {
                //此时p[j]是m的最小质因子,运用反证法p[j]也是m的最小质因子
                //考虑f(m)=...
                //多个质因子的情况
                break;
            }
            else{
                //此时gcd(i,pj)=1,f(m)=f(i)f(pj)
                //新增质因子的情况
            }
        }
    }
    return primes;
}
```

6.16 线性基(gauss)

```
//异或线性基 O(bit*n)
//异或线性基是原数列的一个基向量,
//意味这基向量的线性组合能构造出原数列的任意数
//意味着原数列线性组合构造出的数和线性基线性组合构造出的数是一样的
//xor=mod 2+/GF(2)域
//guass消元法给出的线性基是行最简式
//即形如
//01001
//00100
//00011
//00000
//满足三个性质:
//1.线性基中任意两个基向量的异或结果不会是0
//2.线性基每一个基向量的高位1在别的基向量中都是0
//3.基向量是从大到小存储的
```

```
class basic{
    public:
    vector<int> num,bas;
    int bit,cnt,n;
    basic(vector<int> a,int bit):
    num(a),bit(bit),cnt(0),n(a.size()){
        gauss();
        //usually bit=30/60
        //注意数据范围
    }
    void gauss(){
        for(int i=bit-1;i>=0;i--){
            //把当前第i位是1的数换上去
            for(int j=cnt;j<n;j++){
                if(num[j]>>i&1){
                    swap(num[j],num[cnt]);
                    break;
                }
            }
            //如果这一位全0, 跳过
            if((num[cnt]>>i&1)==0) continue;
            //消去其他数第i位1
            for(int j=0;j<n;j++){
                if(j!=cnt&&(num[j]>>i&1))
                    num[j]^=num[cnt];
            }
            cnt++;
            if(cnt==n) break;
        }
        bas.assign(num.begin(),num.begin()+cnt);
    }
    //求第k小的数 k:1base
    int kth(ll k){
        //k个基向量能构造出2^k-1个数
        //case1 :cnt<n 意味这能构造出 0 所以能构造2^k个数
        //case2 :cnt=n 意味这不能构造出 0 所以只能构造2^k-1个数
        if(cnt<n) k--;
        if(k>=(1ull<<cnt)) return -1;
        int ans=0;
        for(int i=0;i<cnt;i++){
```

```
            if(k>>i&1) ans^=bas[cnt-1-i];
        }
        return ans;
    }
    //求不去重的所有非空子序列异或和的第k小 k:1base
    int kth_dup(ll k){
        if(n<=60&&k>=(1ull<<n)) return -1;
        int z=n-cnt;
        ll rk=z>60?0:k>>z;
        int ans=0;
        for(int i=0;i<cnt;i++){
            if(rk>>i&1) ans^=bas[cnt-1-i];
        }
        return ans;
    }
    //求一个数用一个数列异或得到的方案数
    //约简为0的向量是不必要的 于是可以任选
    ll count(int x){
        for(auto b:bas){
            if((x^b)<x) x^=b;
        }
        return x==0?(1ll<<(n-cnt)):0;
    }
    //求一个数在数列xor和中的排名
    ll rk(int x){
        int tp=x;
        for(auto b:bas){
            if((tp^b)<tp) tp^=b;
        }
        if(tp) return -1;
        int id=0;
        for(int i=0;i<cnt;i++){
            if((x^bas[i])<x)
            {
                id|=(1ll<<(cnt-1-i));
                x^=bas[i];
            }
        }
        if(cnt<n) id++;
        return id;
    }
    //求一个数在xor线性基的数中得到最小值
    int getmin(int x){
        for(auto b:bas){
            if((x^b)<x) x=x^b;
        }
        return x;
    }
};
```

用法示例:

```
    int n;cin>>n;
    vector<int> a(n);
    for(int i=0;i<n;i++) cin>>a[i];
    basic b(a,64);
    int ans=0;
    for(auto i:b.bas) ans^=i;
    cout<<ans<<endl;
```

6.17 线性基(贪心法)

```
//贪心法构造的线性基:
//按照元素顺序构造, 适用于依赖元素顺序的题
```

```
#define int long long //赫赫 要不要龙龙呢
using namespace std;
class basic{
    public:
    using u64=uint64_t;
    struct node
    {
        u64 val;
        int inf;
    };
    vector<node> bas; int bit,tot;
    basic(int bit=64):bas(bit,{0,0}),bit(bit),tot(0){
        assert(1<=bit&&bit<=64);
    }
    bool ins(node x)
    {
        for(int i=bit-1;i>=0;i--)
        {
            if(x.val>>i&1)
            {
                if(bas[i].val==0)
                {
                    bas[i]=x;
                    tot++;
                    return true;
                }
                else x.val^=bas[i].val;
```

```
            }
        }
        return false;
    }
};
```

用法示例:

```
    int n;cin>>n;
    vector<basic::node> a(n);
    for(auto& [x,y]:a) cin>>x>>y;
    sort(a.begin(),a.end(),[](auto& x,auto& y){return x.inf>y.inf;});
    basic b(64);
    int ans=0;
    for(auto& [x,y]:a)
    {
        if(b.ins({x,y})) ans+=y;
    }
    cout<<ans<<endl;
```

6.18 组合数预处理

```
const int mod=1e9+7;
template <int MOD>
struct SMC {
    int64_t val;
    constexpr SMC(int64_t v=0){
        val=(v%MOD+MOD)%MOD;
    }
    SMC& operator=(int64_t v){
        val=(v%MOD+MOD)%MOD;
        return *this;
    }
    SMC& operator+=(const SMC &rhs){
        val+=rhs.val;
        if(val>=MOD) val-=MOD;
        return *this;
    }
    SMC& operator-=(const SMC &rhs){
        val-=rhs.val;
        if(val<0) val+=MOD;
        return *this;
    }
    SMC& operator*=(const SMC &rhs){
        val=1LL*val*rhs.val%MOD;
        return *this;
    }
    static int64_t qpow(int64_t a,int64_t b){
        int64_t res=1;
        while(b){
            if(b&1) res=res*a%MOD;
            a=a*a%MOD;
            b>>=1;
        }
        return res;
    }
    SMC pow(int64_t k) const{
        return SMC(qpow(val,k));
    }
    SMC inv() const{
        return pow(MOD-2);
    }
    SMC& operator/=(const SMC &rhs){
        return *this*=rhs.inv();
    }
    friend SMC operator+(SMC a,const SMC &b){ return a+=b;}
    friend SMC operator-(SMC a,const SMC &b){ return a-=b;}
    friend SMC operator*(SMC a,const SMC &b){ return a*=b;}
    friend SMC operator/(SMC a,const SMC &b){ return a/=b;}
    SMC& operator++() { return *this += 1; }
    SMC& operator--() { return *this -= 1; }
    SMC operator++(int32_t dummy) { SMC t=*this; ++*this; return t; }
    SMC operator--(int32_t dummy) { SMC t=*this; --*this; return t; }
    friend bool operator==(const SMC &a,const SMC &b){ return a.val==b.val;}
    friend bool operator<(const SMC &a,const SMC &b){ return a.val<b.val;}
    friend bool operator>(const SMC &a,const SMC &b){ return a.val>b.val;}
    friend bool operator<=(const SMC &a,const SMC &b){ return a.val<=b.val;}
    friend bool operator>=(const SMC &a,const SMC &b){ return a.val>=b.val;}
    friend bool operator!=(const SMC &a,const SMC &b){ return a.val!=b.val;}

    friend std::istream& operator>>(std::istream &in,SMC &a){
        int64_t v;
        in>>v,a=SMC(v);
        return in;
    }

    friend std::ostream& operator<<(std::ostream &out,const SMC &a){
        out<<a.val;
        return out;
    }
    explicit operator long long() const{
        return val;
    }
    SMC operator-() const{
```

```
        return SMC(-val);
    }
    SMC& operator+=(int64_t x) { return *this+=SMC(x); }
    SMC& operator-=(int64_t x) { return *this-=SMC(x); }
    SMC& operator*=(int64_t x) { return *this*=SMC(x); }
    SMC& operator/=(int64_t x) { return *this/=SMC(x); }

    friend SMC operator+(SMC a, int64_t b) { return a+=b; }
    friend SMC operator-(SMC a, int64_t b) { return a-=b; }
    friend SMC operator*(SMC a, int64_t b) { return a*=b; }
    friend SMC operator/(SMC a, int64_t b) { return a/=b; }

    friend SMC operator+(int64_t a, SMC b) { return b+a; }
    friend SMC operator-(int64_t a, SMC b) { return SMC(a)-b; }
    friend SMC operator*(int64_t a, SMC b) { return b*a; }
    friend SMC operator/(int64_t a, SMC b) { return SMC(a)/b; }
};
using Z=SMC<mod>;
class Pre{
    public:
    int n,m;
    vector<vector<Z>> s,s2;
    vector<Z> inv,fac,invfac,d;
    //c组合数,s第一类斯特林数
    Pre(int n,int m):n(n),m(m){
        preS();
        preC();
        preD();
        preS2();
        preH();
    }
    void preS(){
        s.resize(n+1,vector<Z>(m+1));
        s[0][0]=1;
        for(int i=1;i<=n;i++){
            s[i][0]=0;
            if(i<=m) s[i][i]=1;
            for(int j=1;j<=min(m,i);j++){
                s[i][j]=s[i-1][j]*(i-1)+s[i-1][j-1];
            }
        }
    }
    void preC()
    {
        assert(n<mod);//逆元递推要求 mod 为质数且 1..n 均可逆
        fac.resize(n+1);
        invfac.resize(n+1);
        inv.resize(n+1);
        inv[1]=1;
        for(int i=2;i<=n;i++)
        {
            inv[i]=(mod-mod/i)*inv[mod%i];
        }
        fac[0]=invfac[0]=1;
        for(int i=1;i<=n;i++)
        {
            fac[i]=fac[i-1]*i;
            invfac[i]=invfac[i-1]*inv[i];
        }
    }
    void preD(){
        d.resize(n+1);
        d[0]=1;
        if(n>=1) d[1]=0;
        if(n>=2) d[2]=1;
        for(int i=3;i<=n;i++){
            d[i]=(i-1)*(d[i-1]+d[i-2]);
        }
    }
    void preS2(){
        s2.resize(n+1,vector<Z>(m+1));
        s2[0][0]=1;
        for(int i=1;i<=n;i++){
            s2[i][0]=0;
            if(i<=m) s2[i][i]=1;
            for(int j=1;j<=min(m,i);j++){
                s2[i][j]=s2[i-1][j]*j+s2[i-1][j-1];
            }
        }
    }
    void preH(){
        fac.resize(2*n+1);
        invfac.resize(2*n+1);
        inv.resize(2*n+1);
        inv[1]=1;
        for(int i=2;i<=2*n;i++)
        {
            inv[i]=(mod-mod/i)*inv[mod%i];
        }
        fac[0]=invfac[0]=1;
        for(int i=1;i<=2*n;i++)
        {
            fac[i]=fac[i-1]*i;
            invfac[i]=invfac[i-1]*inv[i];
```

```
        }
    }
    //第一类斯特林数S(n,m) n个不同元素划分为m个非空圆排列的方案数
    Z S(int i,int j){
        return s[i][j];
    }
    //第二类斯特林数S2(n,m) n个不同元素划分为m个非空子集的方案数
    Z S2(int i,int j){
        return s2[i][j];
    }
    //排列数 A(n,m) n个数中选m个的排列
    Z A(int n,int m){
        if(n<0||m<0||n<m)return 0;
        return fac[n]*invfac[n-m];
    }
    //组合数C(n,m) n个数中选m个
    Z C(int n,int m)
    {
        if(n<0||m<0||n<m)return 0;
        return fac[n]*invfac[m]*invfac[n-m];
    }
    //圆排列数 Q(n,m) n个数中选m个，m个数的圆排列
    //Q(n,n)=(n-1)!,n个数的圆排列
    Z Q(int n,int m)
    {
        if(n<0||m<0||n<m)return 0;
        return fac[n]*invfac[n-m]*inv[m];
    }
    //错位排列数 D(n,m) n个数中选m个，m个数的错位排列
    //D(n,n)=d[n],n个数的错位排列
    Z D(int n,int m)
    {
        if(n<0||m<0||n<m)return 0;
        return d[m]*C(n,m);
    }
    //卡特兰数 H(n) n个节点的二叉树个数
    //H(n)=C(2n,n)/(n+1)
    Z H(int n){
        return C(2*n,n)*inv[n+1];
    }
};
```

7 计算几何(全整数版)

7.1 三分

```cpp
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;
using ld=long double;

//在闭整数区间上寻找单峰函数最优点；默认求最小值，求最大值传greater<>。
//时间复杂度：O((T_f+T_b+1) log(r-l+2))；T_f、T_b 分别为单次 f、better 调用耗时。
template<class F,class Better=less<>>
ll ternaryInt(ll l,ll r,F f,Better better={}){
    assert(l<=r);
    while((i128)r-l>6){
        ll d=((i128)r-l)/3;
        ll m1=l+d,m2=r-d;
        auto v1=f(m1),v2=f(m2);
        if(better(v2,v1))l=m1+1;
        else r=m2-1;
    }
    ll ans=l;
    auto val=f(l);
    for(ll x=l;;x++){
        auto cur=f(x);
        if(better(cur,val))ans=x,val=cur;
        if(x==r)break;
    }
    return ans;
}

//在实数区间上寻找连续单峰函数最优点；固定轮数规避eps导致的死循环。
//时间复杂度：O(1+rounds*(T_f+T_b+1))；T_f、T_b 分别为单次 f、better 调用耗时。
template<class F,class Better=less<>>
ld ternaryReal(ld l,ld r,F f,int rounds=200,Better better={}){
    assert(l<=r&&rounds>=0);
    while(rounds--){
        ld m1=l+(r-l)/3,m2=r-(r-l)/3;
        auto v1=f(m1),v2=f(m2);
        if(better(v2,v1))l=m1;
        else r=m2;
    }
    return (l+r)/2;
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.2 三角剖分

```cpp
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;
using ld=long double;
const ld eps=1e-18L;

//整数向量保存平移到圆心后的多边形端点，并精确完成叉积和点积。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return {x+o.x,y+o.y};}
    vec operator-(const vec& o)const{return {x-o.x,y-o.y};}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点保存圆心和原多边形坐标。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
};
//浮点向量仅用于线段与圆交点之间的面积积分。
struct dvec{
    ld x,y;
    //时间复杂度：O(1)。
    dvec(ld x=0,ld y=0):x(x),y(y){}
};

//返回整数向量长度平方，端点是否在圆内用它精确判断。
//时间复杂度：O(1)。
i128 len2(const vec& a){return a&a;}
//返回两个浮点向量的叉积，计算分段三角形面积时使用。
//时间复杂度：O(1)。
ld crossVal(const dvec& a,const dvec& b){return a.x*b.y-a.y*b.x;}
```

```
//返回两个浮点向量的点积，计算有向圆心角时使用。
//时间复杂度：O(1)。
ld dotVal(const dvec& a,const dvec& b){return a.x*b.x+a.y*b.y;}
//返回圆心到两向量的有向扇形面积，角度由atan2稳定计算。
//时间复杂度：O(1)。
ld sector(const dvec& a,const dvec& b,ll r){return (ld)r*r*atan2l(crossVal(a,b),dotVal(a,b))/2;}
//求线段a->b与圆的内部交点参数，返回值包含0和1并保持递增。
//时间复杂度：O(1)；参数至多 4 个，排序和去重规模均为常数。
vector<ld> cutParams(const vec& a,const vec& b,ll r){
    vec d=b-a;
    i128 A=d&d;
    vector<ld> ts={0,1};
    if(A==0)return ts;
    i128 ad=a&d,cr=a*d;
    i128 det=(i128)r*r*A-cr*cr;
    if(det>0){
        ld mid=-(ld)ad/(ld)A,off=sqrtl((ld)det)/(ld)A;
        ld t1=mid-off,t2=mid+off;
        if(t1>eps&&t1<1-eps)ts.push_back(t1);
        if(t2>eps&&t2<1-eps)ts.push_back(t2);
    }
    sort(ts.begin(),ts.end());
    //时间复杂度：lambda 单次比较 O(1)，常数个参数的整次去重也是 O(1)。
    ts.erase(unique(ts.begin(),ts.end(),[](ld x,ld y){return fabsl(x-y)<=eps;}),ts.end());
    return ts;
}
//计算有向三角形Oab与圆的面积交贡献，直线交点和扇形角处才使用浮点数。
//时间复杂度：O(1)；线段与圆至多有两个交点，只需处理常数个子区间。
ld getS(const vec& a,const vec& b,ll r){
    if(r==0||(a.x==b.x&&a.y==b.y))return 0;
    i128 rr=(i128)r*r;
    if(len2(a)<=rr&&len2(b)<=rr)return (ld)(a*b)/2;
    vec d=b-a;
    vector<ld> ts=cutParams(a,b,r);
    ld ans=0;
    for(int i=0;i+1<(int)ts.size();i++){
        ld l=ts[i],h=ts[i+1],m=(l+h)/2;
        dvec u={a.x+d.x*l,a.y+d.y*l};
        dvec v={a.x+d.x*h,a.y+d.y*h};
        ld mx=a.x+d.x*m,my=a.y+d.y*m;
        if(mx*mx+my*my<=(ld)rr+eps)ans+=crossVal(u,v)/2;
        else ans+=sector(u,v,r);
    }
    return ans;
}
//三角剖分累加简单多边形与整数圆的面积交，顶点顺/逆时针均可。
//时间复杂度：O(n)；n 为多边形顶点数。
ld S(const pit& o,ll r,const vector<pit>& p){
    assert(r>=0);
    if(p.size()<3||r==0)return 0;
    ld ans=0;
    for(int i=0;i<(int)p.size();i++)ans+=getS(p[i]-o,p[(i+1)%p.size()]-o,r);
    return fabsl(ans);
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.3 三角剖分 2

```
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;

//整数向量为耳切法提供精确叉积和点积。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点保存简单多边形及剖分得到的三角形顶点。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
    bool operator==(const pit& o)const{return x==o.x&&y==o.y;}
};

//返回三点有向面积的两倍，耳朵判定全程使用该精确值。
//时间复杂度：O(1)。
i128 crossVal(const pit& a,const pit& b,const pit& c){return (b-a)*(c-a);}
//返回多边形有向面积的两倍，用于统一输入方向。
//时间复杂度：O(n)；n 为多边形顶点数。
i128 area2(const vector<pit>& p){
    i128 res=0;
```

```
    for(int i=0;i<(int)p.size();i++)res+=(i128)p[i].x*p[(i+1)%p.size()].y-(i128)p[i].y*p[(i+1)%p.size()].x;
    return res;
}
//判断点p是否位于闭线段ab上，用于清理多边形边上的冗余点。
//时间复杂度：O(1)。
bool onSeg(const pit& a,const pit& b,const pit& p){return crossVal(a,b,p)==0&&((a-p)&(b-p))<=0;}
//去掉连续重复点和共线中间点，并把有效简单多边形调整为逆时针。
//时间复杂度：最坏 O(n^2)；n 为输入顶点数，每轮扫描后至少删除一个点或结束。
vector<pit> simplifyPolygon(vector<pit> p){
    if(p.size()>1&&p.front()==p.back())p.pop_back();
    vector<pit> q;
    for(const auto& x:p)if(q.empty()||!(q.back()==x))q.push_back(x);
    if(q.size()>1&&q.front()==q.back())q.pop_back();
    bool changed=true;
    while(changed&&q.size()>2){
        changed=false;
        vector<pit> r;
        int n=q.size();
        for(int i=0;i<n;i++){
            if(onSeg(q[(i-1+n)%n],q[(i+1)%n],q[i]))changed=true;
            else r.push_back(q[i]);
        }
        q.swap(r);
    }
    if(q.size()>2&&area2(q)<0)reverse(q.begin(),q.end());
    return q;
}
//判断点p是否在逆时针三角形abc内部或边界上。
//时间复杂度：O(1)。
bool inTriangle(const pit& a,const pit& b,const pit& c,const pit& p){return
crossVal(a,b,p)>=0&&crossVal(b,c,p)>=0&&crossVal(c,a,p)>=0;}
//用耳切法把简单多边形剖分为三角形；非法或退化输入返回空集，复杂度O(n^2)。
//时间复杂度：O(n^2)；n 为输入顶点数，含多边形简化及各次耳朵判定。
vector<vector<pit>> triangulate(vector<pit> p){
    p=simplifyPolygon(move(p));
    int n=p.size();
    if(n<3||area2(p)==0)return {};
    if(n==3)return {p};
    vector<int> pre(n),nxt(n),alive(n,1),ear(n);
    for(int i=0;i<n;i++)pre[i]=(i-1+n)%n,nxt[i]=(i+1)%n;
    //时间复杂度：isEar 单次最坏 O(n)，n 为当前存活顶点数。
    auto isEar=[&](int u){
        if(!alive[u])return false;
        int a=pre[u],b=nxt[u];
        if(crossVal(p[a],p[u],p[b])<=0)return false;
        for(int v=nxt[b];v!=a;v=nxt[v])if(inTriangle(p[a],p[u],p[b],p[v]))return false;
        return true;
    };
    queue<int> q;
    for(int i=0;i<n;i++){
        ear[i]=isEar(i);
        if(ear[i])q.push(i);
    }
    vector<vector<pit>> res;
    int cnt=n;
    while(cnt>3&&!q.empty()){
        int u=q.front();q.pop();
        if(!alive[u])continue;
        ear[u]=isEar(u);
        if(!ear[u])continue;
        int a=pre[u],b=nxt[u];
        res.push_back({p[a],p[u],p[b]});
        alive[u]=0;nxt[a]=b;pre[b]=a;cnt--;
        ear[a]=isEar(a);ear[b]=isEar(b);
        if(ear[a])q.push(a);
        if(ear[b])q.push(b);
    }
    if(cnt!=3)return {};
    int a=find(alive.begin(),alive.end(),1)-alive.begin(),b=nxt[a],c=nxt[b];
    res.push_back({p[a],p[b],p[c]});
    return res;
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.4 凸包

```
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;
using ld=long double;

//整数向量用于精确计算叉积、点积和距离平方。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
```

```
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点提供Andrew排序和几何判定所需的基本运算。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
    bool operator==(const pit& o)const{return x==o.x&&y==o.y;}
    bool operator<(const pit& o)const{return x!=o.x?x<o.x:y<o.y;}
};

//返回三点的有向二倍三角形面积，用于所有转向判断。
//时间复杂度：O(1)。
i128 crossVal(const pit& a,const pit& b,const pit& c){return (b-a)*(c-a);}
//返回三点转向符号，逆时针、顺时针、共线分别为1、-1、0。
//时间复杂度：O(1)。
int cross(const pit& a,const pit& b,const pit& c){i128 v=crossVal(a,b,c);return (v>0)-(v<0);}
//判断点p是否在闭线段ab上，供退化凸包和边界查询使用。
//时间复杂度：O(1)。
bool onSeg(const pit& a,const pit& b,const pit& p){return cross(a,b,p)==0&&((a-p)&(b-p))<=0;}
//返回两点距离，只有周长需要真实长度时才开方。
//时间复杂度：O(1)。
ld dis(const pit& a,const pit& b){return sqrtl((ld)((b-a)&(b-a)));}
//Andrew算法返回闭合边界周长和逆时针凸包，不保留边上的共线中间点。
//时间复杂度：O(n log n)；n 为输入点数，排序后构造凸包和计算周长均为 O(n)。
pair<ld,vector<pit>> Andrew(vector<pit> p){
    sort(p.begin(),p.end());
    p.erase(unique(p.begin(),p.end()),p.end());
    if(p.empty())return {0,{}};
    if(p.size()==1)return {0,p};
    vector<pit> lo,up;
    for(const auto& x:p){
        while(lo.size()>1&&crossVal(lo[lo.size()-2],lo.back(),x)<=0)lo.pop_back();
        lo.push_back(x);
    }
    for(int i=(int)p.size()-1;i>=0;i--){
        const pit& x=p[i];
        while(up.size()>1&&crossVal(up[up.size()-2],up.back(),x)<=0)up.pop_back();
        up.push_back(x);
    }
    lo.pop_back();up.pop_back();
    lo.insert(lo.end(),up.begin(),up.end());
    ld peri=0;
    for(int i=0;i<(int)lo.size();i++)peri+=dis(lo[i],lo[(i+1)%lo.size()]);
    return {peri,lo};
}
//返回多边形有向面积的两倍，逆时针为正且结果完全精确。
//时间复杂度：O(n)；n 为多边形顶点数。
i128 TA2(const vector<pit>& p){
    i128 res=0;
    for(int i=0;i<(int)p.size();i++)res+=(i128)p[i].x*p[(i+1)%p.size()].y-(i128)p[i].y*p[(i+1)%p.size()].x;
    return res;
}
//返回多边形面积；除以2时才转换为浮点数。
//时间复杂度：O(n)；n 为多边形顶点数。
ld TA(const vector<pit>& p){i128 s=TA2(p);return (ld)(s<0?-s:s)/2;}
//若多边形为顺时针则翻转，使后续凸包算法统一按逆时针处理。
//时间复杂度：O(n)；n 为多边形顶点数。
void rev(vector<pit>& p){if(TA2(p)<0)reverse(p.begin(),p.end());}
//判断点p是否在逆时针三角形abc内部或边界上。
//时间复杂度：O(1)。
bool isCon(const pit& a,const pit& b,const pit& c,const pit& p){return
crossVal(a,b,p)>=0&&crossVal(b,c,p)>=0&&crossVal(c,a,p)>=0;}
//在逆时针凸包中二分判断点是否在内部或边界，复杂度O(logn)。
//时间复杂度：O(log n)；n 为凸包顶点数，n<=2 时为 O(1)。
bool isConvex(const vector<pit>& p,const pit& a){
    int n=p.size();
    if(n==0)return false;
    if(n==1)return a==p[0];
    if(n==2)return onSeg(p[0],p[1],a);
    i128 lcr=crossVal(p[0],p[1],a),rcr=crossVal(p[0],p[n-1],a);
    if(lcr<0||rcr>0)return false;
    if(lcr==0)return onSeg(p[0],p[1],a);
    if(rcr==0)return onSeg(p[0],p[n-1],a);
    int l=1,r=n-1;
    while(r-l>1){
        int mid=(l+r)>>1;
        if(crossVal(p[0],p[mid],a)>=0)l=mid;
        else r=mid;
    }
    return crossVal(p[l],p[r],a)>=0;
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.5 凸包闵和

```cpp
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;

//整数向量保存凸包边，叉积和点积均在i128中精确完成。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return {x+o.x,y+o.y};}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点用于输入、平移和输出闵可夫斯基和的顶点。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
    pit operator+(const vec& o)const{return {x+o.x,y+o.y};}
    pit operator+(const pit& o)const{return {x+o.x,y+o.y};}
    bool operator==(const pit& o)const{return x==o.x&&y==o.y;}
    bool operator<(const pit& o)const{return x!=o.x?x<o.x:y<o.y;}
};

//返回三点有向面积的两倍，用于检查凸性和删除共线冗余点。
//时间复杂度：O(1)。
i128 crossVal(const pit& a,const pit& b,const pit& c){return (b-a)*(c-a);}
//返回多边形有向面积的两倍，正值表示逆时针。
//时间复杂度：O(n)；n 为多边形顶点数。
i128 area2(const vector<pit>& p){
    i128 res=0;
    for(int i=0;i<(int)p.size();i++)res+=(i128)p[i].x*p[(i+1)%p.size()].y-(i128)p[i].y*p[(i+1)%p.size()].x;
    return res;
}
//在线性时间内规范凸多边形：去重、转为逆时针并删除边上的中间点。
//时间复杂度：O(n)；n 为输入凸多边形顶点数，各点进出栈总计 O(n)。
vector<pit> normalizeConvex(vector<pit> p){
    if(p.size()>1&&p.front()==p.back())p.pop_back();
    vector<pit> q;
    for(const auto& x:p)if(q.empty()||!(q.back()==x))q.push_back(x);
    if(q.size()>1&&q.front()==q.back())q.pop_back();
    if(q.size()<3){
        sort(q.begin(),q.end());
        q.erase(unique(q.begin(),q.end()),q.end());
        return q;
    }
    i128 s=area2(q);
    if(s==0){
        auto [mn,mx]=minmax_element(q.begin(),q.end());
        if(*mn==*mx)return {*mn};
        return {*mn,*mx};
    }
    if(s<0)reverse(q.begin(),q.end());
    vector<pit> r;
    for(const auto& x:q){
        while(r.size()>1&&crossVal(r[r.size()-2],r.back(),x)==0)r.pop_back();
        r.push_back(x);
    }
    while(r.size()>2&&crossVal(r[r.size()-2],r.back(),r[0])==0)r.pop_back();
    while(r.size()>2&&crossVal(r.back(),r[0],r[1])==0)r.erase(r.begin());
    return r;
}
//把方向按[0,2pi)分到两个半平面，供边向量归并使用。
//时间复杂度：O(1)。
int half(const vec& a){return a.y>0||(a.y==0&&a.x>=0)?0:1;}
//严格比较两个非零边向量的极角，不调用atan2。
//时间复杂度：O(1)。
bool angleLess(const vec& a,const vec& b){
    int ha=half(a),hb=half(b);
    if(ha!=hb)return ha<hb;
    i128 cr=a*b;
    if(cr!=0)return cr>0;
    return false;
}
//在线性时间求两个凸集的闵可夫斯基和，输入可顺/逆时针且无需重复首点。
//时间复杂度：O(n+m)；n、m 为两个输入凸集的顶点数，含规范化与边向量归并。
vector<pit> minkowski(vector<pit> a,vector<pit> b){
    a=normalizeConvex(move(a));b=normalizeConvex(move(b));
    if(a.empty()||b.empty())return {};
    //时间复杂度：cmp 单次比较 O(1)。
    auto cmp=[](const pit& A,const pit& B){return A.y!=B.y?A.y<B.y:A.x<B.x;};
    rotate(a.begin(),min_element(a.begin(),a.end(),cmp),a.end());
    rotate(b.begin(),min_element(b.begin(),b.end(),cmp),b.end());
    vector<vec> ea,eb;
    if(a.size()>1)for(int i=0;i<(int)a.size();i++)ea.push_back(a[(i+1)%a.size()]-a[i]);
    if(b.size()>1)for(int i=0;i<(int)b.size();i++)eb.push_back(b[(i+1)%b.size()]-b[i]);
    vector<pit> res={a[0]+b[0]};
```

```
    int i=0,j=0;
    while(i<(int)ea.size()||j<(int)eb.size()){
        vec e;
        if(j==(int)eb.size()||(i<(int)ea.size()&&angleLess(ea[i],eb[j])))e=ea[i++];
        else if(i==(int)ea.size()||angleLess(eb[j],ea[i]))e=eb[j++];
        else e=ea[i++]+eb[j++];
        res.push_back(res.back()+e);
    }
    if(res.size()>1&&res.back()==res.front())res.pop_back();
    return res;
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.6 凸包面积并

```
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;
using ld=long double;
const ld eps=1e-18L;

//整数向量承担半平面裁剪中的叉积和点积判定。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点保存各凸多边形顶点，交点参数才转换为浮点数。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
    bool operator==(const pit& o)const{return x==o.x&&y==o.y;}
};

//返回向量叉积，供边对原点的有向面积贡献使用。
//时间复杂度：O(1)。
i128 crossVal(const vec& a,const vec& b){return a*b;}
//返回三点有向面积的两倍，用于精确的半平面方向判定。
//时间复杂度：O(1)。
i128 crossVal(const pit& a,const pit& b,const pit& c){return (b-a)*(c-a);}
//返回多边形有向面积的两倍，正值表示逆时针。
//时间复杂度：O(n)；n 为多边形顶点数。
i128 area2(const vector<pit>& p){
    i128 res=0;
    for(int i=0;i<(int)p.size();i++)res+=(i128)p[i].x*p[(i+1)%p.size()].y-(i128)p[i].y*p[(i+1)%p.size()].x;
    return res;
}
//清理重复首点并统一为逆时针，退化多边形会被丢弃。
//时间复杂度：O(n)；n 为输入多边形顶点数。
vector<pit> normalizePoly(vector<pit> p){
    if(p.size()>1&&p.front()==p.back())p.pop_back();
    vector<pit> q;
    for(const auto& x:p)if(q.empty()||!(q.back()==x))q.push_back(x);
    if(q.size()>1&&q.front()==q.back())q.pop_back();
    if(q.size()<3||area2(q)==0)return {};
    if(area2(q)<0)reverse(q.begin(),q.end());
    return q;
}
//求若干凸多边形的面积并；半平面判断为整数，只有边上交点参数使用浮点数。
//时间复杂度：O(k+N^2+N*k*log(k+1))；k 为输入多边形数，N 为总顶点数，含逐边裁剪与覆盖区间排序。
ld area_union(vector<vector<pit>> polys){
    vector<vector<pit>> valid;
    for(auto& p:polys){
        p=normalizePoly(move(p));
        if(!p.empty())valid.push_back(move(p));
    }
    polys=move(valid);
    ld ans=0;
    int n=polys.size();
    for(int i=0;i<n;i++){
        int ni=polys[i].size();
        for(int e=0;e<ni;e++){
            pit A=polys[i][e],B=polys[i][(e+1)%ni];
            vec AB=B-A;
            vector<pair<ld,ld>> cov;
            for(int j=0;j<n;j++){
                if(i==j)continue;
                ld L=0,R=1;
                bool ok=true;
                int nj=polys[j].size();
                for(int k=0;k<nj;k++){
                    pit C=polys[j][k],D=polys[j][(k+1)%nj];
                    vec DC=D-C;
```

```
                    i128 c1=DC*(A-C),c2=DC*AB;
                    if(c2>0)L=max(L,-(ld)c1/(ld)c2);
                    else if(c2<0)R=min(R,-(ld)c1/(ld)c2);
                    else if(c1<0||(c1==0&&(AB&DC)>0&&i>j)){ok=false;break;}
                }
                if(ok&&L<=R+eps)cov.push_back({max(0.0L,L),min(1.0L,R)});
            }
            sort(cov.begin(),cov.end());
            ld cur=0,edge=(ld)((vec(A.x,A.y))*vec(B.x,B.y));
            for(auto [l,r]:cov){
                if(r<cur+eps)continue;
                if(l>cur+eps)ans+=(l-cur)*edge;
                cur=max(cur,r);
                if(cur>=1-eps)break;
            }
            if(cur<1-eps)ans+=(1-cur)*edge;
        }
    }
    return ans/2;
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.7 向量

```
//本板子由 AI 生成。
//复杂度按固定宽长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;
using ld=long double;

//整数向量负责精确的加减、叉积和点积，乘积用i128避免常见坐标范围下溢出。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return {x+o.x,y+o.y};}
    vec operator-(const vec& o)const{return {x-o.x,y-o.y};}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点用于所有拓扑判定，只有构造非整数结果时才转换为dpit。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
    pit operator+(const vec& o)const{return {x+o.x,y+o.y};}
    pit operator+(const pit& o)const{return {x+o.x,y+o.y};}
    bool operator==(const pit& o)const{return x==o.x&&y==o.y;}
};
//浮点向量仅承接单位化、旋转等无法保持整数坐标的结果。
struct dvec{
    ld x,y;
    //时间复杂度：O(1)。
    dvec(ld x=0,ld y=0):x(x),y(y){}
    //时间复杂度：O(1)。
    dvec operator+(const dvec& o)const{return {x+o.x,y+o.y};}
    //时间复杂度：O(1)。
    dvec operator-(const dvec& o)const{return {x-o.x,y-o.y};}
    //时间复杂度：O(1)。
    dvec operator*(ld k)const{return {x*k,y*k};}
    //时间复杂度：O(1)。
    dvec operator/(ld k)const{return {x/k,y/k};}
};
//浮点点用于表示直线交点等一般为有理数的坐标。
struct dpit{
    ld x,y;
    //时间复杂度：O(1)。
    dpit(ld x=0,ld y=0):x(x),y(y){}
    //时间复杂度：O(1)。
    dpit operator+(const dvec& o)const{return {x+o.x,y+o.y};}
};

//返回向量长度平方，比较距离时优先使用它以保持全整数。
//时间复杂度：O(1)。
i128 len2(const vec& o){return o&o;}
//返回向量模长，仅在确实需要长度值时进行开方。
//时间复杂度：O(1)。
ld len(const vec& o){return sqrtl((ld)len2(o));}
//返回两点距离平方，适合最近点和大小关系判断。
//时间复杂度：O(1)。
i128 dis2(const pit& a,const pit& b){return len2(b-a);}
//返回两点欧氏距离，输出真实距离时使用。
//时间复杂度：O(1)。
ld dis(const pit& a,const pit& b){return sqrtl((ld)dis2(a,b));}
//将整数向量逆时针旋转theta弧度，结果通常不再是整数。
```

```
//时间复杂度：O(1)。
dvec rotate(const vec& o,ld theta){return {o.x*cosl(theta)-o.y*sinl(theta),o.x*sinl(theta)+o.y*cosl(theta)};}
//返回同方向单位向量，调用者需保证原向量非零。
//时间复杂度：O(1)。
dvec norm(const vec& o){
    assert(len2(o)!=0);
    ld l=len(o);
    return {(ld)o.x/l,(ld)o.y/l};
}
//返回两个非零向量的较小夹角，范围为[0,pi]。
//时间复杂度：O(1)。
ld angle(const vec& a,const vec& b){
    assert(len2(a)!=0&&len2(b)!=0);
    ld val=(ld)(a&b)/len(a)/len(b);
    val=max(-1.0L,min(1.0L,val));
    return acosl(val);
}
//返回有向平行四边形面积，正值表示b在a逆时针方向。
//时间复杂度：O(1)。
i128 area(const vec& a,const vec& b){return a*b;}
//精确判断点c相对有向直线ab的位置，依次返回1、-1、0。
//时间复杂度：O(1)。
int cross(const pit& a,const pit& b,const pit& c){
    i128 val=(b-a)*(c-a);
    return (val>0)-(val<0);
}
//判断点p是否位于闭线段ab上，端点也计入。
//时间复杂度：O(1)。
bool onSeg(const pit& a,const pit& b,const pit& p){return cross(a,b,p)==0&&((a-p)&(b-p))<=0;}
//判断非退化直线ab是否与闭线段cd有公共点。
//时间复杂度：O(1)。
bool lcross(const pit& a,const pit& b,const pit& c,const pit& d){return cross(a,b,c)*cross(a,b,d)<=0;}
//判断两条闭线段是否相交，覆盖端点接触、重合和退化成点的情况。
//时间复杂度：O(1)。
bool scross(const pit& a,const pit& b,const pit& c,const pit& d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    if(c1==0&&onSeg(a,b,c))return true;
    if(c2==0&&onSeg(a,b,d))return true;
    if(c3==0&&onSeg(c,d,a))return true;
    if(c4==0&&onSeg(c,d,b))return true;
    return c1*c2<0&&c3*c4<0;
}
//判断两条非退化直线是否有唯一交点，即方向向量不平行。
//时间复杂度：O(1)。
bool pcross(const pit& a,const pit& b,const pit& c,const pit& d){return (b-a)*(d-c)!=0;}
//求两条非平行直线ab、cd的交点，除法不可避免所以返回浮点点。
//时间复杂度：O(1)。
dpit getNode(const pit& a,const pit& b,const pit& c,const pit& d){
    vec u=b-a,v=d-c;
    i128 den=u*v;
    assert(den!=0);
    ld t=(ld)((c-a)*v)/(ld)den;
    return {a.x+u.x*t,a.y+u.y*t};
}
//用点向式求两条非平行直线的交点，u、v分别是两直线方向。
//时间复杂度：O(1)。
dpit getNode(const pit& a,const vec& u,const pit& c,const vec& v){
    i128 den=u*v;
    assert(den!=0);
    ld t=(ld)((c-a)*v)/(ld)den;
    return {a.x+u.x*t,a.y+u.y*t};
}
//返回点p到闭线段ab的最短欧式距离
//时间复杂度：O(1)。
ld disPtSeg(const pit& p,const pit& a,const pit& b){
    vec ab=b-a,ap=p-a,bp=p-b;
    if(len2(ab)==0)return dis(p,a); //退化成点
    if((ab&ap)<=0)return dis(p,a); //投影在a的外侧
    if(((a-b)&bp)<=0)return dis(p,b); //投影在b的外侧
    i128 cr=ab*ap; //投影在线段上，计算平行四边形面积求高
    return (ld)(cr<0?-cr:cr)/len(ab);
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.8 旋转卡壳

```
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;
using u128=__uint128_t;
using ld=long double;
const ld eps=1e-12L;
```

```
//384位无符号整数只用于比较三个非负i128因子的乘积。
struct u384{
    array<uint32_t,12> d{};
};

//整数向量用于卡壳指针移动时的精确叉积、点积比较。
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    i128 operator*(const vec& o)const{return (i128)x*o.y-(i128)y*o.x;}
    i128 operator&(const vec& o)const{return (i128)x*o.x+(i128)y*o.y;}
};
//整数点保存输入点集和凸包，所有最优性判定均不依赖eps。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return {x-o.x,y-o.y};}
    bool operator==(const pit& o)const{return x==o.x&&y==o.y;}
    bool operator<(const pit& o)const{return x!=o.x?x<o.x:y<o.y;}
};
//浮点点只用于返回最小外接矩形通常不是整数的四个顶点。
struct dpit{
    ld x,y;
    //时间复杂度：O(1)。
    dpit(ld x=0,ld y=0):x(x),y(y){}
};

//返回三点有向面积的两倍，凸包和卡壳高度都用它计算。
//时间复杂度：O(1)。
i128 crossVal(const pit& a,const pit& b,const pit& c){return (b-a)*(c-a);}
//返回ab与ac的点积，用于寻找沿当前边方向的最远投影。
//时间复杂度：O(1)。
i128 dotVal(const pit& a,const pit& b,const pit& c){return (b-a)&(c-a);}
//返回向量与点位置向量的叉积，即该点在法向轴上的未归一化投影。
//时间复杂度：O(1)。
i128 crossOrigin(const vec& a,const pit& p){return (i128)a.x*p.y-(i128)a.y*p.x;}
//返回向量与点位置向量的点积，即该点在边轴上的未归一化投影。
//时间复杂度：O(1)。
i128 dotOrigin(const vec& a,const pit& p){return (i128)a.x*p.x+(i128)a.y*p.y;}
//精确计算三个非负i128的乘积，避免候选矩形面积比较时溢出。
//时间复杂度：O(1)；整数位宽固定，三重循环仅执行 4^3 次乘加。
u384 mul3(i128 a,i128 b,i128 c){
    assert(a>=0&&b>=0&&c>=0);
    array<uint32_t,4> A{},B{},C{};
    u128 x=a,y=b,z=c;
    for(int i=0;i<4;i++)A[i]=x>>(i*32),B[i]=y>>(i*32),C[i]=z>>(i*32);
    array<u128,13> sum{};
    for(int i=0;i<4;i++)for(int j=0;j<4;j++)for(int k=0;k<4;k++)sum[i+j+k]+=(u128)A[i]*B[j]*C[k];
    u384 res;
    for(int i=0;i<12;i++)res.d[i]=sum[i],sum[i+1]+=sum[i]>>32;
    return res;
}
//按数值比较两个384位无符号整数，供分数交叉相乘后判优。
//时间复杂度：O(1)。
bool operator<(const u384& a,const u384& b){
    for(int i=11;i>=0;i--)if(a.d[i]!=b.d[i])return a.d[i]<b.d[i];
    return false;
}
//返回两点欧氏距离，计算凸包周长时使用。
//时间复杂度：O(1)。
ld dis(const pit& a,const pit& b){vec d=b-a;return sqrtl((ld)(d&d));}
//Andrew算法返回闭合边界周长和逆时针凸包，不保留共线中间点。
//时间复杂度：O(n log n)；n 为输入点数，排序后构造凸包和计算周长均为 O(n)。
pair<ld,vector<pit>> Andrew(vector<pit> p){
    sort(p.begin(),p.end());
    p.erase(unique(p.begin(),p.end()),p.end());
    if(p.empty())return {0,{}};
    if(p.size()==1)return {0,p};
    vector<pit> lo,up;
    for(const auto& x:p){
        while(lo.size()>1&&crossVal(lo[lo.size()-2],lo.back(),x)<=0)lo.pop_back();
        lo.push_back(x);
    }
    for(int i=(int)p.size()-1;i>=0;i--){
        const pit& x=p[i];
        while(up.size()>1&&crossVal(up[up.size()-2],up.back(),x)<=0)up.pop_back();
        up.push_back(x);
    }
    lo.pop_back();up.pop_back();
    lo.insert(lo.end(),up.begin(),up.end());
    ld peri=0;
    for(int i=0;i<(int)lo.size();i++)peri+=dis(lo[i],lo[(i+1)%lo.size()]);
    return {peri,lo};
}
//返回多边形有向面积的两倍，用于统一卡壳遍历方向。
//时间复杂度：O(n)；n 为多边形顶点数。
i128 TA2(const vector<pit>& p){
    i128 res=0;
    for(int i=0;i<(int)p.size();i++)res+=(i128)p[i].x*p[(i+1)%p.size()].y-(i128)p[i].y*p[(i+1)%p.size()].x;
```

```
    return res;
}
//返回多边形面积，只有最终除以2时转换为浮点数。
//时间复杂度：O(n)；n 为多边形顶点数。
ld TA(const vector<pit>& p){i128 s=TA2(p);return (ld)(s<0?-s:s)/2;}
//若凸包为顺时针则翻转，保证内部始终位于每条边左侧。
//时间复杂度：O(n)；n 为多边形顶点数。
void rev(vector<pit>& p){if(TA2(p)<0)reverse(p.begin(),p.end());}
//判断点p是否在逆时针三角形abc内部或边界上。
//时间复杂度：O(1)。
bool isCon(const pit& a,const pit& b,const pit& c,const pit& p){return
crossVal(a,b,p)>=0&&crossVal(b,c,p)>=0&&crossVal(c,a,p)>=0;}
//在逆时针凸包内二分判点，边界计入且复杂度为O(logn)。
//时间复杂度：O(log n)；n 为凸包顶点数，n<=2 时为 O(1)。
bool isConvex(const vector<pit>& p,const pit& a){
    int n=p.size();
    if(n==0)return false;
    if(n==1)return a==p[0];
    if(n==2)return crossVal(p[0],p[1],a)==0&&((p[0]-a)&(p[1]-a))<=0;
    i128 x=crossVal(p[0],p[1],a),y=crossVal(p[0],p[n-1],a);
    if(x<0||y>0)return false;
    if(x==0)return ((p[0]-a)&(p[1]-a))<=0;
    if(y==0)return ((p[0]-a)&(p[n-1]-a))<=0;
    int l=1,r=n-1;
    while(r-l>1){
        int mid=(l+r)>>1;
        if(crossVal(p[0],p[mid],a)>=0)l=mid;
        else r=mid;
    }
    return crossVal(p[l],p[r],a)>=0;
}
//由边轴投影u和法轴投影v还原矩形顶点，除法在此处才发生。
//时间复杂度：O(1)。
dpit fromProjection(const vec& e,i128 u,i128 v,i128 q){
    ld X=((ld)e.x*(ld)u-(ld)e.y*(ld)v)/(ld)q;
    ld Y=((ld)e.y*(ld)u+(ld)e.x*(ld)v)/(ld)q;
    return {X,Y};
}
//旋转卡壳求逆时针凸包的最小面积外接矩形，返回面积和逆时针四顶点。
//时间复杂度：O(n)；n 为凸包顶点数，三个卡壳指针各自累计移动 O(n) 次，不含求凸包。
pair<ld,vector<dpit>> rot(vector<pit> p){
    if(p.empty())return {0,{}};
    if(p.size()==1)return {0,vector<dpit>(4,{(ld)p[0].x,(ld)p[0].y})};
    rev(p);
    int n=p.size(),top=0,rgt=0,lft=0;
    vec firstEdge=p[1]-p[0];
    for(int i=1;i<n;i++){
        if(crossVal(p[0],p[1],p[i])>crossVal(p[0],p[1],p[top]))top=i;
        if(dotOrigin(firstEdge,p[i])>dotOrigin(firstEdge,p[rgt]))rgt=i;
        if(dotOrigin(firstEdge,p[i])<dotOrigin(firstEdge,p[lft]))lft=i;
    }
    bool first=true;
    i128 bestH=0,bestW=0,bestQ=1;
    vector<dpit> fin(4);
    for(int i=0;i<n;i++){
        int ni=(i+1)%n;
        vec e=p[ni]-p[i];
        i128 q=e&e;
        while(crossVal(p[i],p[ni],p[(top+1)%n])>crossVal(p[i],p[ni],p[top]))top=(top+1)%n;
        while(dotVal(p[i],p[ni],p[(rgt+1)%n])>dotVal(p[i],p[ni],p[rgt]))rgt=(rgt+1)%n;
        while(dotVal(p[i],p[ni],p[(lft+1)%n])<dotVal(p[i],p[ni],p[lft]))lft=(lft+1)%n;
        i128 h=crossVal(p[i],p[ni],p[top]);
        i128 w=dotOrigin(e,p[rgt])-dotOrigin(e,p[lft]);
        if(first||mul3(h,w,bestQ)<mul3(bestH,bestW,q)){
            first=false;bestH=h;bestW=w;bestQ=q;
            i128 u0=dotOrigin(e,p[lft]),u1=dotOrigin(e,p[rgt]);
            i128 v0=crossOrigin(e,p[i]),v1=crossOrigin(e,p[top]);
            fin[0]=fromProjection(e,u0,v0,q);
            fin[1]=fromProjection(e,u1,v0,q);
            fin[2]=fromProjection(e,u1,v1,q);
            fin[3]=fromProjection(e,u0,v1,q);
        }
    }
    return {(ld)bestH*(ld)bestW/(ld)bestQ,fin};
}
//把极小的输出坐标归零，避免打印出-0.00000。
//时间复杂度：O(1)。
void zero(dpit& a){if(fabsl(a.x)<eps)a.x=0;if(fabsl(a.y)<eps)a.y=0;}
//按x、y字典序比较输出点，用于固定矩形顶点输出起点。
//时间复杂度：O(1)。
bool dcmp(const dpit& a,const dpit& b){return fabsl(a.x-b.x)>eps?a.x<b.x:a.y<b.y-eps;}

//读取整数点集并输出最小外接矩形示例，顶点从最小点起按顺时针排列。
//时间复杂度：O(n log n)；n 为输入点数，主要耗时为 Andrew 求凸包。
```

用法示例:

```
    int n;
    if(!(cin>>n))return 0;
    vector<pit> p(n);
    for(auto& x:p)cin>>x.x>>x.y;
```

```
    auto hull=Andrew(p).second;
    auto [ans,fin]=rot(hull);
    cout<<fixed<<setprecision(5)<<ans<<'\n';
    if(fin.empty())return 0;
    reverse(fin.begin(),fin.end());
    int k=0;
    for(int i=1;i<4;i++)if(dcmp(fin[i],fin[k]))k=i;
    for(int i=0;i<4;i++){
        dpit& x=fin[(k+i)%4];zero(x);
        cout<<fixed<<setprecision(5)<<x.x<<' '<<x.y<<'\n';
    }
```

7.9 极角排序

```
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <bits/stdc++.h>
using namespace std;
using ll=long long;
using i128=__int128_t;

//整数点是排序对象，坐标本身不做浮点化。
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
};

//按atan2的[-pi,pi]顺序划分方向，避免实际计算反三角函数。
//时间复杂度：O(1)。
int half(i128 x,i128 y){return y<0||(y==0&&x>=0)?0:1;}
//比较两个方向的极角；同向时短向量优先，零向量排在最前。
//时间复杂度：O(1)。
bool polarLess(i128 ax,i128 ay,i128 bx,i128 by){
    bool az=ax==0&&ay==0,bz=bx==0&&by==0;
    if(az||bz)return az&&!bz;
    int ha=half(ax,ay),hb=half(bx,by);
    if(ha!=hb)return ha<hb;
    i128 cr=ax*by-ay*bx;
    if(cr!=0)return cr>0;
    return ax*ax+ay*ay<bx*bx+by*by;
}
//以整数点o为极点逆时针排序，角度相同的点按离o由近到远排列。
//时间复杂度：O(n log n)；n 为待排序点数，单次极角比较为 O(1)。
void psort(vector<pit>& a,const pit& o){
    //时间复杂度：lambda 单次比较 O(1)，整次排序 O(n log n)。
    sort(a.begin(),a.end(),[&](const pit& A,const pit& B){
        i128 ax=(i128)A.x-o.x,ay=(i128)A.y-o.y;
        i128 bx=(i128)B.x-o.x,by=(i128)B.y-o.y;
        if(polarLess(ax,ay,bx,by))return true;
        if(polarLess(bx,by,ax,ay))return false;
        return A.x!=B.x?A.x<B.x:A.y<B.y;
    });
}
//以点集重心为极点排序，用n*p-sum精确表示方向而不做除法。
//时间复杂度：O(n log n)；n 为待排序点数，单次极角比较为 O(1)。
void psort(vector<pit>& a){
    if(a.empty())return;
    i128 sx=0,sy=0,n=a.size();
    for(const auto& p:a)sx+=p.x,sy+=p.y;
    //时间复杂度：lambda 单次比较 O(1)，整次排序 O(n log n)。
    sort(a.begin(),a.end(),[&](const pit& A,const pit& B){
        i128 ax=n*A.x-sx,ay=n*A.y-sy;
        i128 bx=n*B.x-sx,by=n*B.y-sy;
        if(polarLess(ax,ay,bx,by))return true;
        if(polarLess(bx,by,ax,ay))return false;
        return A.x!=B.x?A.x<B.x:A.y<B.y;
    });
}

//模板文件入口，不执行额外逻辑。
//时间复杂度：O(1)。
```

7.10 点在多边形内判定

```
//本板子由 AI 生成。
//复杂度按固定字长整数及浮点基本运算为 O(1) 计。
#include <algorithm>
#include <bitset>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <ctime>
#include <deque>
#include <map>
#include <iostream>
#include <queue>
#include <set>
#include <stack>
```

```cpp
#include <vector>
#include <array>
#include <unordered_map>
#include <numeric>
#include <functional>
#include <ranges>
#include <iomanip>
#include <chrono>
#include <random>
#include <cstdint>
//#define int long long //赫赫 要不要龙龙呢
using namespace std;
using ll=long long;
using ld=long double;
using dpit=array<ld,2>;
const ld eps=1e-9;
struct vec{
    ll x,y;
    vec(ll x=0,ll y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    ll operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    ll operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit{
    ll x,y;
    pit(ll x=0,ll y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
};
//时间复杂度：O(1)。
int cross(pit a,pit b,pit c){
    ll val=(b-a)*(c-a);
    if(val>0)return 1;
    if(val<0)return -1;
    return 0;
}
//时间复杂度：O(1)。
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=0;
}
//时间复杂度：O(1)。
bool scross(pit a,pit b,pit c,pit d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    if(c1==0&&onSeg(a,b,c))return 1;
    if(c2==0&&onSeg(a,b,d))return 1;
    if(c3==0&&onSeg(c,d,a))return 1;
    if(c4==0&&onSeg(c,d,b))return 1;
    return c1*c2<0&&c3*c4<0;
}
//时间复杂度：O(1)。
dpit getnode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    ld t=1.0*((c-a)*v)/(u*v);
    return {a.x+u.x*t,a.y+u.y*t};
}
//时间复杂度：O(n)；n 为多边形顶点数。
bool inPoly(pit p,const vector<pit>& poly){
    int n=poly.size();
    if(n==1) return p.x==poly[0].x&&p.y==poly[0].y;
    if(n==2) return onSeg(poly[0],poly[1],p);
    int cnt=0;
    for(int i=0;i<n;++i){
        pit a=poly[i],b=poly[(i+1)%n];
        if(onSeg(a,b,p))return 1;
        int k=cross(a,b,p);
        ll d1=a.y-p.y,d2=b.y-p.y;
        if(k>0&&d1<=0&&d2>0)cnt++;
        if(k<0&&d2<=0&&d1>0)cnt--;
    }
    return cnt!=0;
}
//时间复杂度：O(sum(n+m+q*n*m))；对各测试用例求和，n、m 为两多边形顶点数，q 为查询数。
```

用法示例:

```cpp
    int t;cin>>t;
    while(t--){
        int n,m,q;cin>>n>>m>>q;
        vector<pit> a(n),b(m);
        for(int i=0;i<n;i++) cin>>a[i].x>>a[i].y;
        for(int i=0;i<m;i++) cin>>b[i].x>>b[i].y;
        while(q--){
            pit c;cin>>c.x>>c.y;
            vector<pit> nb(m);
            for(int i=0;i<m;i++) nb[i]=pit(c.x-b[i].x,c.y-b[i].y);
            bool flag=0;
            dpit ans1,ans2;
            for(int i=0;i<n;i++){
                if(flag) break;
                for(int j=0;j<m;j++){
                    if(flag) break;
```

```
                    pit a1=a[i],a2=a[(i+1)%n],b1=nb[j],b2=nb[(j+1)%m];
                    if(scross(a1,a2,b1,b2)){
                        if(cross(a1,a2,b1)==0&&cross(a1,a2,b2)==0){
                            if(onSeg(a1,a2,b1)) ans1={1.0*b1.x,1.0*b1.y};
                            else if(onSeg(a1,a2,b2)) ans1={1.0*b2.x,1.0*b2.y};
                            else if(onSeg(b1,b2,a1)) ans1={1.0*a1.x,1.0*a1.y};
                            else ans1={1.0*a2.x,1.0*a2.y};
                        }
                        else ans1=getnode(a1,a2,b1,b2);
                        ans2={c.x-ans1[0],c.y-ans1[1]};
                        flag=1;
                    }
                }
            }
            for(int i=0;i<n;i++){
                if(inPoly(a[i],nb)){
                    ans1={(ld)a[i].x,(ld)a[i].y};
                    ans2={c.x-ans1[0],c.y-ans1[1]};
                    flag=1;
                    break;
                }
            }
            for(int i=0;i<m;i++){
                if(inPoly(nb[i],a)){
                    ans1={(ld)nb[i].x,(ld)nb[i].y};
                    ans2={c.x-ans1[0],c.y-ans1[1]};
                    flag=1;
                    break;
                }
            }
            if(flag) cout<<fixed<<setprecision(10)<<ans1[0]<<" "<<ans1[1]<<" "<<ans2[0]<<" "<<ans2[1]<<'\n';
            else cout<<"-1 -1 -1 -1\n";
        }
    }
```

8 计算几何(浮点数版)

8.1 三分

```
//注意我们通常不用浮点数三分 而是固定次数 t=100
/*
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3;
    int rmid = r - (r - l) / 3;
    lans = f(lmid),rans = f(rmid);
    // 求凹函数的极小值
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
    // 求凸函数的极大值
    if(lasn >= rans) l = lmid + 1;
    else r = rmid - 1;
}
// 求凹函数的极小值
cout << min(lans,rans) << endl;
// 求凸函数的极大值
cout << max(lans,rans) << endl;
*/
/*
const double EPS = 1e-9;
while(r - l > EPS) {
    double lmid = l + (r - l) / 3;
    double rmid = r - (r - l) / 3;
    lans = f(lmid),rans = f(rmid);
    // 求凹函数的极小值
    if(lans <= rans) r = rmid;
    else l = lmid;
    // 求凸函数的极大值
    if(lans >= rans) l = lmid;
    else r = rmid;
}
// 输出 l 或 r 都可
cout << l << endl;
*/
```

```
#define double long double
const double eps=1e-12;
const double pi=acos(-1);
using namespace std;
//const double eps=1e-8;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
//向量逆时针旋转theta弧度
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//向量单位化；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
}
//向量围成的平行四边形面积,b在a的逆时针方向为正，否则为负
double area(vec a,vec b){return a*b;}
//点线关系(点c,直线ab)
int cross(pit a,pit b,pit c){
    if((b-a)*(c-a)>eps) return 1; //c在ab的逆时针方向
    else if((b-a)*(c-a)<-eps) return -1; //c在ab的顺时针方向
    return 0; //c,a,b共线
}
//判断点在线段上(p在ab上)
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=eps;
}
//线线关系
//case1:直线ab与线段cd
bool lcross(pit a,pit b,pit c,pit d){
```

```
    if(cross(a,b,c)*cross(a,b,d)>0) return 0;//c,d在ab的同一侧 无交点
    return 1; //有交点
}
//case2:线段ab与线段cd
bool scross(pit a,pit b,pit c,pit d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    //共线时还要判断投影是否落在线段上
    if(c1==0&&onSeg(a,b,c)) return 1;
    if(c2==0&&onSeg(a,b,d)) return 1;
    if(c3==0&&onSeg(c,d,a)) return 1;
    if(c4==0&&onSeg(c,d,b)) return 1;
    return c1*c2<0&&c3*c4<0; //严格跨立 有交点
}
//case3:直线ab与直线cd
bool pcross(pit a,pit b,pit c,pit d){
    if(fabs((b-a)*(d-c))<=eps) return 0; //平行 无交点
    return 1; //有交点
}
//求两条不平行直线ab、cd的交点；要求两条方向向量非零且不平行
pit getNode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    //assert(fabs(u*v)<=eps);
    //if(fabs(u*v)<=eps) return pit(NAN,NAN); //平行 无交点
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//点向式求两条不平行直线的交点；要求u、v非零且不平行
pit getNode(pit a,vec u,pit c,vec v){
    //assert(fabs(u*v)<=eps);
    //if(fabs(u*v)<=eps) return pit(NAN,NAN); //平行 无交点
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
```

用法示例:

```
    int t;cin>>t;
    while(t--){
        int x1,y1,x2,y2;cin>>x1>>y1>>x2>>y2;
        int x3,y3,x4,y4;cin>>x3>>y3>>x4>>y4;
        pit s1(x1,y1),t1(x2,y2),s2(x3,y3),t2(x4,y4);
        double T1=min(dis(s1,t1),dis(s2,t2));
        vec v1=norm(t1-s1),v2=norm(t2-s2);
        double l=0,r=T1;
        while(r-l>eps){
            double lmid=l+(r-l)/3,rmid=lmid+(r-l)/3;
            double lans=dis(s1+v1*lmid,s2+v2*lmid);
            double rans=dis(s1+v1*rmid,s2+v2*rmid);
            if(rans-lans>eps) r=rmid;
            else l=lmid;
        }
        //不过应该注意的是
        double ans=dis(s1+v1*l,s2+v2*l);
        if(dis(s1,t1)-dis(s2,t2)>eps) swap(s1,s2),swap(t1,t2),swap(v1,v2);
        // now t1, s2->t2
        s2=s2+v2*T1;
        //cout<<s2.x<<' '<<s2.y<<endl;
        vec v_rot=norm(rotate(v2,pi/2));
        //cout<<v_rot.x<<' '<<v_rot.y<<endl;
        //cout<<t1.x<<' '<<t1.y<<endl;
        pit cropit=getNode(t1,v_rot,s2,v2);
        //cout<<v2.x<<' '<<v2.y<<endl;
        //cout<<cropit.x<<' '<<cropit.y<<endl;
        if(onSeg(s2,t2,cropit))
        {
            //cout<<"Y"<<endl;
            ans=min(ans,dis(t1,cropit));
        }
        ans=min(ans,dis(t1,s2)),ans=min(ans,dis(t1,t2));
        cout<<fixed<<setprecision(12)<<ans<<endl;
    }
```

8.2 三角剖分

```
//三角剖分：将多边形分割成若干边 求边与圆心构成的三角形
//通过这个三角形，求多边形与圆的面积交
//传多边形的时候要逆时针传参(极角排序,当且仅当多边形为凸多边形时成立),同时圆心要平移到(0,0)
```

```
//#define double long double
//const double eps=1e-12;
using namespace std;
struct pit;struct vec;
const double eps=1e-8;
const double pi=acos(-1);
double R;
struct pit;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec(pit a);//点转向量(OA向量)
```

```
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
inline vec::vec(pit a) { x=a.x; y=a.y; }
pit CO;//圆心
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
//向量逆时针旋转theta弧度
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//单位向量；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
}
//用单位圆证明
//向量夹角 dot(a,b)=len(a)*len(b)*cos(θ)
double angle(vec a,vec b){
    double val=(a&b)/len(a)/len(b);
    val=max(-1.0,min(1.0,val));
    return acos(val);
}
//向量围成的平行四边形面积,b在a的逆时针方向为正，否则为负
double area(vec a,vec b){return a*b;}
//点线关系(点c,直线ab)
int cross(pit a,pit b,pit c){
    if((b-a)*(c-a)>eps) return 1; //c在ab的逆时针方向
    else if((b-a)*(c-a)<-eps) return -1; //c在ab的顺时针方向
    return 0; //c,a,b共线
}
//判断点在线段上(p在ab上)
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=eps;
}
//OA OB扇形面积
double sector(vec a,vec b){
    double angle=atan2(a*b,a&b); //有向圆心角，范围为(-pi,pi]
    return angle*R*R/2;
}
//求两条不平行直线ab、cd的交点；要求两条方向向量非零且不平行
pit getNode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    //assert(fabs(u*v)<=eps); //平行 无交点
    //if(fabs(u*v)<=eps) return pit(NAN,NAN); //平行 无交点
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//点向式求两条不平行直线的交点；要求u、v非零且不平行
pit getNode(pit a,vec u,pit c,vec v){
    //assert(fabs(u*v)<=eps); //平行 无交点
    //if(fabs(u*v)<=eps) return pit(NAN,NAN); //平行 无交点
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//计算线段ab与圆的交点和距离(此处的距离是有意义的距离 即线段离圆心的距离)
double getDP2(pit a,pit b,pit& pa,pit &pb){
    vec u=b-a;
    double len2=u&u;
    if(len2<=eps){
        pa=pb=a;
        return dis(a,CO);
    }
    double t=((CO-a)&u)/len2;
    t=max(0.0,min(1.0,t));
    pit e=a+u*t;
    double d=dis(e,CO); //线段到圆心的最短距离
    if(R-d<=-eps) return d; //线段在圆外 0个交点
    vector<pair<double,pit>> its;
    vec ac=a-CO;
    double A=len2,B=2*(ac&u),C=(ac&ac)-R*R;
    double delta=B*B-4*A*C;
    auto add=[&](double t){
        if(t<-eps||t>1+eps) return;
        t=max(0.0,min(1.0,t));
        pit p=a+u*t;
```

```
        if(its.empty()||dis(its.back().second,p)>eps) its.push_back({t,p});
    };
    if(delta>=-eps){
        delta=max(0.0,delta);
        double sq=sqrt(delta);
        add((-B-sq)/(2*A));
        add((-B+sq)/(2*A));
    }
    sort(its.begin(),its.end(),[](auto x,auto y){return x.first<y.first;});
    if(its.empty()) pa=pb=e;
    else if(its.size()==1) pa=pb=its[0].second;
    else pa=its[0].second,pb=its[1].second;//按 a->b 顺序保存交点
    return d;
}
//计算线段ab与圆心构成的三角形与圆的面积交
double getS(pit a,pit b){
    if(cross(a,b,CO)==0) return 0; //case1:三点共线
    double da=dis(a,CO),db=dis(b,CO);
    if(R-da>=-eps&&R-db>=-eps) return (vec(a))*(vec(b))/2; //case2:线段在圆内 构成一个三角形
    pit pa,pb;
    double d=getDP2(a,b,pa,pb);
    if(R-d<=-eps) return sector(vec(a),vec(b)); //case3:线段在圆外 构成一个扇形
    if(R-da>=-eps) return (vec(a))*(vec(pb))/2+sector(vec(pb),vec(b)); //case4.1:a在圆内 一个三角形+扇形
    if(R-db>=-eps) return (vec(pa))*(vec(b))/2+sector(vec(a),vec(pa)); //case4.2:b在圆内 一个三角形+扇形
    return (vec(pa))*(vec(pb))/2+sector(vec(a),vec(pa))+sector(vec(b),vec(pb)); //case5:两个端点都在圆内 一个三角形+两个扇
形
}
//极角排序
void psort(vector<pit>& a)
{
    pit cen(0,0);
    for(auto& i:a) cen=cen+i;
    cen=cen/a.size();
    sort(a.begin(),a.end(),[&](pit a,pit b){
        double angA=atan2(a.y-cen.y,a.x-cen.x);
        double angB=atan2(b.y-cen.y,b.x-cen.x);
        return angA<angB;
    });
}
double S(pit o,double r,vector<pit> a)
{
    for(auto& i:a) i.x-=o.x,i.y-=o.y;
    R=r;CO=pit(0,0);
    //psort(a); //当且仅当多边形为凸多边形且传入顺序不是正/逆时针时
    double res=0;
    for(int i=0;i<a.size();i++)
        res+=getS(a[i],a[(i+1)%a.size()]);
    return fabs(res);
}
```

8.3 三角剖分 2

```
//#define double long double
//const double eps=1e-12;
using namespace std;
const double eps=1e-8;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//向量单位化；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
}
double angle(vec a,vec b){
    double val=(a&b)/len(a)/len(b);
    val=max(-1.0,min(1.0,val));
    return acos(val);
}
double area(vec a,vec b){return a*b;}
```

```
int cross(pit a,pit b,pit c){
    if((b-a)*(c-a)>eps) return 1;
    else if((b-a)*(c-a)<-eps) return -1;
    return 0;
}
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=eps;
}
bool lcross(pit a,pit b,pit c,pit d){
    if(cross(a,b,c)*cross(a,b,d)>0) return 0;
    return 1;
}
bool scross(pit a,pit b,pit c,pit d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    if(c1==0&&onSeg(a,b,c)) return 1;
    if(c2==0&&onSeg(a,b,d)) return 1;
    if(c3==0&&onSeg(c,d,a)) return 1;
    if(c4==0&&onSeg(c,d,b)) return 1;
    return c1*c2<0&&c3*c4<0;
}
bool pcross(pit a,pit b,pit c,pit d){
    if(fabs((b-a)*(d-c))<=eps) return 0;
    return 1;
}
//求两条不平行直线ab、cd的交点；要求两条方向向量非零且不平行
pit getNode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//点向式求两条不平行直线的交点；要求u、v非零且不平行
pit getNode(pit a,vec u,pit c,vec v){
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//O(n^2) 三角剖分 把逆时针传入的一个多边形剖分成on个三角形
vector<vector<pit>> triangulate(vector<pit> p){
    int n=p.size();
    if(n==3) return {p};
    vector<vector<pit>> res;
    vector<int> nxt(n),pre(n),in(n,1),ear(n,0);//in 是否被删除
    for(int i=0;i<n;++i) nxt[i]=(i+1)%n,pre[i]=(i-1+n)%n;
    auto chk=[&](int u){
        int pu=pre[u],nu=nxt[u];
        if(cross(p[pu],p[u],p[nu])!=1) return 0;
        for(int j=nxt[nu];j!=pu;j=nxt[j])
            if(cross(p[pu],p[u],p[j])>=0&&cross(p[u],p[nu],p[j])>=0&&cross(p[nu],p[pu],p[j])>=0) return 0;
        return 1;
    };//判断p是否是耳朵点 即和前驱和后继是否能形成一个凸三角形 且没有点在其内部
    queue<int> q; //维护耳朵点
    for(int i=0;i<n;++i) if(ear[i]=chk(i)) q.push(i);
    int cnt=n;
    while(cnt>2&&!q.empty()){
        int u=q.front();q.pop();
        if(!in[u]||!ear[u])continue;
        int pu=pre[u],nu=nxt[u];
        res.push_back({p[pu],p[u],p[nu]});
        in[u]=0;nxt[pu]=nu;pre[nu]=pu;cnt--; //删除这个点
        if(ear[pu]=chk(pu)) q.push(pu); //判断前驱是否是耳朵点
        if(ear[nu]=chk(nu)) q.push(nu); //判断后继是否是耳朵点
    }
    return res;
}
```

8.4 凸包

```
//凸包：给定点集，求周长最小凸多边形围住它们 Andrew算法
//O(nlogn)，不保证逆时针/顺时针，保证有序
```

```
//#define double long double
//const double eps=1e-12;
using namespace std;
const double eps=1e-8;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
```

```
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
bool cmp(const pit& a,const pit& b){
    if(a.x!=b.x)return a.x<b.x;
    return a.y<b.y;
}
//ab x ac
double cross(pit a,pit b,pit c){
    return (b-a)*(c-a);
}
//要求输入至少能形成非退化凸包；单点/两点/全共线需先特判
pair<double,vector<pit>> Andrew(vector<pit> p)
{
    sort(p.begin(),p.end(),cmp);
    vector<pit> st(p.size()+5,{0,0});
    int top=0;
    for(int i=0;i<p.size();i++)
    {
        while(top>1&&cross(st[top-2],st[top-1],p[i])<=eps)top--;
        //<=eps 三点共线不算 <=-eps 三点共线算
        st[top++]=p[i];
    }//下凸包
    int k=top;
    for(int i=p.size()-2;i>=0;i--)
    {
        while(top>k&&cross(st[top-2],st[top-1],p[i])<=eps)top--;
        st[top++]=p[i];
    }//上凸包
    double res=0;
    for(int i=0;i<top-1;i++)res+=len(st[i+1]-st[i]);
    st.resize(top-1);
    return {res,st};
}
//计算多边形面积
double TA(vector<pit> p)
{
    int n=p.size();
    double res=0;
    for(int i=0;i<n;i++){
        res+=p[i].x*p[(i+1)%n].y-p[i].y*p[(i+1)%n].x;
    }
    return fabs(res)/2;
}
//判断凸包是否逆时针,不然要翻转
void rev(vector<pit>& p)
{
    int n=p.size();
    double res=0;
    for(int i=0;i<n;i++){
        res+=p[i].x*p[(i+1)%n].y-p[i].y*p[(i+1)%n].x;
    }
    if(res<=-eps) reverse(p.begin(),p.end());
}
//判断p点是否在三角形abc内
bool isCon(pit a,pit b,pit c,pit p){
    return cross(a,b,p)>=-eps&&cross(b,c,p)>=-eps&&cross(c,a,p)>=-eps;
}
//二分判断点是否在凸包内O(logn)
//需保证凸包逆时针，边界计入
bool isConvex(const vector<pit>& p,pit a)
{
    int n=p.size();
    if(n<3) return false;
    double c1=(p[1]-p[0])*(a-p[0]);
    double c2=(p[n-1]-p[0])*(a-p[0]);
    if(c1<-eps||c2>eps) return false;
    if(fabs(c1)<=eps) return ((p[0]-a)&(p[1]-a))<=eps;
    if(fabs(c2)<=eps) return ((p[0]-a)&(p[n-1]-a))<=eps;
    int l=1,r=n-1;
    while(r-l>1)
    {
        int mid=(l+r)>>1;
        if((p[mid]-p[0])*(a-p[0])>=-eps)
        {
            l=mid;
        }
        else r=mid;
    }
    return isCon(p[0],p[l],p[r],a);
}
```

8.5 凸包闵和

```
//#define double long double
//const double eps=1e-12;
using namespace std;
const double eps=1e-8;
struct vec{
    double x,y;
```

```
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//向量单位化；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
}
double angle(vec a,vec b){
    double val=(a&b)/len(a)/len(b);
    val=max(-1.0,min(1.0,val));
    return acos(val);
}
double area(vec a,vec b){return a*b;}
int cross(pit a,pit b,pit c){
    if((b-a)*(c-a)>eps) return 1;
    else if((b-a)*(c-a)<-eps) return -1;
    return 0;
}
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=eps;
}
bool lcross(pit a,pit b,pit c,pit d){
    if(cross(a,b,c)*cross(a,b,d)>0) return 0;
    return 1;
}
bool scross(pit a,pit b,pit c,pit d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    if(c1==0&&onSeg(a,b,c)) return 1;
    if(c2==0&&onSeg(a,b,d)) return 1;
    if(c3==0&&onSeg(c,d,a)) return 1;
    if(c4==0&&onSeg(c,d,b)) return 1;
    return c1*c2<0&&c3*c4<0;
}
bool pcross(pit a,pit b,pit c,pit d){
    if(fabs((b-a)*(d-c))<=eps) return 0;
    return 1;
}
//求两条不平行直线ab、cd的交点；要求两条方向向量非零且不平行
pit getNode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//点向式求两条不平行直线的交点；要求u、v非零且不平行
pit getNode(pit a,vec u,pit c,vec v){
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//O(n+m)凸包闵和；要求a、b非空、为凸多边形、按逆时针排列且不带重复首点
//同向边可能保留共线中间点；空输入会在后续访问a[0]或b[0]时崩溃
vector<pit> minkowski(vector<pit> a,vector<pit> b){
    auto cmp=[](const pit& p1,const pit& p2){
        if(p1.y!=p2.y)return p1.y<p2.y;
        return p1.x<p2.x;
    };
    rotate(a.begin(),min_element(a.begin(),a.end(),cmp),a.end());
    rotate(b.begin(),min_element(b.begin(),b.end(),cmp),b.end());//以最左下角的点为起点
    int n=a.size(),m=b.size();
    vector<vec> va(n),vb(m);
    for(int i=0;i<n;++i) va[i]=a[(i+1)%n]-a[i];
    for(int i=0;i<m;++i) vb[i]=b[(i+1)%m]-b[i];
    vector<pit> res;
    res.push_back(a[0]+b[0]);
    int i=0,j=0; //做归并排序
    while(i<n&&j<m)res.push_back(res.back()+(va[i]*vb[j]>=0?va[i++]:vb[j++]));
    while(i<n) res.push_back(res.back()+va[i++]);
    while(j<m) res.push_back(res.back()+vb[j++]);
    res.pop_back(); //闭合会回到起点，弹掉最后一个重复点
    return res;
}
```

8.6 凸包面积并

```
//#define double long double
//const double eps=1e-12;
using namespace std;
const double eps=1e-8;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//向量单位化；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
}
double angle(vec a,vec b){
    double val=(a&b)/len(a)/len(b);
    val=max(-1.0,min(1.0,val));
    return acos(val);
}
double area(vec a,vec b){return a*b;}
int cross(pit a,pit b,pit c){
    if((b-a)*(c-a)>eps) return 1;
    else if((b-a)*(c-a)<-eps) return -1;
    return 0;
}
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=eps;
}
bool lcross(pit a,pit b,pit c,pit d){
    if(cross(a,b,c)*cross(a,b,d)>0) return 0;
    return 1;
}
bool scross(pit a,pit b,pit c,pit d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    if(c1==0&&onSeg(a,b,c)) return 1;
    if(c2==0&&onSeg(a,b,d)) return 1;
    if(c3==0&&onSeg(c,d,a)) return 1;
    if(c4==0&&onSeg(c,d,b)) return 1;
    return c1*c2<0&&c3*c4<0;
}
bool pcross(pit a,pit b,pit c,pit d){
    if(fabs((b-a)*(d-c))<=eps) return 0;
    return 1;
}
//求两条不平行直线ab、cd的交点；要求两条方向向量非零且不平行
pit getNode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//点向式求两条不平行直线的交点；要求u、v非零且不平行
pit getNode(pit a,vec u,pit c,vec v){
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//O(v^2+nvlogn) 凸包面积并(逆时针) v为凸包总点数
//做法是 找出外轮廓 然后用鞋带公式算
double area_union(const vector<vector<pit>>& polys){
    double ans=0;
    int n=polys.size();
    for(int i=0;i<n;++i){
        int szi=polys[i].size();
        for(int e=0;e<szi;++e){
            pit A=polys[i][e],B=polys[i][(e+1)%szi];
            vec AB=B-A;
            vector<pair<double,double>> cov;
            for(int j=0;j<n;++j){
                if(i==j)continue;
                int szj=polys[j].size();
                double L=0,R=1.0;
                bool valid=1;
                for(int k=0;k<szj;++k){
                    pit C=polys[j][k],D=polys[j][(k+1)%szj];
```

```
                    vec DC=D-C,CA=A-C;
                    double c1=DC*CA,c2=DC*AB;
                    if(c2>eps)L=max(L,-c1/c2);
                    else if(c2<-eps)R=min(R,-c1/c2);
                    else{
                        if(c1<-eps){valid=0;break;}
                        else if(c1<=eps&&(AB&DC)>eps&&i>j){valid=0;break;}
                    }
                }
                if(valid&&L<=R)cov.push_back({L,R});
            }
            sort(cov.begin(),cov.end());
            double cur=0;
            for(auto& p:cov){
                if(p.first>cur+eps){
                    pit P1=A+AB*cur,P2=A+AB*p.first;
                    ans+=(P1.x*P2.y-P1.y*P2.x);
                }
                cur=max(cur,p.second);
            }
            if(cur<1.0-eps){
                pit P1=A+AB*cur;
                ans+=(P1.x*B.y-P1.y*B.x);
            }
        }
    }
    return ans/2.0;
}
```

8.7 向量

```
//#define double long double
//const double eps=1e-12;
using namespace std;
const double eps=1e-8;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
//向量逆时针旋转theta弧度
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//向量单位化；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
}
//用单位圆证明
//向量夹角 dot(a,b)=len(a)*len(b)*cos(θ)
double angle(vec a,vec b){
    double val=(a&b)/len(a)/len(b);
    val=max(-1.0,min(1.0,val));
    return acos(val);
}
//向量围成的平行四边形面积,b在a的逆时针方向为正，否则为负
double area(vec a,vec b){return a*b;}
//点线关系(点c,直线ab)
int cross(pit a,pit b,pit c){
    if((b-a)*(c-a)>eps) return 1; //c在ab的逆时针方向
    else if((b-a)*(c-a)<-eps) return -1; //c在ab的顺时针方向
    return 0; //c,a,b共线
}
//判断点在线段上(p在ab上)
bool onSeg(pit a,pit b,pit p){
    return cross(a,b,p)==0&&((a-p)&(b-p))<=eps;
}
//线线关系
//case1:直线ab与线段cd
bool lcross(pit a,pit b,pit c,pit d){
    if(cross(a,b,c)*cross(a,b,d)>0) return 0;//c,d在ab的同一侧 无交点
    return 1; //有交点
}
```

```
//case2:线段ab与线段cd
bool scross(pit a,pit b,pit c,pit d){
    int c1=cross(a,b,c),c2=cross(a,b,d),c3=cross(c,d,a),c4=cross(c,d,b);
    //共线时还要判断投影是否落在线段上
    if(c1==0&&onSeg(a,b,c)) return 1;
    if(c2==0&&onSeg(a,b,d)) return 1;
    if(c3==0&&onSeg(c,d,a)) return 1;
    if(c4==0&&onSeg(c,d,b)) return 1;
    return c1*c2<0&&c3*c4<0; //严格跨立 有交点
}
//case3:直线ab与直线cd
bool pcross(pit a,pit b,pit c,pit d){
    if(fabs((b-a)*(d-c))<=eps) return 0; //平行 无交点
    return 1; //有交点
}
//求两条不平行直线ab、cd的交点；要求两条方向向量非零且不平行
pit getNode(pit a,pit b,pit c,pit d){
    vec u=b-a,v=d-c;
    //assert(fabs(u*v)<=eps);
    //if(fabs(u*v)<=eps) return pit(NAN,NAN); //平行 无交点
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//点向式求两条不平行直线的交点；要求u、v非零且不平行
pit getNode(pit a,vec u,pit c,vec v){
    //assert(fabs(u*v)<=eps);
    //if(fabs(u*v)<=eps) return pit(NAN,NAN); //平行 无交点
    double t=((c-a)*v)/(u*v);
    return a+u*t;
}
//返回点p到闭线段ab的最短距离
double disPtSeg(pit p,pit a,pit b){
    vec ab=b-a,ap=p-a,bp=p-b;
    if(len(ab)<=eps)return dis(p,a);
    if((ab&ap)<=-eps)return dis(p,a);
    if(((a-b)&bp)<=-eps)return dis(p,b);
    return fabs(ab*ap)/len(ab);
}
```

8.8 旋转卡壳

```
//凸包：给定点集，求周长最小凸多边形围住它们 Andrew算法
//O(nlogn)，不保证逆时针/顺时针，保证有序
```

```
#define double long double
const double eps=1e-12;
using namespace std;
//const double eps=1e-8;
const double PI=acos(-1);
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
double len(const vec& o){return sqrt(o.x*o.x+o.y*o.y);} //向量模长
double dis(const pit& a,const pit& b){return len(b-a);} //两点距离
bool cmp(const pit& a,const pit& b){
    if(a.x!=b.x)return a.x<b.x;
    return a.y<b.y;
}
//向量逆时针旋转theta弧度
vec rotate(const vec& o,double theta){
    return vec(o.x*cos(theta)-o.y*sin(theta),o.x*sin(theta)+o.y*cos(theta));
}
//ab x ac
double cross(pit a,pit b,pit c){
    return (b-a)*(c-a);
}
//ab·ac
double dot(pit a,pit b,pit c){
    return (b-a)&(c-a);
}
//向量单位化；要求a为非零向量，否则会产生NaN
vec norm(vec a){
    return a/len(a);
```

```
}
pair<double,vector<pit>> Andrew(vector<pit> p)
{
    sort(p.begin(),p.end(),cmp);
    vector<pit> st(p.size()+5,{0,0});
    int top=0;
    for(int i=0;i<p.size();i++)
    {
        while(top>1&&cross(st[top-2],st[top-1],p[i])<=eps)top--;
        //<=eps 三点共线不算 <=-eps 三点共线算
        st[top++]=p[i];
    }//下凸包
    int k=top;
    for(int i=p.size()-2;i>=0;i--)
    {
        while(top>k&&cross(st[top-2],st[top-1],p[i])<=eps)top--;
        st[top++]=p[i];
    }//上凸包
    double res=0;
    for(int i=0;i<top-1;i++)res+=len(st[i+1]-st[i]);
    st.resize(top-1);
    return {res,st};
}
//计算多边形面积
double TA(vector<pit> p)
{
    int n=p.size();
    double res=0;
    for(int i=0;i<n;i++){
        res+=p[i].x*p[(i+1)%n].y-p[i].y*p[(i+1)%n].x;
    }
    return fabs(res)/2;
}
//判断凸包是否逆时针,不然要翻转
void rev(vector<pit>& p)
{
    int n=p.size();
    double res=0;
    for(int i=0;i<n;i++){
        res+=p[i].x*p[(i+1)%n].y-p[i].y*p[(i+1)%n].x;
    }
    if(res<=-eps) reverse(p.begin(),p.end());
}
//判断p点是否在三角形abc内
bool isCon(pit a,pit b,pit c,pit p){
    return cross(a,b,p)>=-eps&&cross(b,c,p)>=-eps&&cross(c,a,p)>=-eps;
}
//二分判断点是否在凸包内O(logn)
//需保证凸包逆时针, 边界计入
bool isConvex(const vector<pit>& p,pit a)
{
    int n=p.size();
    if(n<3) return false;
    double c1=(p[1]-p[0])*(a-p[0]);
    double c2=(p[n-1]-p[0])*(a-p[0]);
    if(c1<-eps||c2>eps) return false;
    if(fabs(c1)<=eps) return ((p[0]-a)&(p[1]-a))<=eps;
    if(fabs(c2)<=eps) return ((p[0]-a)&(p[n-1]-a))<=eps;
    int l=1,r=n-1;
    while(r-l>1)
    {
        int mid=(l+r)>>1;
        if((p[mid]-p[0])*(a-p[0])>=-eps)
        {
            l=mid;
        }
        else r=mid;
    }
    return isCon(p[0],p[l],p[r],a);
}
//双指针/多指针在凸包上找最优->旋转卡壳 形如一个游标卡尺绕着凸包旋转
//旋转卡壳,用叉积可以找离一条线垂直最高或最低, 用点积可以找离一条线水平最左或最右的点 (点积的几何意义是b在a的投影长度)
//要求传入非退化逆时针凸包, 点数至少为3; 单点/两点/全共线需先特判
pair<double,vector<pit>> rot(vector<pit> p)
{
    double ans=1e14;vector<pit> fin(4);
    int n=p.size(),a=1,b=1,c;
    for(int i=0;i<n;i++)
    {
        while(cross(p[i],p[(i+1)%n],p[a])-cross(p[i],p[(i+1)%n],p[(a+1)%n])<=-eps) a=(a+1)%n;
        while(dot(p[i],p[(i+1)%n],p[b])-dot(p[i],p[(i+1)%n],p[(b+1)%n])<=-eps) b=(b+1)%n;
        if(i==0) c=a;
        while(dot(p[(i+1)%n],p[i],p[c])-dot(p[(i+1)%n],p[i],p[(c+1)%n])<=-eps) c=(c+1)%n;
        double d=dis(p[i],p[(i+1)%n]);
        double H=fabs(cross(p[a],p[i],p[(i+1)%n]))/d;
        double R=dot(p[i],p[(i+1)%n],p[b])/d;
        double L=dot(p[(i+1)%n],p[i],p[c])/d;
        if(ans>(R+L-d)*H)
        {
            ans=(R+L-d)*H;
            vec nor1=norm(p[(i+1)%n]-p[i]);//i->i+1
            vec nor2=norm(p[i]-p[(i+1)%n]);//i+1->i
```

```
            fin[0]=p[i]+nor1*R;
            fin[1]=p[(i+1)%n]+nor2*L;
            fin[2]=fin[1]+rotate(nor1,PI/2)*H;
            fin[3]=fin[0]+rotate(nor1,PI/2)*H;
        }
    }
    return {ans,fin};
}
void zero(pit& a)
{
    if(fabs(a.x)<eps) a.x=0;
    if(fabs(a.y)<eps) a.y=0;
}
```

用法示例:

```
    int n;cin>>n;
    vector<pit> p(n);
    for(int i=0;i<n;i++)cin>>p[i].x>>p[i].y;
    auto [_,st]=Andrew(p);
    auto [ans,fin]=rot(st);
    printf("%.5Lf\n",ans);
    int k=0;
    reverse(fin.begin(),fin.end());
    for(int i=0;i<=3;i++) if(cmp(fin[i],fin[k])) k=i;
    for(int i=k;i<=k+3;i++)
    {
        zero(fin[i%4]);
        printf("%.5Lf %.5Lf\n",fin[i%4].x,fin[i%4].y);
    }
```

8.9 极角排序

```
//极角排序：将点按照极角排序(逆时针)，即以某点为极点，将点按照与x轴的夹角从小到大排序
```

```
//#define double long double
//const double eps=1e-12;
using namespace std;
struct vec{
    double x,y;
    vec(double x=0,double y=0):x(x),y(y){}
    vec operator+(const vec& o)const{return vec(x+o.x,y+o.y);}
    vec operator-(const vec& o)const{return vec(x-o.x,y-o.y);}
    vec operator/(const double& o)const{return vec(x/o,y/o);} //数除
    vec operator*(const double& o)const{return vec(x*o,y*o);} //数乘
    double operator*(const vec& o)const{return x*o.y-y*o.x;} //叉积
    double operator&(const vec& o)const{return x*o.x+y*o.y;} //点积
};
struct pit
{
    double x,y;
    pit(double x=0,double y=0):x(x),y(y){}
    vec operator-(const pit& o)const{return vec(x-o.x,y-o.y);}
    pit operator+(const vec& o)const{return pit(x+o.x,y+o.y);}
    pit operator+(const pit& o)const{return pit(x+o.x,y+o.y);}
    pit operator/(const double& o)const{return pit(x/o,y/o);}
};
void psort(vector<pit>& a)
{
    pit cen(0,0);
    for(auto& i:a) cen=cen+i;
    cen=cen/a.size();
    sort(a.begin(),a.end(),[&](pit a,pit b){
        double angA=atan2(a.y-cen.y,a.x-cen.x);
        double angB=atan2(b.y-cen.y,b.x-cen.x);
        return angA<angB;
    });
}
```

9 其他

9.1 bitset 操作手册

用法示例:

```
    // 竞赛标配 IO 优化
    cin.tie(0);
    // ========================================
    // 1. 初始化构造 (时间复杂度: O(N/w))
    // ========================================
    bitset<16> a;                    // 默认全 0
    bitset<16> b(170);               // 整数初始化: 0000000010101010
    bitset<16> c(string("11001100")); // 字符串初始化
    // ========================================
    // 2. 整体按位运算 (时间复杂度: O(N/w))
    // ========================================
    bitset<16> res;
    res = b & c;    // 按位与
    res = b | c;    // 按位或
    res = b ^ c;    // 按位异或
    res = ~b;       // 按位取反
    res = b << 2;   // 整体左移
    res = b >> 2;   // 整体右移
    // ========================================
    // 3. 整体修改 (时间复杂度: O(N/w))
    // ========================================
    a.set();        // 所有位置 1
    a.reset();      // 所有位置 0
    a.flip();       // 所有位取反
    // ========================================
    // 4. 单点修改与查询 (时间复杂度: 严格 O(1))
    // ========================================
    // 【重点】直接使用 [] 操作符: 不带越界检查, 常数极小, 支持直接修改和相互赋值
    b[2] = 1;       // 直接修改第 2 位为 1
    b[5] = 0;       // 直接修改第 5 位为 0
    b[3] = b[2];    // 支持位与位之间的相互赋值
    b[4] = ~b[2];   // 支持取反后赋值
    // 传统内置函数 (常数稍大):
    b.set(2);       // 将第 2 位设为 1
    b.set(3, 0);    // 将第 3 位设为 0
    b.reset(4);     // 将第 4 位设为 0
    b.flip(5);      // 将第 5 位取反
    bool v = b.test(2); // 查询第 2 位 (自带越界检查, 越界抛异常)
    // ========================================
    // 5. 状态统计与判定 (除 size() 外, 时间复杂度: O(N/w))
    // ========================================
    int cnt = b.count();  // 统计 1 的个数 (底层调用 CPU popcount 指令)
    bool s1 = b.any();    // 是否存在至少一个 1
    bool s2 = b.none();   // 是否全为 0
    bool s3 = b.all();    // 是否全为 1 (C++11起)
    int sz = b.size();    // 返回位数 (此处为 16, 时间复杂度 O(1))
    // ========================================
    // 6. GCC 扩展: 高效遍历 1 的位置 (时间复杂度: O(N/w + c))
    // ========================================
    bitset<1000> mask;
    mask[10] = 1;
    mask[666] = 1;
    mask[999] = 1;
    // _Find_first() 返回最低位的 1 的下标, 若没有则返回 size()
    // _Find_next(i) 返回下标 i 之后第一个 1 的下标, 若没有则返回 size()
    // 这俩个函数, 时间复杂度: O(N/w)
    // 配合 for 循环, 跳跃次数严格等于 1 的个数 c, 扫描空白段的开销被压缩了 64 倍
    for (int i = mask._Find_first(); i < mask.size(); i = mask._Find_next(i)) {
        cout << i << " "; // 将输出 10 666 999
    }
    cout << "\n";
```

9.2 SWAG(滑动窗口聚合)

```
constexpr ll INF=1e18;
struct Matrix{
    ll m[2][2]={{INF,INF},{INF,INF}};
    Matrix():m{{0,INF},{INF,0}}{}
    Matrix(ll a,ll b,ll c,ll d):m{{a,b},{c,d}}{}
    Matrix operator*(const Matrix& o) const{
        Matrix r(INF,INF,INF,INF);
        for(int i:{0,1})
            for(int k:{0,1})
                for (int j:{0,1})
                    r.m[i][j]=min(r.m[i][j],m[i][k]+o.m[k][j]);
        return r;
    }
};
template<typename T,class Op>
struct SWAG{
```

```
    struct Node{
        T val;
        T agg;
    };
    vector<Node> front,back;
    Op op; //满足结合律的运算
    T id;  //单位元
    SWAG(Op op,T id):op(op),id(id){}
    void push(const T& x){
        if(back.empty()) back.push_back({x,x});
        else back.push_back({x,op(back.back().agg,x)});
    }
    void pop(){
        if(front.empty()){
            while(!back.empty()){
                T x=back.back().val;
                back.pop_back();
                if(front.empty()) front.push_back({x,x});
                else front.push_back({x,op(x,front.back().agg)});
            }
        }
        if(!front.empty()) front.pop_back();
    }
    T qry(){
        if(back.empty()&&front.empty()) return id;
        if(back.empty()) return front.back().agg;
        if(front.empty()) return back.back().agg;
        return op(front.back().agg,back.back().agg);
    }
};
//一个比较经典的例子是 front从顶到底是{{B,BCD},{C,CD},{D,D}} back从顶到底是{{G,EFG},{F,EF},{E,E}}
//可以发现front是左乘，back是右乘，这样就可以用SWAG维护有结合律的玩意了
//不知道单元元的时候可以传个异常元进去
//使用例：
// auto mul=[](const Matrix& a,const Matrix& b){
//     return b*a;
// };
// SWAG<Matrix,decltype(mul)> q(mul,Matrix());
// 注意矩阵乘法的顺序QAQ
```

用法示例:

```
    int t;cin>>t;
    while(t--){
        int n,k;cin>>n>>k;
        vector<int> a(n+1);
        for(int i=1;i<=n;i++) cin>>a[i];
        if(k==1){
            cout<<*min_element(a.begin()+1,a.end())<<'\n';
            continue;
        }
        vector<Matrix> m(n+1);
        for(int i=1;i<=n;i++){
            m[i]=Matrix(a[i],a[i],0,INF);
        }
        auto mul=[](const Matrix& a,const Matrix& b){
            return b*a;
        };
        auto slove=[&](int w){
            if(w<1) return INF;
            SWAG<Matrix,decltype(mul)> q(mul,Matrix());
            ll res=INF;
            for(int i=1;i<=n;i++){
                q.push(m[i]);
                if(i>w) q.pop();
                if(i>=w){
                    int l=i-w;
                    if(l>=1){
                        res=min(res,q.qry().m[0][0]+a[l]);
                    }
                }
            }
            return res;
        };
        cout<<min(slove(k),slove(k-1))<<'\n';
    }
```

9.3 动态 bitset(0base)

```
//0-based: 有效下标 [0,n)
struct DyBitset0Base
{
    using ull=uint64_t;
    int n; vector<ull> a;
    struct ref
    {
        ull& block;
        const ull st;
        ref& operator=(bool x){
            if(x) block|=st;
            else block&=~st;
            return *this;
```

```
        }
        ref& operator=(const ref& rhs){ return *this=(bool)rhs; }
        operator bool() const { return block&st; }
    };

    DyBitset0Base(int _n):n(_n){
        assert(n>=0);
        a.resize((n+63)/64,0);
    }
    void sant(){
        if(a.empty()) return;
        if(n%64){
            a.back()&=((1ull<<(n%64))-1);
        }
    }

    void set(int i){ assert(0<=i&&i<n); a[i>>6]|=(1ull<<(i&63)); } //置1
    void reset(int i){ assert(0<=i&&i<n); a[i>>6]&=~(1ull<<(i&63)); } //置0
    void clear(){ fill(a.begin(),a.end(),0); } //清空
    bool test(int i) const{ assert(0<=i&&i<n); return (a[i>>6]>>(i&63))&1; } //查询
    bool operator[](int i) const{ return test(i); } //下标访问
    ref operator[](int i){ assert(0<=i&&i<n); return {a[i>>6],1ull<<(i&63)}; } //下标修改
    int count() const{
        int res=0;
        for(auto x:a) res+=__builtin_popcountll(x);
        return res;
    }
    DyBitset0Base operator~() const {
        DyBitset0Base res(n);
        for(int i=0;i<a.size();i++){
            res.a[i]=~a[i];
        }
        res.sant();
        return res;
    }
    DyBitset0Base operator&(const DyBitset0Base& rhs) const{
        DyBitset0Base res(max(n,rhs.n));
        for(int i=0;i<min(a.size(),rhs.a.size());i++){
            res.a[i]=a[i]&rhs.a[i];
        }
        res.sant();
        return res;
    }
    DyBitset0Base operator|(const DyBitset0Base& rhs) const{
        DyBitset0Base res(max(n,rhs.n));
        for(int i=0;i<min(a.size(),rhs.a.size());i++){
            res.a[i]=a[i]|rhs.a[i];
        }
        const auto& rs=(a.size()>rhs.a.size()?a:rhs.a);
        for(int i=min(a.size(),rhs.a.size());i<rs.size();i++){
            res.a[i]=rs[i];
        }
        res.sant();
        return res;
    }
    DyBitset0Base operator^(const DyBitset0Base& rhs) const{
        DyBitset0Base res(max(n,rhs.n));
        for(int i=0;i<min(a.size(),rhs.a.size());i++){
            res.a[i]=a[i]^rhs.a[i];
        }
        const auto& rs=(a.size()>rhs.a.size()?a:rhs.a);
        for(int i=min(a.size(),rhs.a.size());i<rs.size();i++){
            res.a[i]=rs[i];
        }
        res.sant();
        return res;
    }
    bool operator==(const DyBitset0Base& rhs) const{
        if(n!=rhs.n) return false;
        return a==rhs.a;
    }
    bool operator!=(const DyBitset0Base& rhs) const{
        return !(*this==rhs);
    }
    bool operator<(const DyBitset0Base& rhs) const{
        //按高位到低位比较数值大小，数值相同时再按长度排序
        int sz=max(a.size(),rhs.a.size());
        for(int i=sz-1;i>=0;i--){
            ull x=(i<a.size()?a[i]:0);
            ull y=(i<rhs.a.size()?rhs.a[i]:0);
            if(x!=y) return x<y;
        }
        return n<rhs.n;
    }
    int ctz() const{
        for(int i=0;i<a.size();i++){
            if(a[i]){
                return (i<<6)+__builtin_ctzll(a[i]);
            }
        }
        return n;
    }
};
```

9.4 动态 bitset(1base)

```
//1-based: 有效下标 [1,n]
struct DyBitset1Base
{
    using ull=uint64_t;
    int n; vector<ull> a;
    struct ref
    {
        ull& block;
        const ull st;
        ref& operator=(bool x){
            if(x) block|=st;
            else block&=~st;
            return *this;
        }
        ref& operator=(const ref& rhs){ return *this=(bool)rhs; }
        operator bool() const { return block&st; }
    };

    DyBitset1Base(int _n):n(_n){
        assert(n>=0);
        a.resize((n+64)/64,0);
    }
    void sant(){
        a[0]&=~1ull;
        if((n+1)%64){
            a.back()&=((1ull<<((n+1)%64))-1);
        }
    }

    void set(int i){ assert(1<=i&&i<=n); a[i>>6]|=(1ull<<(i&63)); } //置1
    void reset(int i){ assert(1<=i&&i<=n); a[i>>6]&=~(1ull<<(i&63)); } //置0
    void clear(){ fill(a.begin(),a.end(),0); } //清空
    bool test(int i) const{ assert(1<=i&&i<=n); return (a[i>>6]>>(i&63))&1; } //查询
    bool operator[](int i) const{ return test(i); } //下标访问
    ref operator[](int i){ assert(1<=i&&i<=n); return {a[i>>6],1ull<<(i&63)}; } //下标修改
    int count() const{
        int res=0;
        for(auto x:a) res+=__builtin_popcountll(x);
        return res;
    }
    DyBitset1Base operator~() const {
        DyBitset1Base res(n);
        for(int i=0;i<a.size();i++){
            res.a[i]=~a[i];
        }
        res.sant();
        return res;
    }
    DyBitset1Base operator&(const DyBitset1Base& rhs) const{
        DyBitset1Base res(max(n,rhs.n));
        for(int i=0;i<min(a.size(),rhs.a.size());i++){
            res.a[i]=a[i]&rhs.a[i];
        }
        res.sant();
        return res;
    }
    DyBitset1Base operator|(const DyBitset1Base& rhs) const{
        DyBitset1Base res(max(n,rhs.n));
        for(int i=0;i<min(a.size(),rhs.a.size());i++){
            res.a[i]=a[i]|rhs.a[i];
        }
        const auto& rs=(a.size()>rhs.a.size()?a:rhs.a);
        for(int i=min(a.size(),rhs.a.size());i<rs.size();i++){
            res.a[i]=rs[i];
        }
        res.sant();
        return res;
    }
    DyBitset1Base operator^(const DyBitset1Base& rhs) const{
        DyBitset1Base res(max(n,rhs.n));
        for(int i=0;i<min(a.size(),rhs.a.size());i++){
            res.a[i]=a[i]^rhs.a[i];
        }
        const auto& rs=(a.size()>rhs.a.size()?a:rhs.a);
        for(int i=min(a.size(),rhs.a.size());i<rs.size();i++){
            res.a[i]=rs[i];
        }
        res.sant();
        return res;
    }
    bool operator==(const DyBitset1Base& rhs) const{
        if(n!=rhs.n) return false;
        return a==rhs.a;
    }
    bool operator!=(const DyBitset1Base& rhs) const{
        return !(*this==rhs);
    }
    bool operator<(const DyBitset1Base& rhs) const{
        //按高位到低位比较数值大小，数值相同时再按长度排序
        int sz=max(a.size(),rhs.a.size());
        for(int i=sz-1;i>=0;i--){
```

```
            ull x=(i<a.size()?a[i]:0);
            ull y=(i<rhs.a.size()?rhs.a[i]:0);
            if(x!=y) return x<y;
        }
        return n<rhs.n;
    }
    int ctz() const{
        for(int i=0;i<a.size();i++){
            if(a[i]){
                return (i<<6)+__builtin_ctzll(a[i]);
            }
        }
        return n+1;
    }
};
```

9.5 整体二分

```
//整体二分(离线算法) agc2 d
//把q次二分操作放在一起 对询问整体做二分
//一般的二分是考虑区间[l,r],每次二分后变成[l,mid]和[mid+1,r],对单个询问
//整体二分是考虑区间[l,r],每次二分后变成[l,mid]和[mid+1,r],对多个询问 把询问分到左右区间
//时间复杂度分析我们可以把他变成一颗线段树来看
//通常 我们在一层递归的时候我们要保证[1,l-1]的信息存在
//考虑同层的区间修改是常数的 所以修改操作是O(nlogn)的
```

```
class REDSU{
public:
    vector<int> fa,sz;
    int n;vector<array<int,2>> st;
    REDSU(int n):n(n){
        fa.resize(n+5);
        sz.resize(n+5);
        st.reserve(n+5);
        for(int i=1;i<=n;i++){
            fa[i]=i;
            sz[i]=1;
        }
    }
    int find(int x){
        while(x!=fa[x]) x=fa[x];
        return x;
    }
    bool same(int x,int y){
        return find(x)==find(y);
    }
    void merge(int x,int y){
        x=find(x),y=find(y);
        if(x==y)
        {
            st.push_back({0,y});
            return;
        }
        if(sz[x]<sz[y]) swap(x,y); //sz[x]>=sz[y]
        st.push_back({1,y});sz[x]+=sz[y];fa[y]=x;
    }
    int size(int x){return sz[find(x)];}
    void back(){
        if(!st.empty()){
            auto [fg,y]=st.back();st.pop_back();
            if(!fg) return;
            sz[fa[y]]-=sz[y];fa[y]=y;
        }
    }
    void back_k(int k){
        while(k--) back();
    }
};
```

用法示例:

```
    int n,m;cin>>n>>m;
    REDSU dsu(n+5);
    vector<vector<array<int,2>>> mp(n+1);
    vector<array<int,2>> ed(m+5);
    for(int i=1;i<=m;i++)
    {
        int u,v,w=i;cin>>u>>v;
        mp[u].push_back({v,w});
        mp[v].push_back({u,w});
        ed[w]={u,v};
    }
    int q;cin>>q;
    vector<array<int,4>> qr;
    for(int i=1;i<=q;i++)
    {
        int x,y,z;cin>>x>>y>>z;
        qr.push_back({x,y,z,i});
    }
    vector<int> ans(q+1);
    auto sol=[&](this auto&& sol,int l,int r,vector<array<int,4>>& qx)->void{
        int mid=(l+r)>>1;
```

```
        if(l==r){
            for(auto &[_,__,___,id]:qx){
                ans[id]=l;
            }
            return ;
        }
        for(int i=l;i<=mid;i++){
            dsu.merge(ed[i][0],ed[i][1]);
        }
        vector<array<int,4>> q1,q2;
        for(auto &[x,y,z,id]:qx){
            int now=0;
            if(dsu.same(x,y)) now=dsu.size(x);
            else now=dsu.size(x)+dsu.size(y);
            if(now>=z) q1.push_back({x,y,z,id});
            else q2.push_back({x,y,z,id});
        }
        sol(mid+1,r,q2);
        dsu.back_k(mid-l+1);
        sol(l,mid,q1);
    };
    sol(1,m,qr);
    for(int i=1;i<=q;i++)
    {
        cout<<ans[i]<<'\n';
    }
```

9.6 离散化

用法示例:

```
    vector<int> a;
    sort(a.begin(),a.end());
    a.erase(unique(a.begin(),a.end()),a.end());
    auto getid=[&](int x){return lower_bound(a.begin(),a.end(),x)-a.begin()+1;};
```